

MODEL PREDICTIVE CONTROL

Lecture Handout

Contents

I	Introduction	9
1	Introduction to Model Predictive Control	11
1.1	Optimization-Based Control	11
1.2	Concept of Model Predictive Control	12
1.2.1	Why constraints matter	13
1.3	Applications of MPC	13
1.4	A Brief History of MPC	14
1.5	Summary and Main Aspects	15
2	Systems theory basics	17
2.1	Models of Dynamic Systems	17
2.1.1	Nonlinear continuous-time state-space models	17
2.1.2	Linear time-invariant continuous-time models	18
2.1.3	Linearization around an operating point	18
2.1.4	Discrete-time state-space models	20
2.1.5	Euler discretization	20
2.1.6	Exact discretization of linear systems	21
2.1.7	Solution of linear discrete-time systems	21
2.2	Analysis of LTI Discrete-Time Systems	21
2.2.1	Coordinate transformations	22
2.2.2	Stability of linear discrete-time systems	22
2.2.3	Controllability and stabilizability	22
2.2.4	Observability and detectability	23
2.3	Analysis of Nonlinear Discrete-Time Systems	24
2.3.1	Lyapunov stability of nonlinear systems	24
2.3.2	Lyapunov functions	24
2.3.3	Indirect Lyapunov method	25
2.3.4	Lyapunov stability of linear systems	25
2.4	Summary	26
3	Unconstrained Control	27
3.1	Introduction to Linear Quadratic Optimal Control	27
3.1.1	Finite-horizon linear quadratic regulation	27
3.2	Batch Approach	28
3.3	Recursive Approach and Dynamic Programming	29
3.3.1	Cost-to-go and one-step optimization	29
3.3.2	Bellman's principle of optimality	30
3.3.3	Comparison of batch and recursive viewpoints	31
3.4	Receding Horizon Interpretation	31
3.5	Infinite-Horizon LQR	32
3.5.1	Algebraic Riccati equation	32

3.5.2	Stability of infinite-horizon LQR	32
3.6	Terminal Weight and Quasi-Infinite-Horizon Thinking	34
3.7	Summary	35
4	Introduction to Convex Optimization	37
4.1	Main concepts	37
4.2	Convex sets	38
4.3	Convex functions	39
4.4	Convex optimization problems	40
4.4.1	Linear programs	41
4.4.2	Quadratic programs	41
4.4.3	Equivalent problem formulations	42
4.5	Optimality conditions	42
4.5.1	The Lagrangian and the dual problem	42
4.5.2	Examples: dual of an LP and of a QP	43
4.5.3	Strong duality and Slater's condition	45
4.5.4	Karush–Kuhn–Tucker conditions	45
4.5.5	Example: KKT conditions for a quadratic program	47
4.5.6	Sensitivity analysis	47
4.6	Summary	49
II	Nominal MPC	51
5	Constrained Finite Time Optimal Control	53
5.1	Constrained finite time optimal control	53
5.1.1	General CFTOC formulation	54
5.1.2	Feasible set	54
5.2	Quadratic-cost CFTOC and quadratic programming	55
5.2.1	QP formulation without eliminating the states	55
5.2.2	QP formulation by substitution	57
5.2.3	Quadratic-cost state-feedback solution	58
5.3	CFTOC with 1-norm and ∞ -norm costs	59
5.3.1	ℓ_∞ -minimization	60
5.3.2	ℓ_1 -minimization	61
5.3.3	Construction of the LP with substitution	61
5.3.4	1-/ ∞ -norm state-feedback solution	62
5.3.5	Solution properties: quadratic versus 1-/ ∞ -norm cost	63
5.4	Receding-horizon notation and control law	63
5.4.1	CFTOC problem solved at time k	64
5.4.2	Meaning of the prediction notation	64
5.4.3	From the optimal sequence to the applied input	65
5.4.4	Receding-horizon control law and closed loop	65
5.4.5	Time-invariant systems	65
5.5	Summary	66
6	Invariance	67
6.1	Objectives of constrained control	67
6.2	Invariance	68
6.2.1	Positively invariant sets	68
6.2.2	Pre-sets	68
6.2.3	Linear autonomous systems	69

6.2.4	Conceptual computation of the maximal invariant set	69
6.3	Control invariance	70
6.3.1	Definition and meaning	70
6.3.2	Controlled pre-sets	71
6.3.3	Linear systems and projection	71
6.3.4	From a control invariant set to a control law	72
6.4	Relation to MPC	72
6.5	Practical computation of invariant sets	73
6.5.1	Polyhedra and polytopes	73
6.5.2	Common constraints in MPC	73
6.5.3	Intersection and subset tests	73
6.5.4	Polytopic invariant sets	74
6.5.5	Ellipsoids and Lyapunov-based invariant sets	74
6.5.6	Largest ellipsoid contained in a polytope	75
6.6	Summary	75
7	Feasibility and Stability	77
7.1	Motivation	77
7.2	MPC: Key Points Illustrated	78
7.3	Loss of Feasibility and Stability in Finite-Horizon MPC	82
7.3.1	Loss of feasibility: double integrator example	83
7.3.2	Feasibility and stability depend on the tuning	86
7.3.3	Why finite-horizon MPC is short-sighted	87
7.3.4	The role of terminal cost and terminal constraints	88
7.4	Feasibility and Stability Guarantees in MPC	88
7.4.1	Lyapunov Stability Revisited	88
7.4.2	Zero Terminal State Constraint	91
7.4.3	General Terminal Sets	94
7.5	Region of Attraction and Practical Remarks	99
7.6	Extension to Nonlinear MPC	99
7.7	Summary	100
III	Practical MPC	103
8	Practical Issues in MPC	105
8.1	Motivation	105
8.2	Reference Tracking	105
8.2.1	From regulation to tracking	105
8.2.2	The steady-state target problem	106
8.2.3	MPC for reference tracking	107
8.2.4	Terminal Sets for Tracking	109
8.2.5	Offset-Free Tracking	111
8.3	Enlarging the Feasible Set	118
8.3.1	Why terminal constraints may be conservative	118
8.3.2	MPC without Terminal Set	118
8.3.3	Soft-Constrained MPC	120
8.4	Putting the Practical MPC Architecture Together	129
8.5	Summary	130

IV	Robust MPC	131
9	Robust MPC for Linear Systems	133
9.1	Motivation	133
9.2	Uncertainty Models	133
9.2.1	MPC predictions and the real system	133
9.2.2	Goals of robust constrained control	134
9.2.3	Common uncertainty models	134
9.3	Tools: Robust Invariance, Set Difference and Set Sum	135
9.3.1	Robust invariance and robust pre-sets	135
9.3.2	Minkowski sum and Pontryagin difference	139
9.4	Impact of Bounded Additive Noise	140
9.4.1	Uncertain state evolution	140
9.4.2	Robust constrained control under bounded additive noise	142
9.5	Robust Open-Loop MPC	146
9.5.1	Putting the ingredients together	146
9.5.2	Interpretation and limitations	147
9.6	Closed-Loop Predictions	148
9.6.1	Why open-loop robust predictions are conservative	148
9.6.2	Control policies	148
9.6.3	Structured closed-loop predictions	149
9.7	Error Dynamics	150
9.7.1	Splitting nominal planning and error correction	150
9.7.2	Derivation of the error dynamics	150
9.7.3	Uncertain state evolution of the error	151
9.7.4	Minimum robust invariant set	152
9.7.5	Example: convergence to the minimum robust invariant set	153
9.7.6	From error bounds to robust MPC: two variants	154
9.8	Robust Constraint-Tightening MPC with Error Feedback	155
9.8.1	Noisy trajectory around a nominal trajectory	155
9.8.2	Tightened state and input constraints	156
9.8.3	Robust constraint-tightening MPC problem	157
9.8.4	Properties of robust constraint-tightening MPC	159
9.8.5	Intuition	160
9.8.6	Proof sketch	160
9.9	Robust Tube MPC	162
9.9.1	Idea and construction	162
9.9.2	Tube MPC formulation and assumptions	166
9.9.3	Robust invariance and stability	168
9.9.4	Implementation and example	170
9.9.5	Tube MPC summary	176
9.10	Comparison of Robust Constraint-Tightening MPC and Tube MPC	177
9.11	Summary	178
10	Robustness of Nominal MPC	181
10.1	Motivation	181
10.2	Nominal MPC with Noise	182
10.2.1	Ignoring the noise	182
10.2.2	Example: many disturbance realizations	182
10.3	What Happens to the Lyapunov Function?	183
10.3.1	Nominal Lyapunov decrease	183
10.3.2	Perturbed Lyapunov decrease	184

10.4 Special Case: When Nominal MPC is Robust	185
10.5 Input-to-State Stability Interpretation	185
10.6 Discussion	186
10.7 Summary	187
V Implementation Methods	189
11 Implementation Methods	191
11.1 Motivation	191
11.2 Explicit MPC	192
11.2.1 Parametric viewpoint	192
11.2.2 Active sets and critical regions	193
11.2.3 A scalar example	193
11.2.4 Example: double integrator	195
11.2.5 Norm-based costs	195
11.3 Online Evaluation of Explicit MPC	196
11.3.1 Sequential search	197
11.3.2 Logarithmic search	197
11.3.3 Summary of explicit MPC implementation	199
11.4 Iterative Numerical Optimization	199
11.4.1 Why numerical methods are needed	199
11.5 Unconstrained Minimization	200
11.5.1 Descent methods	201
11.5.2 Gradient method	201
11.5.3 Newton's method	202
11.5.4 Line search	203
11.5.5 Equality constraints in Newton's method	204
11.6 Constrained Minimization	205
11.6.1 Projected gradient method	205
11.6.2 Implications for MPC	206
11.7 Interior-Point Methods	207
11.7.1 Constrained problem and KKT conditions	207
11.7.2 Primal-dual interior-point idea	208
11.7.3 Primal-dual search direction	209
11.7.4 Interior-point methods in MPC	210
11.8 Implementation Tradeoffs	211
11.9 Summary	211

Part I

Introduction

Chapter 1

Introduction to Model Predictive Control

1.1 Optimization-Based Control

A central idea behind model predictive control is that the control action is not generated by a fixed pre-computed feedback law alone, but by repeatedly solving an optimization problem online. This point of view is best understood by comparing it with the classical control loop. In a standard feedback architecture, the controller receives a reference and the measured output of the plant, and then applies an input according to a control law that has been designed in advance. In optimization-based control, this fixed controller block is replaced, at least conceptually, by an optimizer.

The optimizer uses a model of the plant to predict how the system will evolve under candidate future inputs. Based on these predictions, it computes an input sequence that best satisfies the desired control objectives while respecting the relevant constraints. The first important shift is therefore methodological: instead of asking directly for a controller formula, we formulate a decision problem whose solution determines the input to be applied.

This approach is particularly natural when the control objective is tightly coupled with physical limitations and safety requirements. Consider, for instance, the problem of driving a race car around a track. One may wish to minimize the lap time, but this objective cannot be pursued arbitrarily. The car must stay on the road, avoid collisions, respect limitations on tire friction, and remain within admissible acceleration and steering capabilities. The control task is therefore inherently constrained. A meaningful strategy is to look ahead, predict the upcoming road geometry and the effect of future decisions, and then plan a feasible trajectory that achieves the best possible performance. This is precisely the kind of reasoning that optimization-based control formalizes.

The same idea can be described algorithmically as follows. At a given time instant, one solves an optimization problem to compute a planned trajectory and a corresponding sequence of future control actions. From this sequence, only the first control move is actually implemented. At the next sampling instant, the state is measured again, the prediction is updated, and a new optimization problem is solved. In this way, planning and feedback are combined in a single framework.

Optimization-based control

The key idea is to compute the control input by solving, at each sampling instant, a finite-dimensional optimization problem built from three ingredients: a performance objective, a

predictive model of the plant, and a set of constraints.

1.2 Concept of Model Predictive Control

Model predictive control, usually abbreviated as MPC, is the most prominent realization of optimization-based control. The name itself summarizes its defining features. It is *model-based* because future behavior is predicted through an internal model of the plant. It is *predictive* because the controller explicitly reasons over a future horizon. Finally, it is a *control* method because the resulting optimization problem is solved repeatedly in closed loop in order to decide the plant input.

A useful way to understand MPC is to compare it with more classical control design philosophies. In classical control, one typically designs a controller C for a plant P so as to achieve good disturbance rejection, limited sensitivity to measurement noise, and robustness to modeling uncertainty. The dominant language is often that of transfer functions, loop shaping, and frequency-domain specifications. By contrast, MPC is naturally expressed in the **time domain**. Its strength lies in the **explicit treatment of constraints**, both on the manipulated inputs and on the system variables. This makes it especially appealing in applications where safety, actuator saturation, and operating envelopes are of central importance.

Indeed, all physical systems are constrained. Inputs cannot be arbitrarily large because actuators have limits. Outputs and states are often restricted by performance or safety considerations, such as maximum temperature, pressure, velocity, or overshoot. In many practical settings, the economically or operationally optimal regime lies close to such limits. Traditional control methods often handle constraints only indirectly, for example by tuning the set-point conservatively or by adding ad hoc supervisory logic. MPC instead incorporates the constraints directly into the control design. This allows operation closer to the admissible boundaries, while still maintaining systematic decision-making.

Mathematically, MPC can be formulated over a finite prediction horizon of length N . At a generic time instant k , one considers the current state $x(k)$ and searches for a sequence of future control inputs

$$U_k = \{u_k, u_{k+1}, \dots, u_{k+N-1}\}$$

that minimizes a cumulative performance criterion. A standard formulation is

$$U_k^*(x(k)) := \arg \min_{U_k} \sum_{i=0}^{N-1} \ell(x_{k+i}, u_{k+i})$$

subject to the system dynamics

$$x_{k+i+1} = Ax_{k+i} + Bu_{k+i},$$

the initial condition

$$x_k = x(k),$$

and the constraints

$$x_{k+i} \in \mathcal{X}, \quad u_{k+i} \in \mathcal{U},$$

for all relevant indices i .

This compact expression already contains the essential ingredients of MPC. The objective function defines what the controller tries to optimize. The model provides predictions of future behavior. The constraint sets $\mathcal{X}$ and $\mathcal{U}$ encode admissible operating conditions. The decision variables are the future inputs across the prediction horizon.

The online operation of MPC follows a receding-horizon principle. At each sampling time, the current state is measured or estimated, the finite-horizon optimization problem is solved, and an optimal input sequence

$$U_k^* = \{u_k^*, u_{k+1}^*, \dots, u_{k+N-1}^*\}$$

is obtained. However, only the first element u_k^* is applied to the plant. After one sampling interval, the state is updated and the entire optimization is solved again from the new initial condition. The horizon therefore moves forward in time, which is why one also speaks of a *receding horizon* strategy.

This repeated re-optimization is what **introduces feedback**. If the plant is disturbed, if the model is imperfect, or if the measured state differs from the previously predicted one, the next optimization step automatically adjusts the future plan. The controller does not blindly execute an open-loop plan; instead, it continuously replans using fresh information.

Receding horizon principle

MPC computes an *optimal sequence* of future control moves, but implements only the *first* one. Feedback arises because the optimization problem is solved again at the next sampling instant using the newly measured or estimated state.

1.2.1 Why constraints matter

The explicit inclusion of constraints is one of the main reasons for the success of MPC. Suppose one wants to regulate a process output to a desired set-point while an upper safety limit must never be violated. A classical controller may achieve fast convergence but produce overshoot that crosses the admissible bound. To avoid this, one may reduce aggressiveness or move the set-point away from the constraint, which typically leads to suboptimal operation. MPC offers a more systematic alternative: the constraint is included directly in the optimization problem, and the computed control action is chosen with full awareness of that limit.

This ability to reason ahead is crucial. The optimizer can anticipate that a constraint will become active in the near future and can begin adjusting the input before the violation occurs. In this sense, MPC is predictive not only in the use of a model, but also in the way it handles constraints proactively rather than reactively.

1.3 Applications of MPC

One of the remarkable features of MPC is the **breadth of its application domain**. The same conceptual framework appears in problems evolving at extremely different time scales.

At very fast time scales, optimization-based control ideas arise in computer and electronic systems, where decisions may need to be made on the **nanosecond or microsecond** level. Slightly slower, but still extremely fast, are power system applications, where control actions and operational decisions can be taken in the microsecond range depending on the problem formulation and hardware layer under consideration.

At the **millisecond** scale, traction control and automotive systems provide a classical application area. Here, one must coordinate performance objectives with safety and actuator limits in real time. On the scale of **seconds**, building control becomes relevant: thermal dynamics are slow enough to allow relatively long prediction horizons, yet constraints and energy-efficiency objectives play a major role. At the scale of **minutes and hours**, refinery operation and process control have historically been among the main industrial drivers for MPC. At even slower scales, **days or weeks**, one finds planning and scheduling problems such as train scheduling, nurse

rostering, and production planning, where the same optimization-and-prediction philosophy remains useful.

These examples show that MPC should not be viewed only as a single algorithm for fast dynamical systems. Rather, it is a general methodology for repeated decision-making under dynamical models and constraints. The exact mathematical models, optimization problems, and computational requirements vary greatly across applications, but the underlying structure remains the same.

1.4 A Brief History of MPC

The historical roots of MPC go back to the early idea of using optimization methods directly inside control algorithms. An early milestone is the work of A. I. Propoi in 1963, where linear programming ideas were proposed for sampled-data automatic systems. This already contains the seed of a now-familiar idea: control can be posed as the repeated solution of an optimization problem.

The modern industrial development of MPC is commonly traced to the 1970s. A key step was the work of Richalet and co-authors in 1978 on what became known as IDCOM, short for *Identification and Command*. Their approach used an impulse response model of the plant, considered a finite prediction horizon, and formulated a quadratic performance objective. Future plant behavior was described relative to a desired reference trajectory, and practical input/output constraints were included in an ad hoc manner. Because the resulting controller was not expressed as a conventional transfer function but rather as the outcome of an iterative computational procedure, the approach was described as heuristic predictive control. At this stage, the framework was mainly aimed at stable plants and was strongly motivated by industrial needs.

A closely related and highly influential development came from Cutler and collaborators. In the late 1970s, Cutler proposed ideas that were later implemented at Shell under the name *Dynamic Matrix Control* (DMC). The 1979 work by Cutler and Ramaker is particularly important in the industrial history of MPC. DMC used a step response model of the plant, a finite-horizon quadratic objective, and a prediction mechanism that aimed at tracking the set-points as closely as possible. The optimal manipulated variables were computed through a least-squares problem, and additional equations were introduced online to account for constraints. The name “dynamic matrix” refers precisely to the structured matrix that arises in this least-squares formulation. A further important step occurred in 1983, when Cutler, Morshedi, and Haydel emphasized the formulation of the controller as a standard quadratic program. This was conceptually decisive, because it provided a systematic mathematical framework for handling constraints rather than relying only on heuristic modifications. From that point on, MPC gradually evolved from an industrial engineering practice into a discipline with a firm optimization-theoretic foundation.

In the mid 1990s, much theoretical work was devoted to understanding under which conditions one can guarantee recursive feasibility and closed-loop stability. This period established many of the fundamental results that are now part of the standard theory of nominal MPC. During the 2000s, research expanded toward tractable robust MPC methods, nonlinear MPC, hybrid systems, and fast MPC algorithms tailored to increasingly demanding real-time requirements. In the 2010s, important developments included stochastic MPC, distributed and large-scale MPC, and economic MPC, where the objective is not merely set-point tracking but direct optimization of process performance. In the 2020s, a major direction has been the interaction between MPC and learning, leading to a growing body of work on learning-based MPC.

1.5 Summary and Main Aspects

Model predictive control can therefore be summarized as a control methodology in which, at every sampling instant, one measures or estimates the current state, predicts future behavior with an internal model, solves a constrained finite-horizon optimization problem, and applies only the first input of the optimal sequence. The procedure is then repeated at the next sampling instant.

Its main appeal lies in the fact that performance objectives and operating constraints are treated within a single optimization-based framework. This gives the designer a systematic way to handle actuator limits, safety requirements, and multi-variable interactions while maintaining a feedback structure through repeated re-planning.

At the same time, MPC also comes with **important challenges**. First, the optimization problem must be solved reliably and fast enough for real-time use. Second, the finite-horizon problem may become infeasible if the constraints are too restrictive or if the system is driven into an unfavorable region. Third, closed-loop stability is not automatic and must typically be enforced or analyzed through suitable design choices. Finally, the closed-loop behavior may be sensitive to model mismatch and disturbances, which motivates the study of robustness and the many extensions of the nominal MPC framework.

Chapter 2

Systems theory basics

2.1 Models of Dynamic Systems

Model predictive control relies on a mathematical model of the system to predict future behavior. This is one of the fundamental ingredients of the entire framework: without a model, there is no basis for forecasting the effect of future control inputs and therefore no basis for optimization-based planning. In practice, one also needs a state estimator, a suitable optimal control problem formulation, a numerical optimization procedure, and finally a verification that the resulting closed-loop system behaves as desired. In this sense, system theory provides the language and tools on which MPC is built.

Dynamic-system models can be classified in several ways. One may distinguish between state-space and transfer-function descriptions, between linear and nonlinear models, between time-varying and time-invariant dynamics, between continuous-time and discrete-time formulations, and between deterministic and stochastic models. In this handout, as in the lecture, the focus is mainly on deterministic models. Physical systems derived from first principles are typically described by nonlinear, time-invariant, continuous-time state-space models, whereas the standard models used in MPC are most often linear, time-invariant, discrete-time state-space models. A substantial part of the modeling workflow therefore consists in transforming the former into the latter in a meaningful way.

2.1.1 Nonlinear continuous-time state-space models

A very general class of dynamical systems can be written in **continuous-time state-space form** as

$$\dot{x} = g(x, u), \quad y = h(x, u),$$

where $x \in \mathbb{R}^n$ is the state vector, $u \in \mathbb{R}^m$ is the input, and $y \in \mathbb{R}^p$ is the output. The map $g : \mathbb{R}^n \times \mathbb{R}^m \rightarrow \mathbb{R}^n$ describes the system dynamics, while $h : \mathbb{R}^n \times \mathbb{R}^m \rightarrow \mathbb{R}^p$ specifies which quantities are measured or observed. This representation is extremely flexible and can describe a very broad range of physical systems.

An important reason why the state-space formalism is so useful is that higher-order ordinary differential equations can always be rewritten as systems of first-order equations. If one starts from an n th-order differential equation

$$x^{(n)} + g_n(x, \dot{x}, \ddot{x}, \dots, x^{(n-1)}) = 0$$

suitable state variables can be introduced by stacking the variable and its derivatives up to order $n - 1$. This is possible by defining

$$x_{j+1} = x^{(j)}, \quad j = 0, 1, \dots, n - 1,$$

where $x^{(0)}$ is simply x . The original equation can then be rewritten as

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \\ \vdots \\ \dot{x}_{n-1} \\ \dot{x}_n \end{bmatrix} = \begin{bmatrix} x_2 \\ x_3 \\ \vdots \\ x_n \\ -g_n(x_1, x_2, \dots, x_n) \end{bmatrix}.$$

The original equation is then transformed into n coupled first-order equations. This conversion is not merely formal: it provides a standard representation in which analysis and controller design can be carried out systematically.

The **drawback** of such nonlinear descriptions is that, although they are general and physically meaningful, their analysis and controller synthesis are often difficult. For this reason, one often replaces them locally by linear approximations around a chosen operating point.

2.1.2 Linear time-invariant continuous-time models

A continuous-time linear time-invariant system is written as

$$\dot{x} = A_c x + B_c u, \quad y = Cx + Du,$$

where A_c , B_c , C , and D are constant matrices of compatible dimensions. Compared with the nonlinear case, this model class is much more **restrictive**, but it has a decisive **advantage**: a vast mathematical theory is available for its analysis and control design.

One important feature of linear systems is that their solution can be written explicitly. If

$$\dot{x}(t) = A_c x(t) + B_c u(t), \quad x(t_0) = x_0,$$

then the state at time t is given by

$$x(t) = e^{A_c(t-t_0)} x_0 + \int_{t_0}^t e^{A_c(t-\tau)} B_c u(\tau) d\tau,$$

where the matrix exponential is defined by

$$e^{A_c t} := \sum_{n=0}^{\infty} \frac{(A_c t)^n}{n!}.$$

This expression makes the effect of the initial condition and the input signal fully explicit.

2.1.3 Linearization around an operating point

Most real systems are nonlinear, yet linear models are much easier to analyze and use in controller design. The standard compromise is therefore to **linearize the nonlinear dynamics around an operating point**. The idea is that if the system is to operate near a desired equilibrium, then a first-order approximation may be sufficiently accurate in a neighborhood of that point.

The mathematical basis is the first-order Taylor expansion. If a nonlinear function is expanded around a point $\bar{x}$, then near $\bar{x}$ it can be approximated by its value at $\bar{x}$ plus the Jacobian times the deviation from that point. The first order Taylor approximation of a function $f(\cdot)$ around $\bar{x}$ is

$$f(x) \approx f(\bar{x}) + \left. \frac{\partial f}{\partial x^\top} \right|_{x=\bar{x}} (x - \bar{x}) \quad \text{with} \quad \frac{\partial f}{\partial x^\top} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \dots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_n}{\partial x_1} & \dots & \frac{\partial f_n}{\partial x_n} \end{bmatrix}$$

Applied to a nonlinear state-space system

$$\dot{x} = g(x, u), \quad y = h(x, u),$$

and linearized around a stationary operating point (x_s, u_s) satisfying

$$g(x_s, u_s) = 0,$$

one obtains a linear model in deviation variables:

$$\dot{x} = \underbrace{g(x_s, u_s)}_{=0} + \underbrace{\frac{\partial g}{\partial x^\top} \Big|_{x=x_s, u=u_s}}_{=A_c} \underbrace{(x - x_s)}_{\Delta x} + \underbrace{\frac{\partial g}{\partial u^\top} \Big|_{x=x_s, u=u_s}}_{=B_c} \underbrace{(u - u_s)}_{\Delta u}$$

Thus

$$\dot{x} - \underbrace{\dot{x}_s}_{=0} = \Delta \dot{x} = A_c \Delta x + B_c \Delta u$$

Proceeding in the same way for the output equation, one obtains

$$\Delta y = \underbrace{\frac{\partial h}{\partial x^\top} \Big|_{x=x_s, u=u_s}}_{=C} \underbrace{(x - x_s)}_{\Delta x} + \underbrace{\frac{\partial h}{\partial u^\top} \Big|_{x=x_s, u=u_s}}_{=D} \underbrace{(u - u_s)}_{\Delta u}$$

Thus we obtain the linearized model

$$\Delta \dot{x} = A_c \Delta x + B_c \Delta u, \quad \Delta y = C \Delta x + D \Delta u,$$

where

$$A_c = \frac{\partial g}{\partial x^\top} \Big|_{x=x_s, u=u_s}, \quad B_c = \frac{\partial g}{\partial u^\top} \Big|_{x=x_s, u=u_s},$$

$$C = \frac{\partial h}{\partial x^\top} \Big|_{x=x_s, u=u_s}, \quad D = \frac{\partial h}{\partial u^\top} \Big|_{x=x_s, u=u_s}.$$

After linearization, it is common to drop the Δ notation and simply denote deviations again by x , u , and y .

The pendulum example clearly illustrates both the usefulness and the limitations of this procedure. Around a chosen equilibrium, such as a constant nonzero angle with zero angular velocity, the nonlinear sine term is replaced by its local linear approximation. The resulting matrices depend on the selected operating point. The linearized dynamics provide a good approximation only in a small neighborhood: for small angles and velocities the behavior of the linear and nonlinear systems is close, whereas farther away the approximation deteriorates. This local nature of linearization is essential to keep in mind in MPC modeling.

Linearization

Linearization replaces a nonlinear model by a local linear approximation around a stationary operating point. It is extremely useful for analysis and design, but its validity is inherently local: it can only be trusted in a sufficiently small neighborhood of the linearization point.

2.1.4 Discrete-time state-space models

Since controllers are typically implemented on digital hardware, MPC is naturally formulated in discrete time. A **discrete-time system** is described by difference equations of the form

$$x(k+1) = g(x(k), u(k)), \quad y(k) = h(x(k), u(k)),$$

where the input and output are defined only at discrete instants indexed by $k \in \mathbb{Z}_+$. Some systems are inherently discrete, such as an account balance updated every month, but most control systems originate in continuous time and are then discretized for implementation. The sampling instants are typically of the form

$$t_{k+1} = t_k + T_s,$$

where T_s is the sampling time. A standard assumption is that the control input is held constant between two consecutive sampling instants,

$$u(t) = u(t_k), \quad t \in [t_k, t_{k+1}),$$

which is the usual zero-order-hold implementation.

2.1.5 Euler discretization

For nonlinear systems, one simple discretization method is the forward Euler approximation. Starting from the continuous-time model

$$\dot{x}_c(t) = g_c(x_c(t), u_c(t)), \quad y_c(t) = h_c(x_c(t), u_c(t)),$$

one approximates the derivative by

$$\dot{x}_c(t) \approx \frac{x_c(t + T_s) - x_c(t)}{T_s}.$$

where T_s is the sampling time. Introducing the notation

$$x(k) = x_c(t_0 + kT_s), \quad u(k) = u_c(t_0 + kT_s),$$

one obtains the discrete-time approximation

$$x(k+1) = x(k) + T_s g_c(x(k), u(k)), \quad y(k) = h_c(x(k), u(k)).$$

Under suitable regularity assumptions and for sufficiently small sampling times, the discrete-time trajectory approximates the continuous-time one well.

For linear continuous-time systems,

$$\dot{x}(t) = A_c x(t) + B_c u(t), \quad y(t) = C_c x(t) + D_c u(t),$$

Euler discretization yields

$$x(k+1) = A x(k) + B u(k), \quad y(k) = C x(k) + D u(k),$$

with

$$A = I + T_s A_c, \quad B = T_s B_c, \quad C = C_c, \quad D = D_c.$$

This is straightforward and often useful, although it is not exact.

2.1.6 Exact discretization of linear systems

For linear systems under zero-order hold, one can derive an exact discrete-time model. Using the explicit continuous-time solution and assuming that the input remains constant over one sampling interval, one obtains

$$x(t_{k+1}) = e^{A_c T_s} x(t_k) + \left(\int_0^{T_s} e^{A_c(T_s-\tau)} B_c d\tau \right) u(t_k).$$

Hence the exact discrete-time system is

$$x(k+1) = Ax(k) + Bu(k),$$

with

$$A = e^{A_c T_s}, \quad B = \int_0^{T_s} e^{A_c(T_s-\tau)} B_c d\tau.$$

If A_c is invertible, then B can also be written as

$$B = A_c^{-1}(A - I)B_c.$$

This model is **exact** in the sense that it predicts the state of the continuous-time system exactly at the sampling instants, provided the zero-order-hold assumption is satisfied.

2.1.7 Solution of linear discrete-time systems

One major advantage of discrete-time linear systems is that their future state can be written explicitly as a function of the initial condition and the input sequence. Starting from

$$x(k+1) = Ax(k) + Bu(k),$$

successive substitution gives

$$x(k+N) = A^N x(k) + \sum_{i=0}^{N-1} A^i B u(k+N-1-i).$$

Thus, for fixed matrices A and B , the predicted state over a horizon is a **linear function of the current state and of the future inputs**. This observation is fundamental in MPC, because it allows the finite-horizon optimization problem to be written directly in terms of the decision variables and the known initial state.

The modeling discussion can be summarized as follows. Continuous-time nonlinear models are general and physically natural, but they are difficult to analyze and use directly in predictive control. Linearization gives a local continuous-time approximation around an operating point. Discretization then converts the model into a form that is directly compatible with digital control implementation. For linear systems, exact zero-order-hold discretization is available, while for general nonlinear systems one often relies on approximate methods such as Euler discretization, whose quality depends strongly on the sampling time. These are precisely the model transformations that make standard MPC formulations possible.

2.2 Analysis of LTI Discrete-Time Systems

Once a model has been put into discrete-time LTI form, one can study several structural properties that are central in estimation and control. Among the most important are the behavior of the system under coordinate changes, asymptotic stability, controllability, and observability. These notions form the classical backbone of linear systems theory and play a direct role in MPC.

2.2.1 Coordinate transformations

Consider the discrete-time LTI system

$$x(k+1) = Ax(k) + Bu(k), \quad y(k) = Cx(k) + Du(k).$$

The input-output behavior is completely determined by the initial state and the input sequence. However, the choice of state variables is **not unique**. Different state representations may generate exactly the same input-output behavior, and some representations may be more convenient than others for analysis or design.

If one introduces a new state variable

$$\tilde{x}(k) = Tx(k),$$

where T is invertible, then the transformed system becomes

$$\tilde{x}(k+1) = \tilde{A}\tilde{x}(k) + \tilde{B}u(k), \quad y(k) = \tilde{C}\tilde{x}(k) + \tilde{D}u(k),$$

with

$$\tilde{A} = TAT^{-1}, \quad \tilde{B} = TB, \quad \tilde{C} = CT^{-1}, \quad \tilde{D} = D.$$

The input and output remain unchanged; only the internal coordinates have changed. Such transformations are useful because they can simplify the structure of the matrices and thereby reveal important properties more transparently.

2.2.2 Stability of linear discrete-time systems

For the autonomous system

$$x(k+1) = Ax(k),$$

asymptotic stability means that every trajectory converges to the origin as $k \rightarrow \infty$. A fundamental theorem states that this happens if and only if all eigenvalues of A lie strictly inside the unit circle, that is,

$$|\lambda_j| < 1, \quad j = 1, \dots, n.$$

This is the discrete-time analogue of the continuous-time condition $\text{Re}(\lambda_j) < 0$.

The intuition becomes especially clear when the system can be diagonalized. In suitable coordinates, the system matrix becomes diagonal, and each transformed state component evolves independently as multiplication by the corresponding eigenvalue. If the magnitude of every eigenvalue is less than one, repeated multiplication drives each component to zero. If any eigenvalue has magnitude greater than or equal to one, some component fails to decay. Even when diagonalization is not possible and Jordan forms must be used, the same conclusion remains valid.

2.2.3 Controllability and stabilizability

A system

$$x(k+1) = Ax(k) + Bu(k)$$

is **controllable** if, for any initial state $x(0)$ and any desired target state x^* , there exists a finite time N and an input sequence $\{u(0), u(1), \dots, u(N-1)\}$ that steers the system exactly to that target, i.e. $x(N) = x^*$. In discrete time, this notion is often also called reachability. By expanding the state recursively:

$$x^* = x(N) = A^N x(0) + \sum_{i=0}^{N-1} A^i B u(N-1-i) = A^N x(0) + \begin{bmatrix} B & AB & \cdots & A^{N-1}B \end{bmatrix} \begin{bmatrix} u(N-1) \\ u(N-2) \\ \vdots \\ u(0) \end{bmatrix}$$

It follows from the **Carley-Hamilton** theorem that A^k can be expressed as a linear combination of $I, A, \dots, A^{n-1}$, so that for all $N \geq n$

$$\text{range} \begin{bmatrix} B & AB & \dots & A^{N-1}B \end{bmatrix} = \begin{bmatrix} B & AB & \dots & A^{n-1}B \end{bmatrix}$$

We therefore conclude that the reachable states are precisely those that can be generated by the columns of the **controllability matrix**

$$\mathcal{C} = \begin{bmatrix} B & AB & \dots & A^{n-1}B \end{bmatrix},$$

this means that the reachable states are precisely those in the column space of $\mathcal{C}$. The system is controllable **if and only if**

$$\text{rank}(\mathcal{C}) = n.$$

A weaker notion is **stabilizability**. A system is stabilizable if the unstable part of the dynamics can still be influenced by the input, even if some already stable modes are uncontrollable. More precisely, the system is stabilizable if all uncontrollable modes are stable. Controllability always implies stabilizability, but the converse need not hold. In many feedback-design settings, stabilizability is the essential requirement.

Stabilizability can be checked using the following condition

$$\text{if } \text{rank} \left(\begin{bmatrix} \lambda_j I - A & B \end{bmatrix} \right) = n \quad \forall \lambda_j \in \Lambda_A^+ \Rightarrow (A, B) \text{ is stabilizable}$$

where Λ_A^+ is the set of eigenvalues of A lying on or outside the unit circle.

2.2.4 Observability and detectability

Dual to controllability is the notion of **observability**. Consider the zero-input system

$$x(k+1) = Ax(k), \quad y(k) = Cx(k).$$

The system is observable if the initial state can be uniquely reconstructed from a finite sequence of output measurements. By stacking the outputs over time, one obtains the linear relation

$$\begin{bmatrix} y(0) \\ y(1) \\ \vdots \\ y(n-1) \end{bmatrix} = \begin{bmatrix} C \\ CA \\ \vdots \\ CA^{n-1} \end{bmatrix} x(0).$$

The matrix

$$\mathcal{O} = \begin{bmatrix} C \\ CA \\ \vdots \\ CA^{n-1} \end{bmatrix}$$

is called the **observability matrix**, and the system is observable **if and only if**

$$\text{rank}(\mathcal{O}) = n.$$

Again, the use of only the first n blocks is justified by the Cayley-Hamilton theorem.

The weaker property corresponding to stabilizability is **detectability**. A system is detectable if all unobservable modes are stable, so that although the state may not be reconstructed exactly, the estimation error can still be driven to zero asymptotically. Observability implies detectability,

but not every detectable system is observable. In practice, detectability is often the minimal property needed to construct a convergent observer or state estimator. Observability can be checked using the following condition

$$\text{if } \text{rank} \begin{pmatrix} \lambda_j I - A \\ C \end{pmatrix} = n \quad \forall \lambda_j \in \Lambda_A^+ \Rightarrow (A, C) \text{ is detectable}$$

where Λ_A^+ is the set of eigenvalues of A lying on or outside the unit circle.

Structural properties

Controllability and observability are exact structural properties of a model. Their relaxed counterparts, stabilizability and detectability, are often sufficient for controller and observer design and are especially relevant when only the unstable modes must be influenced or reconstructed.

2.3 Analysis of Nonlinear Discrete-Time Systems

The previous section dealt with discrete-time LTI systems, for which many results can be stated in terms of matrices and eigenvalues. For nonlinear systems, the situation is more subtle. Stability is no longer merely a property of the matrix A , but a **property of an equilibrium point** of the nonlinear dynamics. The appropriate conceptual framework is Lyapunov stability theory.

2.3.1 Lyapunov stability of nonlinear systems

Consider an autonomous nonlinear discrete-time system

$$x(k+1) = g(x(k)),$$

and let $\bar{x}$ be an equilibrium point, meaning that

$$g(\bar{x}) = \bar{x}.$$

Lyapunov stability formalizes the intuitive idea that if the system is perturbed only slightly from equilibrium, then it should remain close to equilibrium for all future time. More precisely, $\bar{x}$ is Lyapunov stable if for every $\varepsilon > 0$ there exists a $\delta(\varepsilon) > 0$ such that

$$\|x(0) - \bar{x}\| < \delta(\varepsilon) \implies \|x(k) - \bar{x}\| < \varepsilon \quad \forall k \geq 0.$$

If, in addition, trajectories converge to the equilibrium (i.e. is attractive), meaning that

$$\lim_{k \rightarrow \infty} \|x(k) - \bar{x}\| = 0 \quad \forall x(0) \in \mathbb{R}^n,$$

then the equilibrium is **globally asymptotically stable**.

2.3.2 Lyapunov functions

A standard way to prove stability is to construct a Lyapunov function. The intuition comes from mechanics: if the total energy of a damped system decreases over time, then the system tends to settle to an equilibrium. Lyapunov functions generalize this idea to arbitrary dynamical systems. For an equilibrium at the origin, a **Lyapunov function** is a scalar function

$$V : \mathbb{R}^n \rightarrow \mathbb{R}$$

that is continuous at the origin, finite everywhere, radially unbounded, positive definite, and strictly decreases along system trajectories. In particular, one requires

$$V(0) = 0, \quad V(x) > 0 \text{ for all } x \neq 0,$$

$$\|x\| \rightarrow \infty \implies V(x) \rightarrow \infty,$$

and

$$V(g(x)) - V(x) \leq -\alpha(x),$$

where α is a continuous positive definite function. **If such a function exists, then the origin is globally asymptotically stable.** If the decrease condition holds only with α positive semidefinite, then one obtains Lyapunov stability but not necessarily asymptotic convergence.

2.3.3 Indirect Lyapunov method

Although Lyapunov functions are powerful, they are often **difficult to construct** for general nonlinear systems. A useful alternative is the indirect Lyapunov method, which studies the linearization around the equilibrium. Let

$$A = \left. \frac{\partial g}{\partial x^\top} \right|_{x=0},$$

then the origin is **locally asymptotically stable** if all eigenvalues of A satisfy

$$|\lambda_i| < 1.$$

On the other hand, if at least one eigenvalue satisfies

$$|\lambda_i| > 1,$$

then the origin is unstable. The borderline case in which some eigenvalue lies exactly on the unit circle is **inconclusive**, and one must resort to a more refined analysis, often through a Lyapunov function.

This theorem is especially important because it connects nonlinear local stability with the familiar eigenvalue test from linear systems. It also clarifies why linearization is so useful: it can certify local asymptotic stability when the equilibrium lies safely inside the stable region, but it cannot always settle the question in marginal cases.

2.3.4 Lyapunov stability of linear systems

For **linear systems**, Lyapunov theory becomes constructive. Consider

$$x(k+1) = Ax(k).$$

A natural candidate Lyapunov function is quadratic,

$$V(x) = x^\top Px,$$

with $P = P^\top > 0$. Evaluating the change along trajectories gives

$$V(x(k+1)) - V(x(k)) = x(k)^\top (A^\top PA - P)x(k).$$

If one can choose $P > 0$ such that

$$A^\top PA - P = -Q, \quad Q > 0, \tag{2.1}$$

then

$$V(x(k+1)) - V(x(k)) = -x(k)^\top Q x(k) < 0$$

for all nonzero $x(k)$, which proves asymptotic stability. Equation (2.1) is the **discrete-time Lyapunov equation**.

A central theorem states that, for discrete-time LTI systems, the Lyapunov equation has a **unique positive definite solution** P for every given $Q > 0$ **if and only if** all eigenvalues of A lie inside the unit circle. Thus, in the linear case, eigenvalue stability and the existence of a quadratic Lyapunov function are equivalent. Moreover, because the system is linear, this stability is automatically global (stability is always "global" for linear systems, since the dynamics are the same everywhere in the state space).

There is also an important **interpretation** of the matrix P in terms of infinite-horizon cost. If the system is asymptotically stable and one defines

$$\Psi(x(0)) = \sum_{k=0}^{\infty} x(k)^\top Q x(k),$$

then this infinite sum can be written as

$$\Psi(x(0)) = x(0)^\top P x(0),$$

where P is precisely the solution of the discrete-time Lyapunov equation.

We can prove this by defining $H_k = (A^k)^\top Q (A^k)$ and $P = \sum_{k=0}^{\infty} H_k$. Notice that the limit of the sum exists because the system is asymptotically stable. Then we have that

$$A^\top H_k A = (A^{k+1})^\top Q (A^{k+1}) = H_{k+1}$$

Thus

$$A^\top P A = A^\top \left(\sum_{k=0}^{\infty} H_k \right) A = \sum_{k=0}^{\infty} A^\top H_k A = \sum_{k=0}^{\infty} H_{k+1} = P - H_0 = P - Q$$

In other words, the same quadratic function that certifies stability also represents the infinite-horizon cost-to-go associated with the stage cost $x^\top Q x$. This link is conceptually very important for MPC, because later the optimal cost itself will play the role of a Lyapunov function for the closed-loop system under suitable assumptions.

2.4 Summary

The material of this chapter provides the basic system-theoretic background required for MPC. Starting from nonlinear continuous-time models, one typically linearizes around an operating point and then discretizes the system to obtain a model suitable for digital control implementation. For linear discrete-time systems, explicit predictions over a finite horizon can be written as linear functions of the current state and the future input sequence, which is exactly the structure exploited in predictive control.

On the analysis side, stability of LTI systems is characterized by eigenvalue location, while controllability and observability determine whether the system can be influenced and reconstructed from measurements. Their weaker counterparts, stabilizability and detectability, are often sufficient for design tasks. For nonlinear systems, Lyapunov theory provides the appropriate framework to study stability of equilibria. In the linear case, quadratic Lyapunov functions arise naturally through the discrete-time Lyapunov equation and connect stability analysis with infinite-horizon cost. This bridge between dynamics, stability, and cost is one of the conceptual pillars on which the theory of MPC is built.

Chapter 3

Unconstrained Control

3.1 Introduction to Linear Quadratic Optimal Control

The next step toward model predictive control is to study optimal control problems in a setting where the model is **linear**, the objective is **quadratic**, and **no inequality constraints** are present. Although such problems are **much simpler** than full MPC formulations, they already contain many of the central ideas of predictive control: one optimizes over a finite or infinite horizon, one trades off state regulation against control effort through a cost function, and one exploits the model to predict future trajectories. In addition, this class of problems admits explicit structure and elegant solution methods, which makes it an ideal starting point.

In discrete-time optimal control, the goal is to choose an input sequence

$$U := [u_0^\top, u_1^\top, \dots]^\top$$

so as to minimize a performance criterion starting from a given initial state $x(0)$. In the finite-horizon case, a common objective is a sum of stage costs plus a terminal penalty,

$$J(x_0, U) := \ell(x_N) + \sum_{i=0}^{N-1} \ell(x_i, u_i),$$

subject to the system dynamics

$$x_{i+1} = g(x_i, u_i), \quad i = 0, \dots, N-1,$$

the initial condition

$$x_0 = x(0),$$

and, in the most general setting, also input, state, and terminal constraints. In this chapter, however, we focus on the unconstrained linear quadratic case, which already reveals the main structure of optimal regulation and will later serve as a key building block in MPC.

3.1.1 Finite-horizon linear quadratic regulation

We consider the discrete-time **linear time-invariant** system

$$x(k+1) = Ax(k) + Bu(k),$$

and we seek to **regulate the state to the origin** by minimizing the quadratic cost

$$J^*(x(0)) := \min_U x_N^\top P x_N + \sum_{i=0}^{N-1} (x_i^\top Q x_i + u_i^\top R u_i),$$

subject to

$$x_{i+1} = Ax_i + Bu_i, \quad i = 0, \dots, N-1,$$

and

$$x_0 = x(0).$$

Here $P \succeq 0$ is the terminal weight, $Q \succeq 0$ is the state weight, $R \succ 0$ is the input weight, and N is the horizon length. The matrices Q and R determine how strongly one penalizes state deviations and control effort, while the matrix P determines how the final state at the end of the horizon is valued. Because the problem is unconstrained and the objective is quadratic, one expects a solution with a particularly **simple structure**.

The interpretation of the cost is straightforward but important. The state term penalizes deviations from the origin, while the input term penalizes aggressive control action. The controller therefore balances performance against effort. A large Q relative to R encourages fast regulation at the price of stronger control inputs, whereas a large R makes the controller more conservative. This trade-off is one of the reasons why quadratic optimal control is both practical and conceptually appealing.

3.2 Batch Approach

A first way to solve the finite-horizon problem is to **eliminate the state variables** and express the entire optimization problem directly in terms of the initial state and the input sequence. This is called the batch approach. The key observation is that repeated substitution of the system dynamics allows all future states to be written explicitly as linear functions of $x(0)$ and the decision variables $u_0, \dots, u_{N-1}$.

Starting from $x_0 = x(0)$, one obtains

$$x_1 = Ax(0) + Bu_0,$$

$$x_2 = A^2x(0) + ABu_0 + Bu_1,$$

and, continuing in this way, all states up to x_N . Collecting them into the stacked vector

$$X := [x_0^\top, x_1^\top, \dots, x_N^\top]^\top,$$

and the inputs into

$$U := [u_0^\top, u_1^\top, \dots, u_{N-1}^\top]^\top,$$

one may write

$$X = S_x x(0) + S_u U,$$

where

$$S_x = \begin{bmatrix} I \\ A \\ A^2 \\ \vdots \\ A^N \end{bmatrix} \quad S_u = \begin{bmatrix} 0 & 0 & \dots & 0 \\ B & 0 & \dots & 0 \\ AB & B & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ A^{N-1}B & \dots & AB & B \end{bmatrix}$$

To rewrite the cost compactly, define the block-diagonal matrices

$$\bar{Q} := \text{blockdiag}(Q, \dots, Q, P), \quad \bar{R} := \text{blockdiag}(R, \dots, R).$$

Then the cost becomes

$$J(x(0), U) = X^\top \bar{Q} X + U^\top \bar{R} U.$$

Substituting the expression for X yields

$$J(x(0), U) = (S_x x(0) + S_u U)^\top \bar{Q} (S_x x(0) + S_u U) + U^\top \bar{R} U.$$

After expansion, this can be written as

$$\begin{aligned} J(x(0), U) &= x(0)^\top S_x^\top \bar{Q} S_x x(0) + 2x(0)^\top \textcolor{red}{S}_x^\top \bar{Q} \textcolor{red}{S}_u U + U^\top \textcolor{blue}{S}_u^\top \bar{Q} \textcolor{blue}{S}_u U + U^\top \bar{R} U \\ &= U^\top \textcolor{blue}{H} U + 2x(0)^\top \textcolor{red}{F} U + x(0)^\top S_x^\top \bar{Q} S_x x(0) \end{aligned}$$

where

$$H := S_u^\top \bar{Q} S_u + \bar{R}, \quad F := S_x^\top \bar{Q} S_u.$$

Since $\bar{R} \succ 0$ and $S_u^\top \bar{Q} S_u \succeq 0$, it follows that $H \succ 0$. Therefore the cost is a strictly convex quadratic function of U , and hence admits a unique minimizer.

The optimal input sequence is obtained by differentiating with respect to U and setting the gradient to zero:

$$\nabla_U J(x(0), U) = 2HU + 2F^\top x(0) = 0.$$

Hence

$$U^*(x(0)) = -H^{-1}F^\top x(0).$$

So the optimal sequence is a linear function of the initial state. Substituting this optimizer back into the cost shows that the **optimal value is itself a quadratic function of the initial state**:

$$J^*(x(0)) = x(0)^\top \left(S_x^\top \bar{Q} S_x - S_x^\top \bar{Q} S_u (S_u^\top \bar{Q} S_u + \bar{R})^{-1} S_u^\top \bar{Q} S_x \right) x(0).$$

This is an important structural result: in linear quadratic optimal control, not only the optimal input, but also the optimal cost-to-go, has a quadratic dependence on the state.

Batch solution

In the unconstrained finite-horizon LQ problem, all predicted states can be eliminated in favor of the input sequence, and the resulting optimization becomes a strictly convex quadratic minimization in the stacked input vector U . Its unique solution is linear in the initial state.

The batch method is conceptually direct and easy to derive. However, it returns a numerical sequence of future inputs tailored to the specific initial condition. It does not directly provide a state-feedback law to be used later if the trajectory deviates from the nominal prediction. This limitation motivates a second viewpoint, based on dynamic programming. Plus, if there are state or input constraints, solving this problem by matrix inversion is not guaranteed to result in a feasible solution.

3.3 Recursive Approach and Dynamic Programming

Instead of solving for the entire input sequence at once, one may exploit the recursive structure of the problem. The central idea is to describe the optimal cost from a given time onward as a function of the current state. This leads naturally to dynamic programming and to Bellman's principle of optimality.

3.3.1 Cost-to-go and one-step optimization

For a time index j , define the j -step **optimal cost-to-go** as

$$J_j^*(x(j)) := \min_{U_{j \rightarrow N}} x_N^\top P x_N + \sum_{i=j}^{N-1} (x_i^\top Q x_i + u_i^\top R u_i),$$

subject to the dynamics from time j onward and the initial condition $x_j = x(j)$. This is the minimum cost achievable over the remaining portion of the horizon starting from the state at

time j .

At the last stage, namely $j = N - 1$, the problem reduces to a one-step optimization:

$$J_{N-1}^*(x_{N-1}) = \min_{u_{N-1}} \left(x_{N-1}^\top Q x_{N-1} + u_{N-1}^\top R u_{N-1} + x_N^\top P_N x_N \right),$$

with

$$x_N = A x_{N-1} + B u_{N-1}, \quad P_N = P.$$

Substituting the dynamics into the cost yields a quadratic expression in u_{N-1} :

$$\begin{aligned} J_{N-1}^*(x_{N-1}) &= \min_{u_{N-1}} \left(x_{N-1}^\top Q x_{N-1} + u_{N-1}^\top R u_{N-1} + (A x_{N-1} + B u_{N-1})^\top P_N (A x_{N-1} + B u_{N-1}) \right) \\ &= \min_{u_{N-1}} \left(x_{N-1}^\top (Q + A^\top P_N A) x_{N-1} + u_{N-1}^\top (R + B^\top P_N B) u_{N-1} + 2 x_{N-1}^\top A^\top P_N B u_{N-1} \right) \end{aligned}$$

Setting its gradient to zero leads to the optimal terminal-step control law

$$2(B^\top P_N B + R)u_{N-1} + 2B^\top P_N A x_{N-1} = 0$$

$$\begin{aligned} u_{N-1}^* &= -(B^\top P_N B + R)^{-1} B^\top P_N A x_{N-1} \\ &=: F_{N-1} x_{N-1}. \end{aligned}$$

The corresponding optimal cost-to-go turns out to be quadratic:

$$J_{N-1}^*(x_{N-1}) = x_{N-1}^\top P_{N-1} x_{N-1},$$

where

$$P_{N-1} = A^\top P_N A + Q - A^\top P_N B (B^\top P_N B + R)^{-1} B^\top P_N A.$$

This already reveals the characteristic structure of the recursive solution: the **optimal control law is linear in the state, and the optimal cost-to-go is quadratic**.

3.3.2 Bellman's principle of optimality

Now consider the problem one step earlier. The optimal two-step cost from time $N - 2$ can be written as

$$J_{N-2}^*(x_{N-2}) = \min_{u_{N-2}} \left(x_{N-2}^\top Q x_{N-2} + u_{N-2}^\top R u_{N-2} + J_{N-1}^*(x_{N-1}) \right),$$

subject to

$$x_{N-1} = A x_{N-2} + B u_{N-2}.$$

The reason this is valid is **Bellman's principle of optimality**: any tail of an optimal trajectory must itself be optimal for the subproblem starting at that later point. In other words, if a policy is optimal over the full horizon, then after any intermediate time the remaining decisions must solve the corresponding shorter-horizon optimal control problem.

This principle gives the recursion

$$J_i^*(x_i) = \min_{u_i} \left(x_i^\top Q x_i + u_i^\top R u_i + J_{i+1}^*(x_{i+1}) \right), \quad x_{i+1} = A x_i + B u_i,$$

for all $i = 0, \dots, N - 1$. **Assuming** that

$$J_{i+1}^*(x_{i+1}) = x_{i+1}^\top P_{i+1} x_{i+1},$$

one again obtains by substitution and differentiation the optimal policy

$$u_i^* = -(B^\top P_{i+1} B + R)^{-1} B^\top P_{i+1} A x_i =: F_i x_i,$$

and the recursion

$$P_i = A^\top P_{i+1} A + Q - A^\top P_{i+1} B (B^\top P_{i+1} B + R)^{-1} B^\top P_{i+1} A.$$

This backward recursion, initialized with $P_N = P$, is called the **discrete-time Riccati equation**, or Riccati difference equation. It yields all feedback gains F_i and all cost matrices P_i over the horizon. In particular,

$$J^*(x(0)) = J_0^*(x(0)) = x(0)^\top P_0 x(0).$$

Thus the recursive method computes a **family of feedback laws, one for each stage**, rather than only a single open-loop sequence.

Riccati recursion

For the finite-horizon unconstrained LQ problem, the optimal cost-to-go has the quadratic form

$$J_i^*(x_i) = x_i^\top P_i x_i,$$

and the optimal control law has the linear state-feedback form

$$u_i^* = F_i x_i,$$

where the matrices P_i are computed backward through the Riccati difference equation.

3.3.3 Comparison of batch and recursive viewpoints

The batch and recursive approaches solve the same unconstrained problem, but they do so from different perspectives. The batch method returns the optimal input sequence as a stacked vector of numerical values depending on the initial state. The recursive method returns feedback laws depending on the state at each stage. If the system evolves exactly as predicted, then both methods generate the same control actions along the nominal trajectory.

The recursive viewpoint has two major conceptual advantages. First, it exposes the feedback structure of the solution. If disturbances or model mismatch cause the actual state to deviate from the nominal prediction, the control decision can still be recomputed optimally from the current state using the feedback law. Second, it is computationally more attractive in many settings, because it breaks the large horizon problem into a backward sequence of smaller problems. The batch Hessian grows with the horizon length, while the recursive method exploits the problem structure much more efficiently. On the other hand, once constraints are added, the batch formulation is usually much easier to generalize, which is one reason why MPC is commonly formulated as a finite-horizon constrained optimization over a stacked input sequence.

3.4 Receding Horizon Interpretation

The unconstrained finite-horizon LQ problem also provides a simple instance of receding horizon control. One solves the finite-horizon optimization problem, extracts the first control input from the optimal sequence, applies it to the system, and then repeats the entire procedure at the next sampling instant. This is exactly the same logic that later defines MPC in the constrained case.

For an unconstrained linear quadratic problem, applying this receding-horizon strategy yields a **linear state-feedback controller**. Still, the interpretation is important: the **controller is**

not derived first and optimized later; instead, it emerges from the repeated solution of a finite-horizon optimal control problem. This viewpoint generalizes naturally to more complex models and to constrained problems, where no closed-form feedback law is available.

3.5 Infinite-Horizon LQR

In many situations, the natural control objective is not limited to a finite planning window, but extends indefinitely into the future. This leads to the infinite-horizon linear quadratic regulator problem

$$J_\infty(x(0)) = \min_{u(\cdot)} \sum_{i=0}^{\infty} (x_i^\top Q x_i + u_i^\top R u_i),$$

subject to

$$x_{i+1} = Ax_i + Bu_i, \quad i = 0, 1, 2, \dots,$$

and

$$x_0 = x(0).$$

The infinite horizon removes the arbitrariness of a chosen terminal time and leads to a **time-invariant optimal feedback law**.

3.5.1 Algebraic Riccati equation

By analogy with the finite-horizon recursive solution, one expects the infinite-horizon optimal cost-to-go to have the quadratic form

$$J_\infty^*(x) = x^\top P_\infty x,$$

and the corresponding optimal control law to be

$$u^*(k) = -(B^\top P_\infty B + R)^{-1} B^\top P_\infty A x(k) =: F_\infty x(k).$$

If the finite-horizon Riccati difference equation converges as the horizon tends to infinity, then the limiting matrix P_∞ must satisfy the fixed-point equation

$$P_\infty = A^\top P_\infty A + Q - A^\top P_\infty B (B^\top P_\infty B + R)^{-1} B^\top P_\infty A.$$

This is the **discrete-time algebraic Riccati equation**. Its solution yields the asymptotic LQR feedback gain F_∞ .

Under standard assumptions, namely stabilizability of (A, B) and observability of $(Q^{1/2}, A)$, the Riccati recursion converges to the unique positive definite solution P_∞ of the algebraic Riccati equation. This gives a well-defined infinite-horizon optimal controller. The corresponding **feedback is time-invariant**, in contrast with the finite-horizon case where the gains F_i vary with the remaining length of the horizon.

3.5.2 Stability of infinite-horizon LQR

An important advantage of the infinite-horizon LQR is that, under suitable assumptions, the resulting closed-loop system is asymptotically stable. More precisely, if (A, B) is stabilizable and $Q, R \succ 0$, then the optimal infinite-horizon feedback law

$$u(k) = F_\infty x(k), \quad F_\infty = -(R + B^\top P_\infty B)^{-1} B^\top P_\infty A,$$

renders the closed-loop system

$$x(k+1) = (A + BF_\infty)x(k)$$

asymptotically stable. The proof is based on the observation that the optimal value function of the infinite-horizon problem is a Lyapunov function for the closed-loop system.

Lyapunov function for infinite-horizon LQR

If (A, B) is stabilizable and $Q, R \succ 0$, then the optimal value function

$$J^*(x) = x^\top P_\infty x$$

is a Lyapunov function for the closed-loop system

$$x^+ = (A + BF_\infty)x,$$

where

$$F_\infty = -(R + B^\top P_\infty B)^{-1} B^\top P_\infty A,$$

and $P_\infty \succ 0$ is the solution of the algebraic Riccati equation.

The first two Lyapunov properties follow immediately from $P_\infty \succ 0$. Indeed,

$$J^*(0) = 0, \quad J^*(x) = x^\top P_\infty x > 0 \quad \forall x \neq 0,$$

and, since P_∞ is positive definite,

$$\|x\| \rightarrow \infty \implies J^*(x) = x^\top P_\infty x \rightarrow \infty.$$

It therefore remains to show that J^* decreases strictly along closed-loop trajectories.

To do so, observe that the optimal infinite-horizon cost starting at time k can be written as

$$J^*(x(k)) = x(k)^\top P_\infty x(k) = \sum_{i=k}^{\infty} x(i)^\top (Q + F_\infty^\top R F_\infty) x(i),$$

because along the optimal closed-loop trajectory one has

$$u(i) = F_\infty x(i).$$

Now consider the value of the same function one step later:

$$J^*(x(k+1)) = J^*((A + BF_\infty)x(k)) = \sum_{i=k+1}^{\infty} x(i)^\top (Q + F_\infty^\top R F_\infty) x(i).$$

Comparing the two expressions gives

$$J^*(x(k+1)) = J^*(x(k)) - x(k)^\top (Q + F_\infty^\top R F_\infty) x(k).$$

Since $Q \succ 0$ and $R \succ 0$, the matrix

$$Q + F_\infty^\top R F_\infty$$

is positive definite, and therefore

$$x(k)^\top (Q + F_\infty^\top R F_\infty) x(k) > 0 \quad \forall x(k) \neq 0.$$

Hence

$$J^*(x(k+1)) < J^*(x(k)) \quad \forall x(k) \neq 0.$$

Thus the optimal value function decreases strictly along every nontrivial closed-loop trajectory, and is therefore a Lyapunov function for the system

$$x(k+1) = (A + BF_\infty)x(k).$$

This proves that the origin is asymptotically stable. In particular,

$$\lim_{k \rightarrow \infty} x(k) = 0.$$

The result is conceptually important: the same function that measures the optimal infinite-horizon performance also certifies stability of the optimal closed loop.

The argument above used the stronger assumption $Q \succ 0$. This condition can be relaxed. In particular, the infinite-horizon LQR remains stabilizing under the weaker assumptions

$$Q \succeq 0, \quad R \succ 0,$$

provided that (A, B) is stabilizable and $(Q^{1/2}, A)$ is detectable. Thus detectability plays here the same role that observability played in the stronger result: it is sufficient to ensure that no unstable mode can remain invisible to the cost.

3.6 Terminal Weight and Quasi-Infinite-Horizon Thinking

Since MPC is usually implemented with a finite prediction horizon, one naturally asks how to choose the terminal weight P so that the finite-horizon problem approximates the infinite-horizon behavior as well as possible. There are several important possibilities.

A particularly natural choice is to set the terminal weight equal to the infinite-horizon cost-to-go beyond the horizon. In other words, one chooses

$$P = P_\infty,$$

where P_∞ solves the algebraic Riccati equation. Then the finite-horizon problem exactly matches the infinite-horizon solution from the terminal point onward. This is the cleanest way of embedding the tail of the infinite-horizon problem into a finite-horizon formulation.

A second possibility is to assume that no control action is applied after the end of the horizon, so that for $k \geq N$ the state evolves autonomously according to

$$x(k+1) = Ax(k).$$

In that case, the appropriate terminal weight is obtained from the Lyapunov equation

$$A^\top P A + Q = P.$$

This choice only makes sense if the autonomous system is asymptotically stable, because otherwise no positive definite solution exists.

A third option is to impose that both the state and the input vanish after the horizon. In this case one does not only use a terminal penalty, but rather imposes a terminal constraint such as

$$x_N = 0.$$

This viewpoint is important for later constrained MPC theory, where terminal costs and terminal sets are used precisely to mimic infinite-horizon behavior and recover stability guarantees from a finite optimization horizon.

3.7 Summary

Unconstrained linear quadratic optimal control is the simplest predictive-control setting in which all central ingredients of MPC already appear. One starts from a model, predicts future trajectories over a horizon, and chooses the control sequence that minimizes a quadratic objective balancing state regulation and control effort. In the finite-horizon case, the problem can be solved either in batch form, by eliminating the states and minimizing a quadratic function of the full input sequence, or recursively, through dynamic programming and the Riccati difference equation.

The recursive viewpoint is especially important because it reveals the feedback structure of the solution and leads directly to Bellman's principle of optimality. It also clarifies the meaning of receding-horizon control: at each step, one solves a finite-horizon problem, applies the first input, and repeats the optimization at the next sampling instant. The quality of this strategy depends strongly on the horizon length and on the terminal ingredients used.

Passing to the infinite-horizon problem yields the time-invariant LQR feedback law, determined by the algebraic Riccati equation. Under standard stabilizability and detectability assumptions, the infinite-horizon optimal value function is quadratic and also serves as a Lyapunov function, thereby proving stability of the optimal closed loop. This connection between optimization, value functions, and Lyapunov stability will be one of the central themes throughout the rest of MPC theory.

Chapter 4

Introduction to Convex Optimization

4.1 Main concepts

Before studying constrained MPC in full generality, it is useful to introduce the basic language of mathematical optimization. Indeed, at each sampling instant MPC computes the control action by solving an optimization problem over a finite horizon. In the simplest linear setting, this problem has the form

$$\min_{U_k} \sum_{i=0}^{N-1} \ell(x_{k+i}, u_{k+i})$$

subject to the prediction model, the initial condition, and the state and input constraints. The present chapter abstracts away from the specific control setting and introduces the optimization concepts that will later be needed in a more systematic way.

A general optimization problem can be written as

$$\min_{x \in \text{dom}(f)} f(x)$$

subject to

$$g_i(x) \leq 0, \quad i = 1, \dots, m,$$

and

$$h_i(x) = 0, \quad i = 1, \dots, p.$$

Here, $x \in \mathbb{R}^n$ denotes the decision variable, f is the objective function, the functions g_i represent inequality constraints, and the functions h_i represent equality constraints. The set of all admissible decisions is called the *feasible set*:

$$\mathcal{X} := \{x \in \text{dom}(f) \mid g_i(x) \leq 0, \ i = 1, \dots, m, \ h_i(x) = 0, \ i = 1, \dots, p\}.$$

A point $x \in \mathcal{X}$ is called *feasible*. If, in addition, all inequality constraints are satisfied strictly, that is,

$$g_i(x) < 0, \quad i = 1, \dots, m,$$

then x is called *strictly feasible*. The smallest achievable value of the objective over the feasible set is called the *optimal value* and is denoted by

$$p^* := \min_{x \in \mathcal{X}} f(x).$$

Any feasible point x^* achieving this value is called an *optimizer*. In general, the optimizer need not be unique, and one writes

$$\arg \min_{x \in \mathcal{X}} f(x)$$

for the set of all optimal solutions.

It is also important to distinguish between local and global optimality. A feasible point $x \in \mathcal{X}$ is said to be *locally optimal* if for some $R > 0$ it satisfies

$$y \in \mathcal{X}, \|y - x\| \leq R \Rightarrow f(y) \geq f(x)$$

i.e. if it minimizes the objective in a neighborhood of itself.

Whereas it is *globally optimal* if it satisfies

$$y \in \mathcal{X} \Rightarrow f(y) \geq f(x)$$

i.e. if it minimizes the objective over the entire feasible set.

For a general nonconvex problem, these two notions are very different. One of the main reasons convex optimization is so useful is that, under convexity assumptions, every local optimum is automatically global.

Several pathological situations may also occur. If the feasible set is empty, then the problem is called *infeasible* and by convention we say that $p^* = \infty$. If feasible points exist but the objective can be made arbitrarily small (i.e. $p^* = -\infty$), then the problem is *unbounded below*. Finally, if there are no constraints at all ($\mathcal{X} = \mathbb{R}^n$), the problem is called *unconstrained*.

A further piece of terminology that will later be useful concerns the status of constraints at a given feasible point. An inequality constraint $g_i(x) \leq 0$ is said to be *active* at $\bar{x}$ if

$$g_i(\bar{x}) = 0,$$

and *inactive* otherwise. Equality constraints are always active, since they must hold exactly. A constraint is called *redundant* if removing it does not change the feasible set. In that case it has no effect on the solution of the problem.

4.2 Convex sets

The first geometric concept underlying convex optimization is convexity of sets. A set $\mathcal{X} \subseteq \mathbb{R}^n$ is called *convex* if, by definition, for every pair of points $x, y \in \mathcal{X}$ and every scalar $\lambda \in [0, 1]$, the convex combination

$$\lambda x + (1 - \lambda)y$$

also belongs to $\mathcal{X}$. In words, this means that whenever two points lie in the set, the entire line segment joining them also lies in the set.

This property is fundamental because it excludes holes, inward dents, and disconnected geometries that would make optimization much harder. Convexity of the feasible region is one of the key reasons why convex optimization problems are tractable and why local information is often sufficient to identify global solutions. More generally, given points $x_1, \dots, x_k \in \mathbb{R}^n$, any point of the form

$$x = \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_k x_k, \quad \theta_i \geq 0, \quad \sum_{i=1}^k \theta_i = 1,$$

is called a *convex combination* of these points. A set is convex precisely when it contains all convex combinations of its points.

Among the most important convex sets are hyperplanes and halfspaces. A hyperplane in $\mathbb{R}^n$ is a set of the form

$$\{x \in \mathbb{R}^n \mid a^\top x = b\},$$

where $a \neq 0$. Geometrically, a is normal to the hyperplane. A halfspace is one side of such a hyperplane, for example

$$\{x \in \mathbb{R}^n \mid a^\top x \leq b\}.$$

Hyperplanes are affine sets, and halfspaces are convex sets. Since many constraints in control and optimization are linear inequalities or equalities, these objects appear constantly in MPC formulations.

A particularly important class of convex sets is formed by *polyhedra*, that is, intersections of finitely many closed halfspaces:

$$\{x \in \mathbb{R}^n \mid Ax \leq b\}.$$

If such a set is bounded, it is called a *polytope*. In linear MPC, box constraints and more general linear inequality constraints lead exactly to feasible sets of this type.

Another important family is given by ellipsoids. An ellipsoid centered at x_c can be written as

$$\left\{x \in \mathbb{R}^n \mid (x - x_c)^\top A^{-1}(x - x_c) \leq 1\right\},$$

where $A \succ 0$. The positive definite matrix A determines the shape and orientation of the ellipsoid. The Euclidean ball is a special case obtained when $A = r^2 I$.

More generally, for any norm $\|\cdot\|$, the norm ball

$$\{x \in \mathbb{R}^n \mid \|x - x_c\| \leq r\}$$

is convex. Common examples are the ℓ_2 -ball, the ℓ_1 -ball, and the ℓ_∞ -ball, corresponding respectively to the Euclidean norm, the sum of absolute values, and the maximum absolute component:

$$\|x\|_2 = \left(\sum_i x_i^2\right)^{1/2}, \quad \|x\|_1 = \sum_i |x_i|, \quad \|x\|_\infty = \max_i |x_i|.$$

A basic operation that preserves convexity is intersection. If $\mathcal{X}_1, \mathcal{X}_2, \dots$ are convex, then

$$\bigcap_i \mathcal{X}_i$$

is again convex. This is extremely useful because many feasible regions are built by imposing several convex constraints simultaneously. By contrast, unions of convex sets are in general not convex. This explains why hybrid and mixed-integer problems are typically much harder than standard convex problems: they often involve feasible sets that are unions of several simpler regions rather than a single convex one.

4.3 Convex functions

The second central notion is convexity of functions. A function $f : \text{dom}(f) \rightarrow \mathbb{R}$ is called *convex* if its domain is convex and if for all $x, y \in \text{dom}(f)$ and all $\lambda \in [0, 1]$,

$$f(\lambda x + (1 - \lambda)y) \leq \lambda f(x) + (1 - \lambda)f(y).$$

Geometrically, this means that the graph of the function lies below the straight line segment joining any two points on the graph. If the inequality is strict whenever $x \neq y$ and $\lambda \in (0, 1)$, then the function is called *strictly convex*. A function is called *concave* if its negative is convex.

Convex objective functions are particularly important because they do not have spurious local minima. Their shape is globally well behaved, and this is one of the key reasons why optimization over them is reliable.

For differentiable functions, convexity can be characterized through a first-order condition. A differentiable function f on a convex domain is convex if and only if

$$f(y) \geq f(x) + \nabla f(x)^\top (y - x) \quad \forall x, y \in \text{dom}(f).$$

Thus the first-order Taylor approximation of a convex function always lies below the function itself. In other words, each tangent hyperplane is a global underestimator.

If f is twice differentiable, there is also a second-order characterization. The function is convex if and only if

$$\nabla^2 f(x) \succeq 0 \quad \forall x \in \text{dom}(f),$$

that is, if its Hessian is positive semidefinite everywhere. If

$$\nabla^2 f(x) \succ 0 \quad \forall x \in \text{dom}(f),$$

then f is strictly convex.

A useful geometric notion associated with a function is that of a level set. For a scalar α , the level set is

$$L_\alpha := \{x \in \text{dom}(f) \mid f(x) = \alpha\},$$

while the sublevel set is

$$C_\alpha := \{x \in \text{dom}(f) \mid f(x) \leq \alpha\}.$$

If f is convex, then every sublevel set C_α is convex. This property is very important in optimization because it gives geometric information about the shape of cost contours.

Typical examples of convex functions include affine functions, exponentials, powers x^α on $\mathbb{R}_{++}$ for suitable exponents, and vector norms. Typical concave functions include the logarithm on $\mathbb{R}_{++}$ and certain entropy-type expressions. In control and estimation, quadratic functions occupy a particularly prominent role.

A quadratic function

$$f(x) = \frac{1}{2}x^\top Hx + q^\top x + r$$

is convex when $H \succeq 0$, and strictly convex when $H \succ 0$.

Convexity is also preserved under several useful operations. Nonnegative weighted sums of convex functions remain convex, composition with affine maps preserves convexity, pointwise maxima of convex functions are convex, and partial minimization often preserves convexity as well. These rules make it possible to construct fairly rich objective and constraint functions while retaining tractability.

4.4 Convex optimization problems

We now combine convex sets and convex functions. A problem is called a *convex optimization problem* if it can be written in the form

$$\min_{x \in \text{dom}(f)} f(x)$$

subject to

$$g_i(x) \leq 0, \quad i = 1, \dots, m,$$

and

$$a_i^\top x = b_i, \quad i = 1, \dots, p,$$

where f and the g_i are convex functions, the domain of f is convex, and the equality constraints are affine.

The affine equalities are often collected in matrix form as

$$Ax = b.$$

Because sublevel sets of convex functions are convex, and because affine equality constraints define affine sets, the feasible set of a convex optimization problem is itself convex. This structural property is the source of the remarkable behavior of convex programs.

The most important consequence is the following: every locally optimal solution of a convex optimization problem is also globally optimal. This completely changes the nature of optimization. For a general nonconvex problem one may find many local minima and must still ask whether any of them is globally best. In a convex problem, once a local minimizer has been found, the global problem is solved.

This observation explains why convex optimization occupies such a central place in control. Convex problems can often be solved efficiently and reliably by numerical algorithms, and they offer strong guarantees that are usually unavailable in more general settings.

Two convex problem classes are of particular importance.

4.4.1 Linear programs

A *linear program* (LP) has the form

$$\min_{x \in \mathbb{R}^n} c^\top x$$

subject to

$$Gx \leq h, \quad Ax = b.$$

Here both the cost and the constraints are affine. The feasible set is therefore a polyhedron. If this set is empty, the problem is infeasible. If the cost can decrease indefinitely along a feasible direction, the problem is unbounded below. Otherwise the problem has a finite optimal value. Geometrically, an LP amounts to minimizing a linear function over a polyhedron. Since level sets of the objective are hyperplanes, the optimizer is obtained by shifting these hyperplanes until they first touch the feasible region. This immediately explains why optimal solutions often lie on the boundary, and in many cases at extreme points of the feasible polyhedron.

LPs play an important role in MPC whenever the cost is linear or whenever a problem can be reformulated in linear form by introducing auxiliary variables. Moreover, more complicated problems are often compared against LPs because linear programs are among the most tractable optimization problems.

4.4.2 Quadratic programs

A *quadratic program* (QP) has the form

$$\min_{x \in \mathbb{R}^n} \frac{1}{2} x^\top H x + q^\top x + r$$

subject to

$$Gx \leq h, \quad Ax = b.$$

The constant term r has no effect on the optimizer and may often be omitted. The problem is convex if $H \succeq 0$, and strictly convex if $H \succ 0$.

Quadratic programs are especially important for MPC because linear prediction models together with quadratic stage and terminal costs naturally lead to QPs. In this sense, the finite-horizon control problems encountered later in nominal MPC are not arbitrary optimization problems: they are structured convex quadratic programs.

If the unconstrained minimizer of the quadratic objective happens to lie in the feasible set, then it is also the constrained optimizer. Otherwise the optimizer lies on the boundary of the feasible region, where the constraints interact with the curvature of the cost function. This interplay between a quadratic objective and linear constraints is exactly what gives QPs their characteristic structure.

4.4.3 Equivalent problem formulations

It is often useful to reformulate optimization problems without changing their essential content. Two formulations are called equivalent if the solution of one can be easily recovered from the solution of the other.

One common trick is to introduce auxiliary equality constraints. For example, a composite problem such as

$$\min_x f(A_0x + b_0) \quad \text{subject to} \quad g_i(A_ix + b_i) \leq 0$$

can be rewritten by introducing new variables y_i and imposing

$$A_ix + b_i = y_i.$$

Another standard device is the use of slack variables. A linear inequality

$$A_ix \leq b_i$$

can be rewritten as

$$A_ix + s_i = b_i, \quad s_i \geq 0.$$

Such reformulations are extremely common in optimization algorithms and in the derivation of dual problems.

4.5 Optimality conditions

The final step is to understand how one certifies optimality. For this, the central tool is the Lagrangian.

4.5.1 The Lagrangian and the dual problem

Consider again the general problem

$$\min_{x \in \text{dom}(f)} f(x)$$

subject to

$$g_i(x) \leq 0, \quad i = 1, \dots, m,$$

and

$$h_i(x) = 0, \quad i = 1, \dots, p.$$

Its Lagrangian is defined by

$$L(x, \lambda, \nu) = f(x) + \sum_{i=1}^m \lambda_i g_i(x) + \sum_{i=1}^p \nu_i h_i(x),$$

where λ_i are the multipliers associated with the inequality constraints and ν_i those associated with the equality constraints. Since the inequalities must be penalized only when they are violated, the multipliers λ_i are required to satisfy

$$\lambda_i \geq 0.$$

The associated *dual function* is

$$d(\lambda, \nu) := \inf_{x \in \text{dom}(f)} L(x, \lambda, \nu).$$

This function is always concave in (λ, ν) , even if the primal problem itself is not convex. Moreover, whenever $\lambda \geq 0$, the dual function provides a lower bound on the primal optimal value:

$$d(\lambda, \nu) \leq p^*.$$

This is the principle of *weak duality*.

Maximizing this lower bound leads to the *dual problem*:

$$\max_{\lambda, \nu} d(\lambda, \nu) \quad \text{subject to} \quad \lambda \geq 0.$$

If the optimal value of the dual problem is denoted by d^* , weak duality states that

$$d^* \leq p^*.$$

For linear and quadratic programs, the dual often has a particularly simple form. In fact, the dual of an LP is again an LP, and the dual of a convex QP is again a QP. This is not only mathematically elegant but also practically useful, because sometimes the dual problem is easier to solve or interpret than the primal one.

4.5.2 Examples: dual of an LP and of a QP

It is useful to make the previous discussion concrete by deriving the dual explicitly for two important problem classes.

Example: dual of a linear program

Consider the linear program

$$\begin{aligned} (P) : \quad & \min_{x \in \mathbb{R}^n} c^\top x \\ & \text{subj. to } Ax = b, \\ & \quad \quad \quad Cx \leq e. \end{aligned}$$

Its Lagrangian is

$$L(x, \lambda, \nu) = c^\top x + \nu^\top (Ax - b) + \lambda^\top (Cx - e),$$

where ν is associated with the equality constraint and $\lambda \geq 0$ with the inequality constraint. Rearranging terms gives

$$L(x, \lambda, \nu) = (A^\top \nu + C^\top \lambda + c)^\top x - b^\top \nu - e^\top \lambda.$$

Hence the dual function is

$$d(\lambda, \nu) = \inf_{x \in \mathbb{R}^n} L(x, \lambda, \nu) = \inf_{x \in \mathbb{R}^n} \left[(A^\top \nu + C^\top \lambda + c)^\top x - b^\top \nu - e^\top \lambda \right].$$

Now, if

$$A^\top \nu + C^\top \lambda + c \neq 0,$$

then the expression is affine and nonconstant in x , so its infimum is $-\infty$. Therefore, for the dual function to be finite one must impose

$$A^\top \nu + C^\top \lambda + c = 0.$$

Under this condition, the term depending on x disappears and one obtains

$$d(\lambda, \nu) = -b^\top \nu - e^\top \lambda.$$

Thus the dual problem is

$$\begin{aligned} (D) : \quad & \max_{\lambda, \nu} \quad -b^\top \nu - e^\top \lambda \\ & \text{subj. to } A^\top \nu + C^\top \lambda + c = 0, \\ & \lambda \geq 0. \end{aligned}$$

This confirms an important structural fact: the dual of a linear program is again a linear program.

Example: dual of a quadratic program

Now consider the quadratic program

$$\begin{aligned} (P) : \quad & \min_{x \in \mathbb{R}^n} \quad \frac{1}{2} x^\top Q x + c^\top x \\ & \text{subj. to } Cx \leq e, \end{aligned}$$

where $Q \succ 0$. Its Lagrangian is

$$L(x, \lambda) = \frac{1}{2} x^\top Q x + c^\top x + \lambda^\top (Cx - e), \quad \lambda \geq 0.$$

Collecting terms yields

$$L(x, \lambda) = \frac{1}{2} x^\top Q x + (c + C^\top \lambda)^\top x - e^\top \lambda.$$

The dual function is therefore

$$d(\lambda) = \inf_{x \in \mathbb{R}^n} \left[\frac{1}{2} x^\top Q x + (c + C^\top \lambda)^\top x - e^\top \lambda \right].$$

Since $Q \succ 0$, the minimization over x is unconstrained and strictly convex, so the minimizer is obtained from the stationarity condition

$$Qx + c + C^\top \lambda = 0.$$

Hence

$$x^*(\lambda) = -Q^{-1}(c + C^\top \lambda).$$

Substituting this expression into the dual function gives

$$d(\lambda) = -\frac{1}{2} (c + C^\top \lambda)^\top Q^{-1} (c + C^\top \lambda) - e^\top \lambda.$$

The dual problem is therefore

$$\max_{\lambda \geq 0} \quad -\frac{1}{2} (c + C^\top \lambda)^\top Q^{-1} (c + C^\top \lambda) - e^\top \lambda.$$

Equivalently, by multiplying the objective by -1 , one can write it in minimization form as

$$(D) : \quad \min_{\lambda} \frac{1}{2} \lambda^\top C Q^{-1} C^\top \lambda + (C Q^{-1} c + e)^\top \lambda + \frac{1}{2} c^\top Q^{-1} c$$

subj. to $\lambda \geq 0$.

Thus the dual of a convex quadratic program is again a quadratic program.

These two examples illustrate why duality is so useful in practice. For structured problems such as LPs and QPs, the dual preserves the general problem class, while at the same time providing lower bounds, optimality certificates, and sensitivity information through the associated multipliers.

4.5.3 Strong duality and Slater's condition

Weak duality always holds, but equality

$$d^* = p^*$$

is a stronger property called *strong duality*. Strong duality does not hold in general for non-convex problems. For convex problems, however, it often does hold under additional regularity assumptions.

A particularly important sufficient condition is *Slater's condition*. For a convex problem with affine equality constraints, Slater's condition requires the existence of a strictly feasible point satisfying all equalities and strictly satisfying all inequalities, i.e.

$$\{x | Ax = b, g(x_i) < 0 \forall i \in \{1, \dots, m\}\} \neq \emptyset$$

When this condition holds, strong duality follows:

$$p^* = d^*.$$

This result is fundamental, because it means that the dual problem is not merely producing lower bounds; it is producing the exact optimal value. It is also the basis for the Karush–Kuhn–Tucker conditions.

4.5.4 Karush–Kuhn–Tucker conditions

Assume that f , all g_i , and all h_i are differentiable. The *Karush–Kuhn–Tucker* (KKT) conditions are:

1) Primal feasibility:

$$\begin{aligned} g_i(x^*) &\leq 0, & i &= 1, \dots, m, \\ h_i(x^*) &= 0, & i &= 1, \dots, p, \end{aligned}$$

2) Dual feasibility:

$$\lambda^* \geq 0,$$

3) Complementary slackness:

$$\lambda_i^* g_i(x^*) = 0, \quad i = 1, \dots, m,$$

4) Stationarity:

$$\nabla_x L(x^*, \lambda^*, \nu^*) = \nabla f(x^*) + \sum_{i=1}^m \lambda_i^* \nabla g_i(x^*) + \sum_{i=1}^p \nu_i^* \nabla h_i(x^*) = 0.$$

These four groups of conditions play different roles. Primal feasibility requires that the candidate solution x^* satisfy the original constraints. Dual feasibility requires nonnegativity of the multipliers associated with the inequality constraints. Complementary slackness links the primal and dual solutions, while stationarity states that at the optimum the gradient of the Lagrangian with respect to the primal variable must vanish.

Notes on complementary slackness

Complementary slackness is one of the most characteristic parts of the KKT conditions. Assume that strong duality holds, and let x^* and (λ^*, ν^*) be primal and dual optimal solutions. Then

$$d^* = p^* \quad \implies \quad d(\lambda^*, \nu^*) = f(x^*).$$

From the definition of the dual function,

$$d(\lambda^*, \nu^*) = \inf_x \left(f(x) + \sum_{i=1}^m \lambda_i^* g_i(x) + \sum_{i=1}^p \nu_i^* h_i(x) \right).$$

Evaluating the expression at $x = x^*$ gives

$$d(\lambda^*, \nu^*) \leq f(x^*) + \sum_{i=1}^m \lambda_i^* g_i(x^*) + \sum_{i=1}^p \nu_i^* h_i(x^*).$$

Since x^* is primal feasible, $h_i(x^*) = 0$ and $g_i(x^*) \leq 0$, while $\lambda_i^* \geq 0$. Therefore

$$f(x^*) + \sum_{i=1}^m \lambda_i^* g_i(x^*) + \sum_{i=1}^p \nu_i^* h_i(x^*) \leq f(x^*).$$

Combining the two inequalities with $d(\lambda^*, \nu^*) = f(x^*)$ yields

$$f(x^*) = f(x^*) + \sum_{i=1}^m \lambda_i^* g_i(x^*),$$

and hence

$$\lambda_i^* g_i(x^*) = 0, \quad i = 1, \dots, m.$$

This means that for every inequality constraint, one of the following must happen:

$$g_i(x^*) < 0 \implies \lambda_i^* = 0,$$

or

$$\lambda_i^* > 0 \implies g_i(x^*) = 0.$$

So a strictly inactive inequality must have zero multiplier, while a strictly positive multiplier identifies an active constraint.

When are the KKT conditions necessary or sufficient?

For a general optimization problem, the KKT conditions are in general only a *necessary* condition under suitable assumptions. In other words, if x^* and (λ^*, ν^*) are primal and dual optimal solutions and strong duality holds, then they satisfy the KKT conditions.

For a convex optimization problem satisfying Slater's condition, the situation is much stronger: the KKT conditions become *necessary and sufficient*. Thus, if Slater's condition holds, then x^* and (λ^*, ν^*) are primal and dual optimal if and only if they satisfy the KKT conditions.

KKT conditions for convex problems

For a convex optimization problem, KKT conditions provide a complete characterization of optimality once Slater's condition holds. In that case, checking optimality is equivalent to checking primal feasibility, dual feasibility, complementary slackness, and stationarity.

4.5.5 Example: KKT conditions for a quadratic program

Consider the convex quadratic program

$$(P) : \quad \min_{x \in \mathbb{R}^n} \frac{1}{2} x^\top Q x + c^\top x$$

$$\text{subj. to } Ax = b,$$

$$x \geq 0,$$

with $Q \succeq 0$.

Its Lagrangian is

$$L(x, \lambda, \nu) = \frac{1}{2} x^\top Q x + c^\top x + \nu^\top (Ax - b) - \lambda^\top x.$$

The KKT conditions are then:

Stationarity:	$Qx + A^\top \nu - \lambda + c = 0,$
Primal feasibility:	$Ax = b,$
Primal feasibility:	$x \geq 0,$
Dual feasibility:	$\lambda \geq 0,$
Complementarity:	$x_i \lambda_i = 0, \quad i = 1, \dots, n.$

The last three conditions are often written compactly as

$$0 \leq x \perp \lambda \geq 0.$$

This notation emphasizes that x and λ are both componentwise nonnegative and orthogonal component by component, which is exactly the complementarity relation.

4.5.6 Sensitivity analysis

The dual variables are not only useful for deriving optimality conditions. They also carry important information about how the optimal cost changes when the constraints of the primal problem are perturbed.

Consider the optimization problem

$$\min_x f(x)$$

$$\text{subj. to } g_i(x) \leq 0, \quad i = 1, \dots, m,$$

$$h_i(x) = 0, \quad i = 1, \dots, p,$$

together with its dual

$$\max_{\nu, \lambda} d(\nu, \lambda)$$

$$\text{subj. to } \lambda \geq 0.$$

Now perturb the constraints and consider instead

$$\min_x f(x)$$

$$\text{subj. to } g_i(x) \leq u_i, \quad i = 1, \dots, m,$$

$$h_i(x) = v_i, \quad i = 1, \dots, p,$$

where u and v represent perturbations of the inequality and equality constraints, respectively. The corresponding dual problem becomes

$$\begin{aligned} \max_{\nu, \lambda} \quad & d(\nu, \lambda) - u^\top \lambda - v^\top \nu \\ \text{subj. to} \quad & \lambda \geq 0. \end{aligned}$$

Here, x is the primal decision variable, (λ, ν) are the dual decision variables, and the optimal value of the perturbed primal problem is denoted by

$$p^*(u, v).$$

Global sensitivity analysis

Assume strong duality for the unperturbed problem, and let (ν^*, λ^*) be dual optimal. Then weak duality for the perturbed problem implies

$$p^*(u, v) \geq d^*(\nu^*, \lambda^*) - u^\top \lambda^* - v^\top \nu^* = p^*(0, 0) - u^\top \lambda^* - v^\top \nu^*.$$

This gives a lower bound on the perturbed optimal value in terms of the optimal multipliers of the unperturbed problem.

The interpretation is immediate. If λ_i^* is large and $u_i < 0$, then the i -th inequality constraint is tightened and the lower bound increases significantly. This indicates that the optimal cost $p^*(u, v)$ is likely to increase substantially. By contrast, if λ_i^* is small and $u_i > 0$, then relaxing the constraint does not reduce the lower bound by much, so the gain in optimal performance is expected to be limited.

A similar interpretation applies to the equality multipliers ν_i^* . If ν_i^* is large and positive, then a perturbation $v_i < 0$ tends to increase the optimal cost strongly. If ν_i^* is large and negative, then a perturbation $v_i > 0$ has the same effect. Conversely, if ν_i^* is small in magnitude, the corresponding perturbations typically do not change the optimal value very much.

It is important to keep in mind that this result is not symmetric: it provides only a lower bound on $p^*(u, v)$, not an exact formula.

Local sensitivity analysis

A sharper interpretation is available when the perturbed value function is differentiable at the origin. If $p^*(u, v)$ is differentiable at $(u, v) = (0, 0)$, then the optimal multipliers are precisely the local sensitivities of the optimal value:

$$\lambda_i^* = -\frac{\partial p^*(0, 0)}{\partial u_i}, \quad \nu_i^* = -\frac{\partial p^*(0, 0)}{\partial v_i}.$$

Thus λ_i^* measures the sensitivity of the optimal cost with respect to the i -th inequality constraint, and ν_i^* measures the sensitivity with respect to the i -th equality constraint.

This gives a very useful interpretation of Lagrange multipliers. They are not just auxiliary variables introduced for duality theory, but quantitative indicators of how costly it is to tighten constraints. In particular, a large multiplier means that even a small tightening may produce a significant increase in the optimal value.

In MPC this viewpoint is especially natural. Constraints on states and inputs often reflect physical or safety limitations, and the associated multipliers indicate how strongly these limitations influence achievable performance.

4.6 Summary

We can now summarize the main ideas of this chapter. A convex optimization problem is characterized by a convex objective function, convex inequality constraints, and affine equality constraints. This structure implies that the feasible set is convex and, most importantly, that local optimality coincides with global optimality.

This is the central benefit of convex optimization: one does not need to search among many different local minima, since any local minimum is already globally optimal. As a consequence, convex problems can often be solved efficiently and reliably, which is one of the main reasons they play such an important role in control.

Among the most relevant special cases are linear programs and quadratic programs. Linear programs combine affine objectives with polyhedral feasible sets, while quadratic programs add a convex quadratic cost. Since many finite-horizon optimal control problems with linear dynamics and quadratic costs can be formulated as QPs, these problem classes are fundamental in MPC. We also saw that duality provides much more than an alternative mathematical formulation. The dual problem is always convex, even if the primal is not. It provides lower bounds on the primal optimal value, gives optimality certificates through the KKT conditions in the convex case, and plays a central role in optimization algorithms. Moreover, the optimal multipliers identify active constraints and quantify the sensitivity of the optimal cost to perturbations of the problem data.

Altogether, duality, KKT conditions, and sensitivity analysis show that Lagrange multipliers have a rich interpretation: they are simultaneously certificates of optimality, indicators of active constraints, and local measures of the value of relaxing or tightening constraints.

These ideas will reappear throughout MPC. In later chapters, finite-horizon constrained optimal control problems will be formulated as convex optimization problems, often as QPs, and the concepts developed here will become essential both for theoretical analysis and for numerical solution methods.

Why did we consider the dual problem?

It is worth ending the chapter by emphasizing once more why the dual problem is so useful.

First, the dual problem is convex even when the primal problem is not, so in some cases it may be easier to solve or analyze than the primal formulation.

Second, the dual problem always provides a lower bound on the primal optimal value:

$$d^* \leq p^*,$$

and more generally

$$d(\lambda, \nu) \leq p(x)$$

for every primal feasible x and every dual feasible pair (λ, ν) . This makes the dual valuable as a source of suboptimality bounds.

Third, in convex optimization the dual provides a certificate of optimality through the KKT conditions. Once primal feasibility, dual feasibility, complementarity, and stationarity are verified, optimality is established.

Fourth, the KKT conditions are not only theoretically elegant, but also form the basis of many efficient optimization algorithms.

Finally, the multipliers themselves carry information that is directly meaningful. If

$$\lambda_i^* > 0,$$

then the corresponding inequality constraint must be active at the optimum:

$$g_i(x^*) = 0.$$

And if λ_i^* is large, then tightening that constraint tends to increase the optimal cost significantly. In this way, the dual variables encode both structural and sensitivity information about the primal problem.

Part II

Nominal MPC

Chapter 5

Constrained Finite Time Optimal Control

5.1 Constrained finite time optimal control

In the previous chapter we studied finite- and infinite-horizon linear quadratic control in the absence of inequality constraints. That setting already showed the predictive-control logic very clearly: one optimizes over a future input sequence, predicts the resulting state trajectory, and uses the optimizer to determine the control action. However, one of the main reasons why MPC is so useful in practice is precisely that it can handle constraints explicitly. Real systems cannot operate with arbitrarily large inputs, and states or outputs are often restricted by safety, performance, or physical limitations.

The ideal constrained problem would be an infinite-horizon optimal control problem of the form

$$J_{\infty}^*(x(0)) = \min_{u(\cdot)} \sum_{i=0}^{\infty} \ell(x_i, u_i)$$

subject to

$$x_{i+1} = Ax_i + Bu_i, \quad i = 0, 1, \dots,$$

together with the constraints

$$x_i \in \mathcal{X}, \quad u_i \in \mathcal{U}, \quad i = 0, 1, \dots,$$

and the initial condition

$$x_0 = x(0).$$

This problem is conceptually attractive because it captures the long-term consequences of the control action while respecting the constraints at all times. The difficulty is that it involves an infinite number of decision variables, so it is generally not directly computable online.

The standard MPC idea is therefore to replace this ideal problem by a constrained finite time optimal control problem over a horizon of length N . At time k , one solves

$$J^*(x(k)) = \min_U V_f(x_N) + \sum_{i=0}^{N-1} \ell(x_i, u_i)$$

subject to

$$\begin{aligned} x_{i+1} &= Ax_i + Bu_i, & i &= 0, \dots, N-1, \\ x_i &\in \mathcal{X}, & u_i &\in \mathcal{U}, & i &= 0, \dots, N-1, \end{aligned}$$

$$\begin{aligned} x_N &\in \mathcal{X}_f, \\ x_0 &= x(k), \end{aligned}$$

where

$$U = \{u_0, \dots, u_{N-1}\}.$$

The finite horizon makes the problem computationally tractable, while the terminal cost $V_f(x_N)$ and terminal set $\mathcal{X}_f$ are introduced to approximate the effect of the tail beyond the prediction horizon.

5.1.1 General CFTOC formulation

Throughout this chapter we consider a discrete-time linear system

$$x_{i+1} = Ax_i + Bu_i,$$

with state and input constrained to lie in polyhedral sets

$$\mathcal{X} = \{x \in \mathbb{R}^n \mid A_x x \leq b_x\}, \quad \mathcal{U} = \{u \in \mathbb{R}^m \mid A_u u \leq b_u\},$$

and terminal set

$$\mathcal{X}_f = \{x \in \mathbb{R}^n \mid A_f x \leq b_f\}.$$

The associated cost has the general form

$$J(x_0, U) = V_f(x_N) + \sum_{i=0}^{N-1} \ell(x_i, u_i).$$

Two important classes of cost functions will be considered. In the quadratic case, one uses

$$V_f(x_N) = x_N^\top P x_N, \quad \ell(x_i, u_i) = x_i^\top Q x_i + u_i^\top R u_i,$$

with

$$P \succeq 0, \quad Q \succeq 0, \quad R \succ 0.$$

In the norm-based case, one uses either the 1-norm or the ∞ -norm:

$$V_f(x_N) = \|P x_N\|_p, \quad \ell(x_i, u_i) = \|Q x_i\|_p + \|R u_i\|_p, \quad p \in \{1, \infty\},$$

where the weighting matrices are chosen with appropriate rank assumptions so that the cost meaningfully penalizes the variables.

5.1.2 Feasible set

An important object associated with the constrained finite-horizon problem is its feasible set. It describes from which initial states the optimization problem can actually be solved while respecting all constraints over the horizon. For a given horizon N , the feasible set is

$$\mathcal{X}_N = \left\{ x_0 \in \mathbb{R}^n \mid \exists (u_0, \dots, u_{N-1}) \text{ such that } x_i \in \mathcal{X}, u_i \in \mathcal{U}, i = 0, \dots, N-1, x_N \in \mathcal{X}_f, x_{i+1} = Ax_i + Bu_i \right\}.$$

This set is fundamental because outside $\mathcal{X}_N$ the CFTOC problem is simply infeasible. Notice also that $\mathcal{X}_N$ depends on the dynamics, the constraints, the terminal set, and the horizon length, but not on the specific choice of cost. The cost only selects, among the feasible trajectories, the one that is considered optimal.

This is in clear contrast with the unconstrained LQR setting. There, every initial state is admissible because there are no state or input limits, and the solution has a linear feedback form. Once constraints are introduced, this global linear structure is lost, and the control law becomes more intricate.

5.2 Quadratic-cost CFTOC and quadratic programming

We first consider the constrained finite-horizon problem with quadratic stage and terminal cost:

$$J^*(x(k)) = \min_U x_N^\top P x_N + \sum_{i=0}^{N-1} (x_i^\top Q x_i + u_i^\top R u_i)$$

subject to

$$x_{i+1} = A x_i + B u_i, \quad i = 0, \dots, N-1,$$

$$x_i \in \mathcal{X}, \quad u_i \in \mathcal{U}, \quad i = 0, \dots, N-1,$$

$$x_N \in \mathcal{X}_f, \quad x_0 = x(k).$$

Because the dynamics are linear, the constraints are polyhedral, and the cost is quadratic, this problem can be rewritten as a quadratic program. This is one of the most important structural facts in nominal linear MPC.

5.2.1 QP formulation without eliminating the states

A first way to rewrite the quadratic-cost CFTOC problem as a quadratic program is to *keep the predicted states as decision variables* and impose the system dynamics as explicit equality constraints. This formulation is often attractive in practice because it preserves the sparse structure of the dynamics.

Starting from

$$J^*(x(k)) = \min_U x_N^\top P x_N + \sum_{i=0}^{N-1} (x_i^\top Q x_i + u_i^\top R u_i)$$

subject to

$$x_{i+1} = A x_i + B u_i, \quad i = 0, \dots, N-1,$$

$$x_i \in \mathcal{X}, \quad u_i \in \mathcal{U}, \quad i = 0, \dots, N-1,$$

$$x_N \in \mathcal{X}_f, \quad x_0 = x(k),$$

we introduce the stacked decision vector

$$z = \begin{bmatrix} x_1^\top & \dots & x_N^\top & u_0^\top & \dots & u_{N-1}^\top \end{bmatrix}^\top.$$

With this choice of variables, the CFTOC problem can be transformed into a QP of the form

$$J^*(x(k)) = \min_z \begin{bmatrix} z^\top & x(k)^\top \end{bmatrix} \begin{bmatrix} \bar{H} & 0 \\ 0 & Q \end{bmatrix} \begin{bmatrix} z \\ x(k) \end{bmatrix}$$

subject to

$$G_{\text{in}} z \leq w_{\text{in}} + E_{\text{in}} x(k), \quad G_{\text{eq}} z = E_{\text{eq}} x(k).$$

The equality constraints come directly from the dynamics

$$x_{i+1} = A x_i + B u_i.$$

Collecting them over the whole horizon gives

$$G_{\text{eq}} z = E_{\text{eq}} x(k),$$

with the block structure

$$G_{\text{eq}} = \begin{bmatrix} I & 0 & \cdots & 0 & -B & 0 & \cdots & 0 \\ -A & I & \cdots & 0 & 0 & -B & \cdots & 0 \\ 0 & -A & \ddots & \vdots & \vdots & \ddots & \ddots & \vdots \\ \vdots & \ddots & \ddots & I & 0 & \cdots & 0 & -B \end{bmatrix}, \quad E_{\text{eq}} = \begin{bmatrix} A \\ 0 \\ \vdots \\ 0 \end{bmatrix}.$$

The first block row corresponds to

$$x_1 - Ax(k) - Bu_0 = 0,$$

while the remaining rows encode

$$x_{i+1} - Ax_i - Bu_i = 0, \quad i = 1, \dots, N-1.$$

Next, suppose the constraint sets are given in polyhedral form as

$$\mathcal{X} = \{x \mid A_x x \leq b_x\}, \quad \mathcal{U} = \{u \mid A_u u \leq b_u\}, \quad \mathcal{X}_f = \{x \mid A_f x \leq b_f\}.$$

Then all state, input, and terminal constraints can be collected into

$$G_{\text{in}} z \leq w_{\text{in}} + E_{\text{in}} x(k).$$

The matrix G_{in} has a block-diagonal structure, with the state constraints acting on $x_1, \dots, x_{N-1}$, the terminal constraint acting on x_N , and the input constraints acting on $u_0, \dots, u_{N-1}$. Explicitly,

$$G_{\text{in}} = \begin{bmatrix} A_x & 0 & \cdots & 0 & 0 & 0 & \cdots & 0 \\ 0 & A_x & \cdots & 0 & 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & A_x & 0 & 0 & \cdots & 0 \\ 0 & 0 & \cdots & A_f & 0 & 0 & \cdots & 0 \\ 0 & 0 & \cdots & 0 & A_u & 0 & \cdots & 0 \\ 0 & 0 & \cdots & 0 & 0 & A_u & \cdots & 0 \\ \vdots & \vdots & & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 & 0 & 0 & \cdots & A_u \end{bmatrix},$$

$$w_{\text{in}} = \begin{bmatrix} b_x \\ b_x \\ \vdots \\ b_x \\ b_f \\ b_u \\ b_u \\ \vdots \\ b_u \end{bmatrix}, \quad E_{\text{in}} = \begin{bmatrix} -A_x \\ 0 \\ \vdots \\ 0 \end{bmatrix}.$$

The term $E_{\text{in}} x(k)$ appears because the initial state constraint $x_0 \in \mathcal{X}$ is not part of the decision vector z , so its contribution must be written separately on the right-hand side.

Finally, the cost is obtained directly from

$$x_N^\top P x_N + \sum_{i=0}^{N-1} (x_i^\top Q x_i + u_i^\top R u_i).$$

Since $x_0 = x(k)$ is fixed and not optimized over, the quadratic term in the optimization variables is described by the block-diagonal matrix

$$\bar{H} = \text{blockdiag}(Q, \dots, Q, P, R, \dots, R),$$

that is,

$$\bar{H} = \begin{bmatrix} Q & & & & & & \\ & \ddots & & & & & \\ & & Q & & & & \\ & & & P & & & \\ & & & & R & & \\ & & & & & \ddots & \\ & & & & & & R \end{bmatrix}.$$

So the quadratic-cost CFTOC problem can be written as a standard QP in which the dynamics appear as equality constraints, the state and input limits appear as inequalities, and the cost is a quadratic function of the stacked state-input vector z . This formulation stays very close to the original optimal control problem, since both predicted states and predicted inputs remain explicit throughout.

5.2.2 QP formulation by substitution

A second way to rewrite the quadratic-cost CFTOC problem as a quadratic program is to *substitute the state equations* directly into the cost and the constraints, so that the only decision variables are the future inputs. In this formulation, one starts from

$$x_{i+1} = Ax_i + Bu_i, \quad x_0 = x(k),$$

and eliminates the predicted states $x_1, \dots, x_N$ recursively.

Define the stacked input vector

$$U = \begin{bmatrix} u_0^\top & \cdots & u_{N-1}^\top \end{bmatrix}^\top.$$

Step 1: Rewrite the cost. After substituting the predicted states into the quadratic objective, the cost can be written as

$$J(x(k), U) = U^\top H U + 2x(k)^\top F U + x(k)^\top Y x(k).$$

Equivalently,

$$J(x(k), U) = \begin{bmatrix} U^\top & x(k)^\top \end{bmatrix} \begin{bmatrix} H & F^\top \\ F & Y \end{bmatrix} \begin{bmatrix} U \\ x(k) \end{bmatrix}.$$

Since the stage and terminal costs are assumed nonnegative, this block matrix satisfies

$$\begin{bmatrix} H & F^\top \\ F & Y \end{bmatrix} \succeq 0.$$

Step 2: Rewrite the constraints compactly. The input, state, and terminal constraints can also be expressed directly in terms of U and the current state $x(k)$. This gives the compact inequality description

$$GU \leq w + Ex(k).$$

To make this more explicit, suppose that

$$\mathcal{X} = \{x \mid A_x x \leq b_x\}, \quad \mathcal{U} = \{u \mid A_u u \leq b_u\}, \quad \mathcal{X}_f = \{x \mid A_f x \leq b_f\}.$$

Then the matrix G , the vector E , and the right-hand side w are obtained by stacking the input constraints first, then the state constraints along the horizon, and finally the terminal constraint. In block form,

$$G = \begin{bmatrix} A_u & 0 & \cdots & 0 \\ 0 & A_u & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & A_u \\ 0 & 0 & \cdots & 0 \\ A_x B & 0 & \cdots & 0 \\ A_x A B & A_x B & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ A_f A^{N-1} B & A_f A^{N-2} B & \cdots & A_f B \end{bmatrix},$$

$$E = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ -A_x \\ -A_x A \\ -A_x A^2 \\ \vdots \\ -A_f A^N \end{bmatrix}, \quad w = \begin{bmatrix} b_u \\ b_u \\ \vdots \\ b_u \\ b_x \\ b_x \\ b_x \\ \vdots \\ b_f \end{bmatrix}.$$

This compact form comes from writing each predicted state as an affine function of $x(k)$ and the input sequence U , and then imposing the polyhedral constraints on those expressions.

Step 3: Rewrite the optimal control problem as a QP. The quadratic-cost CFTOC problem therefore becomes

$$J^*(x(k)) = \min_U \begin{bmatrix} U^\top & x(k)^\top \end{bmatrix} \begin{bmatrix} H & F^\top \\ F & Y \end{bmatrix} \begin{bmatrix} U \\ x(k) \end{bmatrix}$$

subject to

$$GU \leq w + Ex(k).$$

This is a quadratic program in the variable U , parameterized by the current state $x(k)$. For a given $x(k)$, the optimal input sequence U^* can therefore be obtained by solving this QP.

5.2.3 Quadratic-cost state-feedback solution

Consider again the quadratic program obtained after substitution:

$$J^*(x(k)) = \min_U \begin{bmatrix} U^\top & x(k)^\top \end{bmatrix} \begin{bmatrix} H & F^\top \\ F & Y \end{bmatrix} \begin{bmatrix} U \\ x(k) \end{bmatrix}$$

subject to

$$GU \leq w + Ex(k).$$

For each fixed current state $x(k)$, this problem can be solved numerically and yields an optimal input sequence

$$U^*(x(k)) = \begin{bmatrix} u_0^*(x(k))^\top & \cdots & u_{N-1}^*(x(k))^\top \end{bmatrix}^\top.$$

As in MPC only the first component of the optimal sequence is applied to the plant, the relevant feedback law is

$$u_0^* = \kappa(x(k)).$$

The important point is that this QP is not just an optimization problem with state $x(k)$ inserted into it numerically. It is a *multiparametric quadratic program* (mp-QP), where the current state plays the role of a parameter. Because of this, the optimizer has a very specific structure as a function of the state.

In particular, on the feasible set $\mathcal{X}_0$, the first component of the optimal solution defines a map

$$\kappa : \mathbb{R}^n \rightarrow \mathbb{R}^m,$$

which is continuous and piecewise affine on polyhedra. More precisely, there exists a finite collection of polyhedral regions

$$CR^j = \{x \in \mathbb{R}^n \mid H^j x \leq K^j\}, \quad j = 1, \dots, N^r,$$

which form a partition of the feasible polyhedron $\mathcal{X}_0$, such that on each region one has

$$\kappa(x) = F^j x + g^j, \quad x \in CR^j, \quad j = 1, \dots, N^r.$$

So even though the original optimal control problem is constrained and does not admit a single global linear feedback law, it still has a highly structured solution: the state space is divided into polyhedral regions, and on each region the optimal controller is affine.

The optimal value function has a similarly regular form. The map $J^*(x(k))$, which assigns to each feasible initial state the optimal cost of the finite-horizon problem, is convex and piecewise quadratic on polyhedra. Thus, both the control law and the value function inherit a polyhedral structure from the parametric QP formulation.

This observation is the starting point for *explicit MPC*. Instead of solving the QP online at every sampling instant, one may compute offline the polyhedral partition together with the corresponding affine control laws. The online step is then reduced to locating the current state in the appropriate region and evaluating the affine formula associated with that region.

Quadratic-cost state-feedback structure

For quadratic-cost CFTOC, the parametric optimization problem is an mp-QP. As a result, the first optimal input defines a continuous piecewise affine state-feedback law on the feasible set, while the associated optimal value function is convex and piecewise quadratic.

5.3 CFTOC with 1-norm and ∞ -norm costs

So far we have considered quadratic-cost CFTOC, which leads naturally to a quadratic program. One can treat alternative cost functions in a very similar predictive-control framework. In particular, if the stage and terminal penalties are based on the 1-norm or the ∞ -norm, the constrained finite time optimal control problem can be rewritten as a linear program. More precisely, consider the finite-horizon problem

$$J^*(x(k)) = \min_U \|Px_N\|_p + \sum_{i=0}^{N-1} (\|Qx_i\|_p + \|Ru_i\|_p), \quad p \in \{1, \infty\},$$

subject to

$$x_{i+1} = Ax_i + Bu_i, \quad i = 0, \dots, N-1,$$

$$x_i \in \mathcal{X}, \quad u_i \in \mathcal{U}, \quad i = 0, \dots, N-1,$$

$$x_N \in \mathcal{X}_f, \quad x_0 = x(k).$$

The key point is that both the 1-norm and the ∞ -norm admit an equivalent *epigraph formulation*, in which the norm is replaced by a linear objective together with additional linear inequalities. This is what makes it possible to transform the CFTOC problem into an LP.

5.3.1 ℓ_∞ -minimization

To see the basic idea, it is useful to start from the simpler constrained minimization problem

$$\min_{x \in \mathbb{R}^n} \|x\|_\infty \quad \text{subject to} \quad Fx \leq g.$$

Since

$$\|x\|_\infty = \max\{|x_1|, \dots, |x_n|\},$$

this problem can be viewed as the minimization of the maximum of finitely many linear functions:

$$\min_{x \in \mathbb{R}^n} \max\{x_1, \dots, x_n, -x_1, \dots, -x_n\} \quad \text{subject to} \quad Fx \leq g.$$

The standard trick is then to introduce a scalar auxiliary variable t that upper-bounds all components in absolute value. This gives the equivalent problem

$$\min_{x, t} t$$

subject to

$$x_i \leq t, \quad i = 1, \dots, n,$$

$$-x_i \leq t, \quad i = 1, \dots, n,$$

$$Fx \leq g.$$

In compact vector form, this can be written as

$$\min_{x, t} t$$

subject to

$$-\mathbf{1} t \leq x \leq \mathbf{1} t, \quad Fx \leq g.$$

Here $\mathbf{1}$ denotes the vector of ones. The constraint

$$-\mathbf{1} t \leq x \leq \mathbf{1} t$$

simply says that every component of x must satisfy $|x_i| \leq t$, so the scalar t bounds the absolute value of all entries simultaneously. Minimizing t is therefore exactly the same as minimizing $\|x\|_\infty$.

The ∞ -norm is replaced by a linear cost in an additional scalar variable together with linear inequalities. The problem is no longer written in terms of a norm objective, but it has become a linear program.

5.3.2 ℓ_1 -minimization

The same idea works for the 1-norm. Consider

$$\min_{x \in \mathbb{R}^n} \|x\|_1 \quad \text{subject to} \quad Fx \leq g.$$

Since

$$\|x\|_1 = \sum_{i=1}^n |x_i|,$$

one may write this as

$$\min_{x \in \mathbb{R}^n} \sum_{i=1}^n \max\{x_i, -x_i\} \quad \text{subject to} \quad Fx \leq g.$$

Now, instead of a single scalar epigraph variable, one introduces a vector

$$t \in \mathbb{R}^n$$

whose components bound the absolute value of the corresponding entries of x . The problem becomes

$$\min_{x,t} t_1 + \cdots + t_n$$

subject to

$$\begin{aligned} x_i &\leq t_i, & i &= 1, \dots, n, \\ -x_i &\leq t_i, & i &= 1, \dots, n, \\ Fx &\leq g. \end{aligned}$$

In compact form,

$$\min_{x,t} \mathbf{1}^\top t$$

subject to

$$-t \leq x \leq t, \quad Fx \leq g.$$

Again, $\mathbf{1}$ denotes the vector of ones. The constraint

$$-t \leq x \leq t$$

means that each component t_i bounds the absolute value of the corresponding component x_i . At the optimum one therefore has $t_i = |x_i|$, and minimizing $\mathbf{1}^\top t$ is equivalent to minimizing $\|x\|_1$. So in both cases the same mechanism is at work: one introduces additional variables and writes the norm minimization problem in epigraph form. This is exactly the step that later allows the finite-horizon control problem to be expressed as an LP.

5.3.3 Construction of the LP with substitution

Returning now to CFTOC, we can proceed exactly as in the quadratic case with substitution: one eliminates the predicted states and writes everything directly in terms of the current state $x(k)$, the future inputs, and the auxiliary epigraph variables.

For the ∞ -norm case, one introduces scalar variables

$$\varepsilon_i^x, \quad i = 0, \dots, N,$$

and

$$\varepsilon_i^u, \quad i = 0, \dots, N-1,$$

which bound the norms of the weighted state and input terms. Using the substituted prediction formula for the states, the problem can be written equivalently as

$$\min_z \varepsilon_0^x + \cdots + \varepsilon_N^x + \varepsilon_0^u + \cdots + \varepsilon_{N-1}^u$$

subject to inequalities of the form

$$\begin{aligned} -\mathbf{1}_n \varepsilon_i^x &\leq \pm Q \left(A^i x_0 + \sum_{j=0}^{i-1} A^j B u_{i-1-j} \right), \\ -\mathbf{1}_r \varepsilon_N^x &\leq \pm P \left(A^N x_0 + \sum_{j=0}^{N-1} A^j B u_{N-1-j} \right), \\ -\mathbf{1}_m \varepsilon_i^u &\leq \pm R u_i, \end{aligned}$$

together with the state, input, and terminal constraints

$$\begin{aligned} A^i x_0 + \sum_{j=0}^{i-1} A^j B u_{i-1-j} &\in \mathcal{X}, \quad u_i \in \mathcal{U}, \\ A^N x_0 + \sum_{j=0}^{N-1} A^j B u_{N-1-j} &\in \mathcal{X}_f, \quad x_0 = x(k). \end{aligned}$$

The detailed block matrices are not especially illuminating in prose, but the important message is that after substitution and introduction of the epigraph variables, the entire problem has only a linear objective and linear constraints.

This yields the standard LP form

$$\min_z c^\top z \quad \text{subject to} \quad \bar{G}z \leq \bar{w} + \bar{S}x(k),$$

where the decision vector contains both the auxiliary variables and the input sequence:

$$z := \{\varepsilon_0^x, \dots, \varepsilon_N^x, \varepsilon_0^u, \dots, \varepsilon_{N-1}^u, u_0^\top, \dots, u_{N-1}^\top\} \in \mathbb{R}^s.$$

The matrices can then be partitioned :

$$\bar{G} = \begin{bmatrix} G_\varepsilon & G_u \\ 0 & G \end{bmatrix}, \quad \bar{S} = \begin{bmatrix} S_\varepsilon \\ S \end{bmatrix}, \quad \bar{w} = \begin{bmatrix} w_\varepsilon \\ w \end{bmatrix}.$$

For a given current state $x(k)$, the optimal sequence U^* can therefore be obtained by solving a linear program. The 1-norm case works in exactly the same spirit; only the epigraph variables are vector-valued instead of scalar.

5.3.4 1-/∞-norm state-feedback solution

Once the norm-based CFTOC problem has been written in the compact LP form

$$\min_z c^\top z \quad \text{subject to} \quad \bar{G}z \leq \bar{w} + \bar{S}x(k),$$

the parametric structure of the optimizer becomes analogous to the quadratic case. The current state $x(k)$ again enters as a parameter, so the problem is a *multiparametric linear program* (mp-LP).

As a consequence, the first component of the multiparametric solution defines a state-feedback law

$$u_0^* = \kappa(x(k)), \quad \forall x(k) \in \mathcal{X}_0,$$

where

$$\kappa : \mathbb{R}^n \rightarrow \mathbb{R}^m$$

is piecewise affine on polyhedra. More precisely, there exists a polyhedral partition

$$CR^j = \{x \in \mathbb{R}^n \mid H^j x \leq K^j\}, \quad j = 1, \dots, N^r,$$

of the feasible polyhedron $\mathcal{X}_0$ such that

$$\kappa(x) = F^j x + g^j, \quad x \in CR^j, \quad j = 1, \dots, N^r.$$

If the optimizer is unique, this control law is continuous and piecewise affine. If the LP has multiple optimizers, one can still construct a piecewise affine control law, but uniqueness is no longer guaranteed automatically.

The value function also reflects the underlying linear structure. In contrast with the quadratic case, where the value function is piecewise quadratic, in the LP case the optimal value

$$J^*(x(k))$$

is convex and piecewise affine on polyhedra.

5.3.5 Solution properties: quadratic versus 1-/∞-norm cost

We conclude this part by contrasting the geometry of quadratic and linear costs. This comparison is useful because it explains why the value functions and optimizer properties differ.

Let n denote the number of optimization variables. If the cost is quadratic and positive definite, then the solution is always unique whenever the problem is feasible. More precisely, the optimizer is either in the interior of the feasible set, in which case no inequality constraint is active, or on the boundary of the feasible set, in which case at least one constraint is active. For a linear cost, the situation is different. The problem may be unbounded. If it is bounded and the optimum is unique, then it is attained at a vertex of the feasible set, which means that at least n constraints are active. It may also happen that there is not a unique minimizer, but rather a whole set of optimal solutions; in that case at least one constraint is active and the optimum is attained along a face of the feasible polyhedron.

So the difference between the two formulations is not only algebraic but also geometric. A strictly convex quadratic objective selects a unique point, while a linear objective may select an entire face. This is why quadratic-cost CFTOC leads to a convex piecewise quadratic value function and typically unique optimizers, whereas 1-/∞-norm CFTOC leads to a convex piecewise affine value function and may admit multiple optimal solutions.

Norm-based CFTOC as an LP

If the stage and terminal costs are based on the 1-norm or the ∞-norm, the CFTOC problem can be rewritten in epigraph form and transformed into a linear program. The resulting parametric problem is an mp-LP, whose first optimal input is piecewise affine on a polyhedral partition of the feasible set, while the value function is convex and piecewise affine.

5.4 Receding-horizon notation and control law

Consider the discrete-time model

$$x(k+1) = Ax(k) + Bu(k), \quad y(k) = Cx(k),$$

with

$$x(k) \in \mathcal{X}, \quad u(k) \in \mathcal{U}, \quad \forall k \geq 0.$$

Up to this point, we have mostly written the constrained finite time optimal control problem in a compact form that suppresses the prediction origin. For MPC, however, it is useful to make this dependence explicit, since the optimization problem is solved repeatedly at successive time instants.

5.4.1 CFTOC problem solved at time k

At time k , the constrained finite time optimal control problem is written as

$$J_{k \rightarrow k+N|k}^*(x(k)) = \min_{U_{k \rightarrow k+N|k}} V_f(x_{k+N|k}) + \sum_{i=0}^{N-1} \ell(x_{k+i|k}, u_{k+i|k})$$

subject to

$$\begin{aligned} x_{k+i+1|k} &= Ax_{k+i|k} + Bu_{k+i|k}, & i &= 0, \dots, N-1, \\ x_{k+i|k} &\in \mathcal{X}, \quad u_{k+i|k} \in \mathcal{U}, & i &= 0, \dots, N-1, \\ x_{k+N|k} &\in \mathcal{X}_f, \\ x_{k|k} &= x(k), \end{aligned}$$

where the decision sequence is

$$U_{k \rightarrow k+N|k} = \{u_{k|k}, \dots, u_{k+N-1|k}\}.$$

So the problem is solved at time k , starting from the currently measured or estimated state $x(k)$, and it produces an optimal input sequence over the horizon $k, \dots, k+N-1$.

5.4.2 Meaning of the prediction notation

The notation is best understood carefully. The symbol $x(k)$ denotes the actual state of the system at time k . By contrast,

$$x_{k+i|k}$$

denotes the state predicted for time $k+i$, where the prediction is made at time k . This predicted state is obtained by starting from the current state

$$x_{k|k} = x(k)$$

and propagating the model forward using the planned inputs

$$u_{k|k}, \dots, u_{k+i-1|k}.$$

For instance, the first predicted step is

$$x_{k+1|k} = Ax_{k|k} + Bu_{k|k}.$$

It is important to keep in mind that the same physical time instant may correspond to different predictions, depending on when the prediction is made. For example, $x_{3|1}$ is the state predicted for time 3 using information available at time 1, whereas $x_{3|2}$ is the state predicted for time 3 using information available at time 2. In general, these two quantities are different because they are based on different initial conditions and different input plans.

Similarly,

$$u_{k+i|k}$$

is read as “the input at time $k+i$, computed at time k .” This notation keeps track of the fact that in MPC one computes an entire future plan, even though not all of it will actually be implemented.

5.4.3 From the optimal sequence to the applied input

Let

$$U_{k \rightarrow k+N|k}^* = \{u_{k|k}^*, \dots, u_{k+N-1|k}^*\}$$

denote the optimal solution of the CFTOC problem solved at time k . Although this optimal sequence contains N future control moves, only its first element is applied to the plant:

$$u(k) = u_{k|k}^*(x(k)).$$

This is the defining feature of receding-horizon control. The optimizer computes a whole sequence, but the controller implements only the first input and discards the rest after one sampling interval.

At the next time instant, the state is updated to

$$x_{k+1|k+1} = x(k+1),$$

and the optimization problem is solved again from this new initial condition. In this way, the controller continuously replans as new state information becomes available.

5.4.4 Receding-horizon control law and closed loop

The first optimal move defines the receding-horizon feedback law

$$\kappa_k(x(k)) = u_{k|k}^*(x(k)).$$

The corresponding closed-loop system is therefore

$$x(k+1) = Ax(k) + B\kappa_k(x(k)) =: g_{cl}(x(k)), \quad k \geq 0.$$

So even though the optimization problem is finite-horizon and open-loop in its internal prediction, the repeated re-solution at each step turns the overall control scheme into a feedback controller.

5.4.5 Time-invariant systems

If the system, the constraints, and the cost function are time-invariant, then the CFTOC problem has the same structure at every time k . In that case, the feedback law obtained by receding-horizon optimization also becomes time-invariant, and the notation can be simplified. One then writes

$$J^*(x(k)) = \min_U V_f(x_N) + \sum_{i=0}^{N-1} \ell(x_i, u_i)$$

subject to

$$\begin{aligned} x_{i+1} &= Ax_i + Bu_i, & i &= 0, \dots, N-1, \\ x_i &\in \mathcal{X}, & u_i &\in \mathcal{U}, & i &= 0, \dots, N-1, \\ x_N &\in \mathcal{X}_f, & x_0 &= x(k), \end{aligned}$$

where

$$U = \{u_0, \dots, u_{N-1}\}.$$

In the same spirit, the control law is written simply as

$$u(k) = \kappa(x(k))$$

instead of $\kappa_k(x(k))$.

Receding-horizon principle

At each sampling instant, MPC solves a finite-horizon optimal control problem, applies only the first element of the optimal input sequence, and then repeats the optimization at the next time step using the new state. This repeated re-optimization is what gives MPC its feedback nature.

5.5 Summary

Constrained finite time optimal control is the optimization problem at the heart of nominal MPC. Starting from a linear discrete-time model, one predicts future trajectories over a finite horizon, penalizes those trajectories through a chosen cost function, and imposes state, input, and terminal constraints explicitly. The resulting feasible set consists of the initial states from which at least one admissible predicted trajectory exists.

When the cost is quadratic and the constraints are polyhedral, the problem can be written as a convex quadratic program. This may be done either by keeping the state variables explicitly and enforcing the dynamics as equality constraints, or by substituting the dynamics and optimizing only over the input sequence. In the parametric viewpoint, the first optimal input is a piecewise affine function of the state and the value function is piecewise quadratic.

When the cost is based on the 1-norm or the ∞ -norm, auxiliary epigraph variables allow the problem to be rewritten as a linear program. In that case the first optimal input is again piecewise affine, but the value function becomes piecewise affine and multiple optimal solutions may occur.

Finally, the finite-horizon problem is embedded into a closed-loop strategy through the receding-horizon principle. At each sampling instant, one solves a CFTOC problem from the current state, applies only the first input of the optimal sequence, and then repeats the same procedure at the next time step. This is the basic operational mechanism of model predictive control.

Chapter 6

Invariance

6.1 Objectives of constrained control

In the previous chapter we formulated the constrained finite time optimal control problem that lies at the core of nominal MPC. However, solving a finite-horizon problem at one instant is only part of the story. In closed loop, one wants the controller to keep generating admissible inputs over time, so that the state and input constraints are never violated. This naturally leads to the question of which initial states are *safe* in the sense that constraint satisfaction can be maintained forever.

Consider a general discrete-time system

$$x(k+1) = g(x(k), u(k)),$$

with state and input constrained to lie in sets

$$x(k) \in \mathcal{X}, \quad u(k) \in \mathcal{U}.$$

A control law $u(k) = \kappa(x(k))$ should ideally achieve several goals at once. First, it should respect the constraints for all time. Second, it should stabilize the system, typically to the origin or to a desired equilibrium. Third, among all admissible closed-loop behaviors, it should optimize a suitable performance criterion. Finally, one would like the controller to work from as large a set of initial conditions as possible.

In this chapter we focus on the first of these goals: persistent constraint satisfaction. This is the role of invariant and control invariant sets. Roughly speaking, an invariant set describes a region in which the system can remain forever once it enters it, while a control invariant set describes a region from which one can always choose an admissible input that keeps the system inside that region. These notions are fundamental in MPC because they formalize the idea of long-term feasibility.

A simple way to understand why such sets matter is to compare linear feedback with predictive control in the presence of hard constraints. Even if an unconstrained linear controller stabilizes the system, its region of safe operation may be very small once actuator limits are imposed. Outside that region, either the input saturates immediately or the state evolves in such a way that future constraint violations become unavoidable. Nonlinear constrained controllers, and in particular MPC, are attractive precisely because they can enlarge the set of initial states from which the constraints can be respected indefinitely.

6.2 Invariance

6.2.1 Positively invariant sets

We begin with the autonomous case. Consider the system

$$x(k+1) = g(x(k)),$$

or, more generally, a closed-loop system obtained after fixing a feedback law κ ,

$$x(k+1) = g(x(k), \kappa(x(k))).$$

The central question is the following: for which initial states does the trajectory remain inside the constraint set $\mathcal{X}$ for all future times?

Positively invariant sets

A set $\mathcal{O} \subseteq \mathbb{R}^n$ is called a *positively invariant set* for the autonomous system $x(k+1) = g(x(k))$ if

$$x(k) \in \mathcal{O} \implies x(k+1) \in \mathcal{O} \quad \forall k \in \{0, 1, 2, \dots\}.$$

Thus, once the state starts inside $\mathcal{O}$, all future states remain inside $\mathcal{O}$ as well. If in addition $\mathcal{O} \subseteq \mathcal{X}$, then $\mathcal{O}$ provides a set of initial conditions from which the trajectory never violates the state constraints.

Among all positively invariant sets contained in the constraint set $\mathcal{X}$, one is especially important.

Maximal positively invariant set

The set $\mathcal{O}_\infty \subseteq \mathcal{X}$ is called the *maximal positively invariant set* with respect to $\mathcal{X}$ if

$$\mathcal{O}_\infty \text{ is positively invariant}$$

and

$$\mathcal{O} \subseteq \mathcal{O}_\infty \quad \text{for every positively invariant set } \mathcal{O} \subseteq \mathcal{X}.$$

This set has a very natural interpretation: it is the set of all states for which the autonomous system can remain feasible forever once it is there. In that sense, $\mathcal{O}_\infty$ is the largest region of perpetual constraint satisfaction for the given dynamics.

6.2.2 Pre-sets

To study invariance, it is useful to introduce a one-step backward reachability notion.

Pre-set

Given a set S and the dynamic system

$$x(k+1) = g(x(k)),$$

the *pre-set* of S is the set of states that evolve into the target set S in one time step:

$$\text{pre}(S) := \{x \mid g(x) \in S\}.$$

So, if S is the set of states we would like to be in at the next time instant, then $\text{pre}(S)$ is the set of states from which this happens in one step. In other words, the pre-set tells us which current states are compatible with reaching the target set immediately under the given dynamics.

This notion gives a very compact way to express invariance. Indeed, a set $\mathcal{O}$ is positively invariant precisely when every state in $\mathcal{O}$ is mapped back into $\mathcal{O}$ after one step. That means exactly that every state in $\mathcal{O}$ must belong to the pre-set of $\mathcal{O}$.

Geometric condition for invariance

A set $\mathcal{O}$ is a positively invariant set if and only if

$$\mathcal{O} \subseteq \text{pre}(\mathcal{O}).$$

Equivalently,

$$\text{pre}(\mathcal{O}) \cap \mathcal{O} = \mathcal{O}.$$

The proof is immediate and is conveniently written through the contrapositive in both directions. If $\mathcal{O} \not\subseteq \text{pre}(\mathcal{O})$, then there exists some $\bar{x} \in \mathcal{O}$ such that $\bar{x} \notin \text{pre}(\mathcal{O})$. By definition of the pre-set, this means $g(\bar{x}) \notin \mathcal{O}$, so $\mathcal{O}$ is not positively invariant. Conversely, if $\mathcal{O}$ is not positively invariant, then there exists some $\bar{x} \in \mathcal{O}$ such that $g(\bar{x}) \notin \mathcal{O}$. Again by definition of the pre-set, this implies $\bar{x} \notin \text{pre}(\mathcal{O})$, and therefore $\mathcal{O} \not\subseteq \text{pre}(\mathcal{O})$.

For general nonlinear systems, computing pre-sets can be very difficult. Even if the target set S has a simple description, the set of all states that reach it in one time step may have a complicated shape. Only in special cases, most notably for linear systems with suitably described constraint sets, does the computation become tractable.

6.2.3 Linear autonomous systems

For linear autonomous dynamics

$$x(k+1) = Ax(k),$$

the pre-set becomes much easier to describe. If the target set is a polyhedron

$$S = \{x \in \mathbb{R}^n \mid Fx \leq f\},$$

then

$$\text{pre}(S) = \{x \in \mathbb{R}^n \mid Ax \in S\} = \{x \in \mathbb{R}^n \mid FAx \leq f\}.$$

So for linear systems the pre-set of a polyhedron is again a polyhedron. This closure property is the key reason why polyhedral invariant-set computations are possible.

6.2.4 Conceptual computation of the maximal invariant set

The maximal positively invariant set can be characterized by a simple iterative construction. Starting from the full constraint set,

$$\Omega_0 := \mathcal{X},$$

one recursively removes all states that cannot remain in the current set after one time step. This leads to the iteration

$$\Omega_{i+1} := \text{pre}(\Omega_i) \cap \Omega_i.$$

Since Ω_{i+1} is obtained by intersecting Ω_i with the set of states that return to Ω_i in one step, the sequence is nested:

$$\Omega_{i+1} \subseteq \Omega_i, \quad \forall i \in \mathbb{N}.$$

Thus, at each step, one removes the states that cannot be kept inside the current candidate set. The interpretation is very natural. At the first iteration one discards all states in $\mathcal{X}$ that leave $\mathcal{X}$ after one transition. At the second iteration one discards all states that enter the already

discarded region after one transition, and so on. If the procedure reaches a fixed point, that fixed point is precisely the maximal positively invariant set.

Conceptual algorithm to compute the maximal invariant set

Input: $g, \mathcal{X}$

Output: $\mathcal{O}_\infty$

```

 $\Omega_0 \leftarrow \mathcal{X}$ 
loop
   $\Omega_{i+1} \leftarrow \text{pre}(\Omega_i) \cap \Omega_i$ 
  if  $\Omega_{i+1} = \Omega_i$  then
    return  $\mathcal{O}_\infty = \Omega_i$ 
  end if
end loop

```

So, if for some finite index i the iteration satisfies

$$\Omega_{i+1} = \Omega_i,$$

then the algorithm has converged and the limit set is

$$\mathcal{O}_\infty = \Omega_i.$$

This set is the largest positively invariant subset of $\mathcal{X}$.

In practice, each iteration requires several set operations: one must represent the current set Ω_i , compute its pre-set, intersect sets, and check whether the procedure has converged. These operations are manageable in some structured settings, especially for linear systems with polyhedral sets, but they can become much more difficult in general nonlinear problems.

6.3 Control invariance

6.3.1 Definition and meaning

In the autonomous setting, the evolution is fixed once the current state is known. In control, by contrast, we can influence the next state through the input. This leads to a more flexible notion. Consider the controlled system

$$x(k+1) = g(x(k), u(k)),$$

subject to the constraints

$$x(k) \in \mathcal{X}, \quad u(k) \in \mathcal{U}.$$

Control invariant sets

A set $\mathcal{C} \subseteq \mathcal{X}$ is called a *control invariant set* if

$$x(k) \in \mathcal{C} \implies \exists u(k) \in \mathcal{U} \text{ such that } g(x(k), u(k)) \in \mathcal{C}$$

for all $k \in \mathbb{N}$.

This definition differs from positive invariance in one essential way: it does not require that the next state remains in the set under a fixed autonomous evolution, but only that there *exists* an admissible control action that keeps the successor state in the set. Therefore control

invariance captures the states from which constraint satisfaction can be maintained indefinitely by choosing the input appropriately.

The largest such set plays the same role as before.

Maximal control invariant set

The set $\mathcal{C}_\infty \subseteq \mathcal{X}$ is called the *maximal control invariant set* for the system $x(k+1) = g(x(k), u(k))$ under the constraints $(x, u) \in \mathcal{X} \times \mathcal{U}$ if it is control invariant and contains every control invariant set contained in $\mathcal{X}$.

Thus $\mathcal{C}_\infty$ is the best any controller can do in terms of perpetual constraint satisfaction. If $x(0) \in \mathcal{C}_\infty$, then there exists at least one admissible control strategy that keeps the system feasible forever. If $x(0) \notin \mathcal{C}_\infty$, then no controller can guarantee this.

6.3.2 Controlled pre-sets

The pre-set concept extends naturally to systems with inputs. Given a target set $S \subseteq \mathbb{R}^n$, we define

$$\text{pre}(S) := \{x \in \mathbb{R}^n \mid \exists u \in \mathcal{U} \text{ such that } g(x, u) \in S\}.$$

So the pre-set now contains all states from which it is *possible* to drive the system into S in one step using an admissible input.

With this definition, the geometric condition for control invariance has exactly the same form as before.

Geometric condition for control invariance

A set $\mathcal{C}$ is control invariant if and only if

$$\mathcal{C} \subseteq \text{pre}(\mathcal{C}).$$

Accordingly, the same conceptual fixed-point iteration can be used:

$$\Omega_0 := \mathcal{X}, \quad \Omega_{i+1} := \text{pre}(\Omega_i) \cap \Omega_i.$$

If the iteration converges in finite time, the fixed point is the maximal control invariant set $\mathcal{C}_\infty$.

Conceptually this is identical to the autonomous case, but computationally it is much harder, because the controlled pre-set involves existential quantification over the input. In other words, one must determine not only where the state goes, but whether there exists an admissible input that drives it into the target set.

6.3.3 Linear systems and projection

For the linear system

$$x(k+1) = Ax(k) + Bu(k),$$

with input constraints

$$\mathcal{U} = \{u \in \mathbb{R}^m \mid Gu \leq g\},$$

and target set

$$S = \{x \in \mathbb{R}^n \mid Fx \leq f\},$$

the controlled pre-set is

$$\text{pre}(S) = \{x \in \mathbb{R}^n \mid \exists u \in \mathcal{U} \text{ such that } Ax + Bu \in S\}.$$

Equivalently,

$$\text{pre}(S) = \left\{ x \in \mathbb{R}^n \mid \exists u \in \mathbb{R}^m : \begin{bmatrix} FA & FB \\ 0 & G \end{bmatrix} \begin{bmatrix} x \\ u \end{bmatrix} \leq \begin{bmatrix} f \\ g \end{bmatrix} \right\}.$$

This description shows that the controlled pre-set is obtained by projecting a higher-dimensional polyhedron in the (x, u) -space onto the x -space. That projection step is the main source of computational difficulty.

Still, the conceptual meaning is very clear. In the autonomous case, each state has only one possible successor. In the controlled case, each state is associated with an entire set of possible successors, one for each admissible input. This makes the controlled pre-set much larger, but also much more difficult to compute.

6.3.4 From a control invariant set to a control law

A control invariant set is not just a geometric object: it can be used directly to synthesize a control law that guarantees constraint satisfaction. Suppose $\mathcal{C}$ is control invariant. Then for every $x \in \mathcal{C}$ there exists at least one input $u \in \mathcal{U}$ such that the next state remains in $\mathcal{C}$. One can therefore define a feedback law by selecting, at each state, one admissible input with this property. A generic construction is

$$\kappa(x) := \arg \min_u \{ f(x, u) \mid g(x, u) \in \mathcal{C}, u \in \mathcal{U} \},$$

where f can be any objective function, even $f(x, u) = 0$ if one only wants feasibility.

If $\kappa(x) \in \mathcal{U}$ for all $x \in \mathcal{C}$ and

$$g(x, \kappa(x)) \in \mathcal{C} \quad \forall x \in \mathcal{C},$$

then the closed-loop system remains in $\mathcal{C}$ forever, and therefore the constraints are satisfied for all time.

This construction guarantees feasibility, but not necessarily convergence. Stability requires additional ingredients. This is precisely where MPC enters: it augments feasibility with an optimal control objective and suitable terminal conditions so as to recover both constraint satisfaction and closed-loop stability.

6.4 Relation to MPC

Invariant and control invariant sets are fundamental in MPC, but one rarely computes the maximal control invariant set explicitly in realistic problems. The reason is not conceptual but computational. Even for constrained linear systems, exact computation of maximal control invariant sets can become very complex. For nonlinear systems it is usually far harder.

This difficulty explains one of the deep ideas behind MPC. Instead of trying to compute the largest control invariant region once and for all, MPC solves a tractable finite-horizon optimization problem at each sampling instant. In doing so, it implicitly describes a control invariant behavior in a form that is much easier to represent and compute online. Terminal ingredients such as invariant terminal sets are then used to connect the finite-horizon problem with the infinite-horizon feasibility and stability properties one would ideally like to have.

Control invariance and MPC

A maximal control invariant set tells us the largest region from which some admissible controller can keep the constraints satisfied forever. MPC can be viewed as a tractable way of realizing this idea indirectly, without having to compute that maximal set explicitly.

6.5 Practical computation of invariant sets

6.5.1 Polyhedra and polytopes

For computational purposes, invariant sets are often represented as polyhedra. A polyhedron is the intersection of finitely many halfspaces and can be written as

$$P = \{x \in \mathbb{R}^n \mid Ax \leq b\}.$$

If P is bounded, it is called a *polytope*. In linear MPC this is the natural set class, because state, input, and terminal constraints are often given by linear inequalities.

A complementary viewpoint is given by the convex hull representation. If $v_1, \dots, v_s \in \mathbb{R}^n$, their convex hull is

$$\text{co}\{v_1, \dots, v_s\} = \left\{ x = \sum_{i=1}^s \lambda_i v_i \mid \lambda_i \geq 0, \sum_{i=1}^s \lambda_i = 1 \right\}.$$

The Minkowski–Weyl theorem states that a set is a polytope if and only if it can be represented both as a bounded intersection of finitely many halfspaces and as the convex hull of finitely many points. In computations, one switches between these two descriptions depending on which set operation must be performed.

6.5.2 Common constraints in MPC

Many constraints that appear in MPC are naturally polytopic. Simple box constraints on inputs and outputs,

$$u_{\text{low}} \leq u(k) \leq u_{\text{high}}, \quad y_{\text{low}} \leq y(k) \leq y_{\text{high}},$$

can be written as linear inequalities in the stacked vector $\begin{bmatrix} x(k) \\ u(k) \end{bmatrix}$. Likewise, magnitude constraints such as

$$\|Cx(k)\|_{\infty} \leq \alpha$$

and rate constraints such as

$$\|x(k+1) - x(k)\|_{\infty} \leq \alpha$$

are also polyhedral. This is why polytopes are such a natural computational language in linear MPC.

6.5.3 Intersection and subset tests

The iterative computation of invariant sets relies repeatedly on two basic operations: intersection and equality testing.

If

$$S = \{x \in \mathbb{R}^n \mid Cx \leq c\}, \quad T = \{x \in \mathbb{R}^n \mid Dx \leq d\},$$

then their intersection is obtained simply by stacking inequalities:

$$S \cap T = \left\{ x \in \mathbb{R}^n \mid \begin{bmatrix} C \\ D \end{bmatrix} x \leq \begin{bmatrix} c \\ d \end{bmatrix} \right\}.$$

So intersection is easy in inequality form.

Testing whether one polytope is contained in another is more subtle. Suppose

$$P = \{x \in \mathbb{R}^n \mid Cx \leq c\}, \quad Q = \{x \in \mathbb{R}^n \mid Dx \leq d\}.$$

To check whether $P \subseteq Q$, it is enough to verify each inequality defining Q . Let D_i denote the i -th row of D . Then $P \subseteq Q$ holds if and only if

$$\max_x D_i x \quad \text{subject to } Cx \leq c$$

is at most d_i for every row i . This maximum is the support function of P in the direction D_i :

$$h_P(D_i) := \max_x D_i x \quad \text{subject to } Cx \leq c.$$

Therefore

$$P \subseteq Q \quad \Longleftrightarrow \quad h_P(D_i) \leq d_i \quad \text{for all } i.$$

This subset test is important because equality of two polytopes can be checked through two containment relations:

$$P = Q \quad \Longleftrightarrow \quad P \subseteq Q \text{ and } Q \subseteq P.$$

6.5.4 Polytopic invariant sets

For linear autonomous systems with polytopic constraints, the conceptual invariant-set algorithm becomes implementable. One represents each iterate Ω_i as a polytope, computes the pre-set through linear inequality manipulation, intersects polytopes by stacking inequalities, and tests convergence through bidirectional subset checks.

In principle, this allows one to obtain the maximal polyhedral invariant set. In practice, however, the number of inequalities can grow quickly with the state dimension and with the number of iterations. This is one of the main limitations of exact polyhedral computations in higher dimensions.

6.5.5 Ellipsoids and Lyapunov-based invariant sets

An alternative is to work with ellipsoids. Given a symmetric positive definite matrix $P \in \mathbb{R}^{n \times n}$ and a center $x_c \in \mathbb{R}^n$, the ellipsoid

$$\mathcal{E} := \{x \in \mathbb{R}^n \mid (x - x_c)^\top P (x - x_c) \leq 1\}$$

has a fixed-complexity description: no matter how large the dimension is, containment can be checked through quadratic expressions rather than through arbitrarily many inequalities.

This connects naturally with Lyapunov theory. Suppose $V : \mathbb{R}^n \rightarrow \mathbb{R}$ is a Lyapunov function for the autonomous system

$$x(k+1) = g(x(k)).$$

Then every sublevel set

$$Y_\alpha := \{x \in \mathbb{R}^n \mid V(x) \leq \alpha\}$$

is invariant. Indeed, if the state starts in Y_α , then the Lyapunov function decreases along the trajectory, so the state can never leave that sublevel set.

For linear systems

$$x(k+1) = Ax(k),$$

a standard quadratic Lyapunov function is

$$V(x) = x^\top P x,$$

where $P \succ 0$ satisfies

$$A^\top P A - P \prec 0.$$

Then the invariant sublevel sets are ellipsoids of the form

$$Y_\alpha = \{x \in \mathbb{R}^n \mid x^\top P x \leq \alpha\}.$$

Such sets are typically not maximal, unlike the exact polyhedral invariant set, but they are much easier to compute and scale better to larger dimensions.

6.5.6 Largest ellipsoid contained in a polytope

Suppose the state constraints are given by the polytope

$$\mathcal{X} = \{x \in \mathbb{R}^n \mid Fx \leq f\},$$

and suppose $V(x) = x^\top Px$ is a Lyapunov function. We want the largest ellipsoidal invariant set of the form

$$Y_\alpha = \{x \in \mathbb{R}^n \mid x^\top Px \leq \alpha\}$$

that is contained in $\mathcal{X}$.

This means choosing the largest $\alpha \geq 0$ such that

$$Y_\alpha \subseteq \mathcal{X}.$$

Using the support function, this is equivalent to requiring

$$h_{Y_\alpha}(F_i) \leq f_i \quad \text{for all rows } F_i \text{ of } F.$$

For an ellipsoid one can compute this support function explicitly:

$$h_{Y_\alpha}(F_i) = \|P^{-1/2}F_i^\top\| \sqrt{\alpha}.$$

Hence the inclusion condition becomes

$$\|P^{-1/2}F_i^\top\| \sqrt{\alpha} \leq f_i \quad \forall i,$$

or, equivalently,

$$\alpha \leq \frac{f_i^2}{F_i P^{-1} F_i^\top} \quad \forall i.$$

Therefore the largest admissible value is

$$\alpha^* = \min_i \frac{f_i^2}{F_i P^{-1} F_i^\top}.$$

This gives the largest Lyapunov ellipsoid of the chosen shape matrix P that is still fully contained in the constraints.

The important message is that ellipsoidal invariant sets trade exactness for tractability. They are usually smaller than the maximal polyhedral invariant set, but they are easy to evaluate and can be computed efficiently even in higher dimensions.

6.6 Summary

Invariant and control invariant sets provide the right geometric language for infinite-horizon constraint satisfaction. A positively invariant set describes a region in which an autonomous or already closed-loop system remains forever once it enters it. A control invariant set describes a region in which one can always choose an admissible input that keeps the state inside the region.

The maximal positively invariant set $\mathcal{O}_\infty$ and the maximal control invariant set $\mathcal{C}_\infty$ are the largest regions with these properties. They can be characterized through pre-sets and, at least conceptually, computed by the same fixed-point iteration

$$\Omega_{i+1} = \text{pre}(\Omega_i) \cap \Omega_i.$$

For autonomous systems this iteration removes states that cannot remain feasible after one step. For controlled systems it removes states from which no admissible input can keep the trajectory feasible.

In linear settings with polyhedral constraints, invariant-set computations can be carried out using polytopes. Intersections are easy in inequality form, while subset tests can be performed through support functions. Exact polyhedral invariant sets are very informative, but their complexity can grow quickly with dimension.

Ellipsoidal invariant sets offer a more tractable alternative. When obtained from quadratic Lyapunov functions, they are invariant by construction and their size inside a polytopic constraint set can be computed explicitly. They are generally conservative compared with maximal polyhedral sets, but their complexity remains fixed and manageable.

From the viewpoint of MPC, the key lesson is that invariant sets are not just auxiliary geometric objects. They are central tools for reasoning about long-term feasibility, and they form the bridge between finite-horizon optimization and infinite-horizon constraint satisfaction. In the next chapter, this bridge will be used to study recursive feasibility and stability of MPC.

Chapter 7

Feasibility and Stability

7.1 Motivation

The previous chapters introduced the constrained finite time optimal control problem and showed how it gives rise to the receding-horizon control law used in MPC. At each sampling instant, one solves a finite-horizon optimization problem, applies only the first input of the optimal sequence, measures the new state, and repeats the procedure. This mechanism introduces feedback, because the optimization is repeatedly updated using the current state.

However, the fact that an optimization problem is solved at every time instant does not by itself guarantee that the resulting closed-loop system behaves well. In particular, two fundamental questions remain open. First, if the MPC problem is feasible at the current state, will it also be feasible at the next state after applying the first optimal input? Second, will the closed-loop trajectory actually converge to the desired equilibrium? These are the questions of recursive feasibility and closed-loop stability.

The ideal objective of constrained control can be described as follows. Given a discrete-time system

$$x(k+1) = g(x(k), u(k)),$$

with state and input constraints

$$x(k) \in \mathcal{X}, \quad u(k) \in \mathcal{U},$$

we would like to design a control law

$$u(k) = \kappa(x(k))$$

such that the closed-loop trajectory satisfies the constraints for all times, converges to the origin, achieves good performance, and does so from as large a set of initial conditions as possible. In other words, the controller should not only compute locally optimal actions, but also preserve the long-term admissibility and stability of the closed loop.

A natural benchmark is the constrained infinite-horizon optimal control problem

$$\begin{aligned} J_{\infty}^*(x(0)) &:= \min_{u(\cdot)} \sum_{i=0}^{\infty} \ell(x_i, u_i) \\ \text{subj. to } & x_{i+1} = Ax_i + Bu_i, \quad i = 0, 1, \dots, \\ & x_i \in \mathcal{X}, \quad u_i \in \mathcal{U}, \quad i = 0, 1, \dots, \\ & x_0 = x(0). \end{aligned}$$

This formulation is conceptually attractive because it optimizes over the entire future. The stage cost $\ell(x, u)$ measures the immediate cost of being in state x and applying input u , while the

infinite sum forces the optimizer to account for the long-term consequences of each decision. In this sense, the infinite-horizon problem naturally balances short-term and long-term benefits. The difficulty is that the infinite-horizon problem contains infinitely many decision variables and constraints. Except in special cases, it cannot be solved directly online. MPC therefore replaces it by a finite-horizon problem of the form

$$\begin{aligned} J^*(x(k)) &:= \min_U V_f(x_N) + \sum_{i=0}^{N-1} \ell(x_i, u_i) \\ \text{subj. to } & x_{i+1} = Ax_i + Bu_i, \quad i = 0, \dots, N-1, \\ & x_i \in \mathcal{X}, \quad u_i \in \mathcal{U}, \quad i = 0, \dots, N-1, \\ & x_N \in \mathcal{X}_f, \\ & x_0 = x(k), \end{aligned}$$

where

$$U := \{u_0, \dots, u_{N-1}\}.$$

The finite horizon makes the optimization problem computable. The terminal cost $V_f(x_N)$ and terminal set $\mathcal{X}_f$ are introduced to approximate the part of the infinite-horizon problem that has been truncated. The main message of this chapter is that these terminal ingredients are not merely optional tuning parameters: they are the standard mechanism through which one recovers feasibility and stability guarantees for finite-horizon MPC.

7.2 MPC: Key Points Illustrated

Before proving recursive feasibility and stability formally, it is useful to look at what can happen in a constrained control problem when different controllers and different MPC formulations are used. We illustrate these issues through the example of a Cessna Citation aircraft. The model is a linearized continuous-time model around an operating point at altitude 5000 m and speed 128.2 m/s. The input is the elevator angle, while the state variables are

$$x_1 = \text{angle of attack}, \quad x_2 = \text{pitch angle}, \quad x_3 = \text{pitch rate}, \quad x_4 = \text{altitude}.$$

The measured outputs are pitch angle and altitude. The linearized model is

$$\dot{x} = \begin{bmatrix} -1.2822 & 0 & 0.98 & 0 \\ 0 & 0 & 1 & 0 \\ -5.4293 & 0 & -1.8366 & 0 \\ -128.2 & 128.2 & 0 & 0 \end{bmatrix} x + \begin{bmatrix} -0.3 \\ 0 \\ -17 \\ 0 \end{bmatrix} u, \quad y = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} x.$$

The open-loop response is unstable, with open-loop poles

$$0, \quad 0, \quad -1.5594 \pm 2.29i.$$

Therefore, some form of feedback stabilization is necessary. At the same time, the aircraft is subject to physical and operational constraints: the elevator angle is bounded, the elevator rate is bounded, and the pitch angle must remain within an admissible range. These constraints are not secondary details; they determine whether the closed-loop behavior is acceptable. A first comparison is between unconstrained LQR and constrained linear MPC with quadratic cost. In the LQR case, one solves the infinite-horizon quadratic problem

$$J_\infty^*(x(k)) = \min \sum_{i=0}^{\infty} (x_i^\top Q x_i + u_i^\top R u_i)$$

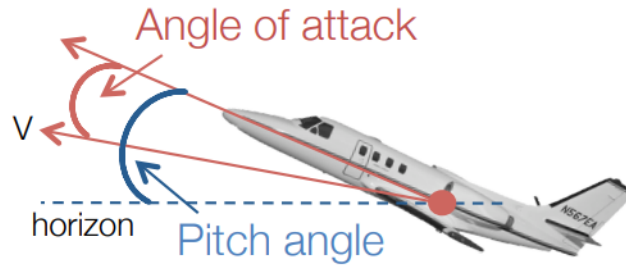


Figure 7.1: Linearized Cessna Citation aircraft model used to illustrate the effect of constraints in MPC. The input is the elevator angle, the states are angle of attack, pitch angle, pitch rate, and altitude, and the outputs are pitch angle and altitude.

subject only to the dynamics

$$x_{i+1} = Ax_i + Bu_i, \quad x_0 = x(k).$$

By contrast, in the finite-horizon MPC formulation one solves

$$J^*(x(k)) = \min_U \sum_{i=0}^{N-1} (x_i^\top Q x_i + u_i^\top R u_i)$$

subject to the same prediction dynamics, but also to the constraints

$$x_i \in \mathcal{X}, \quad u_i \in \mathcal{U}, \quad i = 0, \dots, N-1,$$

with

$$x_0 = x(k).$$

In both cases the cost is quadratic, the dynamics are linear, and the weights satisfy

$$Q = Q^\top \succeq 0, \quad R = R^\top \succ 0.$$

The crucial difference is that LQR does not take the constraints into account during the design, whereas MPC includes them directly in the optimization problem.

This distinction is important because a controller designed without constraints may behave very poorly once constraints are imposed afterwards. For example, one might design an unconstrained LQR controller and then simply saturate the elevator input whenever the actuator limit is exceeded. This may seem reasonable, but it changes the closed-loop system in a way that was not accounted for in the LQR design. In the aircraft example, starting from an altitude deviation of 10 m,

$$x_0 = \begin{bmatrix} 0 & 0 & 0 & 10 \end{bmatrix}^\top,$$

with sampling time 0.25 s, $Q = I$, and $R = 10$, the LQR law with saturated inputs leads to an unstable closed-loop response. Figure 7.2 shows that the altitude and pitch angle do not converge, while the elevator angle repeatedly hits the imposed saturation bounds.

MPC improves on this situation by including actuator bounds directly in the prediction problem. For instance, suppose that the MPC controller enforces the elevator angle constraint

$$|u_k| \leq 0.262,$$

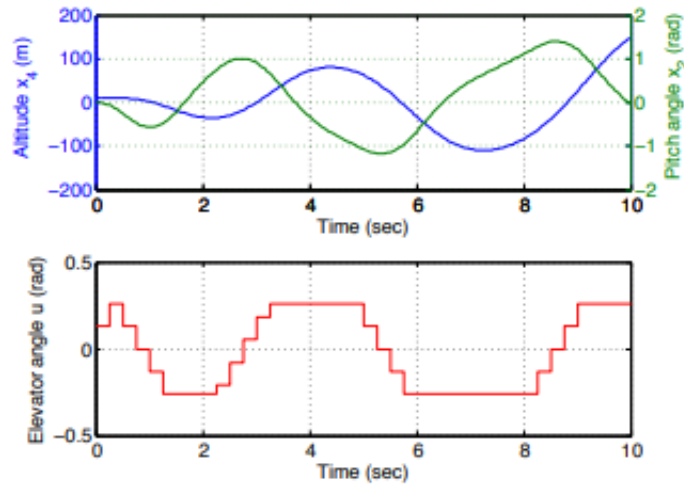


Figure 7.2: LQR with saturated elevator angle. Although the unconstrained LQR law is stabilizing for the nominal unconstrained model, saturating the input afterwards can make the closed-loop system unstable.

corresponding approximately to $\pm 15^\circ$, with the same weights $Q = I$, $R = 10$, and horizon $N = 10$. The controller now knows during the optimization that the elevator cannot exceed its admissible range. However, this is still not enough if the actuator rate constraint is ignored. The prediction model assumes that the input can jump freely within the admissible interval, whereas the real actuator cannot. As shown in Figure 7.3, the resulting closed-loop trajectory does not converge to the desired steady state, but instead approaches a limit cycle.

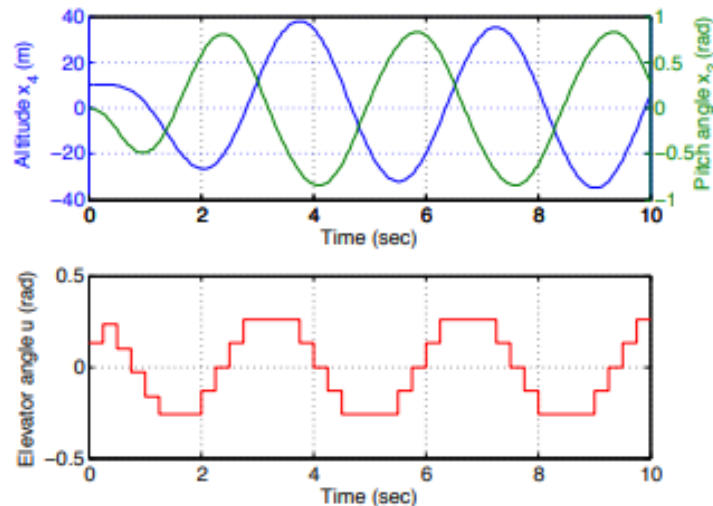


Figure 7.3: MPC with only the elevator angle bound $|u_k| \leq 0.262$. The controller accounts for saturation, but not for the elevator rate constraint. As a consequence, the closed-loop trajectory may fail to converge and can enter a limit cycle.

The next step is to include all relevant actuator constraints. In addition to the magnitude bound on the elevator angle, one imposes a rate constraint. In discrete time, an elevator rate bound can be approximated by

$$|u_i - u_{i-1}| \leq 0.524 T_s,$$

where T_s is the sampling time. With the same horizon $N = 10$, the MPC controller now optimizes

over trajectories that are consistent with both the elevator magnitude and the elevator rate limits. The behavior in Figure 7.4 is then stable: the altitude and pitch angle converge, and the input uses the available control authority efficiently without violating the actuator restrictions.

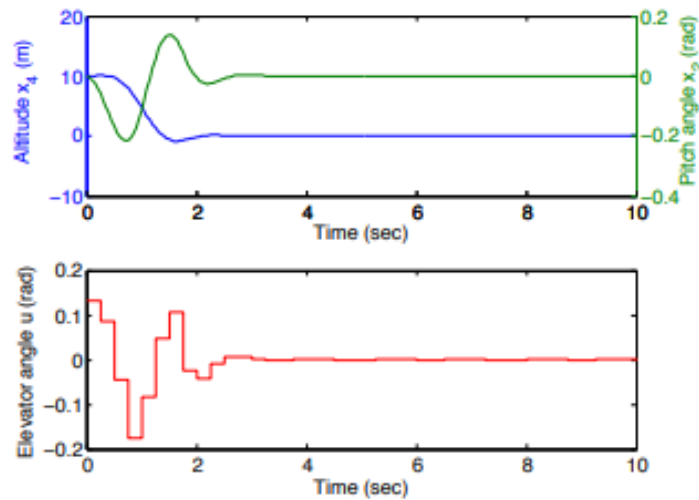


Figure 7.4: MPC with both elevator magnitude and elevator rate constraints. Once all relevant actuator constraints are included in the prediction problem, the closed-loop response becomes stable in this example.

State constraints can be equally important. Consider a larger altitude deviation, for instance

$$x_0 = \begin{bmatrix} 0 & 0 & 0 & 100 \end{bmatrix}^\top.$$

If the MPC controller includes the input magnitude and rate constraints but does not explicitly constrain the pitch angle, it may choose a trajectory that regulates altitude by using a very large pitch transient. In the example shown in Figure 7.5, the pitch angle reaches approximately -0.9 rad, corresponding to about -50° . Such a maneuver may be optimal for the mathematical problem that was solved, but it is not acceptable from the physical or comfort point of view.

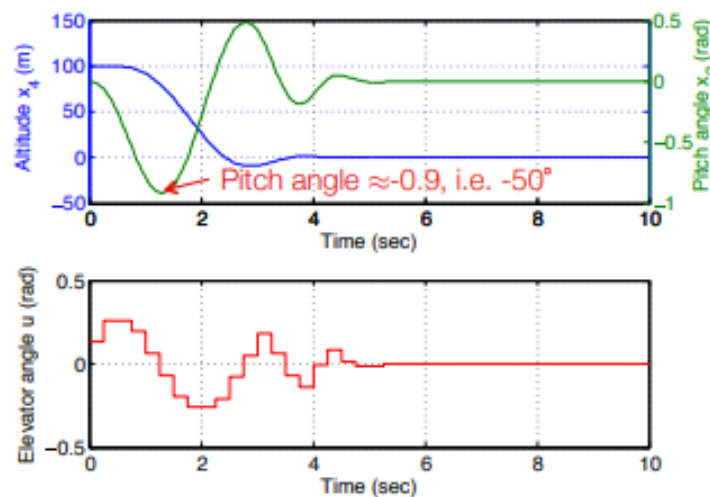


Figure 7.5: MPC with input constraints but without a pitch-angle state constraint. The controller regulates altitude, but it produces an excessive pitch transient.

If one adds the passenger-comfort constraint

$$|x_2| \leq 0.349,$$

then the controller must choose a different trajectory. The pitch angle constraint becomes active, as shown in Figure 7.6. The altitude response is less aggressive, but the closed-loop trajectory respects the admissible pitch range. This example highlights one of the main advantages of MPC: constraints can be encoded directly in the controller design, and the optimizer automatically trades performance against constraint satisfaction.

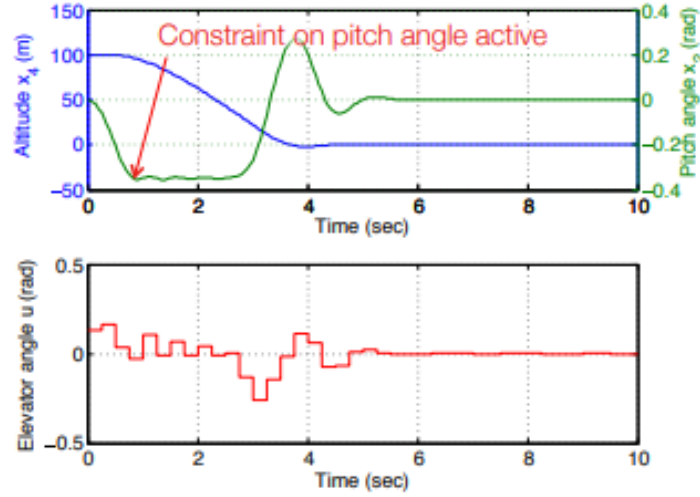


Figure 7.6: MPC with input constraints and the pitch-angle constraint $|x_2| \leq 0.349$. The pitch constraint becomes active during the transient, and the controller modifies the altitude response in order to remain feasible.

The final point illustrated by the aircraft example is more subtle. Even if the correct physical constraints are included, the prediction horizon may still be too short. With the same input constraints and rate constraints, but with horizon reduced to

$$N = 4,$$

the closed-loop response may again lose stability. Figure 7.7 shows that decreasing the prediction horizon can destroy the stabilizing behavior observed for the longer horizon. The controller only optimizes over a short future window, and therefore it may choose inputs that look beneficial over that window but do not lead to a good long-term closed-loop evolution.

The examples therefore show both the strength and the limitation of finite-horizon MPC. The strength is that constraints can be incorporated directly into the control problem, instead of being handled afterwards by saturation or ad hoc logic. This allows the controller to anticipate constraint activation and to use the available control authority systematically. The limitation is that constraint satisfaction over a finite prediction window does not automatically imply constraint satisfaction and stability for all future times. A finite-horizon controller remains short-sighted unless its terminal behavior is designed carefully.

7.3 Loss of Feasibility and Stability in Finite-Horizon MPC

The examples of the previous section suggest that MPC is a powerful framework for constrained control, but they also reveal an important limitation. A standard finite-horizon MPC formulation does not automatically guarantee that the optimization problem will remain feasible at all future time instants, nor does it automatically guarantee that the closed-loop trajectory will converge to the origin.

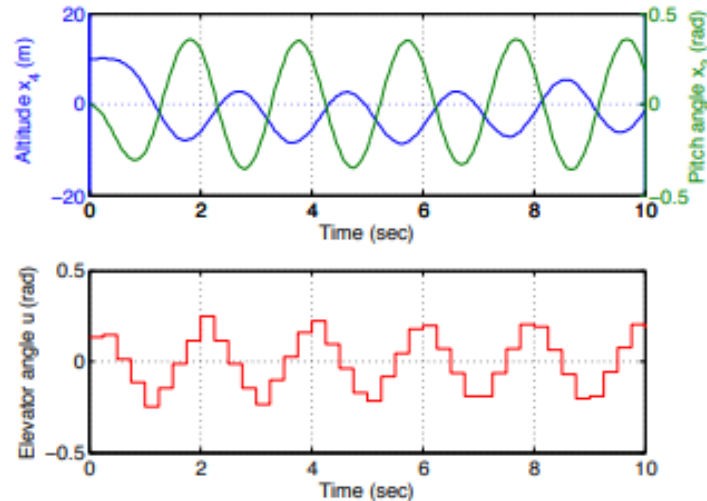


Figure 7.7: MPC with all considered input constraints but short horizon $N = 4$. Reducing the prediction horizon can cause loss of stability, even though the constraints are included in the optimization problem.

In other words, two things can go wrong. First, the MPC problem may become infeasible after some time, even if it was feasible at the initial condition. Second, the optimization problem may remain feasible, but the resulting closed-loop trajectory may fail to converge. These two issues are called loss of feasibility and loss of stability, respectively.

The reason is that finite-horizon MPC is intrinsically a short-sighted strategy. At time k , the controller optimizes an open-loop trajectory over only N future steps. Then, however, only the first input is applied, and the problem is solved again at the next time instant. Thus the actual closed-loop trajectory is not the same object as the open-loop prediction computed at one time instant. A predicted trajectory may be feasible over the finite horizon, but the first input of that trajectory may still drive the system to a state from which the next finite-horizon problem has no solution.

7.3.1 Loss of feasibility: double integrator example

To see how loss of feasibility can occur, consider the constrained double integrator

$$x(k+1) = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} x(k) + \begin{bmatrix} 0 \\ 1 \end{bmatrix} u(k), \quad y(k) = \begin{bmatrix} 1 & 0 \end{bmatrix} x(k).$$

The input is constrained by

$$-0.5 \leq u(k) \leq 0.5,$$

and the state is constrained by the box

$$\begin{bmatrix} -5 \\ -5 \end{bmatrix} \leq x(k) \leq \begin{bmatrix} 5 \\ 5 \end{bmatrix}.$$

We compute a receding-horizon controller with quadratic objective, horizon

$$N = 3,$$

terminal weight and state weight

$$P = Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix},$$

and input weight

$$R = 10.$$

For each measured state $x(k)$, the controller solves the corresponding constrained finite-time optimal control problem, obtains an optimal input sequence

$$U^*(x(k)) = \{u_0^*(x(k)), u_1^*(x(k)), u_2^*(x(k))\},$$

applies only the first input $u_0^*(x(k))$, and repeats the procedure at the next sampling instant. The finite-horizon problem can be written as a quadratic program of the form

$$J^*(x(k)) = \min_U U^\top H U + 2x(k)^\top F U + x(k)^\top Y x(k)$$

subject to

$$G U \leq w + E x(k).$$

The matrices H , F , Y , G , E , and w are obtained by substituting the double integrator dynamics into the cost and into the constraints. The exact entries are not conceptually important here. What matters is that, for each value of the parameter $x(k)$, the QP is either feasible or infeasible. Therefore, the feasible set of the finite-horizon MPC problem is a subset of the state constraint set.

Now consider the receding-horizon implementation. The algorithm is simple: measure the state, solve the CFTOC problem, check whether a feasible optimizer exists, apply the first optimal input, and repeat. Starting from

$$x(0) = \begin{bmatrix} -4 \\ 3 \end{bmatrix},$$

the finite-horizon problem is feasible and gives

$$u_0^*(x(0)) = -0.5.$$

After applying this input, the system evolves to

$$x(1) = \begin{bmatrix} -1 \\ 2.5 \end{bmatrix},$$

and the MPC problem is again feasible, with

$$u_0^*(x(1)) = -0.5.$$

After applying this second input, the system reaches

$$x(2) = \begin{bmatrix} 1.5 \\ 2 \end{bmatrix}.$$

At this point, however, the finite-horizon problem is infeasible. Thus the controller has driven the system from a feasible initial state to a state at which the next optimization problem has no solution.

This example is important because no disturbance or model mismatch is involved. The system evolves exactly according to the model used inside the optimizer. The loss of feasibility is therefore not caused by uncertainty, but by the finite prediction horizon itself. The controller can see only three steps ahead, and the first input of a three-step feasible plan does not necessarily lead to a state from which another three-step feasible plan exists.

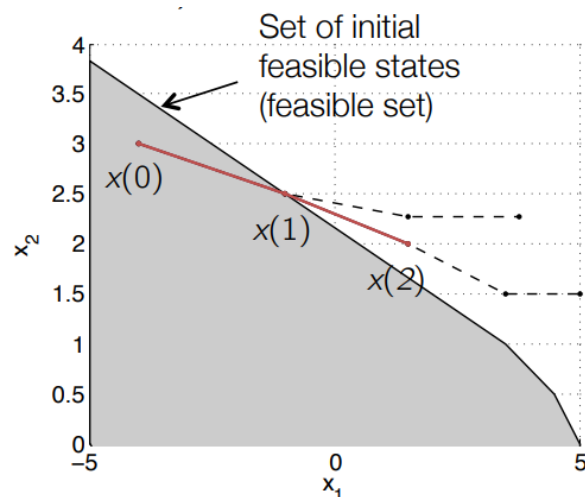


Figure 7.8: Closed-loop evolution of the double integrator under finite-horizon MPC. Although $x(0)$ and $x(1)$ are feasible, the state $x(2)$ lies outside the finite-horizon feasible set, so the next MPC problem is infeasible.

The same phenomenon can be visualized by sampling many initial states. Some initial states lead to closed-loop trajectories for which the MPC problem remains feasible at every time instant; others lead to trajectories that eventually become infeasible. In Figure 7.9, the boxes represent initial conditions that lead to feasible closed-loop trajectories, while the circles represent initial conditions that do not. The figure shows that the set of states from which the finite-horizon problem is feasible is not automatically the same as the set of states from which the receding-horizon closed loop remains feasible forever.

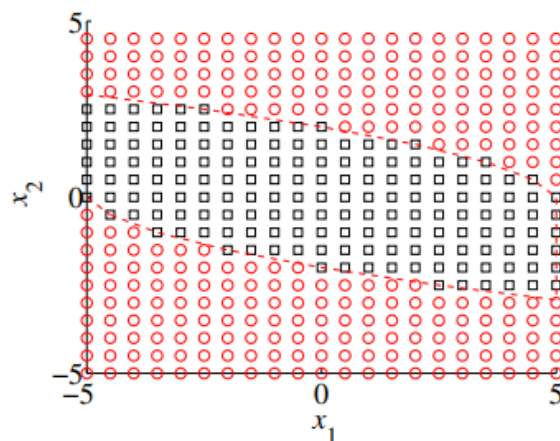


Figure 7.9: Sampled initial conditions for the double integrator. Boxes denote states leading to feasible closed-loop trajectories, while circles denote states for which the MPC problem becomes infeasible after some time.

Recursive feasibility

A finite-horizon MPC controller is recursively feasible on a set Ω if feasibility propagates forward in closed loop. More precisely, whenever the MPC problem is feasible at $x(k) \in \Omega$, the successor state

$$x(k+1) = Ax(k) + Bu_0^*(x(k))$$

must also belong to the feasible set of the MPC problem. The double integrator example

shows that standard finite-horizon MPC does not automatically have this property.

7.3.2 Feasibility and stability depend on the tuning

Loss of feasibility is not the only possible failure. Even when the finite-horizon optimization problem remains feasible, the resulting trajectories may not converge to the origin. The stability properties of finite-horizon MPC can depend strongly on the horizon length and on the cost weights.

Consider the unstable system

$$x(k+1) = \begin{bmatrix} 2 & 1 \\ 0 & 0.5 \end{bmatrix} x(k) + \begin{bmatrix} 1 \\ 0 \end{bmatrix} u(k),$$

with input constraint

$$-1 \leq u(k) \leq 1,$$

state constraint

$$\begin{bmatrix} -10 \\ -10 \end{bmatrix} \leq x(k) \leq \begin{bmatrix} 10 \\ 10 \end{bmatrix},$$

and state weight

$$Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

The purpose of this example is to investigate the stability properties of the finite-horizon receding-horizon controller for different horizons N and different input weights R .

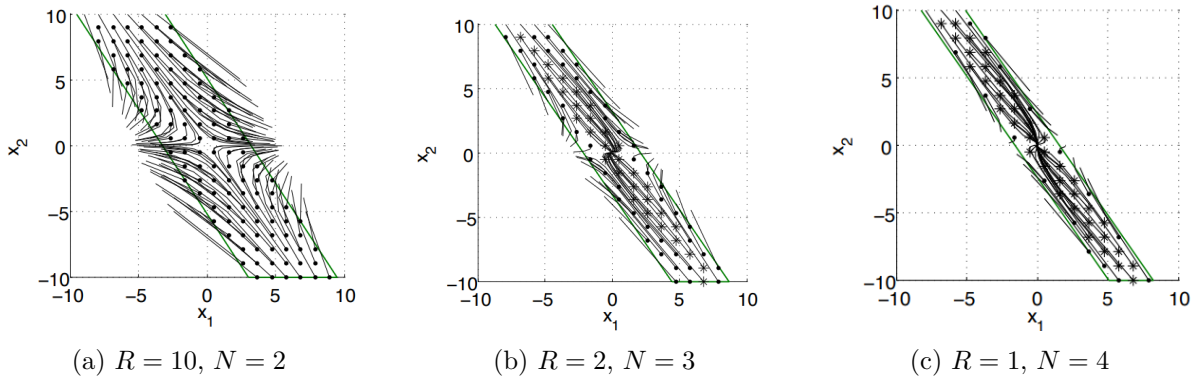


Figure 7.10: Effect of horizon length and input weight on finite-horizon MPC. Stars denote initial points with convergent trajectories, while circles denote initial points that diverge. The green lines denote the feasible set of initial states.

The results are summarized in Figure 7.10. For $R = 10, N = 2$, all sampled trajectories are unstable. In this case the horizon is very short and the input penalty is large, so the controller is too short-sighted and too conservative to stabilize the unstable mode. For $R = 2, N = 3$, some trajectories converge, while others still diverge. Finally, for $R = 1, N = 4$, a larger portion of the sampled initial conditions leads to convergent trajectories.

The green boundary in the plots denotes the set of feasible initial points. This feasible set depends on the horizon N , because a longer horizon changes the set of states from which it is possible to find an admissible predicted trajectory. However, the feasible set does not depend on the input weight R , since feasibility is determined by the dynamics and constraints, not by the objective function. The actual closed-loop stability behavior, on the other hand, depends

on both N and R . Thus the cost affects which feasible trajectory is selected, and therefore it affects the closed-loop trajectory.

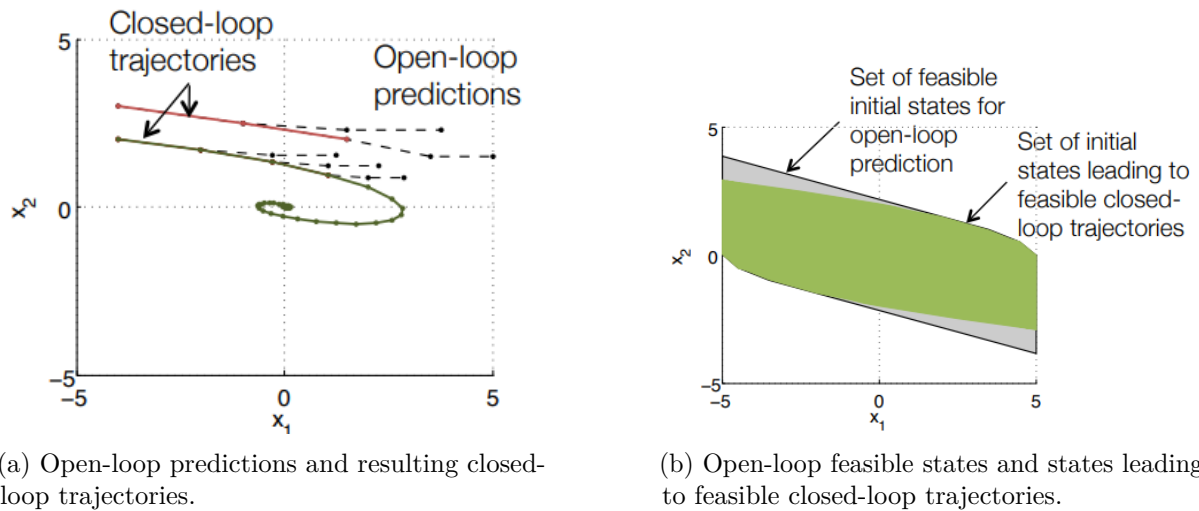


Figure 7.11: Mismatch between finite-horizon open-loop prediction and receding-horizon closed-loop behavior. The left plot shows that the predicted open-loop trajectories need not coincide with the closed-loop trajectories generated by repeatedly resolving the MPC problem. The right plot shows that the set of initial states admitting a feasible finite-horizon open-loop prediction can differ from the set of initial states that lead to feasible closed-loop trajectories.

This example highlights an important point. Feasibility and stability are related, but they are not equivalent. A state may be feasible for the finite-horizon optimization problem, but the resulting closed-loop trajectory may still fail to converge. Conversely, changing the cost weights can change the closed-loop behavior even though it does not change the feasible set. The tuning parameters of finite-horizon MPC therefore have a complex effect on the resulting trajectories.

7.3.3 Why finite-horizon MPC is short-sighted

The previous examples can be summarized geometrically. At time k , the controller computes an open-loop prediction over a finite horizon. This prediction may satisfy all constraints over the predicted window. However, after the first input is applied and the problem is solved again, the closed-loop trajectory may deviate from what would have been expected by considering only one finite open-loop plan.

This is the central mismatch: the set of initial states from which there exists a feasible open-loop prediction over N steps is generally larger than, or at least different from, the set of initial states from which the receding-horizon closed loop remains feasible for all times. The finite-horizon problem certifies only that a feasible path exists over the next N steps. It does not automatically certify that feasibility can be maintained after those N steps.

The ideal solution would be to solve the constrained infinite-horizon problem. If the horizon were infinite, the open-loop plan would already contain the entire future. After applying the first input, the remaining tail of the infinite-horizon plan would still be a feasible candidate at the next time step. Therefore, if the infinite-horizon problem is feasible initially, the closed-loop trajectory remains feasible. Moreover, if the infinite-horizon cost is finite and the stage cost is positive definite, then the state and input must converge asymptotically to the origin.

For a finite horizon, this argument no longer holds automatically. The controller approximates

the infinite-horizon strategy, but because it sees only a finite window, it may make decisions that are locally reasonable and globally poor. This is why standard finite-horizon receding horizon control may lose feasibility or stability.

Finite versus infinite horizon

With an infinite horizon, the tail of the optimal trajectory provides a natural feasible candidate after the first input has been applied. This gives feasibility and stability properties. With a finite horizon, this tail is missing. The controller is short-sighted, and recursive feasibility and stability must be enforced by design.

7.3.4 The role of terminal cost and terminal constraints

The standard solution is to modify the finite-horizon problem so that it mimics the infinite-horizon problem beyond the end of the prediction window. This is done by adding a terminal cost and a terminal constraint:

$$\begin{aligned} J^*(x(k)) = \min_U \quad & V_f(x_N) + \sum_{i=0}^{N-1} \ell(x_i, u_i) \\ \text{subj. to} \quad & x_{i+1} = Ax_i + Bu_i, \quad i = 0, \dots, N-1, \\ & x_i \in \mathcal{X}, \quad u_i \in \mathcal{U}, \quad i = 0, \dots, N-1, \\ & x_N \in \mathcal{X}_f, \\ & x_0 = x(k). \end{aligned}$$

The terminal cost $V_f(x_N)$ approximates the cost of controlling the system after the end of the finite horizon. The terminal set $\mathcal{X}_f$ approximates the set of terminal states from which the constraints can be satisfied indefinitely by a suitable local controller. In this sense, the terminal ingredients are a way of “faking” the infinite horizon while keeping the optimization problem finite-dimensional and computationally tractable.

The rest of the chapter makes this idea precise. We will show that, if the terminal set is chosen to be invariant under a suitable local control law and if the terminal cost decreases along that local closed loop, then the finite-horizon MPC controller is recursively feasible and stabilizing. The proof relies on a simple but powerful idea: shift the previously optimal input sequence by one step and append the terminal control law at the end. This shifted sequence provides recursive feasibility, while the decrease of the optimal cost provides a Lyapunov function.

7.4 Feasibility and Stability Guarantees in MPC

After seeing that finite-horizon MPC may lose feasibility or stability, we now turn to the standard mechanism used to recover guarantees. The key tool is Lyapunov stability theory. The reason is that, in MPC, the optimal value function naturally behaves like an energy function: if it can be shown to decrease along the closed-loop trajectory, then it can be used to prove asymptotic stability.

7.4.1 Lyapunov Stability Revisited

Consider a nonlinear, time-invariant, discrete-time autonomous system

$$x(k+1) = g(x(k)).$$

Let $\bar{x}$ be an equilibrium point, meaning that

$$g(\bar{x}) = \bar{x}.$$

We are interested in understanding whether trajectories starting near $\bar{x}$, or more generally inside a given set Ω , remain well behaved and converge to $\bar{x}$.

Asymptotic stability

An equilibrium point $\bar{x} \in \Omega$ of the system

$$x(k+1) = g(x(k))$$

is called *asymptotically stable in the positive invariant set* $\Omega \subseteq \mathbb{R}^n$ if it is Lyapunov stable and attractive in Ω , that is,

$$\lim_{k \rightarrow \infty} \|x(k) - \bar{x}\| = 0, \quad \forall x(0) \in \Omega.$$

If the same property holds with $\Omega = \mathbb{R}^n$, then the equilibrium is called *globally asymptotically stable*.

The set Ω is important because, in constrained MPC, stability is usually not global in $\mathbb{R}^n$. The state cannot be arbitrary: it must remain inside the constraint set and inside the set of states for which the MPC problem is feasible. Therefore, the natural stability statement for constrained MPC is stability on a positively invariant feasible set, rather than global stability on the whole state space.

A standard way to prove asymptotic stability is to construct a Lyapunov function. The interpretation is the same as in the earlier system-theory chapter: the Lyapunov function is a generalized energy. It is positive away from the equilibrium and decreases along trajectories. For simplicity, and in line with the MPC regulation problem, we state the definition for the equilibrium at the origin.

Lyapunov function

Consider the equilibrium point $\bar{x} = 0$ of

$$x(k+1) = g(x(k)).$$

Let $\Omega \subset \mathbb{R}^n$ be a closed and bounded positive invariant set containing the origin. A function

$$V : \mathbb{R}^n \rightarrow \mathbb{R}$$

is called a Lyapunov function in Ω if it is continuous at the origin, finite for every $x \in \Omega$, and satisfies

$$V(0) = 0, \quad V(x) > 0 \quad \forall x \in \Omega \setminus \{0\},$$

together with the decrease condition

$$V(g(x)) - V(x) \leq -\alpha(x), \quad \forall x \in \Omega \setminus \{0\},$$

where

$$\alpha : \mathbb{R}^n \rightarrow \mathbb{R}$$

is continuous and positive definite.

The first condition says that the function has a strict minimum at the equilibrium. The second condition says that the value of V decreases every time the system evolves, except at the origin. Thus V plays the role of a certificate that trajectories move progressively closer to the equilibrium.

Lyapunov stability theorem

If the system

$$x(k+1) = g(x(k))$$

admits a Lyapunov function $V(x)$ on a positive invariant set Ω , then the equilibrium $x = 0$ is asymptotically stable in Ω .

We now connect this general result to MPC. The closed-loop system generated by MPC can be written as

$$x(k+1) = Ax(k) + Bu_0^*(x(k)),$$

where $u_0^*(x(k))$ is the first element of the optimal input sequence computed at time k . The goal is to prove that this closed-loop system is asymptotically stable on the feasible set of the MPC problem.

The standard proof has two parts. First, one proves recursive feasibility: starting from a feasible initial point, one must show that a feasible control sequence exists again at every future time instant. Second, one proves stability by showing that the optimal cost function decreases along the closed-loop trajectory. In other words, the optimal MPC value function

$$J^*(x(k))$$

will be used as the Lyapunov function.

For the finite-horizon problem, we use the general notation

$$J^*(x(k)) = \min_U V_f(x_N) + \sum_{i=0}^{N-1} \ell(x_i, u_i),$$

where $V_f(x_N)$ is the terminal cost and $\ell(x_i, u_i)$ is the stage cost. The proof will be developed for two increasingly general terminal constraints. The first case is the zero terminal constraint,

$$x_N = 0.$$

The second case is a terminal constraint in a set,

$$x_N \in \mathcal{X}_f,$$

where $\mathcal{X}_f$ is chosen to be invariant under a suitable local terminal controller.

MPC feasibility and stability proof strategy

The stability proof for MPC follows two main steps.

First, prove *recursive feasibility*: if the MPC problem is feasible at time k , construct a feasible candidate sequence for the problem at time $k+1$.

Second, prove *Lyapunov decrease*: show that the optimal cost satisfies

$$J^*(x(k+1)) - J^*(x(k)) \leq -\ell(x(k), u_0^*(x(k))).$$

If the stage cost is positive definite, then J^* decreases strictly away from the origin and can be used as a Lyapunov function.

The essential idea behind both parts is the same. At time k , the MPC problem produces an optimal predicted sequence. After applying the first input, the system moves to the first predicted successor state. At time $k+1$, one constructs a candidate sequence by shifting the old optimal sequence one step forward and appending an appropriate terminal input. If the terminal constraint and terminal cost are designed correctly, this shifted sequence is feasible and gives the desired decrease of the value function.

7.4.2 Zero Terminal State Constraint

We first consider the simplest possible terminal constraint, namely

$$x_N \in \mathcal{X}_f = \{0\}.$$

Thus the finite-horizon problem is feasible at a state $x(k)$ only if there exists an admissible input sequence that drives the predicted state exactly to the origin at the end of the horizon. This is a restrictive choice, but it makes the mechanism behind recursive feasibility and stability particularly clear.

Assume that the MPC problem is feasible at $x(k)$. Let

$$\{u_0^*, u_1^*, \dots, u_{N-1}^*\}$$

be the optimal control sequence computed at $x(k)$, and let

$$\{x(k), x_1^*, \dots, x_N^*\}$$

be the corresponding predicted state trajectory. Since the terminal constraint is $x_N = 0$, we have

$$x_N^* = 0.$$

The receding-horizon controller applies only the first input of the optimal sequence,

$$u(k) = u_0^*,$$

and the real system evolves to

$$x(k+1) = Ax(k) + Bu(k) = Ax(k) + Bu_0^* = x_1^*.$$

Therefore, after one sampling step, the measured state coincides with the first predicted successor state.

To prove recursive feasibility, we now construct an admissible candidate sequence for the optimization problem at the next state $x(k+1) = x_1^*$. The natural candidate is obtained by shifting the previously optimal sequence by one step and appending the zero input:

$$\tilde{U} = \{u_1^*, u_2^*, \dots, u_{N-1}^*, 0\}.$$

The first $N-1$ elements of this sequence simply follow the tail of the old predicted trajectory. After these inputs, the predicted state reaches $x_N^* = 0$. The appended input is then equal to zero, and because the origin is an equilibrium,

$$Ax_N^* + B \cdot 0 = A \cdot 0 + B \cdot 0 = 0.$$

Thus the shifted sequence again satisfies the terminal constraint. Since the original predicted trajectory satisfied all state and input constraints, and since the last step remains at the origin, the sequence $\tilde{U}$ is feasible for the problem initialized at $x(k+1)$. This proves recursive feasibility.

Recursive feasibility for $x_N = 0$

If the MPC problem is feasible at $x(k)$ and the terminal constraint is $x_N = 0$, then the problem is feasible again at

$$x(k+1) = Ax(k) + Bu_0^*.$$

A feasible candidate at time $k+1$ is obtained by shifting the old optimal sequence and appending the input 0.

We now use the same shifted sequence to prove stability. The goal is to show that the optimal cost decreases along the closed-loop trajectory:

$$J^*(x(k+1)) < J^*(x(k)), \quad \forall x(k) \neq 0.$$

Since the terminal state is constrained to be zero, the optimal cost at time k can be written as

$$J^*(x(k)) = V_f(x_N^*) + \sum_{i=0}^{N-1} \ell(x_i^*, u_i^*) = \sum_{i=0}^{N-1} \ell(x_i^*, u_i^*),$$

because

$$x_N^* = 0, \quad V_f(0) = 0.$$

At time $k+1$, the shifted sequence

$$\tilde{U} = \{u_1^*, u_2^*, \dots, u_{N-1}^*, 0\}$$

is feasible, but not necessarily optimal. Therefore, the optimal cost at $x(k+1)$ is no larger than the cost generated by this feasible candidate:

$$\begin{aligned} J^*(x(k+1)) &\leq \tilde{J}(x(k+1)) \\ &= \sum_{i=1}^{N-1} \ell(x_i^*, u_i^*) + \ell(x_N^*, 0). \end{aligned}$$

Using $x_N^* = 0$ and $\ell(0, 0) = 0$, this becomes

$$\begin{aligned} J^*(x(k+1)) &\leq \sum_{i=1}^{N-1} \ell(x_i^*, u_i^*) + \ell(0, 0) \\ &= \sum_{i=0}^{N-1} \ell(x_i^*, u_i^*) - \ell(x_0^*, u_0^*) \\ &= J^*(x(k)) - \ell(x(k), u_0^*). \end{aligned}$$

Hence

$$J^*(x(k+1)) - J^*(x(k)) \leq -\ell(x(k), u_0^*).$$

If the stage cost is positive definite, then

$$\ell(x(k), u_0^*) > 0 \quad \text{for } x(k) \neq 0,$$

and therefore

$$J^*(x(k+1)) < J^*(x(k)), \quad \forall x(k) \neq 0.$$

Thus J^* decreases strictly along every nontrivial closed-loop trajectory. The optimal cost is therefore a Lyapunov function for the MPC closed loop, and the origin is asymptotically stable on the feasible set.

Stability for $x_N = 0$

With the terminal constraint $x_N = 0$, the shifted-sequence argument gives

$$J^*(x(k+1)) - J^*(x(k)) \leq -\ell(x(k), u_0^*).$$

If the stage cost is positive definite, then J^* is a Lyapunov function for the MPC closed-loop system. Hence the origin is asymptotically stable on the feasible set.

The advantage of the zero terminal constraint is that it gives a very simple proof. The drawback is that it can be conservative. Requiring the state to reach the origin exactly at time N may exclude many initial states that could in fact be stabilized if a longer horizon, or a less restrictive terminal set, were used. This effect is especially visible for short horizons.

To illustrate this, consider the unstable system

$$x(k+1) = \begin{bmatrix} 1.2 & 1 \\ 0 & 1 \end{bmatrix} x(k) + \begin{bmatrix} 1 \\ 0.5 \end{bmatrix} u(k),$$

with state constraint

$$\mathcal{X} := \{x \mid -50 \leq x_1 \leq 50, -10 \leq x_2 \leq 10\} = \{x \mid A_x x \leq b_x\},$$

input constraint

$$\mathcal{U} := \{u \mid \|u\|_\infty \leq 1\} = \{u \mid A_u u \leq b_u\},$$

and stage cost

$$\ell(x_i, u_i) = x_i^\top \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} x_i + u_i^\top u_i.$$

For the zero terminal constraint, the feasible set depends strongly on the horizon length. A longer horizon gives the controller more time to steer the state to the origin while respecting the constraints, and therefore increases the region of attraction.

Figure 7.12 shows the feasible regions obtained for different horizon lengths. For $N = 5$, only a relatively small set of initial conditions can be driven to the origin within the horizon while satisfying the constraints. Increasing the horizon to $N = 10$ and $N = 20$ enlarges the region. In the limit of large horizons, the feasible region approaches the maximum control-invariant set associated with the constraints.

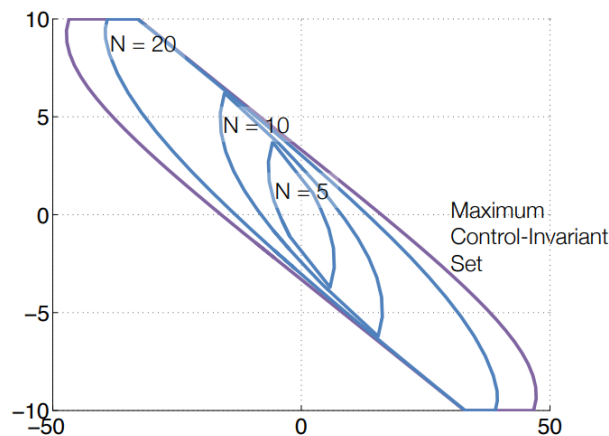


Figure 7.12: Impact of the horizon length with the zero terminal constraint. Increasing N enlarges the feasible region and therefore the certified region of attraction. The figure also shows the maximum control-invariant set, which represents the largest set from which constraint satisfaction can in principle be maintained indefinitely.

This example motivates the next step. Instead of forcing the terminal state to be exactly equal to zero, we can require it to lie in a terminal set $\mathcal{X}_f$. If this terminal set is invariant under a suitable local controller, then recursive feasibility and stability can still be proven, while the region of attraction can be significantly larger.

7.4.3 General Terminal Sets

The zero terminal constraint $x_N = 0$ gives a clean and intuitive stability proof, but it can be very restrictive. Requiring the predicted state to reach the origin exactly at the end of the horizon may considerably reduce the feasible set, especially when the horizon is short. The natural generalization is therefore to replace the terminal equality constraint by a terminal set constraint

$$x_N \in \mathcal{X}_f.$$

The goal is to choose $\mathcal{X}_f$ so that it enlarges the region of attraction, while still allowing the same recursive-feasibility and Lyapunov-stability argument to go through.

The idea is the following. Instead of asking the MPC optimization to drive the state all the way to the origin in N steps, we only ask it to drive the state into a region $\mathcal{X}_f$ where a known local controller can safely take over. This terminal set should be chosen so that, once the state enters $\mathcal{X}_f$, there exists an admissible control law that keeps the state inside $\mathcal{X}_f$ and continues decreasing an appropriate terminal cost. Thus, the terminal set represents a region in which the infinite-horizon tail of the problem is already under control.

For example, in the double integrator case, replacing $x_N = 0$ by $x_N \in \mathcal{X}_f$ can enlarge the feasible set significantly. The optimization problem no longer needs to reach the origin exactly within the horizon; it only needs to reach a set from which constraint satisfaction and stability can be guaranteed afterwards. This is precisely why terminal sets are useful in MPC: they make the finite-horizon problem less conservative while preserving the theoretical guarantees.

Invariant terminal sets

The required property of the terminal set is positive invariance under a local terminal control law. Recall that a set $\mathcal{O}$ is positively invariant for a closed-loop system

$$x(k+1) = g_{\text{cl}}(x(k))$$

if

$$x(0) \in \mathcal{O} \implies x(k) \in \mathcal{O} \quad \forall k \in \mathbb{N}_+.$$

The largest closed positively invariant set, when it exists, is called the maximal positively invariant set and is denoted by $\mathcal{O}_\infty$.

In the MPC terminal-set construction, the closed-loop system inside the terminal set is generated by a local feedback law

$$\kappa_f(x).$$

Thus, for the linear system

$$x^+ = Ax + Bu,$$

we require

$$Ax + B\kappa_f(x) \in \mathcal{X}_f, \quad \forall x \in \mathcal{X}_f.$$

In addition, the terminal controller must satisfy the original constraints in the terminal set:

$$\mathcal{X}_f \subseteq \mathcal{X}, \quad \kappa_f(x) \in \mathcal{U}, \quad \forall x \in \mathcal{X}_f.$$

These conditions mean that, once the predicted state reaches $\mathcal{X}_f$, the terminal controller can keep the state feasible forever. This is the key replacement for the simple condition $x_N = 0$ used in the previous proof.

Invariant terminal set

A terminal set $\mathcal{X}_f$ is suitable for the MPC stability proof if there exists a local controller κ_f such that

$$Ax + B\kappa_f(x) \in \mathcal{X}_f, \quad \forall x \in \mathcal{X}_f,$$

and

$$\mathcal{X}_f \subseteq \mathcal{X}, \quad \kappa_f(x) \in \mathcal{U}, \quad \forall x \in \mathcal{X}_f.$$

Thus, inside $\mathcal{X}_f$, the local controller preserves both invariance and constraint satisfaction.

Main stability result

We now state the standard sufficient conditions for recursive feasibility and asymptotic stability of MPC with a general terminal set. Consider the finite-horizon problem

$$\begin{aligned} J^*(x(k)) = \min_U \quad & V_f(x_N) + \sum_{i=0}^{N-1} \ell(x_i, u_i) \\ \text{subj. to} \quad & x_{i+1} = Ax_i + Bu_i, \quad i = 0, \dots, N-1, \\ & x_i \in \mathcal{X}, \quad u_i \in \mathcal{U}, \quad i = 0, \dots, N-1, \\ & x_N \in \mathcal{X}_f, \\ & x_0 = x(k). \end{aligned}$$

The first assumption is that the stage cost is positive definite, meaning that it is zero at the origin and strictly positive away from it. In the regulation setting this can be expressed as

$$\ell(0,0) = 0, \quad \ell(x,u) > 0 \quad \text{for } (x,u) \neq (0,0).$$

The second assumption is the invariance and constraint-satisfaction property of the terminal set:

$$Ax + B\kappa_f(x) \in \mathcal{X}_f, \quad \forall x \in \mathcal{X}_f,$$

with

$$\mathcal{X}_f \subseteq \mathcal{X}, \quad \kappa_f(x) \in \mathcal{U}, \quad \forall x \in \mathcal{X}_f.$$

The third assumption concerns the terminal cost. The terminal cost must be a Lyapunov function for the terminal closed loop inside $\mathcal{X}_f$. More precisely, it must satisfy

$$V_f(Ax + B\kappa_f(x)) - V_f(x) \leq -\ell(x, \kappa_f(x)), \quad \forall x \in \mathcal{X}_f.$$

This inequality says that the decrease of the terminal cost along the local terminal controller is large enough to pay for the stage cost incurred at the terminal state. Therefore, the terminal cost behaves as an upper approximation of the infinite-horizon cost-to-go after the end of the prediction horizon.

MPC feasibility and stability theorem

Assume that the stage cost is positive definite. Assume also that there exists a local terminal control law κ_f such that the terminal set $\mathcal{X}_f$ is invariant under

$$x^+ = Ax + B\kappa_f(x),$$

and such that all state and input constraints are satisfied in $\mathcal{X}_f$. Finally, assume that the terminal cost satisfies

$$V_f(Ax + B\kappa_f(x)) - V_f(x) \leq -\ell(x, \kappa_f(x)), \quad \forall x \in \mathcal{X}_f.$$

Then the closed-loop system under the MPC law

$$u(k) = u_0^*(x(k))$$

is asymptotically stable on the feasible set X_N . Moreover, X_N is positive invariant for the closed-loop system

$$x(k+1) = Ax(k) + Bu_0^*(x(k)).$$

Proof idea

The proof follows the same structure as in the zero terminal constraint case. The only difference is that the last element of the shifted sequence is no longer the zero input. Instead, it is the terminal control law evaluated at the old terminal state.

Assume that the MPC problem is feasible at $x(k)$, and let

$$\{u_0^*, u_1^*, \dots, u_{N-1}^*\}$$

be the optimal input sequence computed at $x(k)$. Let the associated predicted state trajectory be

$$\{x(k), x_1^*, \dots, x_N^*\}.$$

Since the terminal constraint is satisfied, we have

$$x_N^* \in \mathcal{X}_f.$$

The MPC controller applies

$$u(k) = u_0^*,$$

so that the true successor state is

$$x(k+1) = Ax(k) + Bu_0^* = x_1^*.$$

At time $k+1$, we construct a candidate input sequence by shifting the old optimal sequence and appending the terminal controller:

$$\tilde{U} = \{u_1^*, u_2^*, \dots, u_{N-1}^*, \kappa_f(x_N^*)\}.$$

The first $N-1$ inputs reproduce the tail of the old feasible trajectory. The last input is feasible because $x_N^* \in \mathcal{X}_f$ and, by construction,

$$\kappa_f(x_N^*) \in \mathcal{U}.$$

Moreover, the successor of the old terminal state satisfies

$$Ax_N^* + B\kappa_f(x_N^*) \in \mathcal{X}_f,$$

because $\mathcal{X}_f$ is invariant under the terminal controller. Therefore the shifted sequence satisfies the terminal constraint at time $k+1$. This proves recursive feasibility.

The stability part uses the same candidate sequence. At time k , the optimal cost is

$$J^*(x(k)) = \sum_{i=0}^{N-1} \ell(x_i^*, u_i^*) + V_f(x_N^*).$$

At the next state $x(k+1) = x_1^*$, the shifted sequence is feasible, but not necessarily optimal. Hence the optimal cost at $x(k+1)$ is bounded above by the cost of this candidate:

$$J^*(x(k+1)) \leq \sum_{i=1}^{N-1} \ell(x_i^*, u_i^*) + \ell(x_N^*, \kappa_f(x_N^*)) + V_f(Ax_N^* + B\kappa_f(x_N^*)).$$

We rewrite this expression by adding and subtracting the old cost:

$$J^*(x(k+1)) \leq J^*(x(k)) - \ell(x_0^*, u_0^*) + V_f(Ax_N^* + B\kappa_f(x_N^*)) - V_f(x_N^*) + \ell(x_N^*, \kappa_f(x_N^*)).$$

The last two terms are nonpositive by the terminal Lyapunov condition:

$$V_f(Ax_N^* + B\kappa_f(x_N^*)) - V_f(x_N^*) + \ell(x_N^*, \kappa_f(x_N^*)) \leq 0.$$

Therefore

$$J^*(x(k+1)) - J^*(x(k)) \leq -\ell(x(k), u_0^*).$$

Since the stage cost is positive definite, this is a strict decrease away from the origin. Thus the optimal value function J^* is a Lyapunov function for the MPC closed loop, and the closed-loop system is asymptotically stable on X_N .

Shifted-sequence proof

The proof is based on the candidate sequence

$$\tilde{U} = \{u_1^*, \dots, u_{N-1}^*, \kappa_f(x_N^*)\}.$$

The terminal constraint gives recursive feasibility because $x_N^* \in \mathcal{X}_f$ implies

$$Ax_N^* + B\kappa_f(x_N^*) \in \mathcal{X}_f.$$

The terminal-cost decrease condition gives Lyapunov decrease because it cancels the additional terminal stage cost. Hence J^* becomes a Lyapunov function for the MPC closed loop.

Choice of Terminal Set and Cost for Linear Systems with Quadratic Cost

We now specialize the previous result to the most common linear-quadratic MPC setting. The finite-horizon problem is

$$\begin{aligned} J^*(x(k)) = \min_U \quad & x_N^\top P x_N + \sum_{i=0}^{N-1} (x_i^\top Q x_i + u_i^\top R u_i) \\ \text{subj. to} \quad & x_{i+1} = Ax_i + Bu_i, \quad i = 0, \dots, N-1, \\ & x_i \in \mathcal{X}, \quad u_i \in \mathcal{U}, \quad i = 0, \dots, N-1, \\ & x_N \in \mathcal{X}_f, \\ & x_0 = x(k). \end{aligned}$$

Here

$$Q = Q^\top \succeq 0, \quad R = R^\top \succ 0.$$

A standard design is obtained by using the unconstrained infinite-horizon LQR controller as the terminal controller. Let P_∞ solve the discrete-time algebraic Riccati equation

$$P_\infty = A^\top P_\infty A + Q - A^\top P_\infty B (B^\top P_\infty B + R)^{-1} B^\top P_\infty A.$$

The corresponding LQR feedback gain is

$$F_\infty = -(B^\top P_\infty B + R)^{-1} B^\top P_\infty A.$$

We then choose the terminal cost and terminal controller as

$$P = P_\infty, \quad V_f(x) = x^\top P_\infty x, \quad \kappa_f(x) = F_\infty x.$$

The terminal set $\mathcal{X}_f$ is chosen as an invariant set for the LQR closed-loop system

$$x^+ = (A + BF_\infty)x.$$

In addition, the state and input constraints must be satisfied in $\mathcal{X}_f$. If

$$\mathcal{X} = \{x \mid A_x x \leq b_x\}, \quad \mathcal{U} = \{u \mid A_u u \leq b_u\},$$

then the admissible closed-loop constraint set for the terminal controller is

$$\mathcal{X}_{\text{cl}} := \left\{ x \mid \begin{bmatrix} A_x \\ A_u F_\infty \end{bmatrix} x \leq \begin{bmatrix} b_x \\ b_u \end{bmatrix} \right\}.$$

A natural choice is to take $\mathcal{X}_f$ as the maximal invariant set for

$$x^+ = (A + BF_\infty)x$$

inside $\mathcal{X}_{\text{cl}}$. By construction, this gives

$$(A + BF_\infty)x \in \mathcal{X}_f, \quad \forall x \in \mathcal{X}_f,$$

and

$$\mathcal{X}_f \subseteq \mathcal{X}, \quad F_\infty x \in \mathcal{U}, \quad \forall x \in \mathcal{X}_f.$$

It remains to verify the terminal-cost condition. Since

$$x^+ = (A + BF_\infty)x,$$

we compute

$$\begin{aligned} & (x^+)^\top P_\infty x^+ - x^\top P_\infty x \\ &= x^\top \left((A + BF_\infty)^\top P_\infty (A + BF_\infty) - P_\infty \right) x. \end{aligned}$$

Using the Riccati equation and the definition of F_∞ , this becomes

$$(x^+)^\top P_\infty x^+ - x^\top P_\infty x = -x^\top (Q + F_\infty^\top R F_\infty) x.$$

Since

$$\ell(x, F_\infty x) = x^\top Q x + x^\top F_\infty^\top R F_\infty x,$$

we obtain

$$V_f((A + BF_\infty)x) - V_f(x) = -\ell(x, F_\infty x).$$

Thus the terminal cost is a Lyapunov function for the terminal controller in $\mathcal{X}_f$, and the terminal-cost decrease assumption is satisfied exactly.

Linear-quadratic terminal ingredients

For linear MPC with quadratic cost, a standard stabilizing choice is

$$\kappa_f(x) = F_\infty x, \quad V_f(x) = x^\top P_\infty x,$$

where F_∞ and P_∞ are obtained from the unconstrained infinite-horizon LQR problem. The terminal set $\mathcal{X}_f$ is chosen as an invariant set for

$$x^+ = (A + BF_\infty)x$$

inside the set where both $x \in \mathcal{X}$ and $F_\infty x \in \mathcal{U}$. With these choices, all assumptions of the MPC feasibility and stability theorem are verified.

7.5 Region of Attraction and Practical Remarks

The feasible set of the MPC problem,

$$X_N = \{x_0 \mid \exists u_0, \dots, u_{N-1} \text{ satisfying the dynamics, constraints, and } x_N \in \mathcal{X}_f\},$$

is also the certified region of attraction in the stability theorem. If $x(0) \in X_N$, then recursive feasibility ensures that the closed-loop trajectory remains in X_N , and the Lyapunov decrease of the optimal cost ensures convergence to the origin.

The choice of terminal set affects this region significantly. The terminal constraint $x_N = 0$ is the simplest possible choice, but it can lead to a small feasible set, especially when N is short. A larger invariant terminal set generally increases the feasible set, because the predicted trajectory only needs to reach a region where the local controller can take over, rather than reach the origin exactly.

There is, however, an important practical trade-off. Adding a terminal constraint gives a clean sufficient condition for feasibility and stability, but it may be conservative. It is possible that an MPC controller without a terminal constraint has a larger region of attraction in practice, but characterizing that region is generally difficult. The terminal-set approach sacrifices some flexibility in exchange for a tractable certificate.

For linear systems with quadratic cost, the construction based on LQR and invariant sets gives a systematic design method. In practice, one often enlarges the horizon and checks closed-loop behavior by simulation or sampling. As the horizon increases, the region of attraction of the finite-horizon MPC controller typically approaches the maximal control invariant set associated with the constraints. However, longer horizons increase computational cost, so the horizon must be chosen as a compromise between performance, feasibility, stability margin, and online optimization time.

7.6 Extension to Nonlinear MPC

The stability argument presented above is not fundamentally tied to linear dynamics. Consider a nonlinear discrete-time system

$$x(k+1) = g(x(k), u(k)).$$

The nonlinear MPC problem can be written as

$$\begin{aligned} J^*(x(k)) = \min_U \quad & V_f(x_N) + \sum_{i=0}^{N-1} \ell(x_i, u_i) \\ \text{subj. to} \quad & x_{i+1} = g(x_i, u_i), \quad i = 0, \dots, N-1, \\ & x_i \in \mathcal{X}, \quad u_i \in \mathcal{U}, \quad i = 0, \dots, N-1, \\ & x_N \in \mathcal{X}_f, \\ & x_0 = x(k). \end{aligned}$$

The same proof idea applies if there exists a terminal controller κ_f such that

$$g(x, \kappa_f(x)) \in \mathcal{X}_f \quad \forall x \in \mathcal{X}_f,$$

with

$$\mathcal{X}_f \subseteq \mathcal{X}, \quad \kappa_f(x) \in \mathcal{U} \quad \forall x \in \mathcal{X}_f,$$

and if the terminal cost satisfies

$$V_f(g(x, \kappa_f(x))) - V_f(x) \leq -\ell(x, \kappa_f(x)) \quad \forall x \in \mathcal{X}_f.$$

Then the shifted-sequence argument again proves recursive feasibility, and the optimal value function again decreases along the closed-loop trajectory.

The conceptual extension is therefore direct: Lyapunov stability is a general framework for nonlinear systems, and the terminal-set/terminal-cost argument only relies on invariance and cost decrease. The practical difficulty is computational. For nonlinear systems, finding a suitable terminal set $\mathcal{X}_f$, terminal controller κ_f , and terminal cost V_f can be very challenging. In many applications, one uses local linearization, local LQR design, or other Lyapunov-based constructions to obtain approximate terminal ingredients near the desired equilibrium.

7.7 Summary

Finite-horizon MPC is not automatically recursively feasible and not automatically stable. The reason is that finite-horizon optimization is short-sighted: it optimizes an open-loop prediction over a finite window, whereas the actual controller is implemented in closed loop by repeatedly resolving the problem. As a result, the controller may drive the system to a state from which the next optimization problem is infeasible, or it may generate feasible trajectories that do not converge to the origin.

The infinite-horizon constrained optimal control problem avoids this difficulty conceptually, because the optimized trajectory already accounts for the entire future. If the infinite-horizon problem is feasible and has finite cost, the remaining tail of the optimal trajectory provides a natural feasible continuation after the first input is applied. However, infinite-horizon constrained problems are generally not computable online.

The stabilizing MPC construction mimics the infinite-horizon tail by introducing terminal ingredients. The terminal set $\mathcal{X}_f$ is chosen so that, once the predicted trajectory reaches it, a known local controller κ_f can keep the system feasible forever. The terminal cost V_f is chosen so that it decreases under this terminal controller by at least the incurred stage cost. These two properties allow one to prove recursive feasibility by shifting the previous optimal sequence and appending the terminal control input. They also allow one to prove stability by showing that the optimal MPC value function is a Lyapunov function.

For linear systems with quadratic costs, a standard choice is obtained from the unconstrained infinite-horizon LQR problem. The terminal controller is the LQR feedback $F_\infty x$, the terminal cost is $x^\top P_\infty x$, and the terminal set is an invariant set for the closed-loop system

$x^+ = (A + BF_\infty)x$ satisfying the state and input constraints. This construction verifies the assumptions of the general MPC stability theorem and provides a systematic way to design stabilizing finite-horizon MPC.

The same ideas extend to nonlinear MPC, because the proof relies on invariance and Lyapunov decrease rather than on linearity. The main challenge in the nonlinear case is the computation of suitable terminal sets, terminal controllers, and terminal costs.

Part III

Practical MPC

Chapter 8

Practical Issues in MPC

8.1 Motivation

The previous chapter developed the standard feasibility and stability theory for nominal MPC. The central result was that recursive feasibility and closed-loop stability can be guaranteed by adding suitable terminal ingredients: a terminal set, a terminal controller, and a terminal cost satisfying an invariance and Lyapunov decrease condition. This gives a clean theoretical construction for regulation to the origin.

In practice, however, MPC is rarely used only as a regulator to the origin. The controller is usually required to track a nonzero reference, reject disturbances, operate with measured or estimated outputs rather than full state measurements, and remain usable even when some state or output constraints cannot be satisfied exactly. Moreover, terminal constraints, although very useful for theoretical guarantees, may reduce the feasible set and make the online optimization problem more restrictive than desired.

This chapter discusses these practical issues. We first extend the regulation formulation to reference tracking. The key idea is to compute a steady-state target corresponding to the desired reference and then formulate the MPC problem in deviation variables around that target. We then discuss offset-free tracking, where constant disturbances or model mismatch would otherwise lead to steady-state tracking errors. Finally, we consider two ways of enlarging the practical operating region of MPC: removing terminal constraints, and softening state or output constraints.

8.2 Reference Tracking

8.2.1 From regulation to tracking

Consider the linear discrete-time system

$$x(k+1) = Ax(k) + Bu(k),$$

where $x(k) \in \mathbb{R}^{n_x}$ and $u(k) \in \mathbb{R}^{n_u}$. The state and input are constrained to lie in polyhedral sets

$$\mathcal{X} = \{x \in \mathbb{R}^{n_x} \mid G_x x \leq h_x\}, \quad \mathcal{U} = \{u \in \mathbb{R}^{n_u} \mid G_u u \leq h_u\}.$$

The standard MPC problem studied in the previous chapters is a regulation problem: the origin is the desired equilibrium, and the cost penalizes deviations of x and u from zero.

In many applications, the control goal is instead to track a nonzero output reference. Let

$$z(k) = Hx(k)$$

be the controlled output, and let $r \in \mathbb{R}^{n_r}$ be the desired constant reference. The tracking objective is

$$z(k) = Hx(k) \longrightarrow r \quad \text{as } k \rightarrow \infty.$$

In this chapter we focus on constant references, or on piecewise constant references that change only occasionally. Time-varying reference trajectories require additional prediction information and will not be considered here.

The main conceptual question is how to convert a desired output reference r into a state-input target for the MPC controller. For regulation to the origin, the target is already known: $(x_s, u_s) = (0, 0)$. For tracking, however, the output reference r may correspond to a nonzero equilibrium (x_s, u_s) , and this equilibrium must first be computed.

8.2.2 The steady-state target problem

A target state x_s achieves the output reference if

$$Hx_s = r.$$

However, this condition alone is not sufficient. The state x_s must also be a steady-state of the system under some constant input u_s . Otherwise the system could pass through x_s , but it could not remain there. Therefore the target pair (x_s, u_s) must satisfy

$$x_s = Ax_s + Bu_s, \quad Hx_s = r.$$

Equivalently,

$$\begin{bmatrix} I - A & -B \\ H & 0 \end{bmatrix} \begin{bmatrix} x_s \\ u_s \end{bmatrix} = \begin{bmatrix} 0 \\ r \end{bmatrix}.$$

This is the algebraic steady-state target condition. It says that the desired reference must be compatible with the system dynamics and with the controlled output map.

In the presence of constraints, the target must also satisfy

$$x_s \in \mathcal{X}, \quad u_s \in \mathcal{U}.$$

If several steady-state pairs satisfy the same reference, one can select among them by solving an optimization problem. A common choice is to choose the cheapest steady-state input:

$$\begin{aligned} \min_{x_s, u_s} \quad & u_s^\top R_s u_s \\ \text{s.t.} \quad & \begin{bmatrix} I - A & -B \\ H & 0 \end{bmatrix} \begin{bmatrix} x_s \\ u_s \end{bmatrix} = \begin{bmatrix} 0 \\ r \end{bmatrix}, \\ & x_s \in \mathcal{X}, \quad u_s \in \mathcal{U}. \end{aligned}$$

This problem is called the steady-state target problem. It is solved before, or together with, the MPC optimization problem.

If the desired reference cannot be achieved exactly, one may instead compute a reachable steady-state whose output is as close as possible to r :

$$\begin{aligned} \min_{x_s, u_s} \quad & (Hx_s - r)^\top Q_s (Hx_s - r) \\ \text{s.t.} \quad & x_s = Ax_s + Bu_s, \\ & x_s \in \mathcal{X}, \quad u_s \in \mathcal{U}. \end{aligned}$$

This modified target problem is useful when the requested reference lies outside the set of admissible steady-state outputs. In that case the controller should not attempt to track an impossible reference, but should instead track the closest feasible steady-state output according to the chosen weighting matrix Q_s .

8.2.3 MPC for reference tracking

Once a steady-state target (x_s, u_s) has been computed, the MPC problem can be modified so that the predicted trajectory is penalized relative to this target rather than relative to the origin. A direct tracking formulation is

$$\begin{aligned} \min_U \quad & \|z_N - Hx_s\|_{P_z}^2 + \sum_{i=0}^{N-1} \left(\|z_i - Hx_s\|_{Q_z}^2 + \|u_i - u_s\|_R^2 \right) \\ \text{s.t.} \quad & x_{i+1} = Ax_i + Bu_i, \quad i = 0, \dots, N-1, \\ & x_i \in \mathcal{X}, \quad u_i \in \mathcal{U}, \quad i = 0, \dots, N-1, \\ & x_0 = x(k). \end{aligned}$$

The cost now penalizes output tracking error and deviation of the input from the steady-state input. If the terminal ingredients are included, the terminal penalty is also centered at the target.

This formulation is intuitive, but it is often more convenient to rewrite the problem in deviation variables. Define

$$\Delta x := x - x_s, \quad \Delta u := u - u_s.$$

Because (x_s, u_s) is a steady-state,

$$x_s = Ax_s + Bu_s.$$

Therefore the deviation dynamics are

$$\begin{aligned} \Delta x(k+1) &= x(k+1) - x_s \\ &= Ax(k) + Bu(k) - (Ax_s + Bu_s) \\ &= A(x(k) - x_s) + B(u(k) - u_s) \\ &= A\Delta x(k) + B\Delta u(k). \end{aligned}$$

Thus, for a linear system, the deviation variables satisfy exactly the same system matrices A and B . This is why tracking can be viewed as a regulation problem after a change of coordinates. The constraints are shifted accordingly. Since

$$G_x x \leq h_x \iff G_x(\Delta x + x_s) \leq h_x,$$

the state constraint becomes

$$G_x \Delta x \leq h_x - G_x x_s.$$

Similarly, the input constraint becomes

$$G_u \Delta u \leq h_u - G_u u_s.$$

The tracking MPC problem in deviation form is therefore

$$\begin{aligned} \min_{\Delta U} \quad & \sum_{i=0}^{N-1} \left(\Delta x_i^\top Q \Delta x_i + \Delta u_i^\top R \Delta u_i \right) + V_f(\Delta x_N) \\ \text{s.t.} \quad & \Delta x_0 = x(k) - x_s, \\ & \Delta x_{i+1} = A\Delta x_i + B\Delta u_i, \quad i = 0, \dots, N-1, \\ & G_x \Delta x_i \leq h_x - G_x x_s, \quad i = 0, \dots, N-1, \\ & G_u \Delta u_i \leq h_u - G_u u_s, \quad i = 0, \dots, N-1, \\ & \Delta x_N \in \mathcal{X}_f. \end{aligned}$$

After solving this regulation problem in deviation variables, the input applied to the plant is

$$u(k) = u_s + \Delta u_0^*.$$

Tracking as regulation in deviation variables

For a linear system, tracking a steady-state target (x_s, u_s) is equivalent to regulating the deviation variables

$$\Delta x = x - x_s, \quad \Delta u = u - u_s$$

to the origin. The dynamics remain

$$\Delta x^+ = A\Delta x + B\Delta u,$$

while the constraints are shifted by the target.

MPC for Tracking: Convergence

We now discuss why the tracking MPC formulation inherits the convergence guarantees of the regulation problem. The key observation is that, after introducing the deviation variables

$$\Delta x = x - x_s, \quad \Delta u = u - u_s,$$

the tracking problem becomes a regulation problem for the deviation dynamics

$$\Delta x^+ = A\Delta x + B\Delta u.$$

Therefore, if the terminal ingredients are chosen as in the regulation case, then the same Lyapunov and recursive-feasibility arguments can be applied to the shifted system.

Assume that the target steady-state is feasible, i.e.

$$x_s \in \mathcal{X}, \quad u_s \in \mathcal{U}.$$

Moreover, assume that the terminal cost V_f , the terminal set $\mathcal{X}_f$, and the local terminal controller

$$\Delta u = K\Delta x$$

satisfy the usual regulation assumptions. Namely, the terminal set is invariant under the local closed-loop dynamics,

$$(A + BK)\Delta x \in \mathcal{X}_f \quad \forall \Delta x \in \mathcal{X}_f,$$

and the terminal cost decreases along the terminal controller according to

$$V_f((A + BK)\Delta x) - V_f(\Delta x) \leq -\ell(\Delta x, K\Delta x) \quad \forall \Delta x \in \mathcal{X}_f.$$

These two conditions are exactly the same as in the regulation case. They guarantee that, once the predicted deviation state reaches $\mathcal{X}_f$, the local controller can keep it inside $\mathcal{X}_f$ while providing the required Lyapunov decrease.

For tracking, however, one additional issue appears. The terminal set is expressed in deviation coordinates, but the real constraints are imposed on the original variables x and u . Hence the shifted terminal set must remain inside the original state constraints:

$$x_s \oplus \mathcal{X}_f \subseteq \mathcal{X}.$$

Similarly, the terminal input obtained after shifting back to the original coordinates must satisfy the input constraints:

$$u_s + K\Delta x \in \mathcal{U} \quad \forall \Delta x \in \mathcal{X}_f.$$

These two inclusions are the tracking-specific constraint-compatibility conditions. They say that the terminal controller is not only admissible for the deviation system, but also admissible for the original constrained system around the target (x_s, u_s) .

Under these assumptions, the proof follows the same logic as for stabilizing MPC in the regulation case. At the next sampling instant, the previously optimal sequence can be shifted by one step, and the missing terminal input can be appended using the local law $\Delta u = K\Delta x$. The invariance condition ensures that the new terminal deviation state remains in $\mathcal{X}_f$, while the shifted constraint conditions ensure that the corresponding original state and input remain in $\mathcal{X}$ and $\mathcal{U}$. This proves recursive feasibility.

The Lyapunov argument is also inherited from the regulation case. The optimal value function of the tracking problem is a Lyapunov function for the deviation dynamics. Since the terminal cost decreases sufficiently inside $\mathcal{X}_f$, the closed-loop deviation state converges to the origin:

$$\Delta x(k) \rightarrow 0 \quad \text{as } k \rightarrow \infty.$$

Returning to the original coordinates, this means

$$x(k) \rightarrow x_s.$$

Consequently, since the controlled output is $z(k) = Hx(k)$, we obtain

$$z(k) = Hx(k) \rightarrow Hx_s = r.$$

Tracking convergence

If the target (x_s, u_s) is feasible, if the terminal set and terminal cost satisfy the usual regulation conditions in deviation coordinates, and if

$$x_s \oplus \mathcal{X}_f \subseteq \mathcal{X}, \quad u_s + K\Delta x \in \mathcal{U} \quad \forall \Delta x \in \mathcal{X}_f,$$

then the MPC closed loop converges to the target:

$$x(k) \rightarrow x_s, \quad z(k) = Hx(k) \rightarrow r.$$

8.2.4 Terminal Sets for Tracking

The previous convergence result shows that tracking can be treated as regulation around a shifted steady-state target. However, this shift introduces an important practical difficulty: the terminal set is no longer fixed at the origin, but must be placed around the target state x_s . In the regulation case, the terminal constraint is simply

$$x_N \in \mathcal{X}_f,$$

where $\mathcal{X}_f$ is a set around the origin. In the tracking case, the terminal constraint is imposed on the deviation state:

$$x_N - x_s \in \mathcal{X}_f.$$

Equivalently, in the original state coordinates,

$$x_N \in x_s \oplus \mathcal{X}_f.$$

Thus, the regulation terminal set is translated by the target x_s .

The state constraints define the admissible region $\mathcal{X}$, while the terminal set must fit inside this region after being shifted to the desired target. If the target is well inside the constraint set, the shifted terminal set may fit without difficulty. If, however, the target is close to the boundary of $\mathcal{X}$, then the full terminal set may no longer fit. In that case, even though the steady-state target x_s itself is feasible, the MPC tracking problem may become infeasible because the controller requires the whole terminal region $x_s \oplus \mathcal{X}_f$ to be feasible.

This explains why the set of feasible targets can be significantly smaller than the set of constrained steady-states. A target may satisfy

$$x_s \in \mathcal{X}, \quad u_s \in \mathcal{U},$$

but still fail the stronger terminal-set condition

$$x_s \oplus \mathcal{X}_f \subseteq \mathcal{X}.$$

The terminal set therefore gives a useful stability certificate, but it also introduces conservatism in reference tracking.

Scaling the Terminal Set

A natural way to reduce this conservatism is to scale the terminal set. Instead of using the same terminal set $\mathcal{X}_f$ for all targets, one uses a target-dependent scaled terminal set

$$\mathcal{X}_f^{\text{scaled}} = \alpha \mathcal{X}_f,$$

where $\alpha \geq 0$ is chosen so that the shifted set remains inside the constraints:

$$x_s \oplus \alpha \mathcal{X}_f \subseteq \mathcal{X}.$$

The corresponding terminal input constraint must also hold:

$$u_s + K\Delta x \in \mathcal{U} \quad \forall \Delta x \in \alpha \mathcal{X}_f.$$

The reason why this scaling is admissible is that invariance is preserved under scaling for linear closed-loop dynamics. Indeed, if $\mathcal{X}_f$ is invariant under $A + BK$, then for every $\Delta x \in \mathcal{X}_f$,

$$(A + BK)\Delta x \in \mathcal{X}_f.$$

For a scaled point $\alpha\Delta x$, linearity gives

$$(A + BK)(\alpha\Delta x) = \alpha(A + BK)\Delta x \in \alpha \mathcal{X}_f.$$

Hence $\alpha \mathcal{X}_f$ is also invariant.

The scaling factor α is therefore selected according to the available constraint margin around the target. If x_s is far from the constraint boundary, a large value of α can be used. If x_s is close to the boundary, only a small terminal set may fit. In the limiting case, for a target exactly on the boundary of the admissible set, one may need

$$\alpha = 0.$$

Then the terminal set collapses to a single point and the terminal condition becomes

$$x_N = x_s.$$

This is the tracking analogue of imposing a zero terminal state constraint in the regulation problem.

Target-dependent terminal sets

In tracking MPC, the terminal set must be shifted to the target:

$$x_N \in x_s \oplus \mathcal{X}_f.$$

This can reduce the set of feasible references, because the whole shifted terminal set must fit inside the constraints. Scaling the terminal set,

$$\mathcal{X}_f^{\text{scaled}} = \alpha \mathcal{X}_f,$$

reduces this conservatism while preserving invariance for linear closed-loop dynamics.

Practical Consequences

The main practical consequence is that reference changes can affect feasibility. When the reference changes, the target selector computes a new steady-state pair (x_s, u_s) . This new target changes both the shifted constraints in deviation variables and the location of the terminal set. Therefore, even if the MPC problem was feasible for the previous reference, it may become infeasible for the new one.

This is one of the main limitations of the direct shifted-terminal-set approach to tracking. It provides a clean extension of stabilizing MPC for regulation, but the terminal set becomes a function of the target. In applications with frequent reference changes, this dependence can be restrictive. More advanced tracking MPC formulations address this issue by optimizing over artificial steady-states or by using terminal ingredients that are less directly tied to a fixed target.

8.2.5 Offset-Free Tracking

Why standard tracking can have offset

The tracking formulation developed so far assumes that the model used by the controller describes the plant accurately. Under this assumption, if the target selector computes a steady-state pair (x_s, u_s) satisfying

$$x_s = Ax_s + Bu_s, \quad Hx_s = r,$$

then stabilizing MPC drives the state to x_s and the controlled output to r .

In practice, however, the true plant is rarely described exactly by the nominal model. A particularly important case is the presence of a constant disturbance. The plant may evolve according to

$$x(k+1) = Ax(k) + Bu(k) + B_d d,$$

with measured output

$$y(k) = Cx(k) + C_d d,$$

where d is an unknown constant disturbance. Such a disturbance changes the true steady-state relation between input, state, and output. Consequently, the nominal target (x_s, u_s) computed without accounting for d may no longer correspond to the desired output of the real plant.

The result is a steady-state offset. The MPC controller may stabilize the closed-loop system, but the output can converge to a value different from the desired set point. In other words, stability alone does not imply zero tracking error when the model is affected by constant disturbances.

Objective of offset-free tracking

The objective of offset-free tracking is to design the MPC controller so that, if the closed-loop system is stabilized in the presence of a constant disturbance, then the controlled output still converges to the desired set point with zero steady-state offset.

The classical unconstrained-control solution to constant disturbances is to introduce integral action. The integrator accumulates the tracking error and modifies the input until the steady-state error disappears. In constrained control, however, this idea must be used carefully. Input

constraints and actuator limitations can lead to windup phenomena if integral action is added naively. MPC therefore follows a different but related approach: the disturbance is modeled explicitly, the state and disturbance are estimated from measurements, and the target problem is modified using the disturbance estimate.

Thus the offset-free MPC architecture consists of three conceptual steps. First, augment the prediction model with a constant disturbance model. Second, use the output measurements to estimate both the state and the disturbance. Third, compute the steady-state target and the MPC input using the disturbance estimate.

Augmented disturbance model

To model constant disturbances, we augment the system with an integrating disturbance state:

$$\begin{aligned}x_{k+1} &= Ax_k + Bu_k + B_d d_k, \\d_{k+1} &= d_k, \\y_k &= Cx_k + C_d d_k,\end{aligned}$$

where $d_k \in \mathbb{R}^{n_d}$. The equation

$$d_{k+1} = d_k$$

encodes the assumption that the disturbance is constant over time. This does not mean that the disturbance is known. Rather, d_k is treated as an additional state to be estimated from the measured output.

In compact form, the augmented model is

$$\begin{bmatrix} x_{k+1} \\ d_{k+1} \end{bmatrix} = \begin{bmatrix} A & B_d \\ 0 & I \end{bmatrix} \begin{bmatrix} x_k \\ d_k \end{bmatrix} + \begin{bmatrix} B \\ 0 \end{bmatrix} u_k,$$

with output equation

$$y_k = \begin{bmatrix} C & C_d \end{bmatrix} \begin{bmatrix} x_k \\ d_k \end{bmatrix}.$$

The only essential restriction on the choice of the disturbance matrices B_d and C_d is that the augmented model must be observable, or at least detectable. In the lecture, the following condition is used. The augmented system is observable if and only if (A, C) is observable and

$$\text{rank} \begin{bmatrix} A - I & B_d \\ C & C_d \end{bmatrix} = n_x + n_d.$$

This rank condition has a useful steady-state interpretation. At steady-state,

$$\begin{bmatrix} A - I & B_d \\ C & C_d \end{bmatrix} \begin{bmatrix} x_s \\ d_s \end{bmatrix} = \begin{bmatrix} 0 \\ y_s \end{bmatrix}.$$

Thus, given a measured steady-state output y_s , the disturbance d_s must be uniquely identified by the equations. If two different disturbances could produce the same measured steady-state output, then the estimator would not be able to distinguish them, and offset-free compensation would not be well posed.

This also explains the restriction

$$n_d \leq n_y.$$

Since disturbances are inferred from output measurements, one cannot in general estimate more independent constant disturbances than the number of measured outputs.

Observability of the augmented model

For offset-free tracking, the disturbance model must be chosen so that the augmented state

$$\begin{bmatrix} x \\ d \end{bmatrix}$$

can be reconstructed from output measurements. A standard condition is that (A, C) is observable and

$$\text{rank} \begin{bmatrix} A - I & B_d \\ C & C_d \end{bmatrix} = n_x + n_d.$$

In particular, the number of modeled constant disturbances cannot exceed the number of measured outputs:

$$n_d \leq n_y.$$

Linear state estimation

We now need an estimator for the augmented model. Let $\hat{x}(k)$ and $\hat{d}(k)$ denote the estimates of the state and the disturbance. A linear observer can be written as

$$\begin{bmatrix} \hat{x}(k+1) \\ \hat{d}(k+1) \end{bmatrix} = \begin{bmatrix} A & B_d \\ 0 & I \end{bmatrix} \begin{bmatrix} \hat{x}(k) \\ \hat{d}(k) \end{bmatrix} + \begin{bmatrix} B \\ 0 \end{bmatrix} u(k) + \begin{bmatrix} L_x \\ L_d \end{bmatrix} \left(-y(k) + C\hat{x}(k) + C_d\hat{d}(k) \right).$$

Equivalently, the correction term is proportional to the output prediction error. The innovation is written as

$$-y(k) + C\hat{x}(k) + C_d\hat{d}(k).$$

With the opposite sign convention one would simply change the sign of the observer gain. Let the estimation errors be

$$e_x(k) = x(k) - \hat{x}(k), \quad e_d(k) = d(k) - \hat{d}(k).$$

Using the plant and observer equations, the error dynamics become

$$\begin{bmatrix} e_x(k+1) \\ e_d(k+1) \end{bmatrix} = \left(\begin{bmatrix} A & B_d \\ 0 & I \end{bmatrix} + \begin{bmatrix} L_x \\ L_d \end{bmatrix} \begin{bmatrix} C & C_d \end{bmatrix} \right) \begin{bmatrix} e_x(k) \\ e_d(k) \end{bmatrix}.$$

The observer gain

$$L = \begin{bmatrix} L_x \\ L_d \end{bmatrix}$$

is chosen so that this error system is asymptotically stable. Then

$$\hat{x}(k) \rightarrow x(k), \quad \hat{d}(k) \rightarrow d,$$

at least asymptotically in the constant-disturbance setting. In practice, this gain can be designed by pole placement or through a Kalman filter.

The important point is that the estimator is not only reconstructing the plant state. It is also identifying the constant disturbance that explains the mismatch between the nominal model and the measured output. Once this disturbance has been estimated, it can be used to correct the steady-state target.

A steady-state property of the observer

The previous observer has a crucial steady-state property. Suppose the observer is asymptotically stable, and suppose that the number of outputs equals the number of modeled constant disturbances:

$$n_y = n_d.$$

At steady-state, the observer estimates satisfy

$$\begin{bmatrix} A - I & B \\ C & 0 \end{bmatrix} \begin{bmatrix} \hat{x}_\infty \\ u_\infty \end{bmatrix} = \begin{bmatrix} -B_d \hat{d}_\infty \\ y_\infty - C_d \hat{d}_\infty \end{bmatrix}.$$

This relation says that the estimated state and disturbance are consistent with the steady-state input and the measured steady-state output. In particular, the observer output satisfies

$$C\hat{x}_\infty + C_d\hat{d}_\infty = y_\infty.$$

Thus, at steady-state, the observer output tracks the measured output without offset.

Observer steady-state property

Suppose the augmented observer is asymptotically stable and $n_y = n_d$. Then, at steady-state, the estimated output satisfies

$$C\hat{x}_\infty + C_d\hat{d}_\infty = y_\infty.$$

Thus the observer reproduces the measured output at steady-state without offset.

Disturbance-aware target problem

The control objective is now to track a constant reference

$$z(k) = Hy(k) \rightarrow r.$$

Because the measured output is affected by the disturbance, the steady-state target equations must be modified. If the current disturbance estimate is $\hat{d}$, then the target pair (x_s, u_s) should satisfy

$$x_s = Ax_s + Bu_s + B_d\hat{d},$$

and

$$z_s = H(Cx_s + C_d\hat{d}) = r.$$

The first equation modifies the steady-state condition for the state evolution. The second modifies the target condition for the controlled variables, because the controlled output depends on the measured output and hence also on the disturbance.

Equivalently, the disturbance-aware target condition is

$$\begin{bmatrix} A - I & B \\ HC & 0 \end{bmatrix} \begin{bmatrix} x_s \\ u_s \end{bmatrix} = \begin{bmatrix} -B_d\hat{d} \\ r - HC_d\hat{d} \end{bmatrix}.$$

This is the same target selection idea as in nominal tracking, but with a right-hand side corrected by the disturbance estimate. The best prediction of the steady-state disturbance is the current estimate

$$\hat{d}_\infty \approx \hat{d}(k).$$

The same construction also applies to regulation by setting $r = 0$.

Offset-free MPC problem

At each sampling instant, offset-free MPC combines estimation, target selection, and predictive control. The controller first estimates the current state and disturbance,

$$\hat{x}(k), \quad \hat{d}(k).$$

It then computes a disturbance-corrected steady-state target (x_s, u_s) from the target problem above. Finally, it solves an MPC tracking problem using the disturbance estimate:

$$\begin{aligned} \min_U \quad & \sum_{i=0}^{N-1} \left((x_i - x_s)^\top Q (x_i - x_s) + (u_i - u_s)^\top R (u_i - u_s) \right) + V_f(x_N - x_s) \\ \text{s.t.} \quad & x_0 = \hat{x}(k), \\ & d_0 = \hat{d}(k), \\ & x_{i+1} = Ax_i + Bu_i + B_d d_i, \quad i = 0, \dots, N-1, \\ & d_{i+1} = d_i, \quad i = 0, \dots, N-1, \\ & x_i \in \mathcal{X}, \quad u_i \in \mathcal{U}, \quad i = 0, \dots, N-1, \\ & x_N - x_s \in \mathcal{X}_f. \end{aligned}$$

Since the predicted disturbance satisfies $d_{i+1} = d_i$, the disturbance remains equal to the current estimate along the prediction horizon:

$$d_i = \hat{d}(k).$$

Therefore, as in the nominal tracking case, the problem can also be rewritten in deviation variables around the disturbance-corrected target.

The plant produces the measured output y . The estimator uses y and the applied input u to produce $\hat{x}$ and $\hat{d}$. The target selector uses the reference r and the disturbance estimate $\hat{d}$ to compute (x_s, u_s) . The MPC regulator then computes the input u that drives the estimated state toward the corrected target.

Offset-free MPC algorithm

At each sampling time:

1. Estimate the state and disturbance $\hat{x}(k), \hat{d}(k)$.
2. Compute (x_s, u_s) from the disturbance-aware target problem.
3. Solve the MPC tracking problem using $\hat{x}(k)$ and $\hat{d}(k)$.
4. Apply the first optimal input u_0^* .

Main result

We state the offset-free property in the following form. Denote by

$$\kappa(\hat{x}(k), \hat{d}(k), r) = u_0^*$$

the receding-horizon control law obtained when the estimated state is $\hat{x}(k)$, the estimated disturbance is $\hat{d}(k)$, and the reference is r . The resulting closed-loop system is

$$\begin{aligned} x(k+1) &= Ax(k) + B\kappa(\hat{x}(k), \hat{d}(k), r) + B_d d, \\ \hat{x}(k+1) &= (A + L_x C)\hat{x}(k) + (B_d + L_x C_d)\hat{d}(k) + B\kappa(\hat{x}(k), \hat{d}(k), r) - L_x y(k), \\ \hat{d}(k+1) &= L_d C\hat{x}(k) + (I + L_d C_d)\hat{d}(k) - L_d y(k). \end{aligned}$$

Offset-free tracking theorem

Consider the case in which the number of modeled constant disturbances equals the number of outputs,

$$n_d = n_y.$$

Assume that the receding-horizon controller is recursively feasible and unconstrained after some finite time j , and assume that the closed-loop system converges, i.e.

$$\hat{x}(k) \rightarrow \hat{x}_\infty, \quad \hat{d}(k) \rightarrow \hat{d}_\infty, \quad y(k) \rightarrow y_\infty \quad \text{as } k \rightarrow \infty.$$

Then the controlled output converges to the reference:

$$z(k) = Hy(k) \rightarrow r.$$

The meaning of the theorem is important. Offset-free MPC does not say that disturbances disappear. Rather, it says that if the disturbance can be consistently estimated and if the closed loop converges, then the steady-state target is corrected in such a way that the controlled output tracks the desired reference. The disturbance estimate is used to modify the equilibrium around which MPC regulates.

Example: double integrator

We illustrate the construction on the double integrator. The first set point is $r = 0$, while the second set point is $r = 3$. The system has one output, one input, and one modeled constant disturbance.

For standard tracking, the nominal model is

$$x(k+1) = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} x(k) + \begin{bmatrix} 1 \\ 0.5 \end{bmatrix} u(k),$$

with output

$$z(k) = y(k) = \begin{bmatrix} 1 & 0 \end{bmatrix} x(k),$$

and constraints

$$-0.5 \leq u(k) \leq 0.5, \quad -5 \leq y(k) \leq 5.$$

The nominal target condition is

$$\begin{bmatrix} I - A & -B \\ C & 0 \end{bmatrix} \begin{bmatrix} x_s \\ u_s \end{bmatrix} = \begin{bmatrix} 0 \\ r \end{bmatrix}.$$

For offset-free tracking, the model is augmented with a constant disturbance:

$$x(k+1) = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} x(k) + \begin{bmatrix} 1 \\ 0.5 \end{bmatrix} u(k) + \begin{bmatrix} 1 \\ 1 \end{bmatrix} d(k),$$

with the same measured output

$$z(k) = y(k) = \begin{bmatrix} 1 & 0 \end{bmatrix} x(k).$$

The augmented observability condition requires

$$\begin{bmatrix} I - A & -B_d \\ C & 0 \end{bmatrix}$$

to have full column rank. The target condition becomes

$$\begin{bmatrix} I - A & -B \\ C & 0 \end{bmatrix} \begin{bmatrix} x_s \\ u_s \end{bmatrix} = \begin{bmatrix} B_d \hat{d} \\ r - C_d \hat{d} \end{bmatrix}.$$

Figure 8.1 shows the qualitative difference between standard tracking and offset-free tracking. In the standard case, the controller reacts to the reference change but converges with a steady-state offset because the constant disturbance is not accounted for in the target selection. In the offset-free case, the disturbance estimate modifies the target, and the output converges to the desired set point without offset.

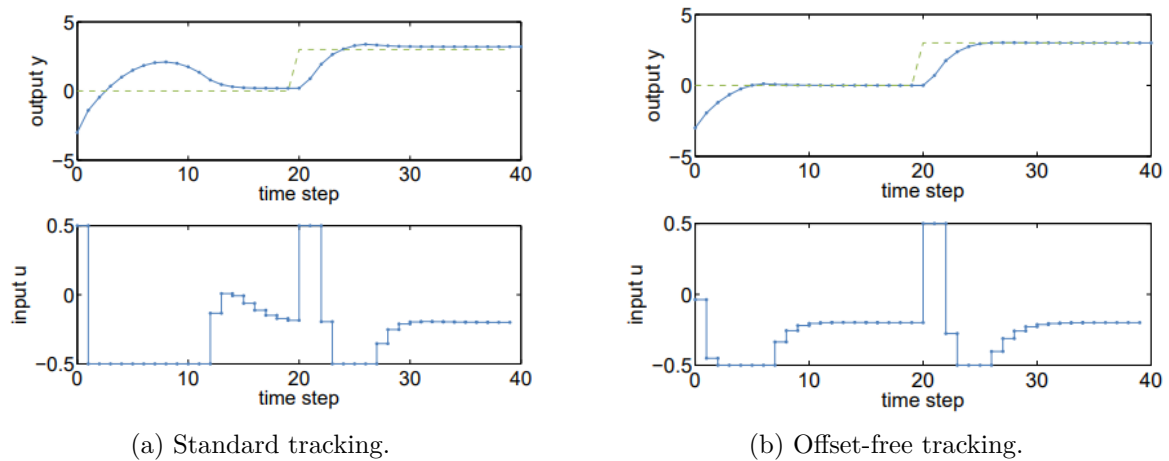


Figure 8.1: Comparison between standard tracking and offset-free tracking for the disturbed double integrator. Standard tracking stabilizes the system but exhibits a steady-state offset. Offset-free tracking estimates the constant disturbance and modifies the target condition, so that the output converges to the desired reference.

Summary of offset-free control

Offset-free MPC can be summarized as follows. A constant disturbance is included in the model through integrating disturbance dynamics. The output measurements are then used to estimate both the state and the disturbance, for example by a linear observer or a Kalman filter. The steady-state target problem is modified using the disturbance estimate, so that the target accounts for the effect of the disturbance on both the state equation and the controlled variables. Finally, the MPC problem is solved for regulation or tracking around this corrected target. The main result is that, if the number of disturbance states equals the number of outputs, if the target steady-state problem is feasible and no constraints are active at steady-state, and if the closed-loop system converges, then the desired target is achieved without offset:

$$z(k) \rightarrow r.$$

Offset-free control: main message

Standard tracking MPC may converge with a steady-state error when constant disturbances or model mismatch are present. Offset-free MPC removes this error by estimating the disturbance and using it in the steady-state target problem. The controller is still an MPC tracking controller, but it regulates around a disturbance-corrected equilibrium.

8.3 Enlarging the Feasible Set

8.3.1 Why terminal constraints may be conservative

The stabilizing MPC construction from the previous chapter uses a terminal constraint

$$x_N \in \mathcal{X}_f.$$

This terminal constraint gives a clean proof of recursive feasibility and stability, but it may also reduce the feasible set. Indeed, the optimization problem is feasible only from states that can be driven into $\mathcal{X}_f$ within N steps while respecting all state and input constraints. If the terminal set is small or if the horizon is short, this requirement can be restrictive.

Removing the terminal constraint generally enlarges the set of states for which the finite-horizon optimization problem has a solution. It also reduces the number of constraints in the optimization problem. In some cases, it avoids adding state constraints to problems that otherwise would have only input constraints. From an implementation point of view, this can be attractive.

However, the price is theoretical. Without a terminal constraint, the feasible set of the MPC problem is not automatically invariant. This means that the optimizer may choose a control action that is feasible over the current horizon but leads to a state from which the next optimization problem is infeasible. Thus, removing the terminal set may enlarge the one-step feasible set, but it removes the simple certificate of recursive feasibility.

8.3.2 MPC without Terminal Set

We now consider the same stabilizing MPC idea, but without imposing the terminal constraint $x_N \in \mathcal{X}_f$. The motivation is practical: as discussed above, the terminal constraint can substantially reduce the set of initial states for which the optimization problem is feasible. Removing it gives the controller more freedom and often enlarges the region in which a feasible input sequence exists.

The finite-horizon MPC problem without terminal set is

$$\begin{aligned} J^*(x(k)) = \min_U \quad & \sum_{i=0}^{N-1} \ell(x_i, u_i) + V_f(x_N) \\ \text{s.t.} \quad & x_{i+1} = Ax_i + Bu_i, \quad i = 0, \dots, N-1, \\ & x_i \in \mathcal{X}, \quad u_i \in \mathcal{U}, \quad i = 0, \dots, N-1, \\ & x_0 = x(k). \end{aligned}$$

Compared with the stabilizing MPC formulation of the previous chapter, the constraint

$$x_N \in \mathcal{X}_f$$

has been removed. The terminal cost $V_f(x_N)$ may still be present, but the optimizer is not forced to end the prediction horizon inside the terminal invariant set.

Geometrically, this can lead to a larger feasible set. With a terminal constraint, the current state must be such that the system can be driven into $\mathcal{X}_f$ in N steps while satisfying all state and input constraints. Without the terminal constraint, it is enough to find a trajectory that satisfies the state and input constraints over the finite prediction horizon. Therefore, the feasible set without terminal constraint is generally larger.

However, this enlargement comes at a price. The feasible set without terminal constraint is not automatically invariant. In the stabilizing construction, recursive feasibility was proved by shifting the optimal input sequence and appending the local terminal control law

$$u = \kappa_f(x).$$

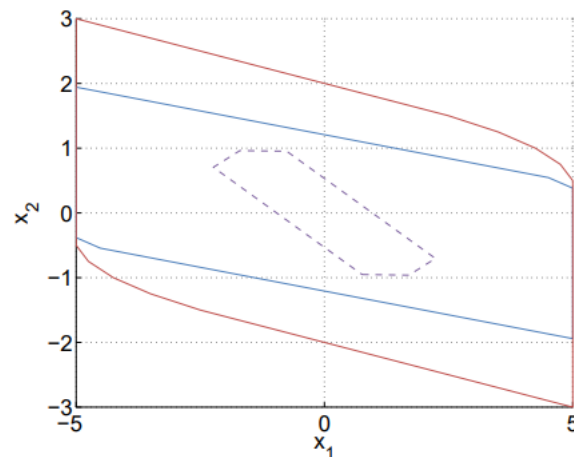


Figure 8.2: Comparison between the feasible set with and without terminal constraint. The blue line is the feasible set with terminal constraint. The red line is the feasible set without terminal constraint. The dashed line is the terminal set.

This argument relied crucially on the fact that the predicted terminal state belonged to the terminal invariant set $\mathcal{X}_f$. Without the terminal constraint, there is no guarantee that the last predicted state lies in a region from which feasibility can be continued. Therefore, the controller may be feasible at time k , but the state reached after applying the first optimal input may no longer admit a feasible solution at time $k + 1$.

The central question is then whether one can remove the terminal constraint while still retaining stability. The answer is yes, but only under stronger and less explicit conditions. The basic idea is that if the horizon N is sufficiently long, and if the initial state lies in a suitable subset of the feasible region, then the optimal finite-horizon solution may naturally drive the terminal state into the region where the terminal controller would have been valid, even though this is not enforced as a constraint.

In this case, the finite-horizon solution behaves like the infinite-horizon solution over the region of interest. The terminal constraint is no longer imposed explicitly, but the horizon is long enough for the optimizer to “see” the long-term consequences of its decisions. Hence the terminal state satisfies the desired terminal condition implicitly.

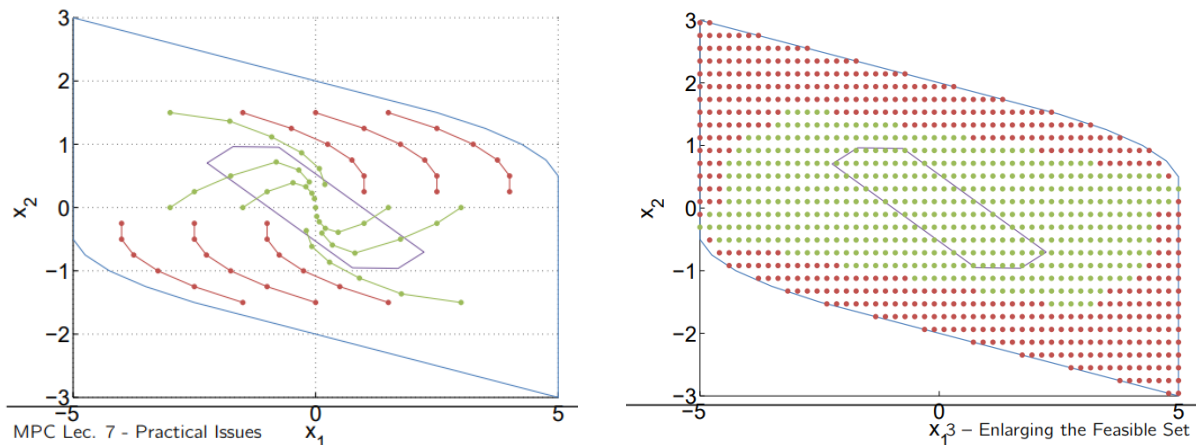
MPC without terminal set

An MPC problem without terminal constraint can still be stabilizing if

$$x(k)$$

belongs to a sufficiently small subset of the feasible set and the horizon N is sufficiently large. In that case, the optimal terminal state may enter the terminal region without this being explicitly enforced, and the finite-horizon MPC solution can coincide with the corresponding infinite-horizon behavior on that region.

This point is illustrated by comparing the behavior of optimal trajectories and the resulting feasible regions. For a short horizon, the optimizer may not look far enough ahead, and the absence of a terminal constraint can lead to poor long-term decisions. For a sufficiently large horizon, instead, the predicted trajectory may approach the terminal region even without being forced to do so.



(a) Predicted trajectories for a sufficiently long horizon.

(b) Region in which the terminal condition is implicitly satisfied.

Figure 8.3: MPC without terminal constraint can be stabilizing when the horizon is sufficiently large. The optimizer is not forced to end in $\mathcal{X}_f$, but for suitable initial conditions the optimal trajectory may still enter the terminal region implicitly.

The advantage of this formulation is clear: the controller may be defined on a larger feasible set, and the optimization problem contains fewer constraints. This can be particularly attractive when the terminal constraint introduces many additional inequalities, or when the original problem has only input constraints and the terminal set would introduce artificial state constraints.

The disadvantage is that the theoretical characterization becomes much harder. With a terminal set, recursive feasibility and stability follow from explicit and checkable sufficient conditions: terminal invariance and terminal Lyapunov decrease. Without a terminal set, the region of attraction is generally difficult to compute, and the horizon length required for stability is hard to specify a priori.

In practice, a common approach is therefore to choose a larger horizon and check the closed-loop behavior by simulation or by sampling many initial conditions. This does not give the same clean guarantee as the terminal-set construction, but it is often effective in applications. As the horizon length increases, the region of attraction of MPC without terminal constraint tends to approach the maximal control invariant set associated with the constraints. Intuitively, a longer horizon gives the optimizer more information about future constraint limitations and makes the finite-horizon problem closer to the infinite-horizon constrained control problem.

Practical trade-off

The terminal constraint is a sufficient condition for recursive feasibility and stability, but it may be conservative. Removing it can enlarge the feasible set and simplify the optimization problem, but the region of attraction and the required horizon length become difficult to characterize. In practice, one usually increases N and verifies stability and constraint satisfaction by sampling or simulation.

8.3.3 Soft-Constrained MPC

Motivation for soft constraints

We now turn to another important practical issue: what should the MPC controller do when some constraints cannot be satisfied exactly? In the nominal formulation, constraints are hard. This means that, if no predicted trajectory satisfies all constraints, the optimization problem

becomes infeasible and the controller is no longer defined. From a theoretical point of view this is clean, but from an implementation point of view it is often too rigid.

A useful distinction must be made between input constraints and state or output constraints. Input constraints are usually dictated by physical actuator limitations. For example, a motor cannot provide more torque than its maximum value, and a valve cannot open beyond its physical range. Such constraints are genuinely hard and should normally never be violated.

State and output constraints often have a different interpretation. They may describe desired operating ranges, comfort bounds, product-quality limits, or safety margins. These constraints are important, but in many applications a small and temporary violation is preferable to losing feasibility completely. This is especially true in the presence of disturbances, measurement noise, or model mismatch, where the measured or estimated state may move outside the nominal feasible range.

This is the motivation for soft constraints. Instead of requiring selected state or output constraints to hold exactly, one allows them to be violated by introducing slack variables. The violation is then penalized in the objective function. In this way the controller remains defined, but violations are made expensive.

The goal is therefore not to ignore constraints, but to handle violations in a controlled way. The controller should still respect hard actuator constraints, while state or output constraints can be softened if this is necessary to maintain feasibility.

Size and duration of constraint violations

When a state or output constraint cannot be satisfied exactly, there are two natural objectives. One would like to minimize the duration of the violation, and one would like to minimize the size of the violation. These two objectives are not always aligned.

A controller that focuses on removing the violation as quickly as possible may allow a large temporary peak violation. Conversely, a controller that keeps the violation very small may need more time to return fully inside the admissible region. Thus, reducing the size of the violation can come at the cost of increasing its duration.

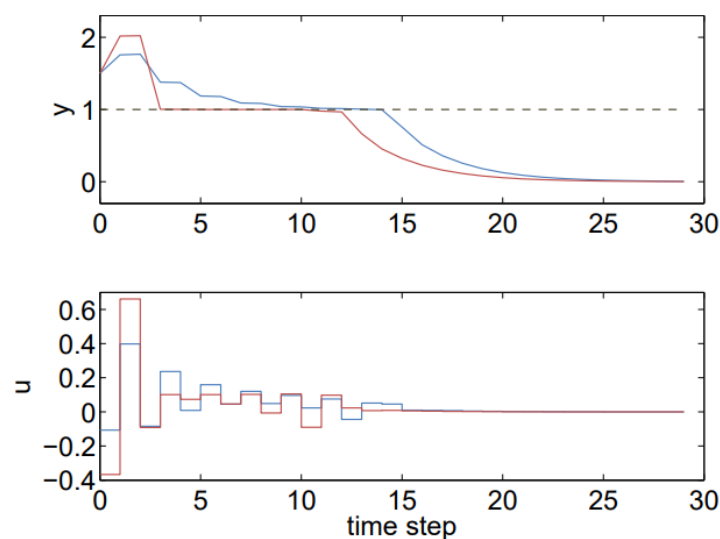


Figure 8.4: Tradeoff between duration and size of a constraint violation. The red response prioritizes minimum violation time, while the blue response prioritizes minimum violation size.

This leads naturally to a multi-objective viewpoint. For a given system, horizon length, and

initial condition, one can imagine plotting the achievable tradeoff between violation size and violation duration. The best operating points lie on a Pareto optimal curve. Points below this curve cannot be attained, while points above it are inferior because another controller achieves both smaller or equal size and smaller or equal duration.

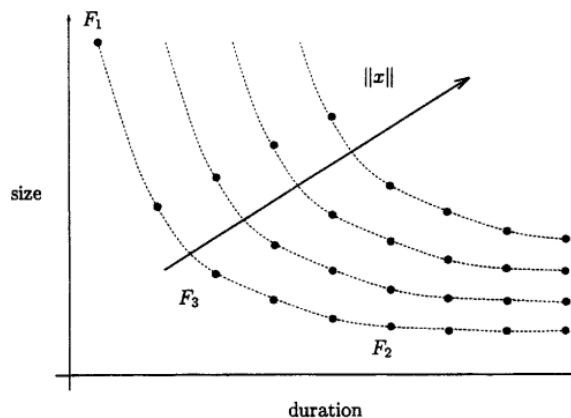


Figure 8.5: Multi-objective interpretation of soft constraints. The best operation points lie on the Pareto optimal size-duration curve. Which point is preferable depends on the application.

The appropriate choice depends on the application. If a product must be discarded whenever a quality constraint is violated, then minimizing the time spent in violation may be the most important objective. If, instead, a large violation can trigger a shutdown or damage the process, then minimizing the peak or total amount of violation may be more important. Soft-constrained MPC provides a systematic way to tune this tradeoff through the penalty assigned to the slack variables.

Soft-constrained MPC formulation

Consider again a linear prediction model

$$x_{i+1} = Ax_i + Bu_i.$$

Suppose that the input constraints are kept hard:

$$H_u u_i \leq k_u.$$

By contrast, suppose that the state constraints

$$H_x x_i \leq k_x$$

are softened. To do this, introduce slack variables

$$\epsilon_i \in \mathbb{R}^p, \quad \epsilon_i \geq 0,$$

and replace the hard state constraint by

$$H_x x_i \leq k_x + \epsilon_i.$$

The slack variable ϵ_i measures how much the state constraint is relaxed at prediction step i . If the original constraint can be satisfied, the desired behavior is $\epsilon_i = 0$. If the original constraint cannot be satisfied, then ϵ_i allows the optimization problem to remain feasible, but the violation

is penalized in the cost.

The soft-constrained MPC problem can be written as

$$\begin{aligned}
 \min_{U, \epsilon_0, \dots, \epsilon_N} \quad & \sum_{i=0}^{N-1} \left(x_i^\top Q x_i + u_i^\top R u_i + \ell_\epsilon(\epsilon_i) \right) + x_N^\top P x_N + \ell_\epsilon(\epsilon_N) \\
 \text{s.t.} \quad & x_{i+1} = A x_i + B u_i, \quad i = 0, \dots, N-1, \\
 & H_x x_i \leq k_x + \epsilon_i, \quad i = 0, \dots, N, \\
 & H_u u_i \leq k_u, \quad i = 0, \dots, N-1, \\
 & \epsilon_i \geq 0, \quad i = 0, \dots, N, \\
 & x_0 = x(k).
 \end{aligned}$$

Here $\ell_\epsilon(\epsilon_i)$ is the slack penalty. It determines how expensive constraint violations are and therefore shapes the closed-loop tradeoff between violation size and violation duration.

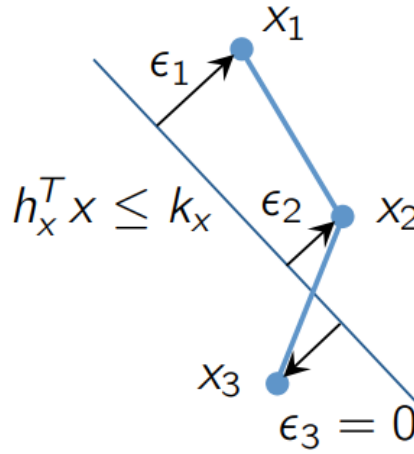


Figure 8.6: Soft-constrained MPC setup. State constraints are relaxed by introducing slack variables ϵ_i , while input constraints remain hard. The slack variables are penalized in the cost.

A purely quadratic slack penalty has the form

$$\ell_\epsilon(\epsilon_i) = \epsilon_i^\top S \epsilon_i, \quad S \succ 0.$$

A commonly used combined penalty adds a linear norm term:

$$\ell_\epsilon(\epsilon_i) = \epsilon_i^\top S \epsilon_i + v \|\epsilon_i\|_{1/\infty}.$$

The quadratic term is useful for tuning the magnitude of the violation and preserving a well-posed quadratic program. The linear term is important because it can make the soft constraint exact: if the original hard constraint is feasible, then the optimizer will choose zero slack, provided that the linear weight is sufficiently large.

Mathematical formulation of softening

To understand the role of the slack penalty, consider the abstract constrained optimization problem

$$\begin{aligned}
 \min_z \quad & f(z) \\
 \text{s.t.} \quad & g(z) \leq 0,
 \end{aligned}$$

where, for simplicity, $g(z)$ is scalar. The corresponding softened problem is

$$\begin{aligned} \min_{z, \epsilon} \quad & f(z) + \ell_\epsilon(\epsilon) \\ \text{s.t.} \quad & g(z) \leq \epsilon, \\ & \epsilon \geq 0. \end{aligned}$$

The slack variable ϵ relaxes the constraint. If $g(z) \leq 0$, then the original constraint is satisfied and one may choose $\epsilon = 0$. If $g(z) > 0$, then one can still make the softened problem feasible by choosing $\epsilon \geq g(z)$.

However, softening should not change the optimizer when violation is unnecessary. Therefore we impose the following requirement.

Requirement on the slack penalty

If the original hard-constrained problem has a feasible optimizer z^* , then the softened problem should have the same optimizer and zero slack:

$$(z^*, \epsilon^*) = (z^*, 0).$$

Thus, soft constraints should recover the original hard-constrained solution whenever the hard constraint can be satisfied.

This requirement is essential. Without it, the controller could decide to violate a constraint even when a nonviolating solution exists, simply because the violation improves the performance cost.

Quadratic penalty

A natural first idea is to penalize the slack quadratically:

$$\ell_\epsilon(\epsilon) = s\epsilon^2, \quad s > 0.$$

Although this is smooth and easy to optimize, it does not satisfy the exactness requirement in general. The reason is that the derivative of $s\epsilon^2$ at $\epsilon = 0$ is zero. Therefore, an infinitesimal violation has almost no first-order cost.

To see the issue, suppose the hard constraint is

$$g(z) := z - z^* \leq 0,$$

so that the feasible region is $z \leq z^*$, and suppose that the hard-constrained optimizer is z^* . If the objective $f(z)$ would decrease by moving slightly to the right of z^* , then the softened problem may prefer a point

$$z = z^* + \epsilon^*$$

with a small but nonzero slack $\epsilon^* > 0$. The improvement in f can dominate the small quadratic penalty near the origin.

Thus, a purely quadratic penalty is useful for tuning, but it is not an exact penalty. It may allow constraint violation even when the hard-constrained solution is feasible.

Linear penalty and exactness

A linear penalty behaves differently:

$$\ell_\epsilon(\epsilon) = v\epsilon, \quad v > 0.$$

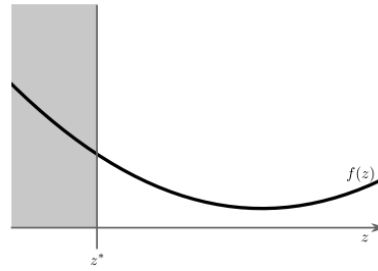


Figure 8.7: Hard-constrained problem. The grey region is the feasible set induced by $g(z) = z - z^* \leq 0$, and the constrained optimizer is z^* .

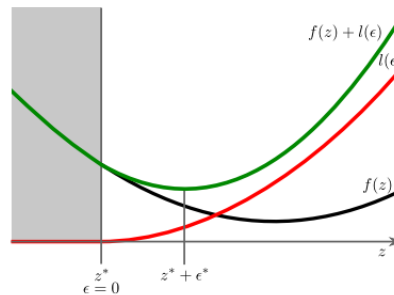


Figure 8.8: Quadratic slack penalty. Since a quadratic penalty has zero slope at the origin, the softened optimizer may move outside the hard feasible set and choose $(z^* + \epsilon^*, \epsilon^*)$ instead of $(z^*, 0)$.

Unlike the quadratic penalty, the linear penalty has nonzero slope at the origin. Therefore, constraint violation immediately carries a first-order cost. If v is chosen sufficiently large, then violating the constraint is more expensive than any local improvement in the original objective f , and the optimizer remains at the hard-constrained solution.

The precise condition is given by the optimal Lagrange multiplier of the original problem. Let $\lambda^* \geq 0$ be the multiplier associated with the constraint $g(z) \leq 0$. Then the linear penalty

$$\ell_\epsilon(\epsilon) = v\epsilon$$

is exact if

$$v > \lambda^*.$$

Exact penalty function

For the scalar constraint $g(z) \leq 0$, the penalty

$$\ell_\epsilon(\epsilon) = v\epsilon$$

satisfies the exactness requirement for any

$$v > \lambda^* \geq 0,$$

where λ^* is the optimal Lagrange multiplier of the original hard-constrained problem.

In practice, one often combines the exactness of the linear penalty with the tuning flexibility of the quadratic penalty:

$$\ell_\epsilon(\epsilon) = v\epsilon + s\epsilon^2, \quad v > \lambda^*, \quad s > 0.$$

For multiple constraints

$$g_j(z) \leq 0, \quad j = 1, \dots, r,$$

with slack vector

$$\epsilon = \begin{bmatrix} \epsilon_1 & \dots & \epsilon_r \end{bmatrix}^\top,$$

one can use

$$\ell_\epsilon(\epsilon) = v \|\epsilon\|_{1/\infty} + \epsilon^\top S \epsilon, \quad S \succ 0.$$

The condition becomes

$$v > \|\lambda^*\|_D,$$

where $\|\cdot\|_D$ is the dual norm of the chosen norm. In particular, the dual of $\|\cdot\|_1$ is $\|\cdot\|_\infty$, and the dual of $\|\cdot\|_\infty$ is $\|\cdot\|_1$.

Example system

We illustrate the effect of different slack penalties on a third-order nonminimum-phase system:

$$x(k+1) = \begin{bmatrix} 2 & -1.45 & 0.35 \\ 1 & 0 & 0 \\ 0 & 1 & 1 \end{bmatrix} x(k) + \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} u(k),$$

with output

$$y(k) = \begin{bmatrix} -1 & 0 & 2 \end{bmatrix} x(k).$$

The output constraint is

$$-1 \leq y(k) \leq 1,$$

and the controller parameters are

$$Q = C^\top C, \quad R = 1, \quad N = 20.$$

For the initial condition

$$x(0) = \begin{bmatrix} 0.8 & 0.8 & 0.8 \end{bmatrix}^\top,$$

the nominal controller can satisfy the output constraint and brings the system back inside the admissible range.

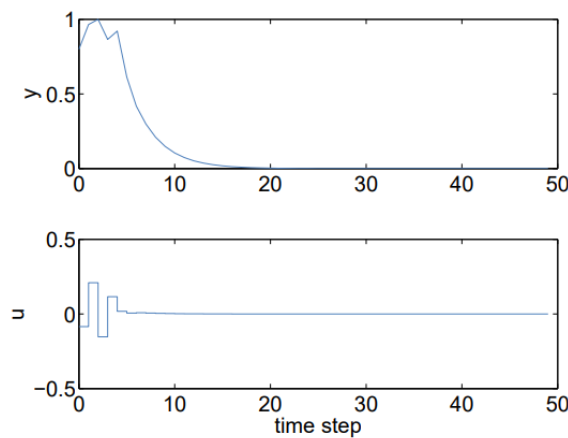


Figure 8.9: Third-order nonminimum-phase example used to illustrate soft constraints. The output constraint is $-1 \leq y \leq 1$, with $Q = C^\top C$, $R = 1$, and horizon $N = 20$.

Soft constraints with quadratic penalty

We first consider a purely quadratic slack penalty:

$$\ell_\epsilon(\epsilon_i) = \epsilon_i^\top S \epsilon_i.$$

This choice gives a well-posed quadratic program, because the cost remains quadratic and the Hessian can be made positive definite. Increasing S makes the violation more expensive and therefore hardens the soft constraint.

The simulation uses the initial condition

$$x(0) = \begin{bmatrix} 1.5 & 1.5 & 1.5 \end{bmatrix}^\top.$$

Different values of S lead to different violation profiles. A larger S reduces the size of the violation, but the violation tends to last longer. This is exactly the size-duration tradeoff discussed above.

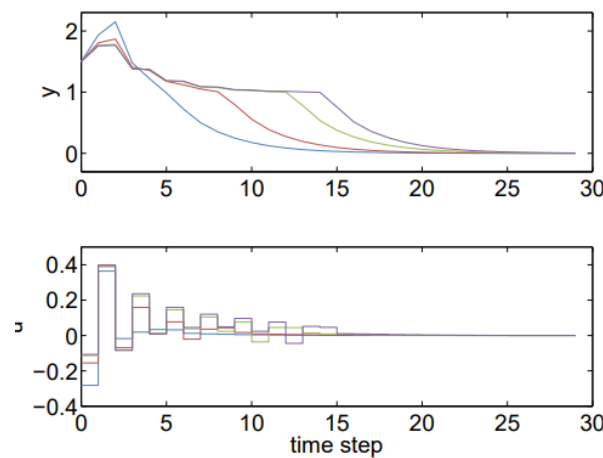


Figure 8.10: Soft constraints with quadratic penalty. Increasing S hardens the soft constraint: the size of the violation is reduced, but the duration of the violation can increase.

Thus, the quadratic weight S is a tuning parameter. Small values of S allow larger violations but may remove them quickly. Large values of S discourage large violations, but can make the controller more conservative and prolong the time spent near or above the constraint.

Soft constraints with quadratic and linear penalty

We now add a linear penalty term:

$$\ell_\epsilon(\epsilon_i) = \epsilon_i^\top S \epsilon_i + v \|\epsilon_i\|_1.$$

The linear term allows exact penalty behavior. If v is sufficiently large and the hard constraint can be satisfied, the optimizer chooses zero slack. Thus, constraints are satisfied whenever possible.

We first show a case with initial condition

$$x(0) = \begin{bmatrix} 0.95 & 0.95 & 0.95 \end{bmatrix}^\top,$$

penalty norm 1-norm, and quadratic weight $S = 20$. Increasing v makes violation more expensive at the origin and pushes the controller toward satisfying the constraint whenever this is possible.

The second case uses the larger initial condition

$$x(0) = \begin{bmatrix} 1.5 & 1.5 & 1.5 \end{bmatrix}^\top,$$

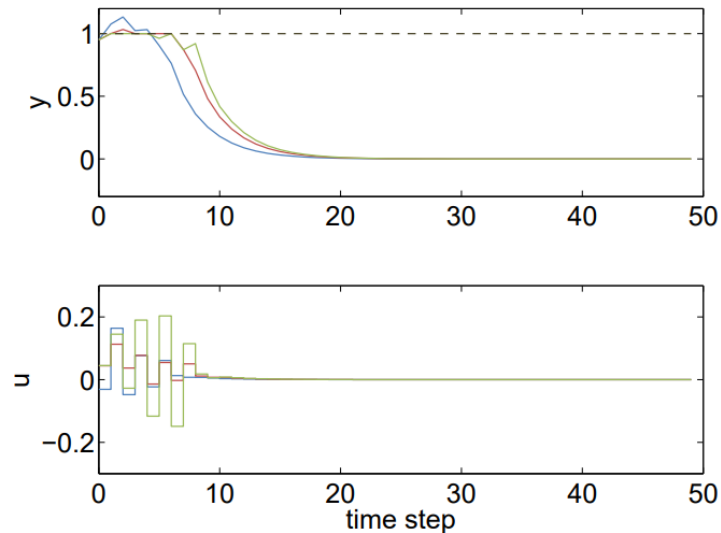


Figure 8.11: Soft constraints with combined quadratic and linear penalty for $x(0) = [0.95 \ 0.95 \ 0.95]^\top$. A sufficiently large linear weight v promotes exact satisfaction of the constraint whenever this is possible.

again with 1-norm penalty and $S = 20$. In this case the violation cannot be avoided completely. Increasing v changes the way the violation is distributed: larger linear penalties tend to increase peak violations but reduce their duration. Very large values of v also make the numerical tuning more delicate and can lead to ill-conditioned optimization problems.

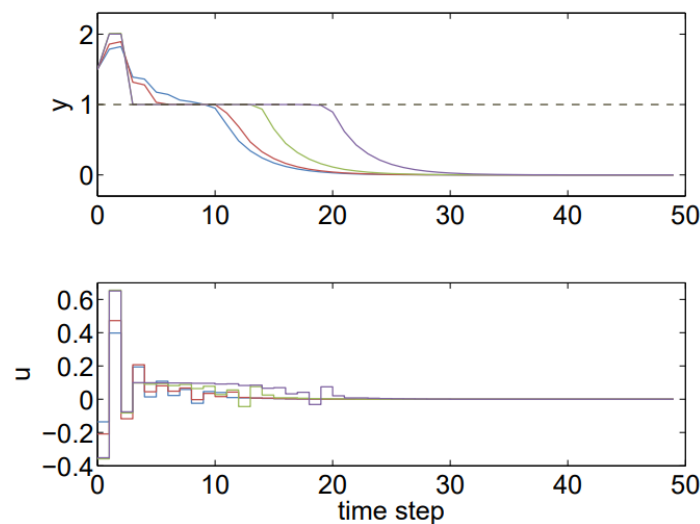


Figure 8.12: Soft constraints with combined quadratic and linear penalty for $x(0) = [1.5 \ 1.5 \ 1.5]^\top$. Increasing v tends to reduce violation duration but may increase peak violation and can make numerical tuning more difficult.

The role of the linear penalty is therefore twofold. It provides exactness when exact constraint satisfaction is possible, and it influences the size-duration tradeoff when constraint violation is unavoidable. However, v should not be chosen blindly large, since excessively large weights can cause numerical problems and make tuning difficult.

Simplification by separation of objectives

Instead of combining performance and violation penalties in one optimization problem, one can separate the objectives into two steps.

First, compute the smallest constraint relaxation over the horizon:

$$\begin{aligned} \epsilon^{\min} = \arg \min_{u, \epsilon} \quad & \sum_{i=0}^{N-1} \left(\epsilon_i^\top S \epsilon_i + v^\top \epsilon_i \right) \\ \text{s.t.} \quad & x_{i+1} = Ax_i + Bu_i, \\ & H_x x_i \leq k_x + \epsilon_i, \\ & H_u u_i \leq k_u, \\ & \epsilon_i \geq 0. \end{aligned}$$

Then, in a second optimization problem, optimize the controller performance subject to the minimal relaxation already computed:

$$\begin{aligned} \min_u \quad & \sum_{i=0}^{N-1} \left(x_i^\top Q x_i + u_i^\top R u_i \right) + x_N^\top P x_N \\ \text{s.t.} \quad & x_{i+1} = Ax_i + Bu_i, \\ & H_x x_i \leq k_x + \epsilon_i^{\min}, \\ & H_u u_i \leq k_u. \end{aligned}$$

The advantage is that tuning becomes simpler. The first problem focuses only on constraint violation, while the second problem focuses on performance given the smallest required relaxation. In particular, constraints are satisfied whenever possible. The disadvantage is that two optimization problems must be solved at each sampling time instead of one.

Soft constraints and stability

Soft constraints are extremely useful in practice because they recover feasibility when some state or output constraints cannot be satisfied exactly. They also provide a systematic way to tune the tradeoff between violation size and violation duration. For this reason, soft constraints are widely used in industrial MPC implementations.

At the same time, standard soft-constrained MPC formulations do not automatically provide the same stability guarantees as the hard-constrained stabilizing MPC construction from the previous chapter. This issue is particularly relevant for open-loop unstable systems. By allowing state or output constraints to be violated, the invariance-based arguments used for recursive feasibility and stability may no longer apply directly. Additional design ingredients are needed if one wants soft constraints together with formal stability guarantees.

Soft constraints: main message

Soft constraints recover feasibility when selected constraints cannot be satisfied exactly and allow a tunable tradeoff between violation size and violation duration. They are generally useful in practice, but standard soft-constrained MPC does not automatically guarantee stability for open-loop unstable systems.

8.4 Putting the Practical MPC Architecture Together

We can now summarize the practical MPC architecture developed throughout this chapter.

In general, the full state is not directly measured. Therefore, the controller is typically combined with an estimator, such as a Kalman filter, which produces an estimate $\hat{x}(k)$ of the current state. The MPC optimization is then initialized at the estimated state.

For reference tracking, the desired reference r is first translated into a steady-state target

$$(x_s, u_s).$$

The MPC problem is then rewritten in deviation variables

$$\Delta x = x - x_s, \quad \Delta u = u - u_s,$$

so that tracking becomes regulation around the target. The terminal weight and terminal constraint are selected as in the regulation case, but the terminal set may need to be scaled so that the shifted terminal set fits inside the constraints.

To achieve offset-free tracking, the model is augmented with a constant disturbance model, and the estimator is also augmented to estimate both $\hat{x}(k)$ and $\hat{d}(k)$. The target steady-state problem is then modified using the disturbance estimate, so that the MPC regulator tracks a disturbance-corrected equilibrium.

The feasible set can be enlarged by removing the terminal constraint, provided that a sufficiently long horizon is chosen and the closed-loop behavior is checked carefully. Finally, soft constraints can be introduced to ensure that the optimization problem remains feasible even when selected state or output constraints cannot be satisfied exactly.

8.5 Summary

This chapter extended the nominal MPC formulation toward the issues that arise in practical applications. The first extension was reference tracking. A desired output reference is converted into a constrained steady-state target (x_s, u_s) , and the MPC problem is written in deviation variables around this target. In this form, tracking becomes regulation, and the same terminal cost and terminal set ideas can be used, provided that the shifted terminal set and shifted terminal control law remain feasible.

The second extension was offset-free tracking. Constant disturbances and model mismatch can cause steady-state tracking errors if the controller uses only the nominal target equation. Offset-free MPC augments the model with integrating disturbance dynamics, estimates both the state and the disturbance, and modifies the target problem using the disturbance estimate. Under the appropriate convergence and feasibility assumptions, the controlled output then converges to the desired reference without offset.

The third topic was enlargement of the feasible set. Terminal constraints provide a powerful sufficient condition for recursive feasibility and stability, but they may reduce the feasible set. Removing the terminal constraint can enlarge the operating region and simplify the optimization problem, but the resulting feasible set is not automatically invariant and the required horizon length is difficult to characterize.

Finally, we introduced soft-constrained MPC. Slack variables relax selected state or output constraints and are penalized in the cost. This recovers feasibility when exact constraint satisfaction is impossible and gives a tunable tradeoff between violation size and violation duration. Quadratic penalties are smooth and easy to tune, while sufficiently large linear penalties can be exact. Soft constraints are therefore a central practical tool, although additional care is needed when formal stability guarantees are required.

Part IV

Robust MPC

Chapter 9

Robust MPC for Linear Systems

9.1 Motivation

The previous chapters developed MPC under the assumption that the prediction model used by the controller describes the plant exactly. At each sampling instant, the controller predicts the future evolution of the system, optimizes a finite-horizon cost, and applies the first input of the optimal sequence. This nominal viewpoint is useful and forms the basis of the theory developed so far, but it hides an important practical difficulty: real systems are never described perfectly by the model used inside the optimizer.

Model errors may have several origins. Some parameters may be unknown or only approximately identified, the mathematical model may neglect nonlinear effects, and the plant may be affected by disturbances or noise. Therefore, even if the nominal prediction satisfies all constraints, the real trajectory may deviate from it. This deviation can lead to constraint violations, degraded performance, or even loss of feasibility of the MPC problem at later time instants.

Robust MPC addresses this issue by explicitly incorporating uncertainty into the control design. The goal is no longer only to optimize the nominal predicted trajectory, but to guarantee that the real system behaves acceptably for all admissible uncertainty realizations. In particular, we would like the closed-loop system to satisfy constraints for all possible disturbances, to remain stable in a suitable sense, and to retain good performance despite model mismatch.

A key difference with nominal MPC is that persistent disturbances generally prevent convergence to a single equilibrium point. If the disturbance does not vanish asymptotically, then the state cannot be expected to converge exactly to the origin. The best one can typically hope for is convergence to a neighborhood of the origin, whose size depends on the magnitude of the uncertainty and on the robustness properties of the controller.

The central question of this chapter is therefore the following. If the controller predicts the future using a nominal model, but the real system is affected by bounded disturbances, how can we modify the MPC formulation so that the real state still respects the constraints?

9.2 Uncertainty Models

9.2.1 MPC predictions and the real system

In nominal predictive control, the system is assumed to evolve according to a deterministic model of the form

$$x(k+1) = g(x(k), u(k)).$$

This means that, once the current state and the future input sequence are fixed, the future state trajectory is uniquely determined. The optimizer can therefore compute a single predicted trajectory and impose constraints directly on that trajectory.

In the real world, however, the evolution of the system is better described by a model of the form

$$x(k+1) = g(x(k), u(k), w(k); \theta),$$

where $w(k)$ represents a time-varying disturbance or noise term, while θ denotes unknown model parameters. The disturbance $w(k)$ changes from one sampling instant to the next, whereas θ is often assumed to be unknown but constant, or slowly varying. This means that the future state is no longer uniquely determined by the current state and the planned input sequence. Instead, each possible realization of the uncertainty gives rise to a different trajectory.

This distinction is essential. In nominal MPC, one checks whether the predicted trajectory belongs to the constraint sets. In robust MPC, one must check whether all possible trajectories generated by the uncertainty belong to the constraint sets. Thus, the objects to be propagated are no longer single trajectories, but sets of possible trajectories.

9.2.2 Goals of robust constrained control

Consider the uncertain constrained system

$$x(k+1) = g(x(k), u(k), w(k); \theta),$$

with constraints

$$x(k) \in \mathcal{X}, \quad u(k) \in \mathcal{U},$$

and uncertainty sets

$$w(k) \in \mathcal{W}, \quad \theta \in \Theta.$$

The robust constrained control problem consists in designing a feedback law

$$u(k) = \kappa(x(k))$$

such that the closed-loop system satisfies the constraints for all admissible uncertainty realizations. More precisely, the first requirement is robust constraint satisfaction:

$$x(k) \in \mathcal{X}, \quad u(k) \in \mathcal{U}, \quad \forall k \geq 0,$$

for all possible disturbance sequences. The second requirement is stability. Since persistent disturbances may prevent convergence exactly to the origin, stability is usually interpreted as convergence to a neighborhood of the origin, or as boundedness of the closed-loop trajectory in a robust invariant region. The third requirement is performance. Depending on the available information, one may optimize an expected cost, a worst-case cost, or a nominal cost. Finally, one would like the controller to work from as large a set of initial states as possible.

These goals cannot be met without assumptions on the uncertainty. If the disturbance can be arbitrarily large, no controller can guarantee bounded constraint satisfaction. Robust MPC therefore requires a mathematical uncertainty model.

9.2.3 Common uncertainty models

Examples of uncertainty models

Several uncertainty models appear commonly in MPC. A first example is a constant measurement or input bias. In this case, the dynamics may be written as

$$g(x(k), u(k), w(k); \theta) = \tilde{g}(x(k), u(k)) + \theta,$$

where θ is unknown but constant. Such a model is useful when the plant has a systematic offset, for instance due to actuator bias or a constant disturbance.

A second important class is given by linear parameter-varying systems. In this case, the system matrices depend on a parameter vector $\theta(k)$, and the dynamics are written as

$$x(k+1) = \left(\sum_{j=0}^{n_\theta} \theta_j(k) A_j \right) x(k) + \left(\sum_{j=0}^{n_\theta} \theta_j(k) B_j \right) u(k),$$

where

$$\mathbf{1}^\top \theta(k) = 1, \quad \theta(k) \geq 0.$$

Thus the system matrices are convex combinations of a finite number of vertex systems. This description is useful when the dynamics vary within a known polytope of models.

A third possibility is additive stochastic noise:

$$x(k+1) = Ax(k) + Bu(k) + w(k),$$

where the probability distribution of $w(k)$ is known. In that case, one can formulate stochastic MPC problems, for example by minimizing expected cost or imposing chance constraints. In this chapter we focus on the deterministic robust counterpart of this last model, namely additive bounded noise.

Additive bounded noise

The uncertainty model considered in this chapter is

$$x(k+1) = Ax(k) + Bu(k) + w(k), \quad w(k) \in \mathcal{W}.$$

Here A and B are known, while the disturbance $w(k)$ is unknown, time-varying, and bounded inside the set $\mathcal{W}$. The set $\mathcal{W}$ is assumed to contain all possible disturbance values that may affect the system.

This model is simple but very important. It captures the idea that the nominal linear dynamics are correct up to an additive perturbation. It can also be used as a conservative approximation of more complicated effects, such as small nonlinearities, discretization errors, or unmodeled dynamics. The price paid for this simplicity is conservatism: if $\mathcal{W}$ is chosen too large, the resulting robust MPC controller may become overly cautious or even infeasible for many initial states.

A crucial feature of this model is that the noise is persistent. The disturbance does not necessarily become smaller over time, and therefore the uncertain trajectory need not approach the nominal trajectory. Instead, the possible deviation between the real and nominal trajectories accumulates through the dynamics.

9.3 Tools: Robust Invariance, Set Difference and Set Sum

9.3.1 Robust invariance and robust pre-sets

Reminder: positive invariance

Before introducing robust invariance, we recall the nominal notion of positive invariance. Consider an autonomous system

$$x(k+1) = g(x(k)),$$

or a closed-loop system obtained by applying a fixed feedback law,

$$x(k+1) = g(x(k), \kappa(x(k))).$$

A set $\mathcal{O}$ is called positively invariant if

$$x(k) \in \mathcal{O} \implies x(k+1) \in \mathcal{O}, \quad \forall k \geq 0.$$

Thus, once the state enters $\mathcal{O}$, it remains in $\mathcal{O}$ forever.

In MPC, invariant sets are useful because they provide terminal regions in which constraint satisfaction can be guaranteed indefinitely. If $\mathcal{X}_f \subseteq \mathcal{X}$ is invariant under a local controller κ , and if $\kappa(x) \in \mathcal{U}$ for all $x \in \mathcal{X}_f$, then reaching $\mathcal{X}_f$ at the end of the prediction horizon is sufficient to guarantee that the constraints can be respected after the horizon.

Robust positive invariant sets

In the presence of disturbances, the autonomous dynamics take the form

$$x(k+1) = g(x(k), w(k)), \quad w(k) \in \mathcal{W}.$$

The corresponding invariance notion must now hold for every possible disturbance.

Robust positive invariant set

A set $\mathcal{O}_{\mathcal{W}}$ is called a robust positive invariant set for

$$x(k+1) = g(x(k), w(k)), \quad w(k) \in \mathcal{W},$$

if

$$x \in \mathcal{O}_{\mathcal{W}} \implies g(x, w) \in \mathcal{O}_{\mathcal{W}} \quad \forall w \in \mathcal{W}.$$

The difference from nominal invariance is the universal quantifier over the disturbance. In the nominal case, the next state must remain in the set under one deterministic successor. In the robust case, every possible successor generated by the disturbance must remain in the set. For a closed-loop system

$$x(k+1) = g(x(k), \kappa(x(k)), w(k)),$$

the definition is applied to the autonomous disturbed dynamics induced by the controller κ . A robust invariant terminal set therefore guarantees that, once the state reaches that set, the controller can keep the state inside it despite all admissible disturbances.

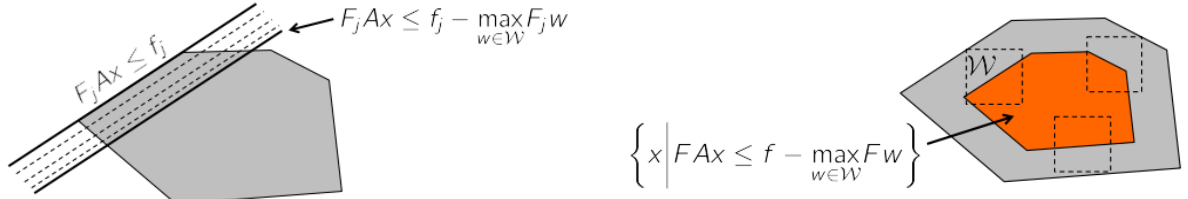
Robust pre-sets

The robust pre-set is the backward-reachability object associated with robust invariance. Given a target set Ω , the robust pre-set is the set of all states from which the next state belongs to Ω for every disturbance value:

$$\text{pre}_{\mathcal{W}}(\Omega) := \{x \mid g(x, w) \in \Omega \text{ for all } w \in \mathcal{W}\}.$$

Thus, $x \in \text{pre}_{\mathcal{W}}(\Omega)$ means that no matter which admissible disturbance occurs at the next step, the successor state is guaranteed to lie in Ω .

This definition is the robust counterpart of the nominal pre-set. The nominal pre-set only asks whether the deterministic successor lies in the target set. The robust pre-set asks whether the entire set of possible successors lies in the target set.



(a) Tightening of one supporting halfspace of the target set.

(b) Robust pre-set obtained after tightening all inequalities.

Figure 9.1: Geometric interpretation of the robust pre-set for the disturbed linear system $x(k+1) = Ax(k) + w(k)$. Each inequality $F_j x \leq f_j$ defining the target set $\Omega = \{x \mid Fx \leq f\}$ is tightened by the worst-case disturbance in the corresponding normal direction, leading to $F_j A x \leq f_j - \max_{w \in \mathcal{W}} F_j w$. Applying this row-wise gives $\text{pre}_{\mathcal{W}}(\Omega) = \{x \mid F A x \leq f - h_{\mathcal{W}}(F)\}$.

Robust pre-sets for linear systems

Consider now the linear disturbed system

$$x(k+1) = Ax(k) + w(k), \quad w(k) \in \mathcal{W},$$

and let the target set be the polyhedron

$$\Omega := \{x \in \mathbb{R}^n \mid Fx \leq f\}.$$

The robust pre-set is

$$\begin{aligned} \text{pre}_{\mathcal{W}}(\Omega) &= \{x \mid Ax + w \in \Omega, \forall w \in \mathcal{W}\} \\ &= \{x \mid F(Ax + w) \leq f, \forall w \in \mathcal{W}\} \\ &= \{x \mid F A x + F w \leq f, \forall w \in \mathcal{W}\}. \end{aligned}$$

For each row F_i of F , the condition becomes

$$F_i A x + F_i w \leq f_i \quad \forall w \in \mathcal{W}.$$

This is equivalent to requiring the inequality to hold for the worst possible disturbance in the direction F_i :

$$F_i A x \leq f_i - \max_{w \in \mathcal{W}} F_i w.$$

Therefore,

$$\text{pre}_{\mathcal{W}}(\Omega) = \{x \mid F A x \leq f - h_{\mathcal{W}}(F)\},$$

where $h_{\mathcal{W}}$ denotes the support function of $\mathcal{W}$, applied row-wise:

$$h_{\mathcal{W}}(F)_i := \max_{w \in \mathcal{W}} F_i w.$$

This formula is one of the central algebraic tools of robust MPC. It shows that robust constraint satisfaction can be converted into nominal constraint satisfaction with tightened right-hand sides. The amount of tightening is determined by the support function of the disturbance set.

Geometric condition for robust invariance

Robust invariance can be expressed through robust pre-sets exactly as in the nominal case.

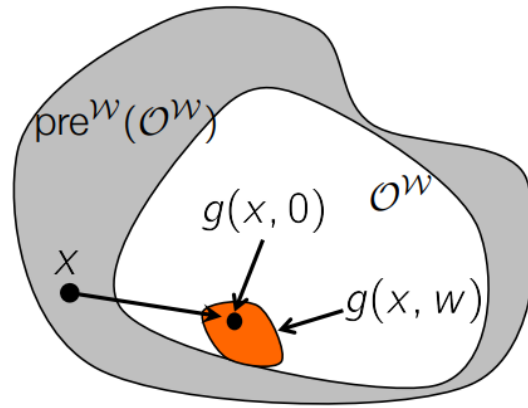


Figure 9.2: Geometric interpretation of the condition $\mathcal{O}_W \subseteq \text{pre}_W(\mathcal{O}_W)$. The white region represents the set $\mathcal{O}_W$, while the gray region represents its robust pre-set $\text{pre}_W(\mathcal{O}_W)$. For a state x to belong to the robust pre-set, all possible successors $g(x, w)$, for $w \in \mathcal{W}$, must remain inside $\mathcal{O}_W$. The orange set illustrates the collection of possible successor states generated by the admissible disturbances. Since this whole set lies inside $\mathcal{O}_W$, the point x belongs to $\text{pre}_W(\mathcal{O}_W)$.

Geometric condition for robust invariance

A set $\mathcal{O}_W$ is robust positively invariant for

$$x(k+1) = g(x(k), w(k)), \quad w(k) \in \mathcal{W},$$

if and only if

$$\mathcal{O}_W \subseteq \text{pre}_W(\mathcal{O}_W).$$

The interpretation is direct. If every point of $\mathcal{O}_W$ belongs to the robust pre-set of $\mathcal{O}_W$, then every point in the set is mapped back into the set for all admissible disturbances. Figure 9.2 illustrates this idea geometrically: a state belongs to the robust pre-set if the entire set of possible one-step successors remains inside $\mathcal{O}_W$.

Conversely, if there exists a point in $\mathcal{O}_W$ that is not in the robust pre-set, then some admissible disturbance drives the successor outside $\mathcal{O}_W$, and the set is not robust invariant.

Computing robust invariant sets

The maximal robust invariant set can be computed conceptually through an iterative procedure. Starting from the constraint set,

$$\Omega_0 := \mathcal{X},$$

one recursively removes all states from which the next state cannot be guaranteed to remain in the current candidate set:

$$\Omega_{i+1} := \text{pre}_W(\Omega_i) \cap \Omega_i.$$

The sequence is nested:

$$\Omega_{i+1} \subseteq \Omega_i.$$

At each iteration, the set is made smaller by discarding states that may leave the current set under some admissible disturbance. If the sequence reaches a fixed point,

$$\Omega_{i+1} = \Omega_i,$$

then the algorithm has found a robust positively invariant set. In fact, under the usual conditions, the fixed point is the maximal robust positively invariant subset of $\mathcal{X}$.

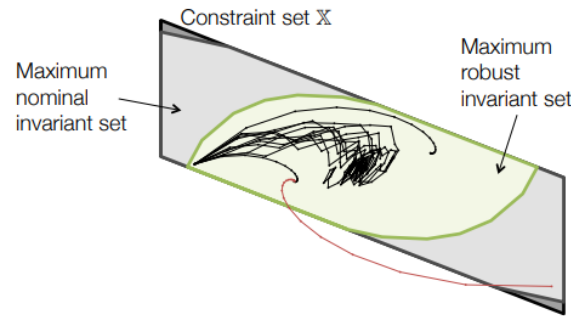


Figure 9.3: Comparison between the constraint set, the maximal nominal invariant set, and the maximal robust invariant set. Robust invariance requires safety for all disturbance realizations, and therefore leads to a smaller admissible set.

Conceptual algorithm for a robust invariant set

Initialize	$\Omega_0 \leftarrow \mathcal{X}.$
For $i = 0, 1, 2, \dots$, compute	$\Omega_{i+1} \leftarrow \text{pre}_{\mathcal{W}}(\Omega_i) \cap \Omega_i.$
If	$\Omega_{i+1} = \Omega_i,$
return	$\mathcal{O}_{\mathcal{W},\infty} = \Omega_i.$

This is the same algorithm used in the nominal case, with the ordinary pre-set replaced by the robust pre-set. The robust version is more conservative because a state is kept only if all possible disturbed successors remain admissible.

The distinction between nominal and robust invariance is illustrated by the closed-loop disturbed system

$$x(k+1) = (A + BK)x(k) + w(k),$$

with

$$A = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}, \quad B = \begin{bmatrix} 1 \\ 0.5 \end{bmatrix},$$

and constraints

$$\mathcal{X} = \{x \mid \|x\|_{\infty} \leq 5, \|Kx\|_{\infty} \leq 1\}, \quad \mathcal{W} = \{w \mid \|w\|_{\infty} \leq 0.3\}.$$

Here K is chosen as an LQR controller. If one ignores the disturbance and computes the maximal nominal invariant set, one obtains a region in which the nominal closed-loop trajectory remains feasible. However, this set is not necessarily safe for the disturbed system. A state may belong to the nominal invariant set and still be driven outside the constraints by an admissible disturbance.

The maximal robust invariant set is therefore smaller. It contains only those states for which the entire disturbance tube remains inside the constraints. This shrinking is not a defect of the method; it is the price paid for a guarantee that holds for all disturbances in $\mathcal{W}$.

9.3.2 Minkowski sum and Pontryagin difference

To describe robust constraint tightening compactly, two set operations are especially useful: the Minkowski sum and the Pontryagin difference.

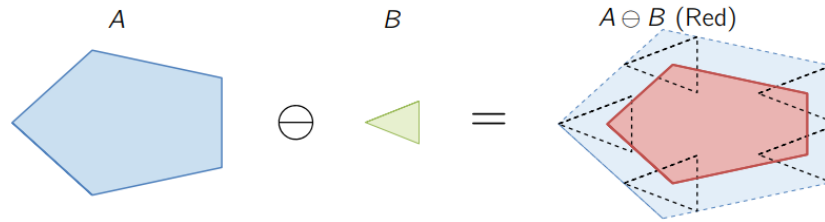


Figure 9.4: The Pontryagin difference tightens a set so that all admissible additions remain feasible.

Minkowski sum

Let $\mathcal{A}, \mathcal{B} \subseteq \mathbb{R}^n$. Their Minkowski sum is

$$\mathcal{A} \oplus \mathcal{B} := \{x + y \mid x \in \mathcal{A}, y \in \mathcal{B}\}.$$

The Minkowski sum describes all possible sums of one element from $\mathcal{A}$ and one element from $\mathcal{B}$. In robust MPC, it naturally appears when adding the nominal state to the possible disturbance offsets.

Pontryagin difference

Let $\mathcal{A}, \mathcal{B} \subseteq \mathbb{R}^n$. Their Pontryagin difference is

$$\mathcal{A} \ominus \mathcal{B} := \{x \mid x + e \in \mathcal{A} \text{ for all } e \in \mathcal{B}\}.$$

The Pontryagin difference is a constraint-tightening operation. If

$$x \in \mathcal{A} \ominus \mathcal{B},$$

then adding any element of $\mathcal{B}$ to x still keeps the result inside $\mathcal{A}$. Therefore, if $\mathcal{B}$ represents the set of possible errors between the real and nominal state, then imposing

$$x_{\text{nom}} \in \mathcal{A} \ominus \mathcal{B}$$

guarantees

$$x_{\text{nom}} + e \in \mathcal{A} \quad \forall e \in \mathcal{B}.$$

9.4 Impact of Bounded Additive Noise

9.4.1 Uncertain state evolution

Set-valued state evolution

We now return to the uncertain linear system

$$x(k+1) = Ax(k) + Bu(k) + w(k), \quad w(k) \in \mathcal{W}.$$

Given the current state $x(0) = x_0$ and a planned input sequence

$$U := \{u_0, u_1, \dots, u_{N-1}\},$$

the future state is not a single point, because it depends on the unknown disturbance sequence

$$W := \{w_0, w_1, \dots, w_{N-1}\}.$$

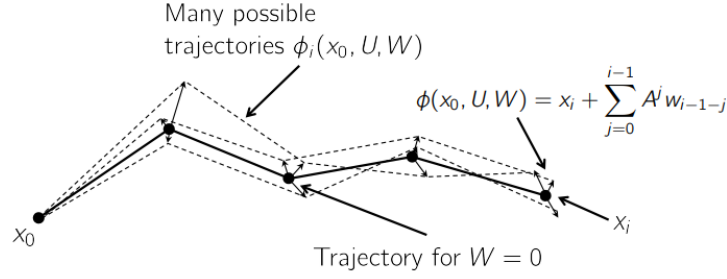


Figure 9.5: Under additive disturbances, a fixed input sequence gives rise to many possible trajectories. The nominal trajectory corresponds to the special case $W = 0$.

We denote by

$$\phi_i(x_0, U, W)$$

the state reached at time i when starting from x_0 , applying the input sequence U , and observing the disturbance sequence W .

For a fixed input sequence, each possible disturbance realization produces a different trajectory. Thus, at time i , the controller must reason about a set of possible states rather than a single predicted state.

Nominal and uncertain predictions

For the nominal system

$$x(k+1) = Ax(k) + Bu(k),$$

successive substitution gives

$$x_1 = Ax_0 + Bu_0,$$

$$x_2 = A^2x_0 + ABu_0 + Bu_1,$$

and, in general,

$$x_i = A^i x_0 + \sum_{j=0}^{i-1} A^j Bu_{i-1-j}.$$

For the uncertain system

$$x(k+1) = Ax(k) + Bu(k) + w(k),$$

the first steps are

$$\phi_1 = Ax_0 + Bu_0 + w_0,$$

$$\phi_2 = A^2x_0 + ABu_0 + Bu_1 + Aw_0 + w_1.$$

Continuing in the same way yields

$$\phi_i(x_0, U, W) = A^i x_0 + \sum_{j=0}^{i-1} A^j Bu_{i-1-j} + \sum_{j=0}^{i-1} A^j w_{i-1-j}.$$

Using the nominal prediction x_i , this can be written compactly as

$$\phi_i(x_0, U, W) = x_i + \sum_{j=0}^{i-1} A^j w_{i-1-j}.$$

Therefore, the uncertain state is equal to the nominal state plus an additive offset caused by the accumulated disturbances.

This decomposition follows from linearity and is the key observation behind constraint tightening. The nominal part depends on the chosen input sequence. The disturbance offset depends only on the disturbance set and on the powers of A .

9.4.2 Robust constrained control under bounded additive noise

Goals under bounded additive noise

We now return to the robust constrained control problem for the uncertain linear system

$$x(k+1) = Ax(k) + Bu(k) + w(k), \quad x(k) \in \mathcal{X}, \quad u(k) \in \mathcal{U}, \quad w(k) \in \mathcal{W}.$$

The objective is to design a feedback law

$$u(k) = \kappa(x(k))$$

such that the closed-loop system satisfies the constraints for all possible disturbance realizations, remains stable in an appropriate robust sense, and achieves good performance.

The essential difficulty is that, once the disturbance is present, the future state is no longer known exactly. Even if the current state x_0 and the future input sequence

$$U = \{u_0, \dots, u_{N-1}\}$$

are fixed, different disturbance sequences

$$W = \{w_0, \dots, w_{N-1}\}$$

produce different future trajectories. Therefore, the controller cannot predict a single future state at time i , but only a set of possible states. The robust MPC design must then ensure that all of these possible states remain inside the constraints.

The guiding idea is to construct a controller that satisfies constraints and stabilizes the system for all admissible disturbances. In this chapter we do this by keeping the nominal prediction model inside the MPC problem, but by tightening the constraints imposed on the nominal trajectory. The tightening is chosen so that the real disturbed trajectory remains feasible even though the optimizer only propagates the nominal dynamics.

Defining a cost to minimize

In nominal MPC, the finite-horizon cost assigns a value to a single predicted trajectory. For example, for a planned input sequence U , one writes

$$J(x_0, U) := \sum_{i=0}^{N-1} \ell(x_i, u_i) + \ell_f(x_N),$$

where x_i denotes the nominal state predicted from

$$x_{i+1} = Ax_i + Bu_i, \quad x_0 = x(0).$$

In the uncertain case, however, the state at each time depends on the disturbance sequence. Consequently, each possible disturbance realization produces a different trajectory and therefore a different cost. The cost becomes

$$J(x_0, U, W) := \sum_{i=0}^{N-1} \ell(\phi_i(x_0, U, W), u_i) + \ell_f(\phi_N(x_0, U, W)),$$

where $\phi_i(x_0, U, W)$ denotes the uncertain state reached at time i .

To obtain an optimization problem that can be solved online, the dependence on the unknown

disturbance sequence W must be removed or summarized. There are several possible choices. One may minimize the expected value

$$J_N(x_0, U) := \mathbb{E}[J(x_0, U, W)],$$

which requires probabilistic information about the disturbance. Alternatively, one may minimize the worst-case cost

$$J_N(x_0, U) := \max_{W \in \mathcal{W}^N} J(x_0, U, W),$$

which leads to a min-max robust optimal control problem. A simpler possibility, and the one used in this lecture, is to optimize the nominal cost

$$J_N(x_0, U) := J(x_0, U, 0).$$

Thus, in the robust open-loop MPC formulation developed here, robustness is enforced through the constraints, while the performance objective is evaluated on the nominal trajectory.

Robust constraint satisfaction

The main robust requirement is that the state and input constraints must be satisfied for every admissible disturbance sequence. For the uncertain system

$$\phi_{i+1} = A\phi_i + Bu_i + w_i,$$

this means that along the prediction horizon one would like to impose

$$\phi_i(x_0, U, W) \in \mathcal{X}, \quad u_i \in \mathcal{U}, \quad \forall W \in \mathcal{W}^N.$$

The input constraint is straightforward in the open-loop formulation, because the planned input sequence U is chosen directly by the optimizer and does not depend on the disturbance. The state constraint is more difficult, because ϕ_i is not a single point but a set of possible states. The key idea is shown in Figure 9.6. Instead of requiring only the nominal predicted state to lie in the original constraint set $\mathcal{X}$, we require the nominal state to lie in a tighter set. The tightening is chosen so that, after adding any admissible disturbance effect, the real state remains inside the original constraint set.

This point of view separates the MPC prediction into two parts. Over the finite horizon $i = 0, \dots, N-1$, the optimizer chooses the input sequence

$$U = \{u_0, \dots, u_{N-1}\}$$

and constraints are enforced explicitly on all possible disturbed predictions. At the end of the horizon, one imposes a terminal constraint

$$\phi_N(x_0, U, W) \in \mathcal{X}_f \quad \forall W \in \mathcal{W}^N,$$

where $\mathcal{X}_f \subseteq \mathcal{X}$ is chosen as a robust positively invariant set for the post-horizon dynamics. In the simple open-loop construction considered here, after the horizon one assumes no further planned input, i.e. $u_i = 0$ for $i \geq N$, and therefore the post-horizon system is

$$\phi_{i+1} = A\phi_i + w_i.$$

If $\mathcal{X}_f$ is robust positively invariant for this system and $0 \in \mathcal{U}$, then once the terminal state is inside $\mathcal{X}_f$, the constraints can be maintained indefinitely despite the disturbance.

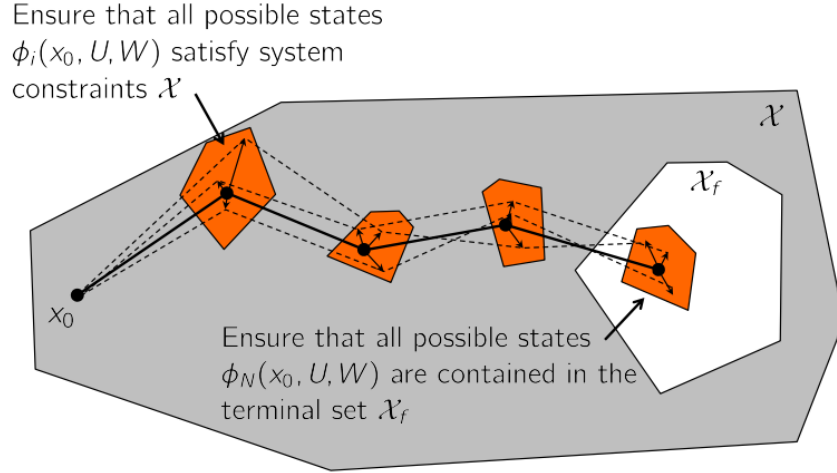


Figure 9.6: Robust constraint satisfaction through constraint tightening. The nominal trajectory is required to satisfy tighter constraints so that the entire set of possible disturbed states remains inside the original constraint set $\mathcal{X}$. At the terminal time, all possible terminal states must be contained in the robust terminal set $\mathcal{X}_f$.

Constraint tightening along the horizon

We now derive the tightened constraints explicitly. From the linearity of the dynamics, the uncertain state at prediction step i can be written as

$$\phi_i(x_0, U, W) = x_i + \sum_{j=0}^{i-1} A^j w_{i-1-j},$$

where x_i is the nominal state obtained from

$$x_{i+1} = Ax_i + Bu_i, \quad x_0 = x(0).$$

The goal is to ensure

$$\phi_i(x_0, U, W) \in \mathcal{X} \quad \forall W \in \mathcal{W}^i.$$

Equivalently, the set

$$\left\{ x_i + \sum_{j=0}^{i-1} A^j w_{i-1-j} \mid w_0, \dots, w_{i-1} \in \mathcal{W} \right\}$$

must be contained in $\mathcal{X}$.

Assume that the state constraint set is polyhedral,

$$\mathcal{X} = \{x \mid Fx \leq f\}.$$

Then robust constraint satisfaction at prediction step i is equivalent to

$$Fx_i + F \sum_{j=0}^{i-1} A^j w_{i-1-j} \leq f \quad \forall W \in \mathcal{W}^i.$$

For each row of F , we must protect against the largest possible disturbance contribution. Thus,

$$Fx_i \leq f - \max_{W \in \mathcal{W}^i} F \sum_{j=0}^{i-1} A^j w_{i-1-j}.$$

The maximization is understood row-wise. In terms of support functions, this can be written as

$$Fx_i \leq f - h_{\mathcal{W}^i} \left(F \sum_{j=0}^{i-1} A^j \right),$$

or, more transparently, as a tightening by the set of all possible disturbance offsets. The important conclusion is that robust satisfaction of the original constraint can be enforced by imposing a tighter constraint on the nominal state x_i . The support-function terms depend only on A , F , and $\mathcal{W}$, and can therefore be precomputed offline.

Constraint tightening

For the disturbed linear system

$$x(k+1) = Ax(k) + Bu(k) + w(k), \quad w(k) \in \mathcal{W},$$

robust satisfaction of $x_i \in \mathcal{X}$ can be enforced by requiring the nominal prediction x_i to lie in a tightened constraint set. The tightening accounts for all disturbance effects that can accumulate during the first i prediction steps.

Terminal state constraint

The same argument applies to the terminal constraint. At the end of the horizon, we require not only the nominal state x_N , but every possible disturbed terminal state, to lie in the terminal set:

$$\phi_N(x_0, U, W) \in \mathcal{X}_f \quad \forall W \in \mathcal{W}^N.$$

Using the same decomposition,

$$\phi_N(x_0, U, W) = x_N + \sum_{j=0}^{N-1} A^j w_{N-1-j},$$

this condition is enforced by tightening the terminal constraint in exactly the same way as the intermediate state constraints. Hence the nominal terminal state must be chosen sufficiently inside $\mathcal{X}_f$ so that all possible disturbed terminal states remain in $\mathcal{X}_f$.

Set notation and disturbance reachable sets

The previous construction can be written compactly using Minkowski sums and the Pontryagin difference. Define the disturbance reachable set at prediction step i as

$$\mathcal{F}_i := \bigoplus_{j=0}^{i-1} A^j \mathcal{W} = \mathcal{W} \oplus A\mathcal{W} \oplus \cdots \oplus A^{i-1}\mathcal{W}, \quad \mathcal{F}_0 = \{0\}.$$

This is the set of all possible deviations between the uncertain and nominal state after i steps. Indeed,

$$\phi_i(x_0, U, W) \in x_i \oplus \mathcal{F}_i.$$

Therefore, the condition that every possible disturbed state lies inside $\mathcal{X}$ becomes

$$x_i \oplus \mathcal{F}_i \subseteq \mathcal{X}.$$

By the definition of the Pontryagin difference, this is equivalent to

$$x_i \in \mathcal{X} \ominus \mathcal{F}_i.$$

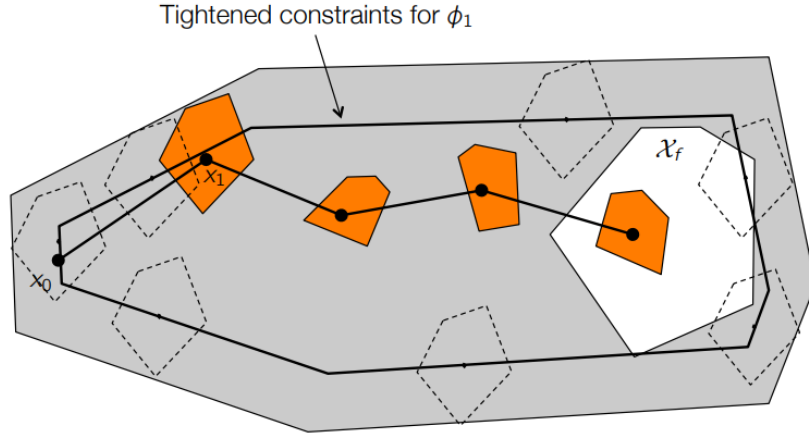


Figure 9.7: Constraint tightening along the prediction horizon. Around each nominal predicted state x_i , the disturbance reachable set describes the possible displacement of the real state. If the nominal state satisfies the tightened constraint $x_i \in \mathcal{X} \ominus \mathcal{F}_i$, then the whole uncertain set $x_i \oplus \mathcal{F}_i$ remains inside the original constraint set $\mathcal{X}$.

Thus the robust state constraint can be written as

$$x_i \in \mathcal{X} \ominus \left(\bigoplus_{j=0}^{i-1} A^j \mathcal{W} \right).$$

Similarly, the robust terminal condition is

$$x_N \in \mathcal{X}_f \ominus \mathcal{F}_N.$$

The disturbance reachable sets satisfy the recursion

$$\mathcal{F}_{i+1} = A\mathcal{F}_i \oplus \mathcal{W}, \quad \mathcal{F}_0 = \{0\}.$$

This recursion is useful in implementations, because the sets $\mathcal{F}_i$, and therefore the tightened constraints, can be computed offline before solving the MPC problem online.

9.5 Robust Open-Loop MPC

9.5.1 Putting the ingredients together

We can now combine the nominal cost, the nominal prediction dynamics, and the tightened constraints. At time k , robust open-loop MPC solves

$$\begin{aligned} \min_U \quad & \sum_{i=0}^{N-1} \ell(x_i, u_i) + \ell_f(x_N) \\ \text{subj. to} \quad & x_{i+1} = Ax_i + Bu_i, \quad i = 0, \dots, N-1, \\ & x_0 = x(k), \\ & x_i \in \mathcal{X} \ominus (\mathcal{W} \oplus A\mathcal{W} \oplus \dots \oplus A^{i-1}\mathcal{W}), \quad i = 1, \dots, N-1, \\ & u_i \in \mathcal{U}, \quad i = 0, \dots, N-1, \\ & x_N \in \mathcal{X}_f \ominus (\mathcal{W} \oplus A\mathcal{W} \oplus \dots \oplus A^{N-1}\mathcal{W}). \end{aligned}$$

Using the shorthand $\mathcal{F}_i = \bigoplus_{j=0}^{i-1} A^j \mathcal{W}$, this becomes

$$\begin{aligned} \min_U \quad & \sum_{i=0}^{N-1} \ell(x_i, u_i) + \ell_f(x_N) \\ \text{subj. to} \quad & x_{i+1} = Ax_i + Bu_i, \quad i = 0, \dots, N-1, \\ & x_0 = x(k), \\ & x_i \in \mathcal{X} \ominus \mathcal{F}_i, \quad i = 1, \dots, N-1, \\ & u_i \in \mathcal{U}, \quad i = 0, \dots, N-1, \\ & x_N \in \mathcal{X}_f \ominus \mathcal{F}_N. \end{aligned}$$

Here $\mathcal{X}_f \subseteq \mathcal{X}$ is a robust positively invariant set for the autonomous disturbed system

$$x(k+1) = Ax(k) + w(k), \quad w(k) \in \mathcal{W}.$$

Robust open-loop MPC

Robust open-loop MPC solves a nominal finite-horizon optimal control problem, but imposes tightened state and terminal constraints. If the nominal trajectory satisfies the tightened constraints, then every disturbed trajectory generated by $w(k) \in \mathcal{W}$ satisfies the original constraints.

The name “open-loop” refers to the structure of the prediction problem. The optimizer chooses an input sequence

$$U = \{u_0, \dots, u_{N-1}\}$$

without allowing the later inputs $u_1, \dots, u_{N-1}$ to depend on future disturbed states. After solving the problem, MPC is still implemented in receding-horizon fashion:

$$u(k) = u_0^*(x(k)).$$

At the next sampling instant, the new state is measured and the optimization problem is solved again.

9.5.2 Interpretation and limitations

The robust open-loop construction has a simple interpretation. We perform nominal MPC, but we do not allow the nominal trajectory to approach the boundary of the true constraint set. Instead, we keep a safety margin around it. This safety margin is exactly the disturbance reachable set $\mathcal{F}_i$. If the nominal state x_i lies in the tightened set $\mathcal{X} \ominus \mathcal{F}_i$, then every possible disturbed state lies in the original set $\mathcal{X}$.

This approach gives a systematic way to guarantee robust constraint satisfaction, but it may be conservative. The reason is that the planned future inputs are open-loop: they cannot react to the disturbances that will be observed in the future. The controller does re-optimize at the next sampling instant, so feedback exists at the closed-loop level, but the prediction problem itself does not optimize over feedback policies.

As a result, robust open-loop MPC can have a small feasible region, or region of attraction, especially when the disturbance set $\mathcal{W}$ is large or the horizon is long. More advanced robust MPC schemes reduce this conservatism by explicitly introducing feedback in the prediction model, for example through tube-based MPC. The tools developed in this chapter, namely robust invariance, disturbance reachable sets, Pontryagin difference, and constraint tightening, are the basic ingredients for those methods.

9.6 Closed-Loop Predictions

9.6.1 Why open-loop robust predictions are conservative

The robust open-loop MPC formulation introduced in the previous section already provides a systematic way to guarantee constraint satisfaction under bounded additive disturbances. However, it does so by optimizing over a fixed future input sequence

$$U = \{u_0, \dots, u_{N-1}\}.$$

This means that, inside the prediction problem, the controller commits today to all future control moves. The future inputs are not allowed to depend on the disturbances that will be observed along the horizon.

This assumption is conservative. In reality, MPC is implemented in closed loop: after applying the first input, the state is measured again and the optimization is solved again. Therefore, if a disturbance pushes the system away from the predicted trajectory, the controller will not blindly continue with the old plan. It will react to the new state. The open-loop robust prediction does not fully exploit this future feedback capability.

A useful way to understand this point is to view robust MPC as a game between two players: the controller and the disturbance. At each time step, the controller selects an input and the disturbance selects an admissible perturbation. For the uncertain system

$$x(k+1) = g(x(k), u(k)) + w(k),$$

the true sequence of events is the following. The controller first applies $u(k)$, the disturbance then realizes $w(k)$, the new state is observed, and the controller chooses the next input based on this new state.

In an open-loop robust prediction, instead, the controller chooses all future moves

$$u_0, \dots, u_{N-1}$$

at once, while the disturbance is allowed to choose an entire disturbance sequence after seeing the whole planned input sequence. In other words, the prediction assumes that the controller will do the same thing in the future no matter what the disturbance does. This is safe, but it does not reflect the way MPC actually uses feedback.

9.6.2 Control policies

A less conservative robust prediction should allow the future control moves to depend on the future states. Instead of optimizing over a sequence

$$\{u_0, u_1, \dots, u_{N-1}\},$$

one would like to optimize over a sequence of feedback policies

$$\{u_0, \mu_1(\cdot), \dots, \mu_{N-1}(\cdot)\},$$

where

$$\mu_i : \mathbb{R}^n \rightarrow \mathbb{R}^m$$

maps the predicted state at time i to the input to be applied at time i .

The first input is still a single value, because the current state is known at the time the optimization problem is solved. The later inputs, however, should be functions. For instance, after the first disturbance w_0 has occurred, the state

$$x_1 = Ax_0 + Bu_0 + w_0$$

is known, and the second input can be chosen as

$$u_1 = \mu_1(x_1).$$

Equivalently, since x_1 is determined by the previous disturbance, one may regard u_1 as a function of w_0 . At the next step, u_2 may depend on w_0 and w_1 , and so on.

This idea captures the actual information pattern of closed-loop control. The difficulty is that one cannot optimize over arbitrary functions online. The set of all possible nonlinear policies is infinite-dimensional and far too complex for a tractable MPC formulation. Therefore, practical robust MPC methods impose a structure on the future policies.

9.6.3 Structured closed-loop predictions

Several structured policy classes can be used to introduce feedback into the prediction problem. A first possibility is to use a fixed **pre-stabilizing** feedback of the form

$$\mu_i(x_i) = Kx_i + v_i,$$

where K is chosen offline such that $A + BK$ is stable, and the optimizer only chooses the feed-forward correction v_i . This is simple and leads to a convex problem, but it can be conservative.

A more general possibility is to optimize over linear feedback gains,

$$\mu_i(x_i) = K_i x_i + v_i.$$

This gives more freedom, but the resulting optimization problem is generally nonconvex and therefore very difficult to solve online.

Another important idea is **disturbance feedback**. In this case the future input is written as an affine function of the past disturbances:

$$\mu_i = v_i + \sum_{j=0}^{i-1} M_{ij} w_j.$$

Although this is equivalent in expressive power to a class of linear state-feedback policies, it can lead to convex formulations. The drawback is that the number of decision variables may become large, making the method computationally demanding.

The approach emphasized in this chapter is **tube MPC**. The future input is decomposed as

$$u_i = v_i + K(x_i - z_i),$$

where z_i is a nominal state, v_i is a nominal input, and K is a fixed stabilizing feedback gain. The optimizer chooses the nominal trajectory and the nominal input sequence, while the feedback term $K(x_i - z_i)$ keeps the real trajectory close to the nominal one.

9.7 Error Dynamics

9.7.1 Splitting nominal planning and error correction

We consider again the disturbed linear system

$$x(k+1) = Ax(k) + Bu(k) + w(k), \quad x(k) \in \mathcal{X}, \quad u(k) \in \mathcal{U}, \quad w(k) \in \mathcal{W}.$$

The central idea is to split the available control authority into two parts. The first part plans a nominal trajectory for the noise-free system

$$z(k+1) = Az(k) + Bv(k).$$

Here $z(k)$ is the nominal state and $v(k)$ is the nominal input. The second part is a local tracking controller that compensates for deviations between the real state and the nominal state. We write

$$u(k) = v(k) + K(x(k) - z(k)),$$

where K is chosen offline so that

$$A_K := A + BK$$

is Schur stable.

The optimization problem will choose the nominal trajectory and the nominal inputs. The feedback gain K is not optimized online. It is used to keep the real trajectory inside a tube around the nominal trajectory. This is what makes the resulting MPC problem tractable: the online optimization remains a nominal finite-horizon optimal control problem, but with tightened constraints that account for the possible tracking error.

9.7.2 Derivation of the error dynamics

Define the error between the real and nominal state as

$$e_i := x_i - z_i.$$

Using the feedback law

$$u_i = v_i + K(x_i - z_i) = v_i + Ke_i,$$

and the nominal dynamics

$$z_{i+1} = Az_i + Bv_i,$$

we obtain

$$\begin{aligned} e_{i+1} &= x_{i+1} - z_{i+1} \\ &= (Ax_i + Bu_i + w_i) - (Az_i + Bv_i) \\ &= Ax_i + B(v_i + K(x_i - z_i)) + w_i - Az_i - Bv_i \\ &= (A + BK)(x_i - z_i) + w_i \\ &= A_K e_i + w_i. \end{aligned}$$

Thus the error evolves according to the autonomous disturbed system

$$e_{i+1} = A_K e_i + w_i, \quad w_i \in \mathcal{W}.$$

Error dynamics under tube feedback

If

$$u_i = v_i + K(x_i - z_i), \quad z_{i+1} = Az_i + Bv_i,$$

then the error $e_i = x_i - z_i$ satisfies

$$e_{i+1} = A_K e_i + w_i, \quad A_K = A + BK.$$

The nominal input v_i disappears from the error dynamics.

This cancellation is fundamental. It means that the possible tracking errors are determined only by the disturbance set $\mathcal{W}$ and by the closed-loop matrix A_K , not by the particular nominal trajectory chosen online. Hence the error bounds can be computed offline.

Furthermore, the dynamics $A + BK$ are stable, and the set $\mathcal{W}$ is bounded, so there is some set $\mathcal{E}$ that e will stay inside for all time.

We wish now to bound the maximum error, or to see how far the real trajectory is from the nominal trajectory.

9.7.3 Uncertain state evolution of the error

We now study more carefully how the error evolves under the feedback law introduced above. The error dynamics are

$$e_{i+1} = A_K e_i + w_i, \quad A_K := A + BK, \quad w_i \in \mathcal{W}.$$

Assume first that the real and nominal trajectories coincide at the beginning of the prediction, so that

$$e_0 = 0.$$

This is the situation that occurs, for instance, when the nominal prediction is initialized at the current measured state. The first prediction steps are then

$$e_1 = w_0,$$

$$e_2 = A_K e_1 + w_1 = A_K w_0 + w_1,$$

and, by repeated substitution,

$$e_i = \sum_{j=0}^{i-1} A_K^j w_{i-1-j}.$$

This formula shows explicitly how the disturbances accumulate through the stable dynamics A_K . The error at time i is not produced only by the last disturbance w_{i-1} , but by all previous disturbances, each propagated forward by the appropriate power of A_K .

Since each disturbance satisfies

$$w_i \in \mathcal{W},$$

we can ask the following set-valued question: where can the error be at prediction step i ? The answer is the disturbance reachable set

$$\mathcal{F}_i = \mathcal{W} \oplus A_K \mathcal{W} \oplus \cdots \oplus A_K^{i-1} \mathcal{W} = \bigoplus_{j=0}^{i-1} A_K^j \mathcal{W}, \quad \mathcal{F}_0 := \{0\}.$$

Thus

$$e_i \in \mathcal{F}_i$$

for every disturbance sequence $w_0, \dots, w_{i-1} \in \mathcal{W}$. The set $\mathcal{F}_i$ therefore contains all possible errors at time i , assuming the prediction starts from zero error.

Disturbance reachable set

For the error dynamics

$$e_{i+1} = A_K e_i + w_i, \quad w_i \in \mathcal{W},$$

with initial condition $e_0 = 0$, the set of all possible errors at prediction step i is

$$\mathcal{F}_i = \bigoplus_{j=0}^{i-1} A_K^j \mathcal{W}, \quad \mathcal{F}_0 = \{0\}.$$

The notation used above is the Minkowski sum:

$$\mathcal{P} \oplus \mathcal{Q} := \{x + y \mid x \in \mathcal{P}, y \in \mathcal{Q}\}.$$

Hence $\mathcal{F}_i$ is obtained by adding all disturbance sets after propagation through the closed-loop matrix A_K .

9.7.4 Minimum robust invariant set

As the prediction step i increases, the disturbance reachable set contains more and more terms:

$$\mathcal{F}_i = \mathcal{W} \oplus A_K \mathcal{W} \oplus \cdots \oplus A_K^{i-1} \mathcal{W}.$$

If A_K is Schur stable and $\mathcal{W}$ is bounded, these accumulated disturbance effects remain bounded. In the limit, one obtains

$$\mathcal{F}_\infty = \bigoplus_{j=0}^{\infty} A_K^j \mathcal{W}, \quad \mathcal{F}_0 := \{0\}.$$

This limiting set is the minimum robust positively invariant set associated with the error dynamics

$$e^+ = A_K e + w.$$

It is “minimum” in the sense that it is the smallest set that necessarily contains all possible steady-state error values generated by the persistent disturbance.

The invariance property can be seen directly from the definition. If

$$e \in \mathcal{F}_\infty,$$

then, after one disturbed step,

$$e^+ = A_K e + w \in A_K \mathcal{F}_\infty \oplus \mathcal{W}.$$

But

$$\begin{aligned} A_K \mathcal{F}_\infty \oplus \mathcal{W} &= A_K \left(\mathcal{W} \oplus A_K \mathcal{W} \oplus A_K^2 \mathcal{W} \oplus \cdots \right) \oplus \mathcal{W} \\ &= A_K \mathcal{W} \oplus A_K^2 \mathcal{W} \oplus A_K^3 \mathcal{W} \oplus \cdots \oplus \mathcal{W} \\ &= \mathcal{W} \oplus A_K \mathcal{W} \oplus A_K^2 \mathcal{W} \oplus \cdots \\ &= \mathcal{F}_\infty. \end{aligned}$$

Therefore,

$$e \in \mathcal{F}_\infty \implies A_K e + w \in \mathcal{F}_\infty \quad \forall w \in \mathcal{W}.$$

Minimum robust positively invariant set

For the error dynamics

$$e^+ = A_K e + w, \quad w \in \mathcal{W},$$

with A_K Schur stable, the set

$$\mathcal{F}_\infty = \bigoplus_{j=0}^{\infty} A_K^j \mathcal{W}$$

is the minimum robust positively invariant set generated by the disturbance.

A conceptual algorithm for computing this set is obtained by accumulating the terms in the Minkowski sum:

$$\Omega_0 \leftarrow \{0\},$$

and then, for $i = 0, 1, 2, \dots$,

$$\Omega_{i+1} \leftarrow \Omega_i \oplus A_K^i \mathcal{W}.$$

If there exists a finite index n such that

$$\mathcal{F}_n = \mathcal{F}_{n+1},$$

then no new error directions are generated by further disturbances and

$$\mathcal{F}_n = \mathcal{F}_\infty.$$

In general, such a finite n does not always exist. In that case, a sufficiently large n gives an approximation, or one computes a robust invariant outer approximation that is slightly larger than $\mathcal{F}_\infty$.

Conceptual computation of $\mathcal{F}_\infty$

Initialize

$$\Omega_0 \leftarrow \{0\}.$$

For $i = 0, 1, 2, \dots$, compute

$$\Omega_{i+1} \leftarrow \Omega_i \oplus A_K^i \mathcal{W}.$$

If

$$\Omega_{i+1} = \Omega_i,$$

then return

$$\mathcal{F}_\infty = \Omega_i.$$

If finite termination does not occur, one uses a finite approximation or an invariant outer approximation.

9.7.5 Example: convergence to the minimum robust invariant set

The construction can be illustrated on the closed-loop disturbed system

$$x(k+1) = \left(\begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} + \begin{bmatrix} 1 \\ 0.5 \end{bmatrix} K \right) x(k) + w(k),$$

with disturbance set

$$\mathcal{W} := \{w \mid |w_1| \leq 0.01, |w_2| \leq 0.1\}.$$

Here K is chosen as the LQR controller for

$$Q = I, \quad R = 10.$$

The sets

$$A_K^j \mathcal{W}$$

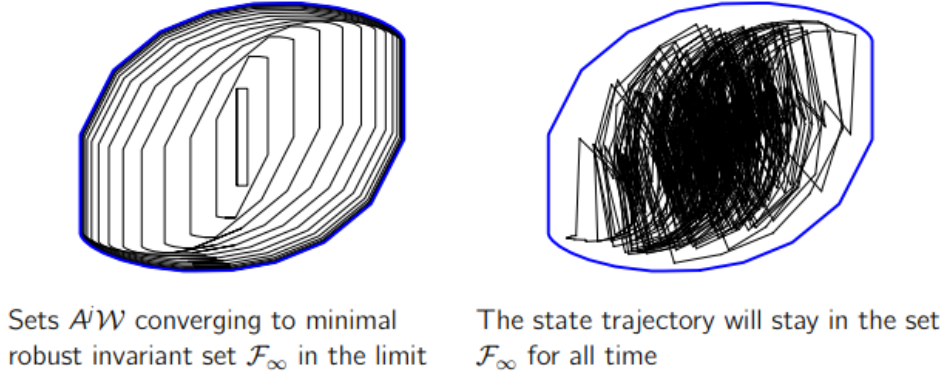


Figure 9.8: Example of the minimum robust invariant set for the disturbed closed-loop system $x(k+1) = A_K x(k) + w(k)$. The left plot shows the sets $A_K^j \mathcal{W}$ accumulating toward $\mathcal{F}_\infty$. The right plot shows that disturbed trajectories remain inside the invariant set once the disturbance effects have accumulated.

are propagated disturbance sets. Their Minkowski sum grows toward the limiting set

$$\mathcal{F}_\infty.$$

Once the error is inside this set, every subsequent disturbed evolution remains inside the same set. Therefore, the state trajectory of the error system stays in $\mathcal{F}_\infty$ for all time.

This example emphasizes the role of the stabilizing feedback K . The disturbance is persistent, so the error cannot be forced to zero. However, if A_K is stable, the effect of old disturbances decays, and the total error remains confined to a bounded robust invariant set.

9.7.6 From error bounds to robust MPC: two variants

The error dynamics

$$e_{i+1} = (A + BK)e_i + w_i = A_K e_i + w_i, \quad w_i \in \mathcal{W},$$

provide the basis for two robust MPC constructions. Both use the same feedback gain K , and both plan a nominal trajectory. The difference is in how the error between the real and nominal states is bounded.

The first possibility is to use the disturbance reachable sets

$$\mathcal{F}_i = \bigoplus_{j=0}^{i-1} A_K^j \mathcal{W}, \quad e_0 = 0.$$

In this case the error bound depends on the prediction step. At the beginning of the prediction the error is zero, while later along the horizon the set $\mathcal{F}_i$ becomes larger as more disturbances may have accumulated. This leads to robust constraint-tightening MPC. The tube around the nominal trajectory is time-varying:

$$x_i \in z_i \oplus \mathcal{F}_i.$$

The second possibility is to use a robust positively invariant set $\mathcal{E}$, satisfying

$$e_i \in \mathcal{E} \implies e_{i+1} \in \mathcal{E} \quad \forall w_i \in \mathcal{W}.$$

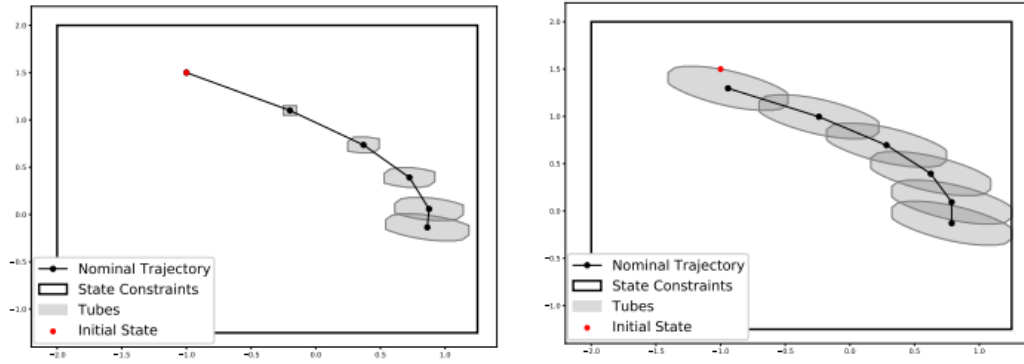


Figure 9.9: From error bounds to robust MPC. On the left, disturbance reachable sets $\mathcal{F}_i$ give a time-varying tube and lead to robust constraint-tightening MPC. On the right, a robust positively invariant set $\mathcal{E}$ gives a constant tube and leads to tube MPC.

Then, for any initial error $e_0 \in \mathcal{E}$, one has

$$e_i \in \mathcal{E} \quad \forall i \geq 0.$$

This leads to tube MPC. Instead of a growing tube, one uses a constant tube cross-section:

$$x_i \in z_i \oplus \mathcal{E}.$$

Two robust MPC variants from the error dynamics

The error dynamics can be used in two main ways:

$$e_i \in \mathcal{F}_i = \bigoplus_{j=0}^{i-1} A_K^j \mathcal{W}, \quad e_0 = 0,$$

which leads to robust constraint-tightening MPC, or

$$e_i \in \mathcal{E}, \quad \mathcal{E} \text{ RPI for } e^+ = A_K e + w,$$

which leads to tube MPC.

The disturbance-reachable-set approach is naturally tied to predictions initialized with zero error. It can be less conservative near the beginning of the horizon, because

$$\mathcal{F}_0 = \{0\}.$$

The RPI-set approach is more uniform: it assumes that the initial error may already lie anywhere inside $\mathcal{E}$, and then guarantees that it remains there forever. This is the viewpoint used in tube MPC.

9.8 Robust Constraint-Tightening MPC with Error Feedback

9.8.1 Noisy trajectory around a nominal trajectory

We now use the error dynamics to obtain a less conservative version of constraint tightening than the open-loop method discussed before. The real and nominal trajectories are related by

$$x_i = z_i + e_i.$$

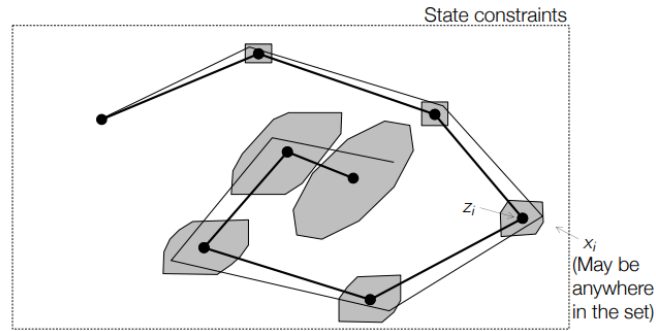


Figure 9.10: At prediction step i , the real state satisfies $x_i \in z_i \oplus \mathcal{F}_i$. The nominal state z_i is the planned tube center, while $\mathcal{F}_i$ bounds the possible deviation caused by disturbances.

If the prediction starts from

$$z_0 = x(k),$$

then

$$e_0 = x(k) - z_0 = 0.$$

Therefore, under the feedback law

$$u_i = v_i + K(x_i - z_i),$$

the error satisfies

$$e_i \in \mathcal{F}_i.$$

Consequently,

$$x_i \in z_i \oplus \mathcal{F}_i.$$

The nominal state z_i is the center of a possible set of real states. The set $\mathcal{F}_i$ describes the maximum possible deviation from this center at prediction step i .

9.8.2 Tightened state and input constraints

The robust state constraint requires

$$x_i \in \mathcal{X} \quad \forall w_0, \dots, w_{i-1} \in \mathcal{W}.$$

Since

$$x_i \in z_i \oplus \mathcal{F}_i,$$

this is guaranteed if and only if

$$z_i \oplus \mathcal{F}_i \subseteq \mathcal{X}.$$

By the definition of the Pontryagin difference, this is equivalent to

$$z_i \in \mathcal{X} \ominus \mathcal{F}_i.$$

Thus, instead of imposing the original state constraint on the nominal trajectory, we impose the tightened constraint

$$z_i \in \mathcal{X} \ominus \mathcal{F}_i.$$

The same reasoning applies to the input. Since

$$u_i = v_i + K e_i$$

and $e_i \in \mathcal{F}_i$, the set of possible inputs at time i is

$$u_i \in v_i \oplus K\mathcal{F}_i.$$

Robust satisfaction of the input constraint is therefore guaranteed by

$$v_i \oplus K\mathcal{F}_i \subseteq \mathcal{U},$$

or equivalently

$$v_i \in \mathcal{U} \ominus K\mathcal{F}_i.$$

Constraint tightening with disturbance reachable sets

If

$$e_i \in \mathcal{F}_i,$$

then robust satisfaction of the original constraints

$$x_i \in \mathcal{X}, \quad u_i \in \mathcal{U}$$

is guaranteed by imposing the nominal constraints

$$z_i \in \mathcal{X} \ominus \mathcal{F}_i, \quad v_i \in \mathcal{U} \ominus K\mathcal{F}_i.$$

The sets $\mathcal{F}_i$ depend only on A_K and $\mathcal{W}$. Hence the tightened sets

$$\mathcal{X} \ominus \mathcal{F}_i, \quad \mathcal{U} \ominus K\mathcal{F}_i$$

can be computed offline.

9.8.3 Robust constraint-tightening MPC problem

We now combine the nominal system dynamics with the disturbance reachable sets derived above. The idea is to optimize a nominal trajectory

$$z_0, z_1, \dots, z_N$$

and a nominal input sequence

$$v_0, v_1, \dots, v_{N-1},$$

while tightening the state and input constraints so that the real disturbed trajectory remains feasible for all admissible disturbances.

Recall that the error dynamics are governed by

$$e_{i+1} = A_K e_i + w_i, \quad A_K := A + BK,$$

and that, when the prediction starts from zero error, the error at step i belongs to the disturbance reachable set

$$\mathcal{F}_i := \mathcal{W} \oplus A_K \mathcal{W} \oplus \dots \oplus A_K^{i-1} \mathcal{W} = \bigoplus_{j=0}^{i-1} A_K^j \mathcal{W}, \quad \mathcal{F}_0 := \{0\}.$$

Therefore, if the nominal state is z_i , then the real state can lie anywhere in

$$z_i \oplus \mathcal{F}_i.$$

Likewise, since the control law is

$$u_i = v_i + K(x_i - z_i),$$

the real input lies in

$$v_i \oplus K\mathcal{F}_i.$$

Robust satisfaction of the original constraints is then enforced by requiring

$$z_i \in \mathcal{X} \ominus \mathcal{F}_i, \quad v_i \in \mathcal{U} \ominus K\mathcal{F}_i.$$

At time k , robust constraint-tightening MPC solves the nominal optimization problem

$$\begin{aligned} \min_{Z,V} \quad & \sum_{i=0}^{N-1} \ell(z_i, v_i) + \ell_f(z_N) \\ \text{subj. to} \quad & z_{i+1} = Az_i + Bv_i, \quad i = 0, \dots, N-1, \\ & z_i \in \mathcal{X} \ominus \mathcal{F}_i, \quad i = 0, \dots, N-1, \\ & v_i \in \mathcal{U} \ominus K\mathcal{F}_i, \quad i = 0, \dots, N-1, \\ & z_N \in \mathcal{X}_f^{\text{ct}} \ominus \mathcal{F}_N, \\ & z_0 = x(k). \end{aligned}$$

Robust constraint-tightening MPC problem

The robust constraint-tightening MPC problem optimizes the nominal dynamics

$$z_{i+1} = Az_i + Bv_i$$

subject to the tightened constraints

$$z_i \in \mathcal{X} \ominus \mathcal{F}_i, \quad v_i \in \mathcal{U} \ominus K\mathcal{F}_i,$$

and the tightened terminal constraint

$$z_N \in \mathcal{X}_f^{\text{ct}} \ominus \mathcal{F}_N.$$

Here

$$\mathcal{F}_i = \bigoplus_{j=0}^{i-1} A_K^j \mathcal{W}, \quad \mathcal{F}_0 = \{0\},$$

and the prediction is initialized with

$$z_0 = x(k).$$

The initialization

$$z_0 = x(k)$$

is important. It implies that the initial error is zero, and therefore the first applied input is simply

$$u(k) = v_0^* + K(x(k) - z_0^*) = v_0^*.$$

Thus the feedback term does not affect the first move, but it is essential in the prediction of the future error bounds through the sets $\mathcal{F}_i$. The terminal set $\mathcal{X}_f^{\text{ct}}$ is chosen so as to be robustly invariant under the terminal controller.

9.8.4 Properties of robust constraint-tightening MPC

The main property of this formulation is recursive feasibility. Intuitively, if the optimization problem is feasible at the current state, then after one disturbed step it remains feasible at the next state as well. This is the robust counterpart of the standard feasibility result in nominal MPC.

To guarantee this property, the terminal ingredients $(\ell_f, \mathcal{X}_f^{\text{ct}})$ must be chosen so that the terminal set is compatible with the terminal controller. More precisely, we assume

$$\mathcal{X}_f^{\text{ct}} \subseteq \mathcal{X},$$

and that for every

$$z \in \mathcal{X}_f^{\text{ct}},$$

the corresponding terminal input satisfies

$$Kz \in \mathcal{U},$$

while the terminal set is robust positively invariant under the disturbed closed-loop dynamics, that is,

$$(A + BK)z + w \in \mathcal{X}_f^{\text{ct}} \quad \forall w \in \mathcal{W}.$$

Robust invariance of the terminal set

Suppose the terminal ingredients $(\ell_f, \mathcal{X}_f^{\text{ct}})$ are designed such that

$$\mathcal{X}_f^{\text{ct}} \subseteq \mathcal{X},$$

and for all $z \in \mathcal{X}_f^{\text{ct}}$,

$$Kz \in \mathcal{U},$$

$$(A + BK)z + w \in \mathcal{X}_f^{\text{ct}} \quad \forall w \in \mathcal{W}.$$

Then $\mathcal{X}_f^{\text{ct}}$ is a robust positively invariant terminal set for the terminal controller.

Let $\mathcal{X}_N$ denote the feasible set of the robust constraint-tightening MPC problem, and let $V^*(x(k))$ denote the corresponding optimizer for the state $x(k) \in \mathcal{X}_N$. Then, after applying the first optimal nominal input $v_0^*(x(k))$, the next real state is

$$x(k+1) = Ax(k) + Bv_0^*(x(k)) + w(k).$$

Since the optimization problem was formulated so that the predicted tube is feasible for every possible disturbance realization, it is in particular feasible for the disturbance that is actually observed. Hence

$$x(k+1) \in \mathcal{X}_N \quad \forall w(k) \in \mathcal{W}.$$

Recursive feasibility

Let $\mathcal{X}_N$ be the feasible set of the robust constraint-tightening MPC problem. Under the robust terminal invariance assumptions above, if

$$x(k) \in \mathcal{X}_N,$$

then

$$x(k+1) = Ax(k) + Bv_0^*(x(k)) + w(k) \in \mathcal{X}_N \quad \forall w(k) \in \mathcal{W}.$$

Therefore the robust constraint-tightening MPC problem is recursively feasible.

This result is the essential robustness guarantee of the scheme. It says that once the problem is feasible, feasibility is preserved for all future times despite the disturbance. Stability analysis, however, requires a more refined notion, typically input-to-state stability, and goes beyond the basic recursive-feasibility argument.

9.8.5 Intuition

The geometric interpretation of the method is very useful. Around each nominal prediction z_i , one considers the disturbance reachable set $\mathcal{F}_i$. Hence the real state at step i may lie anywhere in the set

$$z_i \oplus \mathcal{F}_i.$$

If this whole set remains inside the state constraints, then the real state is guaranteed to satisfy the original constraints for every admissible disturbance realization.

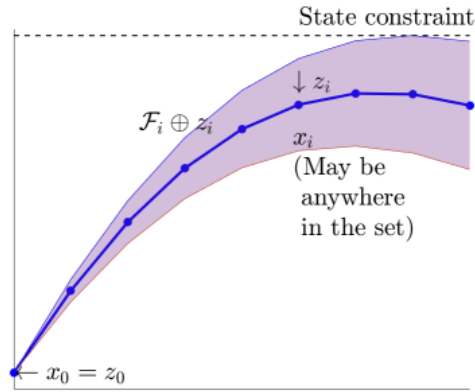


Figure 9.11: Intuition behind robust constraint-tightening MPC. Around each nominal state z_i , the disturbance reachable set $\mathcal{F}_i$ describes the possible location of the real state x_i . The nominal trajectory must therefore remain sufficiently far from the boundary of the constraint set.

After one control step, the actual disturbed state is not exactly at the nominal successor z_1^* , but rather at

$$x^+ = z_1^* + w,$$

or, more generally, inside the corresponding disturbed tube. The candidate nominal trajectory at the next time step is then obtained by shifting the previous optimal trajectory forward and accounting for the effect of the disturbance. The key geometric fact is that the disturbance reachable set around this shifted candidate remains contained in the previously feasible tube.

This picture explains why the formulation is less conservative than purely open-loop robust MPC. The future tube is not built only from the open-loop system matrix A , but from the stabilized error dynamics

$$e_{i+1} = A_K e_i + w_i,$$

so the feedback correction K already enters the prediction through the sets $\mathcal{F}_i$.

9.8.6 Proof sketch

We now sketch the recursive-feasibility argument more explicitly. Let

$$\{z_0^*, \dots, z_N^*\}, \quad \{v_0^*, \dots, v_{N-1}^*\}$$

be the optimal nominal state and input sequences at time k . Because the problem is initialized with

$$z_0^* = x(k),$$

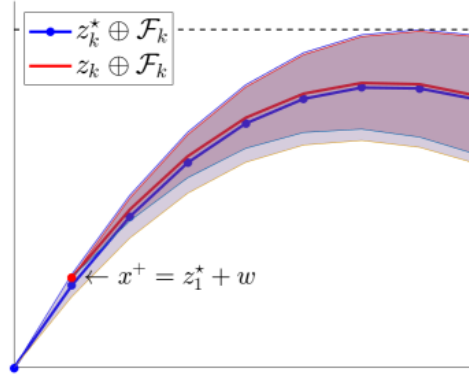


Figure 9.12: After the first disturbance is realized, the next nominal candidate is perturbed with respect to the previous optimal nominal trajectory. The new disturbance reachable tube is contained in the previously feasible one, which is the key idea behind recursive feasibility.

the first applied control is

$$u(k) = v_0^*.$$

After the disturbance $w(k)$ is realized, the next state is

$$x^+ = x(k+1) = z_1^* + w(k).$$

To prove recursive feasibility, we construct a candidate solution for the optimization problem at time $k+1$. This candidate is based on the error feedback and on the terminal controller:

$$v_i = v_{i+1}^* + K(z_i - z_{i+1}^*), \quad i = 0, \dots, N-1,$$

with

$$v_N^* = Kz_N^*, \quad z_{N+1}^* = A_K z_N^*.$$

Recursive application yields the candidate state and input sequences

$$z_i = z_{i+1}^* + A_K^i w(k), \quad i = 0, \dots, N,$$

and

$$v_i = v_{i+1}^* + K A_K^i w(k), \quad i = 0, \dots, N-1.$$

One verifies by induction that the candidate satisfies the nominal dynamics. Indeed,

$$\begin{aligned} z_{i+1} &= A z_i + B v_i \\ &= A(z_{i+1}^* + A_K^i w(k)) + B(v_{i+1}^* + K(z_i - z_{i+1}^*)) \\ &= z_{i+2}^* + A_K^{i+1} w(k). \end{aligned}$$

This matches exactly the recursive construction above.

The next step is the nested-tube argument. For each prediction step,

$$\begin{aligned} z_i \oplus \mathcal{F}_i &= z_{i+1}^* + A_K^i w(k) \oplus \mathcal{F}_i \\ &\subseteq z_{i+1}^* \oplus A_K^i \mathcal{W} \oplus \mathcal{F}_i \\ &= z_{i+1}^* \oplus \mathcal{F}_{i+1}. \end{aligned}$$

Hence the candidate prediction tube at time $k+1$ is contained in the previously feasible tube at time k . This proves that all tightened state constraints remain satisfied. The input constraints are shown in exactly the same way.

At the terminal step, one obtains

$$\begin{aligned} z_N \oplus \mathcal{F}_N &\subseteq z_{N+1}^* \oplus \mathcal{F}_{N+1} \\ &= A_K(z_N^* \oplus \mathcal{F}_N) \oplus \mathcal{W} \\ &\subseteq A_K \mathcal{X}_f^{\text{ct}} \oplus \mathcal{W} \\ &\subseteq \mathcal{X}_f^{\text{ct}}, \end{aligned}$$

where the last inclusion follows from robust positive invariance of the terminal set. Therefore the terminal constraint is also satisfied. This completes the proof sketch: the shifted candidate is feasible, and so the optimization problem remains feasible at the next step.

Idea of the proof

The recursive-feasibility proof is based on three ingredients: the shifted nominal candidate, the containment

$$z_i \oplus \mathcal{F}_i \subseteq z_{i+1}^* \oplus \mathcal{F}_{i+1},$$

and robust invariance of the terminal set. Together, these show that the candidate built at time $k + 1$ satisfies all tightened constraints.

9.9 Robust Tube MPC

9.9.1 Idea and construction

Tube MPC: the idea

The robust constraint-tightening MPC formulation used disturbance reachable sets $\mathcal{F}_i$, which depend on the prediction step. Tube MPC takes a slightly different viewpoint. Instead of using a different error set at every step, it bounds the error by a single robust positively invariant set $\mathcal{E}$.

The nominal system is still planned without disturbances:

$$z(k+1) = Az(k) + Bv(k).$$

The real system is affected by the disturbance:

$$x(k+1) = Ax(k) + Bu(k) + w(k), \quad w(k) \in \mathcal{W}.$$

The controller is assumed to have the form

$$u_i = v_i + K(x_i - z_i),$$

where K is a fixed stabilizing feedback gain. The nominal input v_i steers the nominal trajectory, while the feedback term $K(x_i - z_i)$ keeps the real trajectory close to the nominal one.

The error

$$e_i = x_i - z_i$$

then satisfies

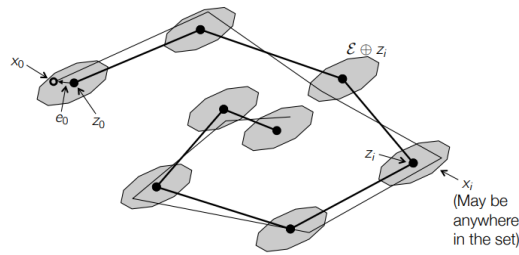
$$e_{i+1} = A_K e_i + w_i, \quad A_K := A + BK.$$

If $\mathcal{E}$ is a robust positively invariant set for this error dynamics, then

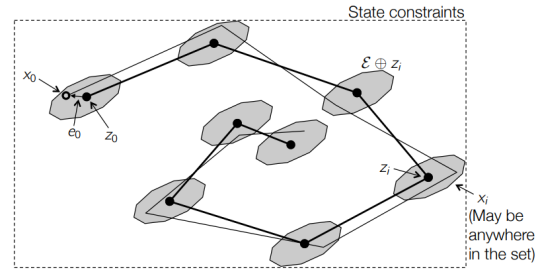
$$e_i \in \mathcal{E} \implies e_{i+1} \in \mathcal{E} \quad \forall w_i \in \mathcal{W}.$$

Therefore, once the error lies in $\mathcal{E}$, it remains in $\mathcal{E}$ for all future times. As a consequence, the real state remains inside the tube

$$x_i \in z_i \oplus \mathcal{E}.$$



(a) Nominal trajectory with tube cross-sections. The real trajectory stays close to the nominal one.



(b) The whole tube must satisfy the state constraints.

Figure 9.13: Tube MPC idea. The optimizer plans a nominal trajectory z_i , while the feedback law $u_i = v_i + K(x_i - z_i)$ keeps the real state x_i inside the tube $z_i \oplus \mathcal{E}$. The tube must remain inside the admissible state constraint set.

In this sense, tube MPC ignores the noise when planning the nominal trajectory, but accounts for the noise by surrounding the nominal trajectory with an invariant tube. Ideally, one chooses

$$\mathcal{E} = \mathcal{F}_\infty = \bigoplus_{j=0}^{\infty} A_K^j \mathcal{W},$$

namely the minimal robust positively invariant set of the error dynamics. In practice, $\mathcal{F}_\infty$ may be difficult to compute exactly, and one often uses a robust invariant outer approximation.

Tube MPC idea

Tube MPC plans a nominal trajectory z_i for the disturbance-free system and applies the feedback law

$$u_i = v_i + K(x_i - z_i).$$

If the error set $\mathcal{E}$ is robust positively invariant for

$$e^+ = A_K e + w,$$

then

$$x_i \in z_i \oplus \mathcal{E}$$

for all admissible disturbance realizations.

What must be designed

The construction of tube MPC consists of two parts. First, one must build the tube and the corresponding tightened nominal constraints. Second, one must prove that the resulting closed-loop controller has the desired robust properties.

The design therefore requires the following ingredients. One must compute a set $\mathcal{E}$ such that the error remains inside it. Then one must modify the constraints on the nominal trajectory z_i so that the entire tube $z_i \oplus \mathcal{E}$ remains feasible. Finally, one must formulate the resulting problem as a tractable optimization problem.

After the formulation is defined, two properties have to be established. The first is robust constraint satisfaction: for all admissible disturbances, the real state and input must satisfy

$$x_i \in \mathcal{X}, \quad u_i \in \mathcal{U}.$$

The second is robust stability: because the disturbance is persistent, the state cannot generally converge exactly to the origin, but it should converge to the invariant tube around the origin.

Tube MPC design tasks

To make tube MPC work, one needs to:

compute $\mathcal{E}$, tighten the nominal constraints, formulate the nominal MPC problem.

Then one proves robust constraint satisfaction and robust stability of the closed-loop system.

Recall: robust invariant set

Before using the set $\mathcal{E}$, it is useful to recall the notion of robust positive invariance. Consider an autonomous disturbed system

$$x(k+1) = g(x(k), w(k)), \quad w(k) \in \mathcal{W},$$

or a closed-loop disturbed system

$$x(k+1) = g(x(k), \kappa(x(k)), w(k)),$$

where the feedback law κ is fixed. A set $\mathcal{O}_{\mathcal{W}}$ is robust positively invariant if, whenever the current state belongs to the set, the next state belongs to the same set for every admissible disturbance.

Robust positive invariant set

A set $\mathcal{O}_{\mathcal{W}}$ is a robust positive invariant set for

$$x(k+1) = g(x(k), w(k)), \quad w(k) \in \mathcal{W},$$

if

$$x \in \mathcal{O}_{\mathcal{W}} \implies g(x, w) \in \mathcal{O}_{\mathcal{W}} \quad \forall w \in \mathcal{W}.$$

There is an important difference between the invariant sets used previously as terminal sets and the invariant set used here. Previously, one was often interested in a large robust invariant set, because the goal was to maximize the terminal region in which a local controller can keep the state feasible. In tube MPC, instead, one wants a small robust invariant set for the error dynamics, because $\mathcal{E}$ measures the size of the tube. A smaller $\mathcal{E}$ means less constraint tightening and therefore a larger feasible region for the nominal MPC problem.

Noisy system trajectory: constant tube

Given a nominal trajectory z_i , the real trajectory can be written as

$$x_i = z_i + e_i.$$

The exact error e_i is not known in advance, because it depends on the disturbance sequence. However, if the error set $\mathcal{E}$ is robust positively invariant and $e_0 \in \mathcal{E}$, then

$$e_i \in \mathcal{E} \quad \forall i \geq 0.$$

Therefore the real state satisfies

$$x_i \in z_i \oplus \mathcal{E} = \{z_i + e \mid e \in \mathcal{E}\}.$$

This is why the construction is called tube MPC: the nominal trajectory is surrounded by a constant tube with cross-section $\mathcal{E}$.

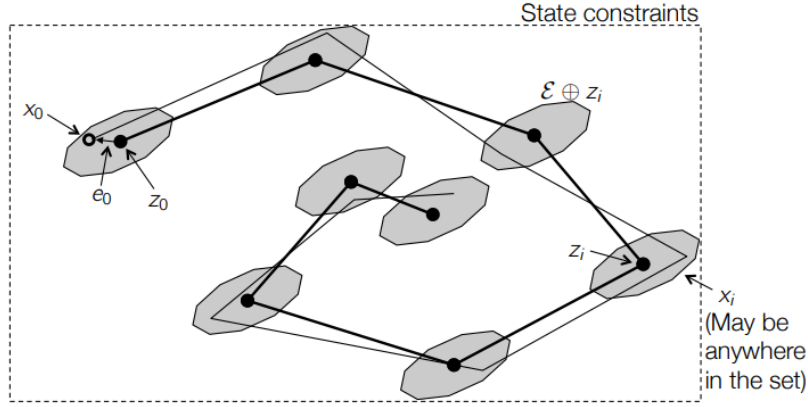


Figure 9.14: Noisy system trajectory with a constant tube. Given a nominal trajectory z_i , the real state can be anywhere in $z_i \oplus \mathcal{E}$. The set $\mathcal{E}$ bounds the error between the real and nominal trajectories at all prediction steps.

Constraint tightening

The goal is to guarantee

$$x_i \in \mathcal{X}, \quad u_i \in \mathcal{U},$$

for all disturbance sequences

$$\{w_0, \dots, w_{i-1}\} \in \mathcal{W}^i.$$

Tube MPC achieves this while optimizing only the nominal dynamics

$$z(k+1) = Az(k) + Bv(k).$$

The noisy system

$$x(k+1) = Ax(k) + Bu(k) + w(k)$$

is kept feasible by tightening the constraints imposed on z_i and v_i .

For the state constraint, since

$$x_i \in z_i \oplus \mathcal{E},$$

we need

$$z_i \oplus \mathcal{E} \subseteq \mathcal{X}.$$

By definition of the Pontryagin difference, this is equivalent to

$$z_i \in \mathcal{X} \ominus \mathcal{E}.$$

Thus the nominal state must satisfy the tightened state constraint

$$z_i \in \mathcal{X} \ominus \mathcal{E}.$$

The same argument applies to the input. Since

$$u_i = v_i + K(x_i - z_i) = v_i + Ke_i$$

and

$$e_i \in \mathcal{E},$$

the possible inputs satisfy

$$u_i \in v_i \oplus K\mathcal{E}.$$

Therefore robust satisfaction of $u_i \in \mathcal{U}$ is equivalent to

$$v_i \oplus K\mathcal{E} \subseteq \mathcal{U},$$

or

$$v_i \in \mathcal{U} \ominus K\mathcal{E}.$$

Constraint tightening for tube MPC

The necessary and sufficient tightening conditions are

$$z_i \oplus \mathcal{E} \subseteq \mathcal{X} \iff z_i \in \mathcal{X} \ominus \mathcal{E},$$

and

$$v_i \oplus K\mathcal{E} \subseteq \mathcal{U} \iff v_i \in \mathcal{U} \ominus K\mathcal{E}.$$

Since $\mathcal{E}$ is computed offline, the tightened constraints

$$\mathcal{X} \ominus \mathcal{E}, \quad \mathcal{U} \ominus K\mathcal{E}$$

can also be computed offline.

9.9.2 Tube MPC formulation and assumptions

Optimization problem

We can now formulate the tube MPC optimization problem. The decision variables are the nominal state sequence

$$Z = \{z_0, \dots, z_N\}$$

and the nominal input sequence

$$V = \{v_0, \dots, v_{N-1}\}.$$

For a measured state x_0 , the feasible set is defined as

$$\begin{aligned} \mathcal{Z}(x_0) := \left\{ (Z, V) \mid \right. & z_{i+1} = Az_i + Bv_i, \quad i = 0, \dots, N-1, \\ & z_i \in \mathcal{X} \ominus \mathcal{E}, \quad i = 0, \dots, N-1, \\ & v_i \in \mathcal{U} \ominus K\mathcal{E}, \quad i = 0, \dots, N-1, \\ & z_N \in \mathcal{X}_f, \\ & \left. x_0 \in z_0 \oplus \mathcal{E} \right\}. \end{aligned}$$

The cost function is evaluated on the nominal tube centers:

$$J(Z, V) := \sum_{i=0}^{N-1} \ell(z_i, v_i) + \ell_f(z_N).$$

The optimization problem is

$$(V^*(x_0), Z^*(x_0)) = \arg \min_{V, Z} \{J(Z, V) \mid (Z, V) \in \mathcal{Z}(x_0)\}.$$

The applied control law is

$$\mu_{\text{tube}}(x) := K(x - z_0^*(x)) + v_0^*(x).$$

Tube MPC problem

Tube MPC solves

$$(V^*(x_0), Z^*(x_0)) = \arg \min_{V, Z} \left\{ \sum_{i=0}^{N-1} \ell(z_i, v_i) + \ell_f(z_N) \mid (Z, V) \in \mathcal{Z}(x_0) \right\},$$

where $\mathcal{Z}(x_0)$ is defined by the nominal dynamics, the tightened constraints

$$z_i \in \mathcal{X} \ominus \mathcal{E}, \quad v_i \in \mathcal{U} \ominus K\mathcal{E},$$

the terminal condition $z_N \in \mathcal{X}_f$, and the tube-inclusion condition

$$x_0 \in z_0 \oplus \mathcal{E}.$$

The implemented input is

$$u = \mu_{\text{tube}}(x) = K(x - z_0^*(x)) + v_0^*(x).$$

This formulation differs from robust constraint-tightening MPC in an important way. The first tube center z_0 is an optimization variable; it is not forced to be equal to the measured state. The only requirement is that the measured state lies inside the first tube:

$$x_0 \in z_0 \oplus \mathcal{E}.$$

The cost is with respect to the nominal tube centers, while robustness is enforced through the tightened constraints and the invariant error tube.

Terminal assumptions

The assumptions needed for tube MPC are essentially the nominal MPC assumptions applied to the tightened nominal system. First, the stage cost is positive definite: it is strictly positive away from the origin and equal to zero at the origin.

Second, the terminal set $\mathcal{X}_f$ is invariant for the nominal system under a local terminal controller

$$\kappa_f(z).$$

That is,

$$Az + B\kappa_f(z) \in \mathcal{X}_f \quad \forall z \in \mathcal{X}_f.$$

Moreover, the terminal set and terminal controller must satisfy the tightened constraints:

$$\mathcal{X}_f \subseteq \mathcal{X} \ominus \mathcal{E},$$

$$\kappa_f(z) \in \mathcal{U} \ominus K\mathcal{E} \quad \forall z \in \mathcal{X}_f.$$

Third, the terminal cost is a continuous Lyapunov function in the terminal set:

$$\ell_f(Az + B\kappa_f(z)) - \ell_f(z) \leq -\ell(z, \kappa_f(z)), \quad \forall z \in \mathcal{X}_f.$$

Tube MPC assumptions

Tube MPC assumes:

$$\ell(z, v) > 0 \quad \text{for } (z, v) \neq (0, 0),$$

$$Az + B\kappa_f(z) \in \mathcal{X}_f \quad \forall z \in \mathcal{X}_f,$$

$$\mathcal{X}_f \subseteq \mathcal{X} \ominus \mathcal{E}, \quad \kappa_f(z) \in \mathcal{U} \ominus K\mathcal{E} \quad \forall z \in \mathcal{X}_f,$$

and

$$\ell_f(Az + B\kappa_f(z)) - \ell_f(z) \leq -\ell(z, \kappa_f(z)) \quad \forall z \in \mathcal{X}_f.$$

9.9.3 Robust invariance and stability

Robust invariance

We now prove the first main property: robust invariance of the feasible set. Define

$$\mathcal{Z} := \{x \mid \mathcal{Z}(x) \neq \emptyset\}.$$

This is the set of all states for which the tube MPC problem is feasible.

Robust invariance of tube MPC

The set

$$\mathcal{Z} = \{x \mid \mathcal{Z}(x) \neq \emptyset\}$$

is a robust positively invariant set for the closed-loop system

$$x(k+1) = Ax(k) + B\mu_{\text{tube}}(x(k)) + w(k), \quad w(k) \in \mathcal{W},$$

subject to the constraints

$$x(k) \in \mathcal{X}, \quad u(k) \in \mathcal{U}.$$

Let

$$(\{v_0^*, \dots, v_{N-1}^*\}, \{z_0^*, \dots, z_N^*\})$$

be the optimal solution for $x(k)$. The applied input is

$$\mu_{\text{tube}}(x(k)) = K(x(k) - z_0^*) + v_0^*.$$

The next state is therefore

$$\begin{aligned} x(k+1) &= Ax(k) + BK(x(k) - z_0^*) + Bv_0^* + w(k) \\ &= Az_0^* + Bv_0^* + A_K(x(k) - z_0^*) + w(k) \\ &= z_1^* + A_K e_0 + w(k), \end{aligned}$$

where

$$e_0 = x(k) - z_0^*.$$

Because the initial tube constraint gives

$$x(k) \in z_0^* \oplus \mathcal{E},$$

we have

$$e_0 \in \mathcal{E}.$$

Since $\mathcal{E}$ is robust positively invariant,

$$A_K e_0 + w(k) \in \mathcal{E}.$$

Hence

$$x(k+1) \in z_1^* \oplus \mathcal{E}.$$

This means that z_1^* is a valid first tube center for the next measured state.

As in nominal MPC, we can therefore construct a feasible candidate for the next optimization problem by shifting the previous nominal sequence and appending the terminal controller:

$$\{v_1^*, \dots, v_{N-1}^*, \kappa_f(z_N^*)\},$$

$$\{z_1^*, \dots, z_N^*, Az_N^* + B\kappa_f(z_N^*)\}.$$

The shifted sequence satisfies the nominal dynamics. The tightened state and input constraints remain satisfied because they were already satisfied by the previous optimal solution, and the terminal assumptions ensure that the appended terminal point and terminal input are feasible. Therefore the tube MPC problem is feasible for every possible value of $x(k+1)$ generated by an admissible disturbance.

Robust stability

The second main property is robust stability. Since the disturbance is persistent, the real state does not generally converge to the origin. Instead, tube MPC guarantees convergence to the invariant error set $\mathcal{E}$.

Robust stability of tube MPC

For the closed-loop system

$$x(k+1) = Ax(k) + B\mu_{\text{tube}}(x(k)) + w(k), \quad w(k) \in \mathcal{W},$$

the state $x(k)$ converges in the limit to the set $\mathcal{E}$:

$$\lim_{k \rightarrow \infty} \text{dist}(x(k), \mathcal{E}) = 0.$$

The proof follows the standard MPC Lyapunov argument, but applied to the nominal tube centers. Let

$$J^*(x(k)) = \sum_{i=0}^{N-1} \ell(z_i^*, v_i^*) + \ell_f(z_N^*)$$

be the optimal value at time k . Using the shifted feasible candidate at time $k+1$, we get

$$\begin{aligned} J^*(x(k+1)) &\leq \sum_{i=1}^{N-1} \ell(z_i^*, v_i^*) + \ell(z_N^*, \kappa_f(z_N^*)) + \ell_f(z_{N+1}) \\ &= J^*(x(k)) - \ell(z_0^*, v_0^*) - \ell_f(z_N^*) + \ell_f(z_{N+1}) + \ell(z_N^*, \kappa_f(z_N^*)). \end{aligned}$$

By the terminal Lyapunov condition,

$$\ell_f(z_{N+1}) - \ell_f(z_N^*) \leq -\ell(z_N^*, \kappa_f(z_N^*)).$$

Therefore,

$$J^*(x(k+1)) \leq J^*(x(k)) - \ell(z_0^*, v_0^*).$$

The optimal value decreases by at least the first nominal stage cost. Since the stage cost is positive definite, this implies

$$\lim_{k \rightarrow \infty} z_0^*(x(k)) = 0.$$

The real state, however, satisfies

$$x(k) \in z_0^*(x(k)) \oplus \mathcal{E}.$$

Thus, as the tube center converges to the origin, the real state converges to the tube cross-section centered at the origin:

$$\lim_{k \rightarrow \infty} \text{dist}(x(k), \mathcal{E}) = 0.$$

This is the correct robust stability statement in the presence of persistent disturbances.

9.9.4 Implementation and example

Offline and online implementation

The final tube MPC algorithm can be separated into an offline design phase and an online implementation phase. Offline, one first chooses a stabilizing feedback gain K such that

$$A + BK$$

is Schur stable. Then one computes a robust positively invariant set $\mathcal{E}$, ideally the minimal set

$$\mathcal{E} = \mathcal{F}_\infty = \bigoplus_{j=0}^{\infty} A_K^j \mathcal{W}$$

for the error dynamics

$$e(k+1) = A_K e(k) + w(k).$$

Next, one computes the tightened constraints

$$\tilde{\mathcal{X}} := \mathcal{X} \ominus \mathcal{E}, \quad \tilde{\mathcal{U}} := \mathcal{U} \ominus K\mathcal{E}.$$

Finally, one chooses the terminal cost ℓ_f and terminal constraint $\mathcal{X}_f$ satisfying the tube MPC assumptions above.

Online, at each sampling instant, the current state x is measured or estimated. Then the tube MPC problem is solved:

$$(V^*(x), Z^*(x)) = \arg \min_{V, Z} \{J(Z, V) \mid (Z, V) \in \mathcal{Z}(x)\}.$$

The applied input is

$$u = K(x - z_0^*(x)) + v_0^*(x).$$

Tube MPC algorithm

Offline:

$$K \implies \mathcal{E} \implies \tilde{\mathcal{X}} = \mathcal{X} \ominus \mathcal{E}, \quad \tilde{\mathcal{U}} = \mathcal{U} \ominus K\mathcal{E} \implies \ell_f, \mathcal{X}_f.$$

Online:

$$x \implies (V^*(x), Z^*(x)) \implies u = K(x - z_0^*(x)) + v_0^*(x).$$

Remark. *It is often not possible to compute the minimal robust invariant set exactly, because it may have infinitely many facets. In practice, one therefore uses a robust invariant outer approximation. This approximation is slightly larger than the minimal set, which makes the tube larger and the tightened constraints more conservative.*

Example data

We now illustrate the tube MPC construction on the double-integrator. The uncertain system is

$$x(k+1) = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} x(k) + \begin{bmatrix} 1 \\ 0.5 \end{bmatrix} u(k) + w(k),$$

where the disturbance is bounded in the set

$$\mathcal{W} := \{w \mid |w_1| \leq 0.01, |w_2| \leq 0.1\}.$$

The state and input constraints are

$$\mathcal{X} := \{x \mid \|x\|_\infty \leq 3\}, \quad \mathcal{U} := \{u \mid \|u\| \leq 0.5\}.$$

The stage cost is quadratic:

$$\ell(z, v) := z^\top Q z + v^\top R v,$$

with

$$Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad R = 10.$$

Tube MPC example data

The example uses a disturbed double integrator with bounded additive disturbance

$$w \in \mathcal{W},$$

box state constraints

$$\|x\|_\infty \leq 3,$$

a bounded input constraint

$$\|u\| \leq 0.5,$$

and quadratic nominal stage cost

$$\ell(z, v) = z^\top Q z + v^\top R v.$$

This example is useful because it shows all the main ingredients of tube MPC: the computation of an invariant error set, the tightening of state and input constraints, the choice of terminal ingredients, and finally the behavior of the planned and realized tubes.

Computing the invariant error set

The first offline step is to choose a stabilizing feedback gain K such that

$$A_K := A + BK$$

is Schur stable. In the example, K is chosen as the LQR gain for

$$Q = I, \quad R = 10.$$

This gives

$$K = \begin{bmatrix} -0.2311 & -0.7011 \end{bmatrix}.$$

The corresponding error dynamics are

$$e(k+1) = A_K e(k) + w(k), \quad w(k) \in \mathcal{W}.$$

The next step is to compute a robust positively invariant set $\mathcal{E}$ for this error system. Ideally, one would choose the minimal RPI set

$$\mathcal{E} = \mathcal{F}_\infty = \bigoplus_{j=0}^{\infty} A_K^j \mathcal{W}.$$

This set contains all possible steady-state errors generated by the persistent disturbance under the stabilizing feedback K .

The role of this set is fundamental. Once

$$e(k) \in \mathcal{E},$$

robust invariance guarantees

$$e(k+1) = A_K e(k) + w(k) \in \mathcal{E} \quad \forall w(k) \in \mathcal{W}.$$

Thus the real state remains inside the tube

$$x(k) \in z(k) \oplus \mathcal{E}.$$

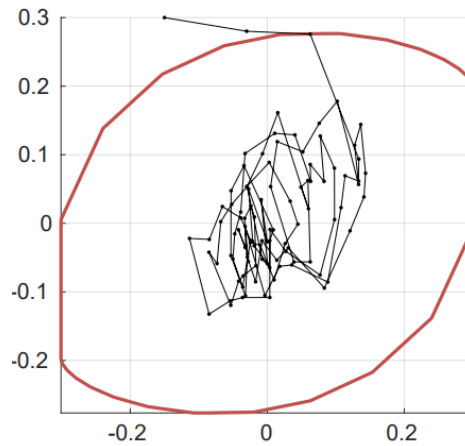


Figure 9.15: Offline computation of the robust invariant error set. The LQR feedback gain $K = [-0.2311 \ -0.7011]$ stabilizes $A + BK$. The red set represents an invariant error bound for the disturbed error dynamics $e(k+1) = A_K e(k) + w(k)$.

Remark. *The minimal robust invariant set may not be exactly computable with a finite number of polyhedral facets. In implementations one often uses an invariant outer approximation. This outer approximation is slightly larger than the minimal set, which makes the tube larger and the tightened constraints more conservative.*

Tightening the state constraints

After computing the error set $\mathcal{E}$, the next offline step is to tighten the state constraints. In the example,

$$\mathcal{X} = \{x \mid \|x\|_\infty \leq 3\}.$$

Equivalently,

$$\mathcal{X} = \left\{ x \mid \begin{bmatrix} I \\ -I \end{bmatrix} x \leq 3 \begin{bmatrix} \mathbf{1} \\ \mathbf{1} \end{bmatrix} \right\}.$$

If the error set is described as

$$\mathcal{E} = \{e \mid Fe \leq f\},$$

then the tightened state constraint is

$$\mathcal{X} \ominus \mathcal{E} = \{x \mid x + e \in \mathcal{X} \ \forall e \in \mathcal{E}\}.$$

Substituting the halfspace description of $\mathcal{X}$, this becomes

$$\mathcal{X} \ominus \mathcal{E} = \left\{ x \mid \begin{bmatrix} I \\ -I \end{bmatrix} x + \begin{bmatrix} I \\ -I \end{bmatrix} e \leq 3 \begin{bmatrix} \mathbf{1} \\ \mathbf{1} \end{bmatrix} \quad \forall e \in \mathcal{E} \right\}.$$

Equivalently, each inequality is tightened by the worst possible error in the corresponding normal direction:

$$\mathcal{X} \ominus \mathcal{E} = \left\{ x \mid \begin{bmatrix} I \\ -I \end{bmatrix} x \leq \begin{bmatrix} 3 - \max\{[1 \ 0]e \mid e \in \mathcal{E}\} \\ 3 - \max\{[0 \ 1]e \mid e \in \mathcal{E}\} \\ 3 - \max\{[-1 \ 0]e \mid e \in \mathcal{E}\} \\ 3 - \max\{[0 \ -1]e \mid e \in \mathcal{E}\} \end{bmatrix} \right\}.$$

The maximizations are linear programs and can be computed offline. The result is a polytope with a smaller right-hand side than the original constraint set.

Polyhedral state constraint tightening

If

$$\mathcal{X} = \{x \mid H_x x \leq h_x\}, \quad \mathcal{E} = \{e \mid Fe \leq f\},$$

then

$$\mathcal{X} \ominus \mathcal{E} = \{x \mid H_x x \leq h_x - \eta_x\},$$

where the j -th component of η_x is

$$(\eta_x)_j = \max_{e \in \mathcal{E}} H_{x,j} e.$$

Each component is obtained by solving an offline linear program.

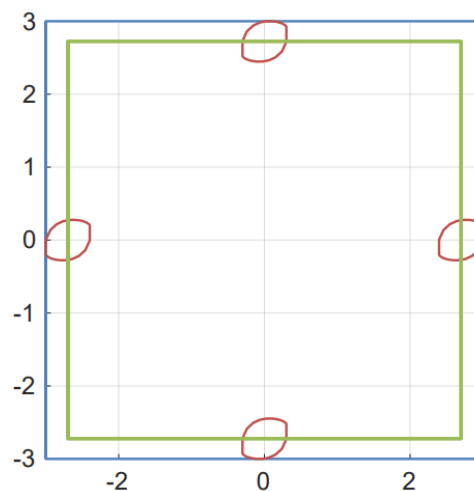


Figure 9.16: State constraint tightening. The original constraint set $\mathcal{X}$ (blue) is shrunk by the error set $\mathcal{E}$ (red). The tightened set $\mathcal{X} \ominus \mathcal{E}$ contains the admissible nominal tube centers. If $z_i \in \mathcal{X} \ominus \mathcal{E}$ (green), then $z_i \oplus \mathcal{E} \subseteq \mathcal{X}$.

Tightening the input constraints

The input constraint is tightened in the same way. The real input is

$$u_i = v_i + Ke_i, \quad e_i \in \mathcal{E}.$$

Therefore the set of possible real inputs around the nominal input v_i is

$$v_i \oplus K\mathcal{E}.$$

To guarantee $u_i \in \mathcal{U}$ for all admissible errors, we impose

$$v_i \in \mathcal{U} \ominus K\mathcal{E}.$$

For the scalar input constraint in the example,

$$\mathcal{U} = \left\{ u \mid \begin{bmatrix} 1 \\ -1 \end{bmatrix} u \leq \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix} \right\},$$

and if

$$\mathcal{E} = \{e \mid Fe \leq f\},$$

then

$$\begin{aligned} \mathcal{U} \ominus K\mathcal{E} &= \left\{ u \mid \begin{bmatrix} 1 \\ -1 \end{bmatrix} u \leq \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix} \right\} \ominus \{Ke \mid Fe \leq f\} \\ &= \left\{ u \mid \begin{bmatrix} 1 \\ -1 \end{bmatrix} u \leq \begin{bmatrix} 0.5 - \max\{Ke \mid Fe \leq f\} \\ 0.5 + \min\{Ke \mid Fe \leq f\} \end{bmatrix} \right\}. \end{aligned}$$

Again, the required extrema are computed offline by linear programs.

Input constraint tightening

The tightened input set is

$$\mathcal{U} \ominus K\mathcal{E} = \{v \mid v + Ke \in \mathcal{U} \forall e \in \mathcal{E}\}.$$

For polyhedral $\mathcal{U}$ and $\mathcal{E}$, this is obtained by tightening each input inequality by the worst possible value of Ke in that inequality direction.

Terminal weights and terminal constraints

The last offline step is to choose the terminal cost ℓ_f and the terminal set $\mathcal{X}_f$. These terminal ingredients must satisfy the same kind of conditions as in nominal MPC, but now with respect to the tightened constraints.

First, the terminal set must be invariant for the nominal system under a local controller κ_f :

$$Az + B\kappa_f(z) \in \mathcal{X}_f \quad \forall z \in \mathcal{X}_f.$$

Second, the terminal set must lie inside the tightened state constraint and the terminal input must lie inside the tightened input constraint:

$$\mathcal{X}_f \subseteq \mathcal{X} \ominus \mathcal{E},$$

$$\kappa_f(z) \in \mathcal{U} \ominus K\mathcal{E} \quad \forall z \in \mathcal{X}_f.$$

Third, the terminal cost must be a Lyapunov function on $\mathcal{X}_f$:

$$\ell_f(Az + B\kappa_f(z)) - \ell_f(z) \leq -\ell(z, \kappa_f(z)) \quad \forall z \in \mathcal{X}_f.$$

Terminal ingredients for tube MPC

The terminal pair $(\ell_f, \mathcal{X}_f)$ must be chosen so that

$$Az + B\kappa_f(z) \in \mathcal{X}_f,$$

$$\mathcal{X}_f \subseteq \mathcal{X} \ominus \mathcal{E}, \quad \kappa_f(z) \in \mathcal{U} \ominus K\mathcal{E},$$

and

$$\ell_f(Az + B\kappa_f(z)) - \ell_f(z) \leq -\ell(z, \kappa_f(z)).$$

These are the standard nominal MPC terminal conditions applied to the tightened nominal system.

A standard choice is to base the terminal ingredients on the LQR controller. One sets

$$\kappa_f(z) = Kz,$$

where the same weights Q and R used in the MPC stage cost are used to compute K . The infinite-horizon cost of the LQR closed loop is

$$J^*(z_0) = \sum_{i=0}^{\infty} z_i^\top (Q + K^\top RK) z_i = z_0^\top P z_0,$$

where P is the solution of the discrete-time Riccati equation. Therefore one can choose

$$\ell_f(z) = z^\top P z.$$

The set $\mathcal{X}_f$ is then chosen as a positively invariant set for

$$z^+ = (A + BK)z$$

contained inside the tightened constraints. This is exactly the same construction used in nominal MPC, but applied after state and input tightening.

Online implementation

Once all offline ingredients have been computed, the online implementation is simple. At each sampling time, one measures or estimates the current state x , solves the tube MPC problem

$$(V^*(x), Z^*(x)) = \arg \min_{V, Z} \{J(Z, V) \mid (Z, V) \in \mathcal{Z}(x)\},$$

and applies

$$u = K(x - z_0^*(x)) + v_0^*(x).$$

Thus the online problem remains a nominal MPC problem in the variables Z, V , but the implemented input contains the feedback correction around the optimized tube center.

Offline/online tube MPC implementation

Offline:

$$K \implies \mathcal{E} \implies \tilde{\mathcal{X}} := \mathcal{X} \ominus \mathcal{E}, \quad \tilde{\mathcal{U}} := \mathcal{U} \ominus K\mathcal{E} \implies \ell_f, \mathcal{X}_f.$$

Online:

$$x \implies (V^*(x), Z^*(x)) \implies u = K(x - z_0^*(x)) + v_0^*(x).$$

Planned tubes and possible trajectories

The effect of the design can be visualized by plotting the planned tubes and the possible future trajectories. The nominal tube centers are generated by the optimized nominal dynamics

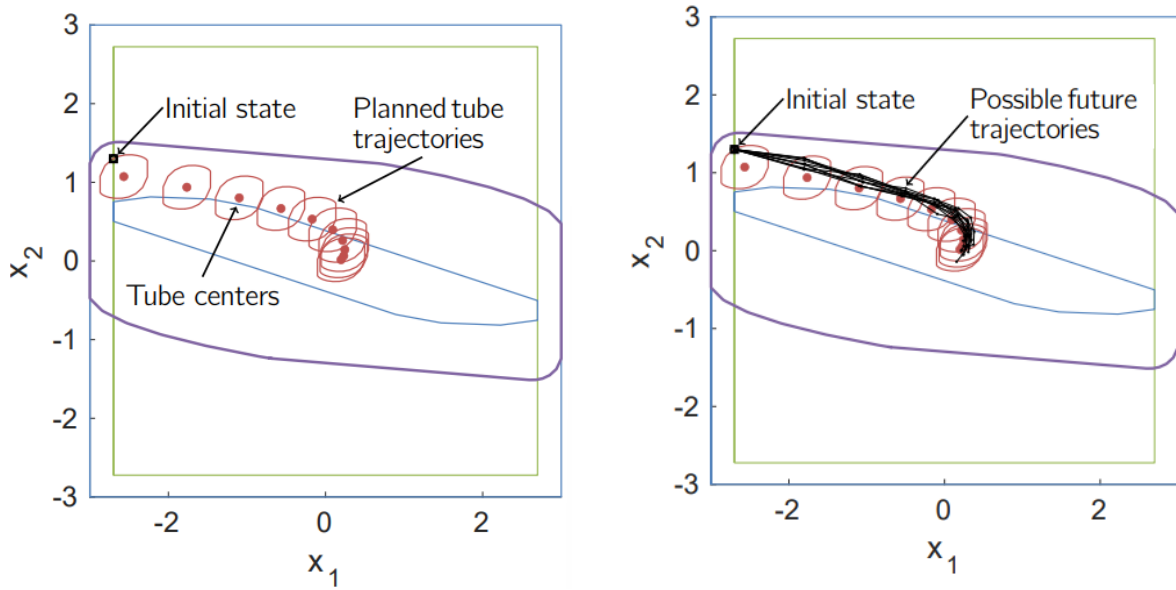
$$z_{i+1} = Az_i + Bv_i.$$

Around each center z_i , the tube cross-section is the invariant error set $\mathcal{E}$. Hence the possible real states lie in

$$z_i \oplus \mathcal{E}.$$

The planned tubes are contained in the original state constraints because the nominal centers satisfy the tightened constraints

$$z_i \in \mathcal{X} \ominus \mathcal{E}.$$



(a) Initial state, tube centers, and planned tube trajectories.

(b) Possible future trajectories inside the planned tubes.

Figure 9.17: Tube MPC example. The nominal trajectory defines the tube centers, while the set $\mathcal{E}$ defines the tube cross-section. Different disturbance realizations generate different real trajectories, but all remain inside the tube and therefore inside the original constraints.

As time evolves and the optimization is repeated, the nominal tube center moves toward the origin. The real disturbed state does not converge exactly to zero, but it remains inside the tube and approaches the robust invariant error set around the origin.

9.9.5 Tube MPC summary

Tube MPC can be summarized as a structured way to include feedback inside robust MPC while keeping the online optimization problem simple. The input is split into two parts:

$$u_i = v_i + Ke_i, \quad e_i = x_i - z_i.$$

The nominal input v_i is optimized online and steers the nominal system. The feedback part Ke_i compensates for disturbances and keeps the real trajectory close to the nominal one.

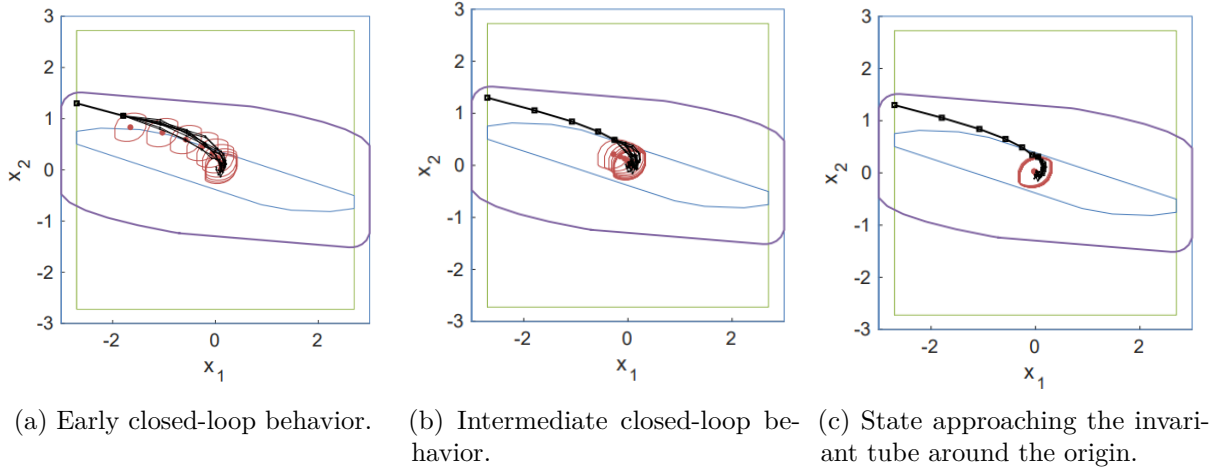


Figure 9.18: Closed-loop tube MPC evolution. The controller repeatedly replans the nominal tube centers, while the feedback correction keeps the real trajectory inside the invariant tube.

The main benefit is that, after the offline computation of $\mathcal{E}$, only one constraint tightening is needed:

$$\tilde{\mathcal{X}} = \mathcal{X} \ominus \mathcal{E}, \quad \tilde{\mathcal{U}} = \mathcal{U} \ominus K\mathcal{E}.$$

The online problem has essentially the same structure as nominal MPC, and the same terminal cost and terminal set ideas can be used. Moreover, nominal MPC theory applies to the tube centers, which gives a clean proof of convergence of the nominal trajectory.

The price is conservatism. Tube MPC is generally suboptimal compared with a fully optimal closed-loop robust controller, because optimizing over arbitrary feedback policies is extremely difficult. The feasible set is also smaller than in nominal MPC, since constraints must be tightened. Finally, one must know a disturbance set $\mathcal{W}$, and this is often unrealistic or conservative in practice.

Tube MPC: main message

Tube MPC splits the input into

$$u_i = v_i + K(x_i - z_i).$$

It optimizes the nominal trajectory and uses the feedback term to keep all disturbed trajectories inside a tube. The method is simple online and has strong stability properties, but it is conservative and requires a known disturbance set $\mathcal{W}$.

9.10 Comparison of Robust Constraint-Tightening MPC and Tube MPC

The two robust MPC approaches described in this chapter have the same objective: compensate for disturbances in the prediction so that all constraints are satisfied by the real system. They differ in how they represent the possible deviation between the nominal and real trajectories.

Robust constraint-tightening MPC uses disturbance reachable sets

$$\mathcal{F}_k = \bigoplus_{j=0}^{k-1} A_K^j \mathcal{W}.$$

The nominal MPC problem uses time-varying tightened constraints

$$\mathcal{X} \ominus \mathcal{F}_k,$$

and the initial nominal state is fixed to the measured state:

$$z_0 = x(k).$$

This approach exploits the fact that the initial prediction error is zero. The tube grows along the prediction horizon as more disturbances may accumulate.

Tube MPC instead uses a robust positively invariant set

$$\mathcal{E}.$$

The nominal MPC problem uses constant tightened constraints

$$\mathcal{X} \ominus \mathcal{E},$$

and the initial tube center is not necessarily equal to the measured state. Instead, it must satisfy

$$x(k) \in z_0 \oplus \mathcal{E}.$$

This formulation is simple and has strong recursive feasibility and stability properties, but the constant tube can be more conservative.

Robust MPC for uncertain systems

Constraint-tightening MPC:

$$\text{DRS } \mathcal{F}_k \implies \text{nominal MPC with } \mathcal{X} \ominus \mathcal{F}_k, \quad z_0 = x(k).$$

Tube MPC:

$$\text{RPI set } \mathcal{E} \implies \text{nominal MPC with } \mathcal{X} \ominus \mathcal{E}, \quad x(k) \in z_0 \oplus \mathcal{E}.$$

The use of an RPI set $\mathcal{E}$ increases the complexity of the offline design and may also complicate the online implementation, depending on the representation of the set. Tube MPC can also have a smaller feasible set, because the same tube cross-section is imposed throughout the horizon. On the other hand, the concept is simple, the online problem remains close to nominal MPC, and the robust stability properties are strong.

9.11 Summary

The main idea of robust MPC for uncertain systems is to compensate for noise in the prediction so that the real state and input satisfy the constraints for all admissible disturbances. This requires replacing single predicted trajectories with sets of possible trajectories, or tubes.

The robust methods discussed here are conservative because they require a disturbance set $\mathcal{W}$ that contains all possible disturbances. If $\mathcal{W}$ is chosen too small, the guarantees may fail. If it is chosen too large, the controller becomes overly cautious and the feasible set may become small. Moreover, robust MPC formulations are generally more complex than nominal MPC, even when the final online problem is simple.

The main benefit is that the feasible set becomes robust invariant. Therefore one knows exactly from which states the controller can guarantee constraint satisfaction for all future times. The methods are also easier to tune in a systematic way, because the designer can trade off robustness against performance through the choice of $\mathcal{W}$, the feedback gain K , and the tube size.

For tube MPC in particular, the central construction is

$$u_i = v_i + K(x_i - z_i),$$

with an invariant error set

$$\mathcal{E}$$

and tightened nominal constraints

$$z_i \in \mathcal{X} \ominus \mathcal{E}, \quad v_i \in \mathcal{U} \ominus K\mathcal{E}.$$

Under the terminal assumptions, the nominal tube center converges to the origin, while the real state converges to the robust invariant set $\mathcal{E}$. Thus robust MPC replaces exact convergence to the origin by convergence to a guaranteed neighborhood of the origin, which is the natural stability objective in the presence of persistent disturbances.

Chapter 10

Robustness of Nominal MPC

10.1 Motivation

The previous chapter introduced robust MPC methods that explicitly account for bounded disturbances in the controller design. In particular, we studied robust open-loop MPC, constraint-tightening MPC, and tube MPC. In all those approaches, the controller is modified so that the predicted trajectories contain safety margins against the effect of disturbances.

In this chapter we consider a different question. Suppose the real system is affected by noise, but the controller ignores this fact and solves the standard nominal MPC problem. This is very common in practice. Engineers often use a nominal model, implement MPC in receding horizon, and rely on feedback and repeated re-optimization to compensate for small disturbances. The question is then: can we say anything rigorous about this strategy?

We consider the noisy linear system

$$x(k+1) = Ax(k) + Bu(k) + w(k),$$

where $w(k)$ is an additive disturbance. Instead of designing a robust MPC controller, we solve the nominal finite-horizon problem

$$\begin{aligned} J^*(x_0) = \min_U \quad & \sum_{i=0}^{N-1} \ell(x_i, u_i) + \ell_f(x_N) \\ \text{subj. to} \quad & x_{i+1} = Ax_i + Bu_i, \quad i = 0, \dots, N-1, \\ & x_i \in \mathcal{X}, \quad u_i \in \mathcal{U}, \quad i = 0, \dots, N-1, \\ & x_N \in \mathcal{X}_f, \\ & x_0 = x(k). \end{aligned}$$

The controller applies only the first element of the optimal input sequence:

$$u(k) = u_0^*(x(k)).$$

Since the plant is disturbed, the true closed-loop system is not the nominal one used in the prediction. It is instead

$$x(k+1) = Ax(k) + Bu_0^*(x(k)) + w(k).$$

Nominal MPC applied to a noisy system

Nominal MPC ignores the disturbance in the prediction model, but the real closed-loop

system is

$$x(k+1) = Ax(k) + Bu_0^*(x(k)) + w(k).$$

Thus the controller is designed for the nominal dynamics, while the plant evolves according to the disturbed dynamics.

The main result is not a full robust constraint-satisfaction guarantee. Unlike tube MPC, nominal MPC does not explicitly enforce tightened constraints, and therefore one cannot generally certify that all constraints are satisfied for all disturbance realizations. However, under suitable regularity assumptions, one can prove a weaker but important property: the closed-loop state converges to a neighborhood of the origin whose size depends on the disturbance magnitude. This is the idea of input-to-state stability.

10.2 Nominal MPC with Noise

10.2.1 Ignoring the noise

The nominal MPC problem assumes that the future states satisfy

$$x_{i+1} = Ax_i + Bu_i.$$

The optimizer therefore computes a trajectory that is feasible and stabilizing for the nominal model. But after applying the first input, the real system evolves as

$$x(k+1) = Ax(k) + Bu_0^*(x(k)) + w(k).$$

Thus, even if the nominal successor

$$Ax(k) + Bu_0^*(x(k))$$

lies in the desired feasible region, the actual successor may be shifted by the disturbance.

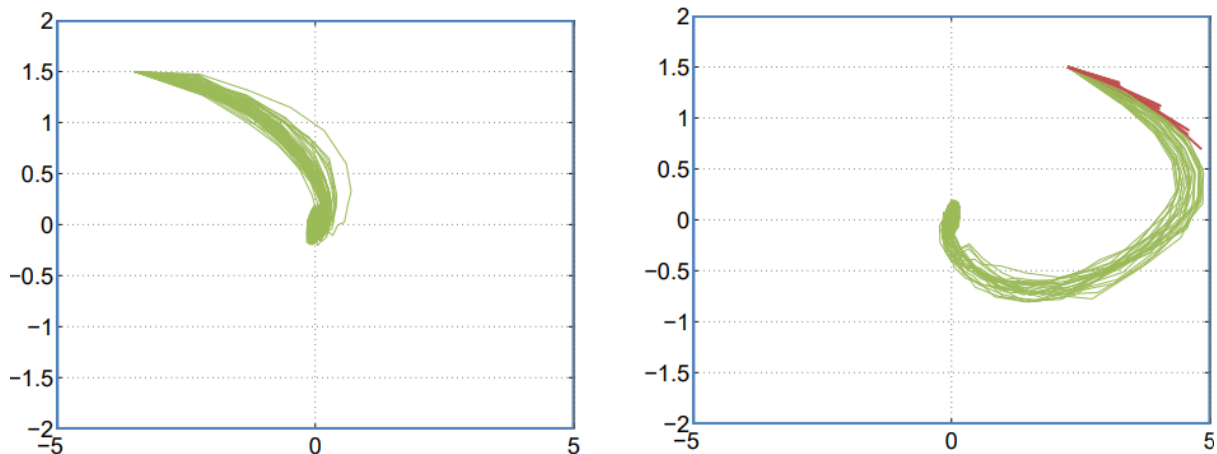
This is precisely the situation that robust MPC was designed to avoid. In robust MPC, one accounts for $w(k)$ explicitly by tightening constraints or propagating uncertainty sets. In nominal MPC, no such margin is introduced. Nevertheless, because MPC is implemented in receding horizon, the controller will remeasure the disturbed state at the next time step and solve a new optimization problem from there. This feedback effect may be enough to compensate for small disturbances in many practical cases.

10.2.2 Example: many disturbance realizations

The behavior of nominal MPC under disturbances can be illustrated by simulating many closed-loop trajectories with different disturbance realizations. The controller is the same in all cases: it is the nominal MPC controller that pretends the noise is absent. The only difference between the trajectories is the disturbance sequence acting on the real system.

In some simulations, all trajectories appear to move toward the origin and remain well behaved. This suggests that nominal MPC can be robust in practice, at least for small disturbances and suitable initial states. However, this observation alone is not a guarantee. Other initial states, or other disturbance realizations, may lead to constraint violations or to poor closed-loop behavior. The important point is therefore not only whether the controller works in a simulation, but how to formalize the region in which it works.

This example motivates the central question of the chapter. If we do not explicitly design a robust controller, can we still prove that the closed-loop system remains bounded or converges near the origin? The answer relies on the behavior of the nominal MPC value function under small perturbations.



(a) For some initial conditions and disturbance realizations, nominal MPC appears to work well despite the noise.

(b) For other initial conditions, the behavior is less certain: the same nominal MPC law may work for some disturbance realizations but not for all.

Figure 10.1: Nominal MPC applied to a noisy system. The controller ignores the disturbance, while the closed-loop trajectories correspond to different noise realizations. The simulations may look acceptable, but they do not provide the same guarantees as robust MPC.

10.3 What Happens to the Lyapunov Function?

10.3.1 Nominal Lyapunov decrease

Recall the standard nominal MPC stability argument. Under the usual terminal ingredients, the optimal cost

$$J^*(x)$$

acts as a Lyapunov function for the nominal closed-loop system

$$x^+ = Ax + Bu^*(x),$$

where $u^*(x)$ denotes the first input selected by the nominal MPC controller. More precisely, one obtains the decrease condition

$$J^*(Ax + Bu^*(x)) - J^*(x) \leq -\ell(x, u^*(x)).$$

This inequality says that the optimal value decreases along nominal closed-loop trajectories by at least the stage cost. Since the stage cost is positive definite, this gives convergence to the origin in the nominal case.

Nominal MPC Lyapunov decrease

For the nominal closed-loop system

$$x^+ = Ax + Bu^*(x),$$

the optimal value function satisfies

$$J^*(Ax + Bu^*(x)) - J^*(x) \leq -\ell(x, u^*(x)).$$

Thus J^* is a Lyapunov function for the nominal closed loop.

In the noisy case, however, the next state is not

$$Ax + Bu^*(x),$$

but

$$Ax + Bu^*(x) + w.$$

The question is therefore whether the Lyapunov decrease survives this perturbation.

10.3.2 Perturbed Lyapunov decrease

Assume that the optimal value function J^* is Lipschitz continuous on the relevant feasible set. This means that there exists a constant $\gamma > 0$ such that

$$|J^*(x_1) - J^*(x_2)| \leq \gamma \|x_1 - x_2\|$$

for all states x_1, x_2 in that set. Applying this property to the nominal successor and the disturbed successor gives

$$\begin{aligned} & |J^*(Ax + Bu^*(x) + w) - J^*(Ax + Bu^*(x))| \\ & \leq \gamma \|Ax + Bu^*(x) + w - (Ax + Bu^*(x))\| = \gamma \|w\|. \end{aligned}$$

Thus the disturbance can increase the Lyapunov function, but the increase is bounded by a term proportional to the disturbance size.

We can now estimate the change of J^* along the disturbed closed loop:

$$\begin{aligned} & J^*(Ax + Bu^*(x) + w) - J^*(x) \\ & = J^*(Ax + Bu^*(x) + w) - J^*(Ax + Bu^*(x)) \\ & \quad + J^*(Ax + Bu^*(x)) - J^*(x) \\ & \leq \gamma \|w\| - \ell(x, u^*(x)). \end{aligned}$$

Hence

$$J^*(Ax + Bu^*(x) + w) - J^*(x) \leq -\ell(x, u^*(x)) + \gamma \|w\|.$$

Perturbed Lyapunov decrease

If J^* is Lipschitz continuous with constant γ , then along the disturbed closed-loop system

$$x^+ = Ax + Bu^*(x) + w$$

one has

$$J^*(x^+) - J^*(x) \leq -\ell(x, u^*(x)) + \gamma \|w\|.$$

This inequality explains the qualitative behavior of nominal MPC under bounded disturbances. The negative term

$$-\ell(x, u^*(x))$$

becomes larger in magnitude when the state is far from the origin, while the positive term

$$\gamma \|w\|$$

is bounded by the size of the disturbance. Therefore, far from the origin the Lyapunov function still decreases. Close to the origin, however, the nominal decrease becomes small and may be balanced by the disturbance effect. The state then cannot be expected to converge exactly to zero, but it can converge to a neighborhood of zero.

10.4 Special Case: When Nominal MPC is Robust

The previous argument depends on the Lipschitz continuity of the optimal value function. This property is not automatic for arbitrary nonlinear MPC problems, but it holds in important linear settings. In particular, if the nominal model is linear, the cost is linear or quadratic, and the constraint sets are polytopes, then the finite-horizon optimal value function

$$V_N^*(x)$$

is Lipschitz continuous on the feasible set $\mathcal{X}_N$.

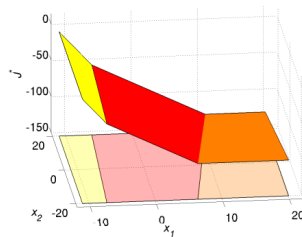
This is consistent with the explicit MPC results discussed earlier. For linear MPC with polyhedral constraints, the optimal control law and value function have a piecewise structure. With linear costs, the value function is piecewise affine. With quadratic costs, the value function is piecewise quadratic. On compact polytopic regions, these piecewise affine or piecewise quadratic functions are Lipschitz continuous.

Lipschitz value function for linear nominal MPC

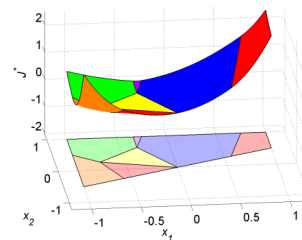
If the model is linear, the constraints are polytopic, and the finite-horizon cost is linear or quadratic, then the nominal MPC value function

$$V_N^*(x)$$

is Lipschitz continuous on the feasible set $\mathcal{X}_N$.



Example optimal cost for linear cost function



Example optimal cost for quadratic cost function

Figure 10.2: Piecewise structure of the nominal MPC value function. For linear MPC with polytopic constraints, a linear cost leads to a piecewise affine value function, while a quadratic cost leads to a piecewise quadratic value function. On compact feasible regions, such functions are Lipschitz continuous.

This special case is important because it gives a rigorous explanation for why nominal MPC may remain well behaved under small disturbances. The nominal Lyapunov decrease is not destroyed completely by the disturbance; it is only perturbed by a bounded term.

10.5 Input-to-State Stability Interpretation

The perturbed Lyapunov inequality

$$J^*(x^+) - J^*(x) \leq -\ell(x, u^*(x)) + \gamma \|w\|$$

is the key to the input-to-state stability interpretation. In nominal asymptotic stability, the Lyapunov function decreases monotonically to zero, and the state converges to the origin. With

persistent disturbances, the decrease is interrupted by the additive term depending on $\|w\|$.

If the state is large, the stage cost $\ell(x, u^*(x))$ is large, and the Lyapunov function decreases. As the state becomes smaller, the stage cost becomes smaller as well. Eventually, the nominal decrease and the disturbance-induced increase balance each other. At that point, one can no longer guarantee convergence closer to the origin. Instead, one guarantees convergence to a set around the origin whose size depends on the maximum disturbance magnitude:

$$\max_{w \in \mathcal{W}} \|w\|.$$

Input-to-state stability intuition

Nominal MPC applied to a disturbed system may still be input-to-state stable. The state is driven toward the origin until the nominal Lyapunov decrease is balanced by the disturbance effect. Therefore, the closed-loop state converges to a neighborhood of the origin whose size is determined by the size of the disturbance.

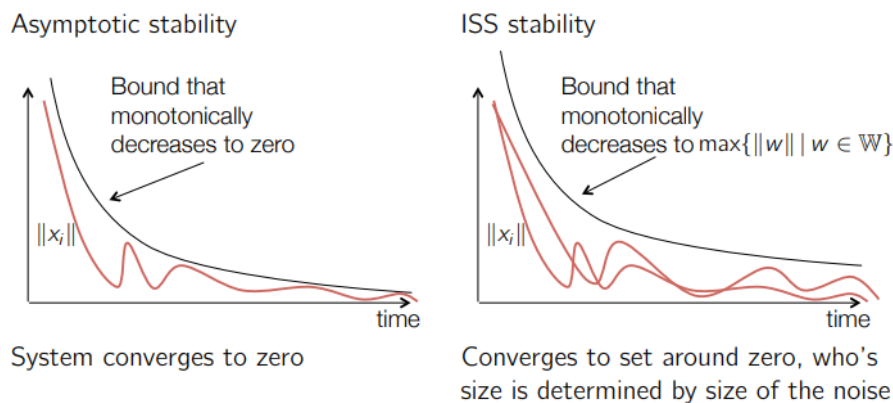


Figure 10.3: Comparison between asymptotic stability and input-to-state stability. In the disturbance-free case, the bound on the state decreases to zero. In the disturbed case, the state converges to a neighborhood of the origin, whose size depends on the disturbance magnitude.

This is weaker than the guarantees obtained with robust constraint tightening or tube MPC. In tube MPC, the invariant tube is explicitly constructed, and constraints are tightened to guarantee robust satisfaction. For nominal MPC, the result is more qualitative: small disturbances may lead only to small steady-state deviations, but the controller does not explicitly certify robust constraint satisfaction for all states and all disturbances.

10.6 Discussion

The robustness of nominal MPC is both practically important and conceptually subtle. On the one hand, nominal MPC is simple. It does not require the designer to know the disturbance set $\mathcal{W}$, and it often works well in practice. This is one reason why nominal MPC is so widely used: repeated measurement and re-optimization already provide some disturbance rejection.

On the other hand, nominal MPC does not provide the same guarantees as robust MPC. Since the controller ignores the disturbance in the optimization problem, the feasible nominal trajectory may lie close to the boundary of the constraints. A disturbance can then push the real state outside the admissible set. Moreover, the region of attraction is difficult to characterize. Some initial states may work for many disturbance realizations, while others may fail for only a

few.

Thus nominal MPC has a large apparent feasible set, because the optimization problem is easier to satisfy. But this feasible set should not be confused with a robust feasible set. It contains states from which the nominal problem has a solution, not necessarily states from which the real disturbed closed loop is guaranteed to satisfy constraints for all time.

Nominal MPC versus robust MPC

Nominal MPC may be robust in the sense of input-to-state stability, but it does not generally provide robust constraint satisfaction. Robust MPC explicitly accounts for the disturbance, whereas nominal MPC relies on feedback and repeated re-optimization.

10.7 Summary

Nominal MPC for uncertain systems is based on a simple idea: ignore the noise in the prediction model and rely on receding-horizon feedback to compensate for it. The nominal controller solves

$$\begin{aligned} \min_U \quad & \sum_{i=0}^{N-1} \ell(x_i, u_i) + \ell_f(x_N) \\ \text{subj. to} \quad & x_{i+1} = Ax_i + Bu_i, \\ & x_i \in \mathcal{X}, \quad u_i \in \mathcal{U}, \\ & x_N \in \mathcal{X}_f, \end{aligned}$$

but the actual closed-loop system is

$$x(k+1) = Ax(k) + Bu_0^*(x(k)) + w(k).$$

The main benefit of this approach is simplicity. No disturbance set $\mathcal{W}$ is required, the online optimization problem is the standard nominal MPC problem, and the method is often effective in practice. The nominal feasible set may also be larger than the feasible set of robust MPC methods, because no constraints are tightened.

The main limitation is the absence of explicit robust guarantees. It is difficult to determine the true region of attraction, hard to tune the tradeoff between robustness and performance, and constraint satisfaction cannot generally be guaranteed for all disturbance realizations. For linear systems with polytopic constraints and linear or quadratic costs, the nominal MPC value function is Lipschitz continuous, and this allows one to prove an input-to-state stability type result:

$$J^*(x^+) - J^*(x) \leq -\ell(x, u^*(x)) + \gamma \|w\|.$$

Thus the system moves toward the origin until the nominal decrease is balanced by the disturbance effect. The correct conclusion is convergence to a neighborhood of the origin, not necessarily convergence to the origin itself.

Part V

Implementation Methods

Chapter 11

Implementation Methods

11.1 Motivation

The previous chapters focused mainly on how an MPC problem is formulated and on which properties its closed loop may have. We studied nominal MPC, practical extensions, robust constraint satisfaction, tube MPC, and robustness properties of nominal MPC. In all these cases, however, one essential issue remains: at every sampling instant the controller must actually compute a control input.

At time k , a general finite-horizon MPC problem has the form

$$U_k^* = \arg \min_{U_k} V_f(x_{k+N}) + \sum_{i=0}^{N-1} \ell(x_{k+i}, u_{k+i})$$

subject to

$$x_k = x(k), \quad x_{k+i+1} = g(x_{k+i}, u_{k+i}),$$

and to the constraints

$$x_{k+i} \in \mathcal{X}, \quad u_{k+i} \in \mathcal{U}, \quad x_{k+N} \in \mathcal{X}_f.$$

The optimizer returns an optimal input sequence

$$U_k^* = \{u_k^*, u_{k+1}^*, \dots, u_{k+N-1}^*\},$$

but only the first element u_k^* is applied to the plant. At the next sampling instant, the state is measured again and the optimization problem is solved again.

From an implementation point of view, this means that the theoretical MPC law is useful only if this optimization problem can be solved fast enough and reliably enough for the desired sampling time. In slow process-control applications, one may have seconds or minutes available. In fast mechanical or electrical systems, the admissible computation time may be milliseconds, microseconds, or even less. Thus implementation is not a secondary detail: it is part of the controller design.

There are two main ways to implement the MPC law. The first is to solve the optimization problem online at every sampling instant using an iterative optimization algorithm. The second is to solve the parametric optimization problem offline and store the resulting feedback law. The online computation is then reduced to evaluating a precomputed piecewise affine function.

The purpose of this chapter is to explain these two viewpoints. We first discuss explicit MPC, which exploits the parametric structure of linear MPC problems. Then we discuss iterative optimization methods, with particular attention to gradient methods, Newton-type methods, projected-gradient methods, and primal-dual interior-point methods.

11.2 Explicit MPC

11.2.1 Parametric viewpoint

Consider the linear quadratic MPC problem

$$\begin{aligned}
 U^*(x(k)) = \arg \min_U \quad & x_N^\top P x_N + \sum_{i=0}^{N-1} (x_i^\top Q x_i + u_i^\top R u_i) \\
 \text{subj. to} \quad & x_0 = x(k), \\
 & x_{i+1} = A x_i + B u_i, \quad i = 0, \dots, N-1, \\
 & x_i \in \mathcal{X}, \quad u_i \in \mathcal{U}, \quad i = 0, \dots, N-1, \\
 & x_N \in \mathcal{X}_f.
 \end{aligned}$$

For a fixed value of $x(k)$, this is an ordinary quadratic program. However, from the controller-design viewpoint, $x(k)$ is not just a number: it is the parameter with respect to which the controller must be evaluated. The MPC feedback law is therefore the map

$$x(k) \mapsto u_0^*(x(k)).$$

After eliminating the predicted states, the quadratic-cost CFTOC problem can be written as

$$J^*(x) = \min_U \begin{bmatrix} U \\ x \end{bmatrix}^\top \begin{bmatrix} H & F^\top \\ F & Y \end{bmatrix} \begin{bmatrix} U \\ x \end{bmatrix}$$

subject to

$$GU \leq w + Ex.$$

This is a multiparametric quadratic program, or mp-QP, where the parameter is the current state x . The matrices H, F, Y, G, w, E are fixed offline once the model, horizon, constraints, and cost weights have been chosen.

The central result is that the solution of this parametric problem has a finite polyhedral structure. The feasible set can be partitioned into polyhedral regions, called critical regions, and on each region the optimal control law is affine.

Explicit MPC structure for quadratic-cost linear MPC

For linear dynamics, polyhedral constraints, and quadratic cost, the MPC problem is an mp-QP. On the feasible set X_N , the first optimal input has the form

$$u_0^* = \kappa(x),$$

where

$$\kappa(x) = F_j x + g_j \quad \text{if } x \in CR_j, \quad j = 1, \dots, N_r.$$

The sets

$$CR_j = \{x \in \mathbb{R}^n \mid H_j x \leq K_j\}$$

are polyhedra and form a partition of the feasible set. The value function $J^*(x)$ is convex and piecewise quadratic on the same type of polyhedral partition.

Thus explicit MPC replaces the repeated solution of a QP by the evaluation of a piecewise affine feedback law. The work is shifted offline: one computes all critical regions and all affine laws in advance. Online, one only needs to determine which region contains the measured state and then evaluate the corresponding affine expression.

11.2.2 Active sets and critical regions

The polyhedral regions in explicit MPC are directly related to active sets. To see this, consider the parametric QP

$$\begin{aligned} \min_U \quad & U^\top H U + 2x^\top F U + x^\top Y x \\ \text{subj. to} \quad & G U \leq w + E x. \end{aligned}$$

Let

$$I := \{1, \dots, m_c\}$$

be the set of indices of all inequality constraints, where m_c denotes the number of scalar constraints. For a given parameter x , the active set is the set of constraints that hold with equality at the optimizer:

$$A(x) := \{j \in I \mid G_j U^*(x) - E_j x = w_j\}.$$

The inactive set is its complement:

$$NA(x) := \{j \in I \mid G_j U^*(x) - E_j x < w_j\}.$$

Here G_j , E_j , and w_j denote the j -th row of G , the j -th row of E , and the j -th component of w , respectively.

A critical region is the set of parameters for which the optimizer has the same active set. If $\bar{x}$ is a feasible parameter and

$$A := A(\bar{x}),$$

then the corresponding critical region is

$$CR_A := \{x \in X_N \mid A(x) = A\}.$$

Inside such a region, the same constraints are active and inactive. Therefore the KKT system has the same algebraic structure throughout the whole region. This is the reason why the optimizer is affine on each critical region.

The intuition is the following. If the active constraints are known, then they can be treated as equalities. Solving the equality-constrained KKT system gives an affine expression in the parameter x . The region of validity of this expression is then obtained by imposing two conditions: the inactive constraints must remain inactive, and the Lagrange multipliers of the active constraints must remain nonnegative. Both conditions are linear inequalities in x , which explains why the critical regions are polyhedral.

11.2.3 A scalar example

The main idea of parametric programming can already be seen in a one-dimensional example. Consider

$$J^*(x) = \min_z \left(\frac{1}{2} z^2 + 2xz + 2x^2 \right)$$

subject to

$$z \leq 1 + x,$$

where $x \in \mathbb{R}$ is a parameter. The goal is to find the optimizer $z^*(x)$, the set of parameters for which the problem has a solution, and the value function $J^*(x)$.

The Lagrangian is

$$L(z, x, \lambda) = \frac{1}{2}z^2 + 2xz + 2x^2 + \lambda(z - x - 1).$$

The KKT conditions are

$$z + 2x + \lambda = 0,$$

$$z - x - 1 \leq 0,$$

$$\lambda(z - x - 1) = 0,$$

and

$$\lambda \geq 0.$$

There are two possible cases. First, suppose that the constraint is active:

$$z - x - 1 = 0.$$

Then

$$z^*(x) = x + 1.$$

Using stationarity,

$$x + 1 + 2x + \lambda = 0,$$

so

$$\lambda = -3x - 1.$$

Dual feasibility requires

$$-3x - 1 \geq 0,$$

and therefore

$$x \leq -\frac{1}{3}.$$

In this region,

$$J^*(x) = \frac{9}{2}x^2 + 3x + \frac{1}{2}.$$

Second, suppose that the constraint is inactive:

$$z - x - 1 < 0.$$

Then $\lambda = 0$, and stationarity gives

$$z^*(x) = -2x.$$

The inactive constraint condition becomes

$$-2x - x - 1 < 0,$$

or

$$x > -\frac{1}{3}.$$

In this region,

$$J^*(x) = 0.$$

Hence the optimizer is the piecewise affine function

$$z^*(x) = \begin{cases} x + 1, & x \leq -\frac{1}{3}, \\ -2x, & x \geq -\frac{1}{3}, \end{cases}$$

and the value function is

$$J^*(x) = \begin{cases} \frac{9}{2}x^2 + 3x + \frac{1}{2}, & x \leq -\frac{1}{3}, \\ 0, & x \geq -\frac{1}{3}. \end{cases}$$

This simple example shows the same structure as explicit MPC: different active sets define different parameter regions, the optimizer is affine on each region, and the value function is quadratic on each region.

11.2.4 Example: double integrator

A standard example used to visualize explicit MPC is the constrained double integrator

$$x(t+1) = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} x(t) + \begin{bmatrix} 0 \\ 1 \end{bmatrix} u(t),$$

with output

$$y(t) = \begin{bmatrix} 1 & 0 \end{bmatrix} x(t).$$

The system is subject to input constraints

$$-1 \leq u(k) \leq 1, \quad k = 0, \dots, 5,$$

and state constraints

$$\begin{bmatrix} -10 \\ -10 \end{bmatrix} \leq x(k) \leq \begin{bmatrix} 10 \\ 10 \end{bmatrix}, \quad k = 0, \dots, 5.$$

One may compute the explicit state-feedback solution for a horizon $N = 6$, weights

$$Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad R = 0.1,$$

terminal weight P given by the solution of the algebraic Riccati equation, and terminal set

$$\mathcal{X}_f = \mathbb{R}^2.$$

The resulting state space is partitioned into finitely many critical regions. The first optimal input is represented by a piecewise affine surface over the state space. The important point is not the exact number of regions, but the qualitative structure: the optimal feedback is not a single linear gain, as in unconstrained LQR. Instead, the constraints divide the state space into many regions, and the controller has a different affine expression in each of them.

This example also illustrates the main limitation of explicit MPC. Even for a two-dimensional system and a moderate horizon, the number of regions can already be significant. As the number of states, inputs, constraints, and horizon length increase, the number of critical regions may grow very quickly. This growth is one of the main reasons why explicit MPC is mostly limited to small systems.

11.2.5 Norm-based costs

The same explicit-control idea also applies to MPC problems with 1-norm or ∞ -norm costs. As discussed in the chapter on constrained finite time optimal control, these problems can be reformulated as linear programs of the form

$$\min_z c^\top z$$

subject to

$$\bar{G}z \leq \bar{w} + \bar{S}x(k).$$

This is a multiparametric linear program, or mp-LP. Its solution has a structure analogous to the mp-QP case.

Explicit MPC structure for norm-based costs

For linear dynamics, polyhedral constraints, and 1-norm or ∞ -norm costs, the MPC problem is an mp-LP. A piecewise affine control law exists:

$$\kappa(x) = F_j x + g_j \quad \text{if } x \in CR_j.$$

The feasible set is partitioned into polyhedral critical regions. The value function is convex and piecewise affine on these regions.

The main difference from the quadratic case is the shape of the value function. With a strictly convex quadratic cost, the value function is piecewise quadratic and the optimizer is typically unique. With a linear cost, the value function is piecewise affine and multiple optimizers may occur. Nevertheless, a piecewise affine feedback law can still be selected.

11.3 Online Evaluation of Explicit MPC

Once an explicit MPC controller has been computed offline, the online task is no longer the solution of an optimization problem. The controller is already available as a piecewise affine function of the state. Therefore, at each sampling instant, one only has to evaluate this piecewise affine map at the current measured state $x(k)$.

More precisely, the explicit control law has the form

$$u^*(x) = F_j x + g_j \quad \text{if } x \in CR_j, \quad j = 1, \dots, N_r,$$

where the critical regions

$$CR_j = \{x \in \mathbb{R}^n \mid H_j x \leq K_j\}$$

form a polyhedral partition of the feasible set. Hence the online evaluation naturally separates into two steps. First, one has to determine which critical region contains the current state. Second, once the correct region has been found, one evaluates the corresponding affine control law.

The important observation is that the affine evaluation itself is very cheap: it only requires a matrix-vector multiplication and an addition. Thus the computational bottleneck is usually not the formula

$$u(k) = F_j x(k) + g_j,$$

but rather the point-location problem:

$$\text{find } j \text{ such that } x(k) \in CR_j.$$

On a boundary between two critical regions, more than one region may contain the same state. For quadratic-cost explicit MPC, the control law is continuous, so this ambiguity does not change the applied input.

11.3.1 Sequential search

The most direct way to solve the point-location problem is sequential search. Each critical region is tested one after the other until the region containing the current state is found. If the regions are stored as

$$CR_j = \{x \in \mathbb{R}^n \mid A_j x + b_j \leq 0\},$$

then the algorithm is simply

for each j :

if $A_j x + b_j \leq 0$, then x is in region j .

Once the correct region has been found, the corresponding affine law is evaluated.

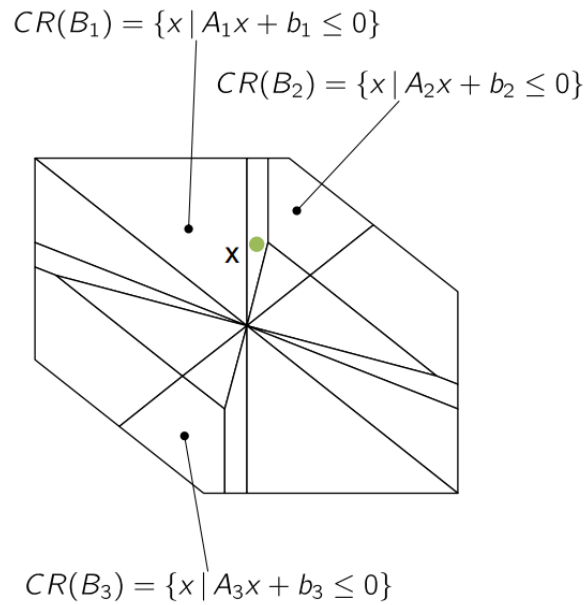


Figure 11.1: Sequential search for point location. The regions are checked one by one by testing their defining inequalities. The method is very simple and works for arbitrary explicit MPC partitions, but its online cost grows linearly with the number of regions.

The advantage of sequential search is its simplicity. It requires almost no additional offline processing beyond storing the polyhedral regions themselves, and it works for all explicit MPC problems. The disadvantage is that, in the worst case, all regions may have to be tested before the correct one is found. Therefore, if the explicit solution contains N_r regions, the online complexity is proportional to

$$N_r.$$

This is acceptable for small explicit controllers, but it becomes inefficient when the number of critical regions is large.

11.3.2 Logarithmic search

A faster method can be obtained by constructing a search tree offline. The idea is to avoid testing the regions one by one. Instead, one chooses a hyperplane that separates the set of regions into two groups of roughly equal size. Then the same procedure is repeated recursively on the two subsets. In this way, the offline phase builds a binary decision tree.

At each node of the tree, the online controller only has to test on which side of a separating hyperplane the current state lies. For example, if the separating hyperplane is described by

$$h^\top x + c = 0,$$

then the test is

$$h^\top x + c \leq 0 \quad \text{or} \quad h^\top x + c > 0.$$

The result determines whether the search continues in the left or in the right branch of the tree.

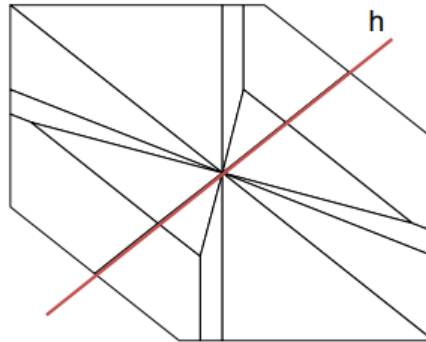


Figure 11.2: Offline construction of a search tree. A separating hyperplane is chosen so as to split the set of critical regions into two subsets of comparable size. The same construction is then repeated recursively on the two subsets.

The online procedure is then very similar to a binary search. Starting from the root node, the current state is tested against the first separating hyperplane. This discards approximately half of the candidate regions. A second hyperplane is then tested in the remaining subset, discarding again a large portion of the remaining candidates. Repeating this process leads to a leaf of the tree, which identifies the critical region containing the state.

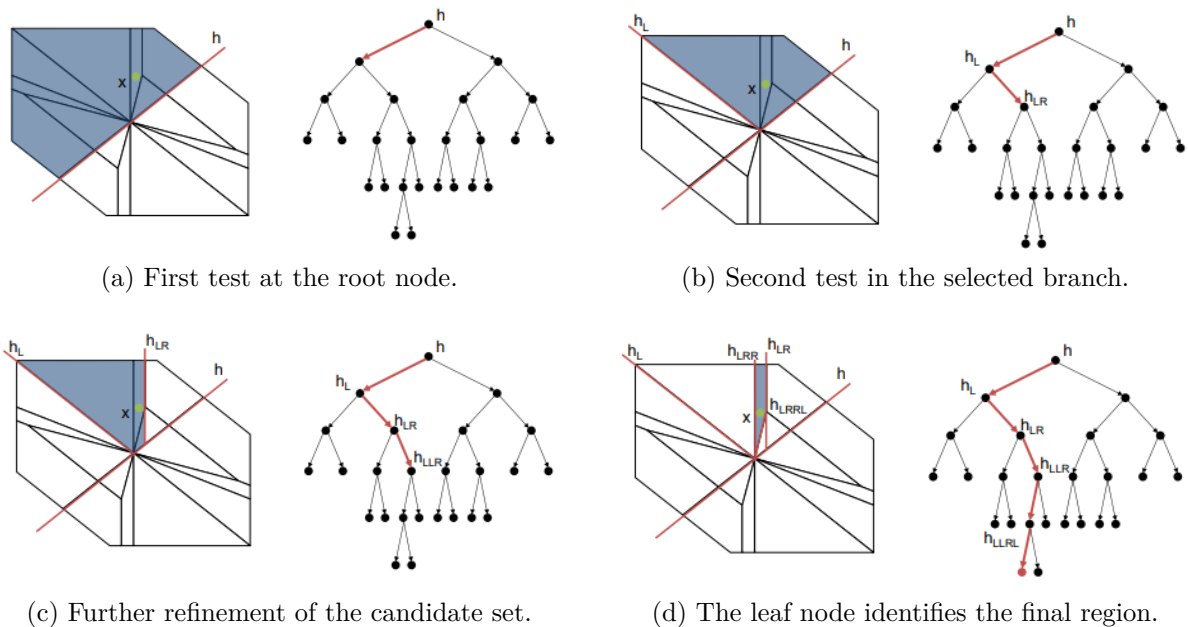


Figure 11.3: Logarithmic search for explicit MPC. The left part of shows the progressive localization of the state in the polyhedral partition. The right part shows the corresponding path in the binary search tree. Each hyperplane test eliminates a large subset of candidate regions.

If the tree is well balanced, the number of online tests grows approximately as

$$\log_2(N_r),$$

rather than as N_r . This is the reason for the name logarithmic search. The improvement can be substantial when the explicit controller contains many regions.

The price to pay is offline complexity. The search tree must be built before the controller is implemented. This requires selecting separating hyperplanes, organizing the regions into a tree, and storing the resulting data structure. Therefore logarithmic search is most useful when the explicit controller is not too large to preprocess, but large enough that sequential search would be inefficient. In practice, this is reasonable for explicit controllers with up to roughly a few hundred or a few thousand regions, depending on the implementation platform and memory constraints.

11.3.3 Summary of explicit MPC implementation

The main advantage of explicit MPC is that the optimization problem is solved before runtime. For linear MPC with polyhedral constraints and quadratic or linear-norm costs, the controller is piecewise affine. Therefore, after offline computation, the online controller only evaluates a piecewise affine function.

The online evaluation is extremely fast. Once the correct critical region is known, the input is computed as

$$u(k) = F_j x(k) + g_j.$$

This can often be done in nanoseconds or microseconds on suitable hardware. This makes explicit MPC attractive for applications with very small sampling times.

However, explicit MPC is limited by the size of the polyhedral partition. As the state dimension, input dimension, horizon length, and number of constraints increase, the number of regions may grow very quickly. The controller may then become too expensive to compute offline, too large to store, or too slow to evaluate by point location. For this reason, explicit MPC is mainly used for small systems, typically with a few states.

In summary, explicit MPC replaces online optimization by offline parametric optimization and online point location. This is extremely effective for small problems, but online optimization methods are usually preferable for medium- and large-scale MPC problems.

11.4 Iterative Numerical Optimization

11.4.1 Why numerical methods are needed

Explicit MPC shifts the computational burden offline, but this is not always possible. In most practical MPC applications, the optimization problem is solved online at every sampling instant. This requires efficient numerical optimization algorithms.

Recall the general convex optimization problem

$$\begin{aligned} \min_x \quad & f(x) \\ \text{subj. to} \quad & x \in Q. \end{aligned}$$

The problem is convex if f and Q are convex. The most important special cases for MPC are linear programs,

$$\begin{aligned} \min_x \quad & c^\top x \\ \text{subj. to} \quad & Ax = b, \\ & Gx \leq h, \end{aligned}$$

and quadratic programs,

$$\begin{aligned} \min_x \quad & \frac{1}{2}x^\top Hx + c^\top x \\ \text{subj. to} \quad & Ax = b, \\ & Gx \leq h, \end{aligned}$$

where the QP is convex if

$$H \succeq 0.$$

In all but the simplest cases, an analytical optimizer cannot be written down explicitly. A numerical method therefore starts from an initial guess $x^{(0)}$ and produces a sequence of iterates

$$x^{(i+1)} = \Psi(x^{(i)}, f, Q), \quad i = 0, 1, \dots, m-1.$$

The goal is to obtain an approximate solution $x^{(m)}$ satisfying

$$|f(x^{(m)}) - f(x^*)| \leq \varepsilon$$

and

$$\text{dist}(x^{(m)}, Q) \leq \delta,$$

where ε and δ are tolerances chosen by the user.

In MPC, these tolerances have a control interpretation. Solving the optimization problem only approximately means that the applied input may not be exactly optimal, and may even be slightly infeasible if constraint tolerances are not handled carefully. Therefore solver accuracy, computation time, and closed-loop performance are directly linked.

11.5 Unconstrained Minimization

We now turn to the first family of iterative optimization methods. Before considering constrained problems, it is useful to understand the simpler unconstrained case

$$\min_x f(x), \quad f: \mathbb{R}^n \rightarrow \mathbb{R}.$$

Throughout this section we assume that f is convex and twice continuously differentiable, and that the optimal value

$$p^* = \min_x f(x)$$

is finite. These assumptions are stronger than what is needed for all algorithms, but they allow us to present the main ideas cleanly.

For unconstrained convex minimization, optimality is characterized by a very simple condition. If f is differentiable, then a point x^* is a global minimizer if and only if

$$\nabla f(x^*) = 0.$$

Thus the problem of minimizing f can be interpreted as the problem of solving the nonlinear system of equations

$$\nabla f(x) = 0.$$

In general, this system has no closed-form analytical solution. Numerical methods therefore construct a sequence of iterates that hopefully converges to a point satisfying this optimality condition.

Necessary and sufficient optimality condition

Assume that f is convex and differentiable. Then

$$x^* \in \arg \min_x f(x) \quad \Longleftrightarrow \quad \nabla f(x^*) = 0.$$

Unconstrained minimization methods can therefore be interpreted as iterative methods for solving the optimality condition $\nabla f(x^*) = 0$.

11.5.1 Descent methods

The basic idea behind descent methods is to move from the current iterate to a new point with a smaller objective value. Starting from $x^{(i)}$, one chooses a step, or search direction, $\Delta x^{(i)}$, and a step size, or step length, $h^{(i)} > 0$. The next iterate is then defined by

$$x^{(i+1)} = x^{(i)} + h^{(i)} \Delta x^{(i)}.$$

The direction $\Delta x^{(i)}$ is called a descent direction if the step can be chosen so that

$$f(x^{(i+1)}) < f(x^{(i)}).$$

A useful sufficient condition is

$$\nabla f(x^{(i)})^\top \Delta x^{(i)} < 0.$$

Indeed, the quantity $\nabla f(x^{(i)})^\top \Delta x^{(i)}$ is the directional derivative of f at $x^{(i)}$ in the direction $\Delta x^{(i)}$. If this number is negative, then moving a small enough positive distance in that direction decreases the objective.

A general descent method has the following structure:

Input: starting point $x^{(0)} \in \text{dom } f$,
repeat
 1. Compute a descent direction $\Delta x^{(i)}$,
 2. Choose a step size $h^{(i)} > 0$ such that
 $f(x^{(i)} + h^{(i)} \Delta x^{(i)}) < f(x^{(i)})$,
 3. Update $x^{(i+1)} := x^{(i)} + h^{(i)} \Delta x^{(i)}$,
until a termination condition is satisfied.

Typical stopping criteria are based on the objective decrease or on the change between successive iterates, for instance

$$f(x^{(m)}) - f(x^*) \leq \varepsilon_1 \quad \text{or} \quad \|x^{(m)} - x^{(m-1)}\| \leq \varepsilon_2.$$

In practice, $f(x^*)$ is usually not known, so stopping conditions are often expressed in terms of computable quantities such as gradient norms, primal and dual residuals, or relative changes in the objective.

11.5.2 Gradient method

The simplest way to choose a descent direction is to use the gradient. Since $\nabla f(x)$ points in the direction of steepest local ascent, the negative gradient points in the direction of steepest local descent. The gradient descent update is therefore

$$x^{(i+1)} = x^{(i)} - h^{(i)} \nabla f(x^{(i)}).$$

This formula has an immediate interpretation: at each iteration, one moves against the local slope of the objective. The remaining question is how to choose the step size $h^{(i)}$. If the step is too small, convergence is slow. If the step is too large, the method may overshoot and increase the objective.

A standard way to obtain a safe constant step size is to assume that f is L -smooth. This means that the gradient is Lipschitz continuous with constant L :

$$\|\nabla f(x) - \nabla f(y)\| \leq L \|x - y\|, \quad \forall x, y \in \mathbb{R}^n.$$

Equivalently, f can be upper bounded by a quadratic approximation:

$$f(x) \leq f(y) + \nabla f(y)^\top (x - y) + \frac{L}{2} \|x - y\|^2, \quad \forall x, y \in \mathbb{R}^n.$$

This inequality says that, around every point y , the function lies below a quadratic model with curvature L . The larger L is, the more conservative the step size must be.

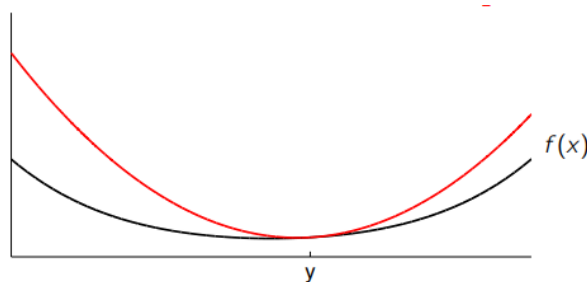


Figure 11.4: L -smoothness and constant step size. If the gradient is Lipschitz continuous with constant L , the function can be upper bounded by a quadratic model. This motivates the constant step size $h^{(i)} = 1/L$.

Under this smoothness assumption, a natural constant choice is

$$h^{(i)} = \frac{1}{L}.$$

The main advantage of gradient descent is that each iteration is cheap: only the gradient has to be evaluated. The main disadvantage is that many iterations may be needed, especially for ill-conditioned problems. This is important in MPC, where one must often trade cheap iterations against the strict real-time requirement imposed by the sampling period.

11.5.3 Newton's method

Newton's method chooses the search direction in a more sophisticated way. Instead of using only the first-order information contained in the gradient, it builds a second-order approximation of the objective around the current iterate $x^{(i)}$. The local quadratic model is

$$\begin{aligned} \tilde{f}(x, x^{(i)}) &= f(x^{(i)}) + \nabla f(x^{(i)})^\top (x - x^{(i)}) \\ &\quad + \frac{1}{2} (x - x^{(i)})^\top \nabla^2 f(x^{(i)}) (x - x^{(i)}). \end{aligned}$$

Newton's idea is to minimize this second-order approximation:

$$x^{(i+1)} = \arg \min_x \tilde{f}(x, x^{(i)}).$$

The first-order condition for the minimizer of the quadratic model is

$$\nabla_x \tilde{f}(x, x^{(i)})|_{x=x^{(i+1)}} = 0.$$

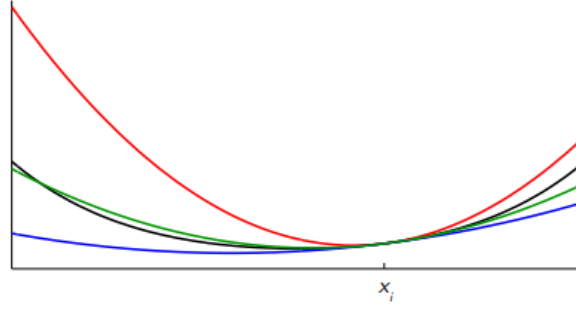


Figure 11.5: Newton's method. At the current iterate $x^{(i)}$, the function is approximated by a second-order Taylor model. The Newton step minimizes this local quadratic model. Unlike the L -smoothness upper bound, this quadratic approximation is not necessarily a global upper bound on f .

Computing the derivative gives

$$\nabla f(x^{(i)}) + \nabla^2 f(x^{(i)})(x^{(i+1)} - x^{(i)}) = 0.$$

Therefore,

$$x^{(i+1)} = x^{(i)} - \left(\nabla^2 f(x^{(i)}) \right)^{-1} \nabla f(x^{(i)}).$$

The corresponding Newton direction is

$$\Delta x_{\text{nt}}^{(i)} = - \left(\nabla^2 f(x^{(i)}) \right)^{-1} \nabla f(x^{(i)}).$$

Hence the full Newton update can be written as

$$x^{(i+1)} = x^{(i)} + \Delta x_{\text{nt}}^{(i)}.$$

However, the quadratic model $\tilde{f}$ is not necessarily an upper bound on f . Therefore, a full Newton step does not always decrease the original objective. It may happen that

$$f(x^{(i+1)}) > f(x^{(i)}),$$

especially when the current iterate is far from the minimizer. For this reason, one often uses a damped Newton step:

$$x^{(i+1)} = x^{(i)} + h^{(i)} \Delta x_{\text{nt}}^{(i)} = x^{(i)} - h^{(i)} \left(\nabla^2 f(x^{(i)}) \right)^{-1} \nabla f(x^{(i)}),$$

where $h^{(i)} > 0$ is chosen so that the step yields descent.

Newton's method is more expensive per iteration than gradient descent because it requires the Hessian and the solution of a linear system. On the other hand, when the iterate is sufficiently close to the optimum, Newton's method can converge much faster. This is one of the reasons why Newton-type ideas appear inside many high-performance optimization algorithms for MPC.

11.5.4 Line search

The step size in a descent method is often computed by line search. Given a direction Δx , for instance the Newton direction Δx_{nt} , one wants to find

$$h^{(i)} > 0$$

such that

$$f(x^{(i)} + h^{(i)} \Delta x_{\text{nt}}) < f(x^{(i)}).$$

The most direct possibility is exact line search, where one solves the one-dimensional optimization problem

$$h^{(i)\star} = \arg \min_{h>0} f(x^{(i)} + h\Delta x_{\text{nt}}).$$

This gives the best step size along the chosen direction. However, it can be time consuming because it may require many evaluations of f . For this reason, exact line search is often not the preferred choice in real-time MPC algorithms.

A cheaper alternative is inexact line search. The goal is not to find the best step size, but only to find a step size that decreases the objective by a sufficient amount. A standard example is backtracking line search. Choose parameters

$$\alpha \in (0, 0.5), \quad \beta \in (0, 1).$$

Initialize

$$h^{(i)} = 1.$$

Then repeat the update

$$h^{(i)} \leftarrow \beta h^{(i)}$$

as long as

$$f(x^{(i)} + h^{(i)}\Delta x_{\text{nt}}) > f(x^{(i)}) + \alpha h^{(i)} \nabla f(x^{(i)})^\top \Delta x_{\text{nt}}.$$

Once this inequality is no longer true, the step size is accepted. This condition is known as a sufficient-decrease condition. It guarantees that the step is not merely decreasing the objective, but decreasing it by an amount compatible with the local slope in the chosen direction.

11.5.5 Equality constraints in Newton's method

Newton's method can also be adapted to equality-constrained problems. Consider

$$\begin{aligned} \min_x \quad & f(x) \\ \text{subj. to} \quad & Ax = b, \end{aligned}$$

where $A \in \mathbb{R}^{m \times n}$. At the current iterate $x^{(i)}$, the Newton direction is obtained by minimizing the quadratic model of f subject to the equality constraint on the next iterate. Written in terms of the step Δx , this gives

$$\begin{aligned} \Delta x_{\text{nt}}(x^{(i)}) \in \arg \min_{\Delta x} \quad & \frac{1}{2} \Delta x^\top \nabla^2 f(x^{(i)}) \Delta x + \nabla f(x^{(i)})^\top \Delta x \\ \text{subj. to} \quad & A \Delta x = -Ax^{(i)} + b. \end{aligned}$$

If the current iterate is feasible, namely

$$Ax^{(i)} = b,$$

then the constraint on the Newton direction becomes

$$A \Delta x = 0.$$

Thus the Newton direction lies in the null space of A . Consequently, for any step size $h^{(i)}$,

$$\begin{aligned} Ax^{(i+1)} &= A(x^{(i)} + h^{(i)}\Delta x_{\text{nt}}) \\ &= Ax^{(i)} + h^{(i)}A\Delta x_{\text{nt}} \\ &= b. \end{aligned}$$

Therefore, if the initial iterate is feasible, all subsequent iterates remain feasible.

The computation of the equality-constrained Newton direction amounts to solving a linear system of equations. The optimality conditions of the quadratic subproblem, with multiplier $\nu \in \mathbb{R}^m$, are

$$\begin{aligned}\nabla^2 f(x^{(i)})\Delta x + \nabla f(x^{(i)}) + A^\top \nu &= 0, \\ A\Delta x &= 0.\end{aligned}$$

Equivalently,

$$\begin{bmatrix} \nabla^2 f(x^{(i)}) & A^\top \\ A & 0 \end{bmatrix} \begin{bmatrix} \Delta x \\ \nu \end{bmatrix} = \begin{bmatrix} -\nabla f(x^{(i)}) \\ 0 \end{bmatrix}.$$

This saddle-point system is an important building block for constrained optimization. In particular, closely related KKT systems appear in interior-point methods and active-set methods for the quadratic programs that arise in MPC.

11.6 Constrained Minimization

We now move from unconstrained minimization to constrained optimization. This is the setting that is most relevant for MPC, since the finite-horizon problem contains input constraints, state constraints, and often terminal constraints. In the convex case, the most common numerical methods can be grouped into three broad families.

Gradient-descent methods are based on the same idea as in the unconstrained case: the gradient gives the direction of steepest local ascent, so one tries to move in the anti-gradient direction. The main difference is that the constraints must now be respected. This can be done, for example, by projecting the tentative gradient step back onto the feasible set.

Interior-point methods take a different viewpoint. Instead of explicitly guessing which constraints will be active at the solution, they solve a sequence of relaxed KKT systems. These systems are usually solved using Newton's method. As the relaxation is reduced, the iterates approach the true constrained optimum.

Active-set methods are based on the idea that the main difficulty is to identify which inequality constraints are active at the optimum. If the active set were known, the problem would locally reduce to an equality-constrained optimization problem. Active-set methods therefore try to identify this set iteratively. This is especially useful for MPC QPs, because consecutive problems are often similar and the active set from the previous sampling instant can be used as a warm start.

11.6.1 Projected gradient method

Consider the constrained convex optimization problem

$$\begin{aligned}\min_x \quad & f(x) \\ \text{subj. to} \quad & x \in Q,\end{aligned}\tag{P}$$

where Q is a convex set and f is convex and L -smooth. In unconstrained gradient descent, one would take the step

$$x^{(i)} - \frac{1}{L} \nabla f(x^{(i)}).$$

With constraints, this tentative point may lie outside the feasible set. The projected-gradient method therefore incorporates the constraints by projecting the tentative step back onto Q :

$$x^{(i+1)} = \pi_Q \left(x^{(i)} - \frac{1}{L} \nabla f(x^{(i)}) \right).$$

Here π_Q denotes the Euclidean projection onto Q , defined as

$$\pi_Q(y) := \arg \min_x \frac{1}{2} \|x - y\|_2^2 \quad \text{subj. to } x \in Q.$$

Thus the method has a very intuitive interpretation. First, one moves in the anti-gradient direction, exactly as in the unconstrained case. Then, if the tentative point is infeasible, one replaces it by the closest feasible point.

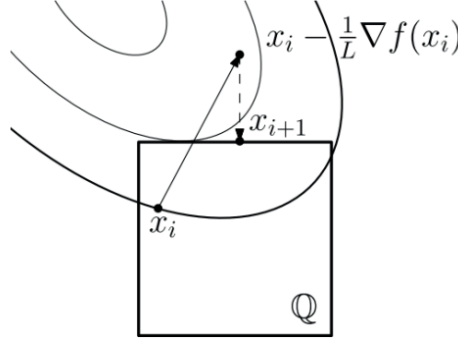


Figure 11.6: Projected-gradient method for constrained minimization. The method first takes an unconstrained gradient step and then projects the result back onto the feasible set Q . The projection $\pi_Q(y)$ is the closest feasible point to the tentative point y .

If f is L -smooth, one can choose the same constant step size as in the unconstrained case,

$$h^{(i)} = \frac{1}{L}.$$

The convergence rates are then analogous to those of the unconstrained gradient method, at least when the projection can be computed efficiently.

The practical usefulness of projected gradient depends almost entirely on the projection. If Q is simple, such as a box, a ball, or another set with a closed-form projection, then the method is very attractive. If Q is a complicated polyhedron or an intersection of many coupled constraints, then computing π_Q may itself require solving a nontrivial optimization problem. In that case, the simplicity of the method can be lost.

11.6.2 Implications for MPC

The projected-gradient idea has a direct interpretation in MPC. Consider first a condensed MPC formulation in which the predicted states have been eliminated and the only decision variable is the stacked input vector u . If the only constraints are simple input constraints, then the problem may have the form

$$\begin{aligned} \min_u \quad & \frac{1}{2} u^\top H u + x_0^\top F u \\ \text{subj. to} \quad & u \in \mathcal{U}. \end{aligned}$$

If $\mathcal{U}$ is a simple set, for example a box constraint on the inputs, then projection onto $\mathcal{U}$ is cheap. Projected-gradient methods are then natural: one takes a gradient step in the input sequence and projects the result back onto the admissible input set.

The situation becomes more subtle when state constraints are present. In a sparse formulation, one keeps both states and inputs as decision variables and writes

$$z := (x, u).$$

The simple constraints can be collected into the product set

$$K := \mathcal{X} \times \mathcal{U}.$$

Projection onto K may be easy if $\mathcal{X}$ and $\mathcal{U}$ are simple sets. However, the dynamics also impose equality constraints of the form

$$Az = b,$$

where this compact equation represents the stacked system dynamics and the initial condition. The true feasible set is therefore

$$K \cap \{z \mid Az = b\}.$$

Even if projection onto K is easy and projection onto the affine set $\{z \mid Az = b\}$ is manageable, projection onto their intersection is generally not easy.

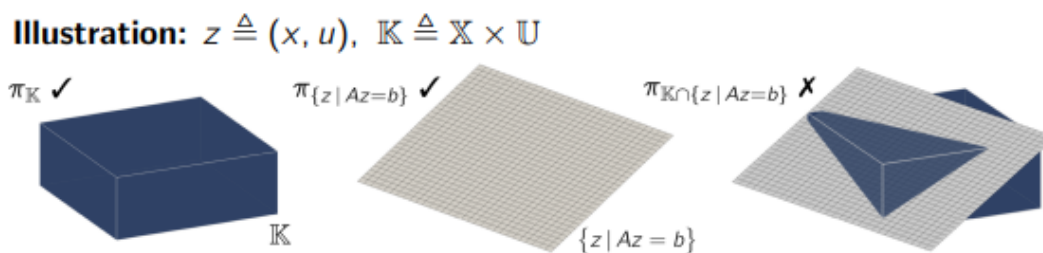


Figure 11.7: Implications of projected-gradient methods for MPC. With simple input constraints in a condensed formulation, projections may be cheap. With state constraints and dynamics, the feasible set becomes an intersection between simple constraint sets and the affine dynamics constraint, and projection onto the intersection is no longer straightforward.

This explains why the formulation of the MPC problem matters so much. A condensed formulation may lead to a smaller problem, but it can destroy sparsity. A sparse formulation preserves the dynamical structure, but it introduces equality constraints and coupled feasibility conditions. Depending on the problem, one may prefer primal methods, dual methods, projected-gradient methods, interior-point methods, or active-set methods.

In the presence of state constraints, one common strategy is to dualize some constraints or the dynamics. The resulting algorithms do not require projection onto the full intersection directly; instead, they alternate between simpler primal operations and dual updates. The key message is that first-order methods are attractive for MPC only when the constraint handling can be made cheap.

11.7 Interior-Point Methods

11.7.1 Constrained problem and KKT conditions

We now consider interior-point methods. The starting point is a constrained convex optimization problem with inequality and equality constraints:

$$\begin{aligned} \min_x \quad & f(x) \\ \text{subj. to} \quad & g_i(x) \leq 0, \quad i = 1, \dots, m, \\ & Cx = d. \end{aligned}$$

We assume that f and the functions g_i are convex and twice continuously differentiable. We also assume that the optimal value $f(x^*)$ is finite and attained, that the feasible set is closed and compact, and that strict feasibility holds. Strict feasibility means that there exists a point $\tilde{x}$ such that

$$\tilde{x} \in \text{dom } f, \quad g_i(\tilde{x}) < 0, \quad i = 1, \dots, m,$$

and

$$C\tilde{x} = d.$$

Under these assumptions, the KKT conditions can be used to characterize optimality.

To prepare the problem for a primal-dual interior-point method, we introduce slack variables

$$s \in \mathbb{R}^m, \quad s_i \geq 0.$$

The inequalities are rewritten as equalities:

$$g_i(x) + s_i = 0, \quad i = 1, \dots, m.$$

The standard KKT conditions then consist of stationarity,

$$\nabla f(x^*) + \sum_{i=1}^m \lambda_i^* \nabla g_i(x^*) + C^\top \nu^* = 0,$$

primal feasibility,

$$Cx^* = d, \quad g_i(x^*) + s_i^* = 0, \quad i = 1, \dots, m,$$

dual feasibility,

$$\lambda_i^* \geq 0, \quad s_i^* \geq 0,$$

and complementarity,

$$\lambda_i^* s_i^* = 0, \quad i = 1, \dots, m.$$

Complementarity is the difficult part. At the optimum, for each inequality constraint, either the slack is positive and the multiplier is zero, or the slack is zero and the multiplier may be positive. Thus complementarity encodes the active-set structure of the solution.

11.7.2 Primal-dual interior-point idea

Interior-point methods avoid deciding explicitly which constraints are active. Instead, they replace the exact complementarity condition

$$\lambda_i s_i = 0$$

by a relaxed condition

$$\lambda_i s_i = \kappa, \quad \kappa > 0.$$

Equivalently, the relaxed system is written as

$$\begin{aligned} \nabla f(x^*) + \sum_{i=1}^m \lambda_i^* \nabla g_i(x^*) + C^\top \nu^* &= 0, \\ Cx^* &= d, \\ g_i(x^*) + s_i^* &= 0, \quad i = 1, \dots, m, \\ \lambda_i^* g_i(x^*) &= -\kappa, \quad i = 1, \dots, m, \\ \lambda_i^*, s_i^* &\geq 0, \quad i = 1, \dots, m. \end{aligned} \tag{**}$$

The form $\lambda_i^* g_i(x^*) = -\kappa$ is equivalent to $\lambda_i^* s_i^* = \kappa$, since $g_i(x^*) + s_i^* = 0$.

For a fixed positive value of κ , the solution of the relaxed equations lies strictly in the interior with respect to the nonnegativity constraints:

$$\lambda_i > 0, \quad s_i > 0.$$

As κ is reduced toward zero, the relaxed solution approaches the true KKT point. The set of relaxed solutions obtained for different positive values of κ is called the primal-dual central path:

$$\mathcal{C} := \{(x, \nu, \lambda, s) \mid (**) \text{ holds for some } \kappa > 0\}.$$

A primal-dual interior-point method follows this central path toward the optimum while gradually reducing κ .

The word primal-dual emphasizes that the method solves for the primal variables x , the equality multipliers ν , the inequality multipliers λ , and the slack variables s simultaneously. The relaxed KKT system is solved by Newton's method, together with additional safeguards and line search to preserve positivity and to ensure progress.

Interior-point principle

Primal-dual interior-point methods solve the primal and dual problems simultaneously by following the relaxed KKT equations. The complementarity condition is relaxed from $\lambda_i s_i = 0$ to $\lambda_i s_i = \kappa$, and κ is reduced to zero as the method approaches the optimum.

11.7.3 Primal-dual search direction

At each iteration, the relaxed KKT system is linearized and a Newton direction is computed. Define

$$S := \text{diag}(s_1, \dots, s_m), \quad \Lambda := \text{diag}(\lambda_1, \dots, \lambda_m),$$

and

$$G(x)^\top := \begin{bmatrix} \nabla g_1(x) & \cdots & \nabla g_m(x) \end{bmatrix}.$$

The Hessian of the Lagrangian with respect to x is

$$H(x, \lambda) := \nabla^2 f(x) + \sum_{i=1}^m \lambda_i \nabla^2 g_i(x).$$

Linearizing the relaxed KKT equations gives the Newton system

$$\begin{bmatrix} H(x, \lambda) & C^\top & G(x)^\top & 0 \\ C & 0 & 0 & 0 \\ G(x) & 0 & 0 & I \\ 0 & 0 & S & \Lambda \end{bmatrix} \begin{bmatrix} \Delta x \\ \Delta \nu \\ \Delta \lambda \\ \Delta s \end{bmatrix} = - \begin{bmatrix} \nabla f(x) + C^\top \nu + G(x)^\top \lambda \\ Cx - d \\ g(x) + s \\ S\lambda - v \end{bmatrix}.$$

The vector $v \in \mathbb{R}^m$ modifies the right-hand side of the complementarity equation. We denote the resulting direction by

$$\Delta[x, \nu, \lambda, s](v).$$

Different choices of v give different search directions.

If

$$v = 0,$$

one obtains the pure Newton direction for the uncentered complementarity equations. This is often called the predictor direction or affine-scaling direction. It points aggressively toward the boundary and predicts how the method would move if it tried to reduce complementarity rapidly.

If

$$v = \kappa \mathbf{1},$$

one obtains a centering direction. This direction pulls the iterate toward the central path, preventing it from approaching the boundary too aggressively.

In practice, primal-dual methods combine these ideas. One chooses

$$v = \sigma \kappa \mathbf{1}, \quad \sigma \in (0, 1),$$

where σ is a centering parameter. This gives a search direction that both reduces complementarity and keeps the iterates sufficiently close to the central path.

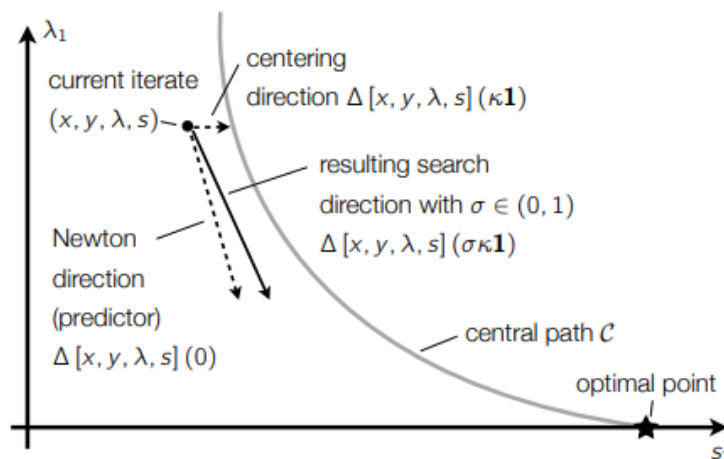


Figure 11.8: Search directions in primal-dual interior-point methods. The predictor direction aims toward the optimum by reducing complementarity aggressively. The centering direction pulls the iterate back toward the central path. Practical methods combine the two effects using a centering parameter $\sigma \in (0, 1)$.

After computing the direction, a step size must be chosen. The step cannot be arbitrary, because the method must preserve

$$s + \alpha \Delta s > 0, \quad \lambda + \alpha \Delta \lambda > 0.$$

Therefore the Newton direction is combined with a line search and additional safeguards. The goal is to decrease residuals and complementarity while remaining in the interior of the nonnegative orthant.

11.7.4 Interior-point methods in MPC

Interior-point methods are widely used in MPC because the optimization problems arising from linear MPC are typically LPs or QPs with a highly structured KKT system. In a sparse MPC formulation, the equality constraints come from the dynamics over the prediction horizon. Consequently, the Newton systems have a block structure that can be exploited computationally.

The main strength of interior-point methods is that they handle many inequality constraints without explicitly identifying the active set. This makes them robust and reliable for large convex optimization problems. The main computational cost is the solution of a linear system at each Newton step. Therefore, efficient MPC implementations exploit sparsity, block structure, fixed horizon dimensions, and warm starts whenever possible.

From the MPC viewpoint, interior-point methods are attractive when the problem is too large for explicit MPC and when one wants a reliable online solver. They are especially effective when the solver is tailored to the optimal-control structure rather than treating the MPC problem as a generic dense optimization problem.

11.8 Implementation Tradeoffs

The choice between explicit MPC and online optimization depends on the application. Explicit MPC moves the main computational effort offline. The online controller only performs point location and affine evaluation, so the runtime can be extremely small. This is ideal for very fast small-scale systems. The limitation is that the number of critical regions can grow rapidly, and the offline computation and memory storage may become prohibitive.

Online optimization avoids storing the full explicit controller. This makes it much more scalable with respect to state dimension, input dimension, horizon length, and number of constraints. The price is that an optimization problem must be solved at every sampling instant. Therefore the solver must be fast, reliable, and well matched to the structure of the MPC formulation.

Gradient-based methods are attractive when each iteration is very cheap, especially when projections are simple. Active-set methods are attractive for QPs with warm starts, because successive MPC problems tend to have similar active sets. Interior-point methods are powerful for larger convex problems and are especially effective when they exploit the sparse optimal control structure. Embedded solvers and code generation are important because they turn these algorithmic ideas into real-time implementations.

11.9 Summary

This chapter discussed how MPC can be implemented computationally. The starting point is that at each sampling instant MPC requires the solution of a finite-horizon optimization problem. There are two main implementation philosophies.

The first is explicit MPC. For linear MPC with polyhedral constraints and quadratic or linear-norm costs, the finite-horizon problem can be interpreted as a multiparametric optimization problem with the current state as parameter. The resulting feedback law is piecewise affine on a polyhedral partition of the feasible set. Online evaluation then consists of point location followed by affine evaluation. This can be extremely fast, but it is practical only for relatively small systems.

The second is online numerical optimization. In this case, the optimization problem is solved at every sampling instant. We first reviewed unconstrained minimization, where descent methods, gradient descent, Newton's method, and line search provide the basic algorithmic ideas. We then considered constrained minimization. Projected-gradient methods incorporate constraints by projecting tentative gradient steps back onto the feasible set. This is effective when the projection is simple, but may be difficult when the feasible set is an intersection of dynamics, state constraints, and input constraints.

Finally, we discussed primal-dual interior-point methods. These methods solve relaxed KKT systems, follow the central path, and reduce the complementarity parameter toward zero. They are powerful tools for the convex optimization problems that arise in MPC, especially when the structure of the finite-horizon problem is exploited.

The main message is that MPC design is not complete until the optimization problem can be solved reliably within the sampling time. Theoretical guarantees, problem formulation, algorithm choice, and implementation platform must therefore be considered together.